

The Backend Engineer's Library

Networking

Volume 3

Contents

The Journey of a Packet: The Stack End to End

IP, Routing, Subnetting, and NAT

TCP in Depth: Handshake, State Machine, Flow and Congestion Control

UDP and QUIC

DNS in Depth

TLS 1.3 and the Web PKI

HTTP/1.1, HTTP/2, and HTTP/3

gRPC and RPC Framework Internals

Load Balancing: L4, L7, and Algorithms

Proxies, Reverse Proxies, Service Mesh, and CDNs

Network Reliability: Timeouts, Retries, Backoff, and Hedging

Debugging and Observing Networks

Chapter 1 — The Journey of a Packet: The Stack End to End

What this chapter covers. Before we spend eleven chapters dissecting TCP congestion control, QUIC's loss recovery, the TLS 1.3 handshake, and the internals of a service mesh sidecar, we need one shared mental model that the rest of the volume can elaborate: what actually happens, mechanically, when your service executes `client.Get("https://api.example.com/v1/orders")`. That single line of code sets off a cascade of DNS lookups, address resolution, a transport handshake, a cryptographic handshake, an application exchange, and an orderly teardown — and underneath all of it, bytes are being wrapped in headers, handed down through kernel layers, serialized onto a wire, switched and routed hop by hop across a data-center fabric or the public Internet, and unwrapped in the reverse order on the far side. This chapter traces that journey end to end. It is deliberately broad: almost every topic here gets its own dedicated chapter later, and I forward-reference aggressively so you know where the deep dive lives. The goal is not to master any one layer but to be able to *place* any networking phenomenon — a timeout, a black hole, a throughput ceiling, a partial outage — at the correct layer, because knowing which layer a problem lives in is most of the work of fixing it.

This chapter builds directly on Volume 2, Chapter 10 (The Linux Network Stack), which traced a packet through the kernel of a *single host*. Here we zoom out: that per-host stack is one node in a path of many, and we follow the bytes across the wire between nodes.

Learning goals — after this chapter you should be able to:

- Explain the OSI 7-layer reference model and the practical TCP/IP model, and map real protocols and real failures onto specific layers.
- Trace encapsulation and decapsulation precisely: which header is added and stripped at each layer, and why each field exists.
- Walk a concrete HTTPS request from `connect()` to teardown, naming DNS, ARP, the TCP three-way handshake, the TLS handshake, the HTTP exchange, and connection close.

- Distinguish what a **switch** does (L2 MAC learning and forwarding) from what a **router** does (L3 longest-prefix forwarding and TTL decrement), and predict a frame's path hop by hop.
- Reason correctly about MTU, IP fragmentation, and Path MTU Discovery, and recognize the PMTUD black-hole failure mode.
- Use the performance vocabulary precisely — bandwidth, latency, RTT, throughput, goodput, bandwidth-delay product — and decompose end-to-end delay into serialization, propagation, queuing, and processing components.
- Apply the distributed-systems lens: understand that one user request fans out into dozens of internal RPCs, each traversing the full stack twice, and that per-hop RTT and BDP, not raw link bandwidth, govern what you actually observe.

Two models: OSI reference vs. the practical TCP/IP stack

Networking is taught with two layered models, and you need both because engineers use both in conversation. The **OSI (Open Systems Interconnection) 7-layer model**, standardized as ISO/IEC 7498-1, is a reference framework — a vocabulary — not a description of any protocol stack anyone actually ships. From the bottom: (1) Physical, the electrical/optical signaling; (2) Data Link, framing and local addressing; (3) Network, global addressing and routing; (4) Transport, end-to-end delivery and multiplexing; (5) Session, dialog management; (6) Presentation, serialization and encryption; (7) Application. The value of OSI is shared shorthand: when someone says "that's an L7 load balancer" or "we lost L2 connectivity" or "it's a layer-3 routing problem," they are pointing at a specific band of responsibility, and the number communicates it unambiguously.

The problem is that the real Internet protocol suite does not have seven layers. The stack that actually runs — the **TCP/IP model**, described conceptually in RFC 1122 (Requirements for Internet Hosts) — collapses into four or five layers, and the OSI session and presentation layers essentially do not exist as separate layers at all. TLS, which does OSI's presentation-layer job (encryption) and some session work, is in practice implemented *inside* the application, riding on top of TCP. So working engineers use a pragmatic 5-layer model:

#	Practical TCP/IP layer	Job	Addressing / unit	Example protocols
5	Application	Semantics of the exchange	— / message	HTTP, gRPC, DNS, SMTP
4	Transport	End-to-end delivery, multiplexing	Port / segment or datagram	TCP, UDP, QUIC ¹
3	Network (Internet)	Global addressing, routing	IP address / packet	IPv4, IPv6, ICMP
2	Data Link	Local hop delivery, framing	MAC address / frame	Ethernet, Wi-Fi, ARP
1	Physical	Signaling on the medium	— / bits & symbols	10GBASE-T, fiber, PAM4

¹ QUIC is an interesting wrinkle: it runs *over* UDP (L4) but internally provides streams, reliability, and its own TLS-integrated handshake, so it occupies transport and part of the application band simultaneously. We give it a full treatment in Chapter 4 (UDP and QUIC).

Two properties make layering powerful. First, **encapsulation**: each layer treats the layer above’s output as opaque payload and prepends its own header. Second, **peer-to-peer abstraction**: each layer on the sender conceptually talks to the same layer on the receiver, even though the actual bytes travel all the way down one stack, across the wire (possibly through many intermediate devices), and all the way up the other. TCP on your host behaves as if it has a direct conversation with TCP on the server, exchanging sequence numbers and ACKs, blissfully unaware that between them sit a dozen routers that never looked past the IP header.

The layering also tells you *what each device on the path is allowed to look at*. A plain Ethernet switch reads only L2. A router reads L3. A stateful firewall or NAT reads L3 and L4. An L7 load balancer or reverse proxy terminates all the way up to the application. This is not academic: it is exactly why a TLS-terminating proxy can route by HTTP path but a bare L4 load balancer cannot, and why a network partition at L2 looks completely different from a routing loop at L3. Hold onto this — it is the through-line of the whole volume.

Encapsulation and decapsulation

When your application hands a buffer of HTTP bytes to the kernel via `write()` on a socket, that buffer descends the stack. Each layer wraps it. By the time it reaches the wire it is a nested set of headers around your original payload, like envelopes inside envelopes.

```
flowchart TB
    subgraph TX["Sender: encapsulation (top to bottom)"]
        A["L7 App data: HTTP request bytes"]
        B["L4 TCP: prepend TCP header (ports, seq, ack, flags, window)"]
        C["L3 IP: prepend IP header (src IP, dst IP, TTL, protocol)"]
        D["L2 Ethernet: prepend frame header (src MAC, dst MAC, EtherType) + FCS trailer"]
        E["L1 Physical: encode frame as symbols → bits on the wire"]
        A --> B --> C --> D --> E
    end
    E -->| "medium" | F
    subgraph RX["Receiver: decapsulation (bottom to top)"]
        F --> G["L1: recover bits → frame"]
        G --> H["L2: check FCS, strip Ethernet header → packet"]
        H --> I["L3: check dst IP, strip IP header → segment"]
        I --> J["L7: HTTP request delivered to server process"]
        F --> G --> H --> I --> J
    end
```

Walk the headers concretely for an HTTP-over-TCP-over-IPv4-over-Ethernet request. Your payload starts as, say, 800 bytes of `GET /v1/orders HTTP/1.1\r\nHost: api.example.com\r\n...`

- **TCP (20 bytes minimum, often 32 with options)** prepends source and destination *port* numbers (the multiplexing keys), a 32-bit *sequence number* naming the first byte of this segment in the byte stream, a 32-bit *acknowledgment number*, control flags (`SYN` , `ACK` , `FIN` , `RST` , `PSH`), a *window* field for flow control, and a checksum. The result is a **segment**. Chapter 3 (TCP in Depth) dissects every field and the state machine behind them.
- **IP (20 bytes for IPv4, 40 for IPv6)** prepends the source and destination *IP addresses*, a *TTL* (time-to-live, decremented at each router), a *protocol* field (6 = TCP, 17 = UDP, 1 = ICMP) telling the receiver which L4 to hand the payload to, and — for IPv4 —

fragmentation fields. The result is a **packet** (or *datagram*). Chapter 2 (IP, Routing, Subnetting, and NAT) is the deep dive.

- **Ethernet (14-byte header + 4-byte FCS trailer)** prepends the *destination* and *source* MAC addresses and a 2-byte *EtherType* (0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP), and appends a 4-byte *Frame Check Sequence* (a CRC-32). The result is a **frame**.
- **Physical** encodes the frame's bits as line symbols — voltage transitions on copper, light pulses on fiber, PAM4 or similar modulation at high rates.

On the receiver each layer performs the inverse — **decapsulation** — validating and stripping its own header before passing the remaining payload up. The Ethernet layer verifies the FCS and discards corrupt frames silently (this is why L2 corruption shows up as loss, not errors, to the layers above). The IP layer confirms the destination address is local and reads the protocol field to choose the L4 handler. The TCP layer uses the four-tuple (*source IP, source port, destination IP, destination port*) to demultiplex the segment to the correct socket, reorders by sequence number, and delivers an in-order byte stream to `read()`.

Two observations that matter for the rest of the volume. First, **every header is overhead**. A minimal TCP/IPv4/Ethernet stack costs 54 bytes of headers per packet before any payload; on a 1500-byte frame that is ~3.6% off the top, and it is much worse for tiny packets — a bare ACK is all header and no payload. This is why chatty, small-message protocols waste bandwidth and why techniques like TCP segmentation offload and jumbo frames exist. Second, **the layer that added a header is the only one that should interpret it**. When you debug with `tcpdump`, you are reading these nested headers directly; learning to read them top-down is the single most useful network-debugging skill, and Chapter 12 (Debugging and Observing Networks) builds it systematically.

A concrete request: calling `https://api.example.com/v1/orders`

Now the main event. Your service, running in a pod on a Linux host, executes an HTTPS GET. We will follow it from the first system call to the last, noting where each step gets its own chapter. Assume the client has never contacted this host before, so nothing is cached — the cold path is the instructive one.

```
sequenceDiagram
    participant App as Client app
    participant Res as Stub resolver
    participant DNS as DNS server
    participant OS as Client kernel
    participant GW as Default gateway (router)
    participant Srv as api.example.com

    App->>Res: getaddrinfo("api.example.com")
    Res->>DNS: A / AAAA query (UDP 53)
    DNS-->>Res: 203.0.113.10
    Res-->>App: 203.0.113.10
    Note over OS,GW: Not on local subnet → send to gateway. ARP to find gateway MAC.
    App->>OS: connect() to 203.0.113.10:443
    OS->>GW: TCP SYN
    GW->>Srv: (routed hop by hop)
    Srv-->>OS: SYN-ACK
    OS->>Srv: ACK (handshake complete)
    OS->>Srv: TLS ClientHello
    Srv-->>OS: ServerHello, cert, Finished
    OS->>Srv: Finished (TLS established)
    App->>Srv: HTTP GET /v1/orders (encrypted)
    Srv-->>App: HTTP 200 + body (encrypted)
    App->>Srv: FIN
    Srv-->>App: FIN-ACK
```

Step 1 — DNS resolution: name to address

`https://api.example.com/...` names a *host*, but IP can only route to *addresses*. The client calls `getaddrinfo()`, which consults the stub resolver (configured via `/etc/resolv.conf`, `/etc/nsswitch.conf`, or on a modern Linux desktop `systemd-resolved`). If the answer is not cached, the resolver sends a DNS query — typically a UDP datagram to port 53 of a recursive resolver — asking for the `A` record (IPv4) and/or `AAAA` record (IPv6). The recursive resolver, if it too lacks a cached answer, walks the delegation hierarchy: root servers → `.com` TLD servers → the authoritative servers for `example.com`, following referrals until it gets the authoritative answer, say `203.0.113.10`. That answer comes back with a TTL governing how long it may be cached.

DNS is a distributed, hierarchical, cached database with its own failure modes — stale records, negative caching, split-horizon views, the interaction with TTLs during failover — and it gets a full treatment in Chapter 5 (DNS in Depth). For now the essential point is that name resolution is a *separate network round-trip (or several)* that happens

before your connection even begins, and its latency and reliability are part of your request's latency and reliability. A slow or flapping resolver is a classic invisible cause of tail latency.

Step 2 — Is the destination local? Routing decision and ARP

Now the kernel has a destination IP, `203.0.113.10`, and must decide how to send the first packet. It consults the routing table (`ip route`). It compares the destination against each route's prefix. If the destination is on a directly connected subnet, the next hop is the destination itself. If not — as here, since `203.0.113.10` is not on the pod's `/24` — the packet goes to the **default gateway**, the router whose address is the `default` route's next hop.

Either way, to build the Ethernet frame the kernel needs the *MAC address* of the next hop on the local link — the gateway's MAC, not the server's. IP addresses route across the Internet; MAC addresses deliver across a single link. To translate the gateway's IP into its MAC, the host uses **ARP** (Address Resolution Protocol, RFC 826) on IPv4, or Neighbor Discovery (NDP, ICMPv6) on IPv6. ARP is a broadcast: the host sends an Ethernet frame to `ff:ff:ff:ff:ff:ff` asking "who has `10.0.0.1`? tell `10.0.0.5`," and the gateway replies with its MAC. The result is cached in the neighbor table (`ip neigh`) for reuse.

```
$ ip route get 203.0.113.10
203.0.113.10 via 10.0.0.1 dev eth0 src 10.0.0.5 uid 1000

$ ip neigh
10.0.0.1 dev eth0 lladdr 52:54:00:a1:b2:c3 REACHABLE
```

Note the crucial asymmetry: the frame's destination MAC is the gateway (`52:54:00:a1:b2:c3`), but the packet's destination IP is the far-off server (`203.0.113.10`). At each hop, the L2 addresses are rewritten while the L3 addresses stay the same — the mechanism we dissect under "hop by hop" below.

Step 3 — TCP three-way handshake: establishing the connection

With an address resolved and a next-hop MAC in hand, `connect()` triggers the TCP three-way handshake. The client sends a **SYN** (an empty segment with the SYN flag set and an initial sequence number). The

server replies with **SYN-ACK** (SYN flag plus an ACK of the client's SYN, and its own initial sequence number). The client replies with **ACK**. After this one round-trip the connection is **ESTABLISHED** on both ends and either side may send data. During the handshake both sides also negotiate options in the SYN/SYN-ACK: the Maximum Segment Size (MSS), window scaling, selective acknowledgment (SACK) support, and timestamps.

The handshake costs one RTT before any application byte moves. That is a floor on connection setup latency and a major reason connection pooling and keep-alive matter so much — you amortize this cost across many requests. The full state machine (`SYN_SENT`, `SYN_RECV`, `ESTABLISHED`, `FIN_WAIT`, `TIME_WAIT`, and the rest), the SYN/accept queues, and congestion control are the subject of Chapter 3 (TCP in Depth). On the kernel side, this is exactly the SYN-queue / accept-queue machinery from Volume 2, Chapter 10.

Step 4 — TLS handshake: establishing a secure channel

Because the scheme is `https`, the client now performs a **TLS handshake** on top of the established TCP connection, before any HTTP is exchanged. In TLS 1.3 (RFC 8446) the handshake is one round-trip: the client sends `ClientHello` with its supported cipher suites and a key-share for (EC)DHE key agreement; the server responds with `ServerHello`, its certificate chain, a `CertificateVerify` signature, and `Finished`; the client verifies the certificate against the Web PKI, checks the name matches `api.example.com`, and sends its own `Finished`. Both sides derive symmetric session keys, and all subsequent traffic is encrypted and authenticated. (TLS 1.2 needed two round-trips; TLS 1.3 also offers 0-RTT resumption for returning clients, with a replay-safety caveat.)

So on a cold `https` connection you have already paid, in sequence: DNS lookup (one or more RTTs), the TCP handshake (one RTT), and the TLS handshake (one RTT in 1.3) — all before the first HTTP byte. On a cross-continent path with an ~80 ms RTT, that is easily 240+ ms of pure setup. This latency stack is *the* argument for connection reuse, session resumption, and edge termination. Chapter 6 (TLS 1.3 and the Web PKI) covers the handshake, certificate validation, and PKI trust in depth.

Step 5 — The HTTP request and response

Finally, application data. The client writes the HTTP request — request line, headers, optional body — into the (now encrypted) TCP stream. On HTTP/1.1 this is plaintext framing inside the TLS record layer; on HTTP/2 it is a `HEADERS` frame with HPACK-compressed headers on a new stream ID, multiplexed over the single connection; on HTTP/3 it rides QUIC streams over UDP. The server processes the request and streams back a status line/frame, response headers, and the body. Chapter 7 (HTTP/1.1, HTTP/2, and HTTP/3) covers all three wire formats and their very different multiplexing and head-of-line-blocking behavior; Chapter 8 (gRPC and RPC Framework Internals) covers gRPC, which is HTTP/2 with a specific message framing and trailer convention layered on top.

Each HTTP message is, of course, just payload as far as TCP is concerned: it is chopped into MSS-sized segments, each of which becomes an IP packet, each of which becomes an Ethernet frame, each of which is switched and routed independently to the destination and reassembled in order by the receiving TCP.

Step 6 — Teardown

When the exchange is done (or the connection idles past keep-alive), it is closed. TCP's graceful close is a four-way exchange: each side sends a **FIN** and receives an **ACK**, independently, because a TCP connection is a pair of independent byte streams (full-duplex). The side that closes actively enters `TIME_WAIT` and lingers there for a duration meant to be $2 \times \text{MSL}$ (maximum segment lifetime) — long enough to absorb any delayed duplicate segments and to ensure the final ACK was received. The RFC's nominal MSL is 2 minutes (so $2 \times \text{MSL}$ is 4 minutes), but Linux does not use MSL directly: it fixes `TIME_WAIT` at 60 seconds (the compiled-in `TCP_TIMEWAIT_LEN`), which is not runtime-tunable via a `sysctl`. `TIME_WAIT` accumulation on a busy client is a real operational concern — it can exhaust ephemeral ports — which is why keep-alive and connection pooling, and options like `SO_REUSEADDR`, matter, as covered in Volume 2, Chapter 10 and again in Chapter 3 here.

That is the whole life of a request. Everything below re-examines the individual layers this walk-through passed through quickly.

Layer 2: Ethernet, MACs, switches, VLANs, and the MTU

The data link layer moves frames across a *single* link or LAN segment. Its addressing is the **MAC address**: a 48-bit identifier, usually burned into (or assigned to) a NIC, written as six hex octets like `52:54:00:a1:b2:c3`. The least-significant bit of the first octet distinguishes unicast from multicast; the next bit up distinguishes globally-unique (OUI-assigned) from locally-administered addresses (which is what your virtual NICs and containers use — note the `2` in `52`, whose low bits mark it locally-administered unicast). MAC addresses are *flat* — they have no hierarchy and carry no location information — which is precisely why they cannot scale to the global Internet and why we need L3 on top.

A **switch** is an L2 device. Its entire job is to forward frames within a broadcast domain, and it does so by **MAC learning**. The switch maintains a *forwarding table* (also called a CAM or MAC table) mapping MAC addresses to physical ports. When a frame arrives, the switch records "source MAC X is reachable via port P" (learning), then looks up the destination MAC: if known, it forwards only out the corresponding port; if unknown, it **floods** the frame out every port except the ingress one, and the eventual reply teaches it the location. Broadcast frames (like ARP requests) are always flooded. This is why a switch is transparent — hosts do not address the switch, they address each other, and the switch silently learns the topology from traffic.

A **broadcast domain** is the set of ports across which a broadcast frame propagates. Left alone, a physical switch is one broadcast domain, which does not scale and does not isolate. The fix is the **VLAN** (IEEE 802.1Q): a 12-bit tag inserted into the Ethernet header that partitions one physical switch into multiple logical broadcast domains. Traffic in VLAN 10 cannot reach VLAN 20 without passing through a router, even on the same physical switch. VLANs are how a single physical fabric is sliced into isolated tenant or tier networks, and their logical descendant — VXLAN — is how overlay networks in Kubernetes and cloud VPCs tunnel L2 semantics across an L3 fabric. We touch VXLAN and overlays again in Chapter 10 (Proxies, Reverse Proxies, Service Mesh, and CDNs) and Volume 2, Chapter 9 (Namespaces and cgroups).

Finally, L2 imposes the **MTU** (Maximum Transmission Unit): the largest payload a single frame can carry. For standard Ethernet the MTU is **1500**

bytes. This one number ripples all the way up the stack: it bounds the IP packet size, which (minus IP and TCP headers) yields the TCP MSS — typically 1460 bytes for IPv4 (1500 – 20 IP – 20 TCP). "Jumbo frames" raise the MTU to ~9000 bytes, cutting per-byte header overhead and per-packet processing cost, and are common inside data centers where every device on the path can be configured consistently. The danger, as we will see, is a path where the MTU is *not* consistent.

Layer 3: IP addressing and routing

The network layer provides what L2 cannot: **global, hierarchical addressing** and **routing** across independently administered networks. An **IP address** is not flat; it is structured as a *network prefix* plus a *host part*, and that hierarchy is what makes routing tractable. IPv4 addresses are 32 bits (`203.0.113.10`); IPv6 addresses are 128 bits (`2001:db8::a`). A prefix like `203.0.113.0/24` means "the first 24 bits identify the network; the remaining 8 identify hosts within it." Routers reason about prefixes, not individual addresses, so one routing-table entry can represent millions of hosts.

Routing is the process each router performs on every packet: examine the destination IP, find the most specific matching prefix in the forwarding table, and send the packet out toward that prefix's next hop. "Most specific" means **longest-prefix match** — if both `10.0.0.0/8` and `10.1.2.0/24` match the destination, the `/24` wins because it is more specific. The default route `0.0.0.0/0` is the least specific possible match, the catch-all when nothing else fits. Addressing structure (subnetting, CIDR), how routers learn prefixes (static routes, and dynamic protocols like BGP and OSPF), and address translation (NAT, essential to how cloud VPCs and home networks work) are the entire subject of Chapter 2 (IP, Routing, Subnetting, and NAT).

The key L3 field for our end-to-end story is the **TTL (IPv4) / Hop Limit (IPv6)**: an 8-bit counter the source sets (commonly 64) and *every router decrements by one*. When it hits zero, the router drops the packet and sends back an ICMP "Time Exceeded" message. This is not a timer; it is a hop counter, and it exists to guarantee that a packet caught in a routing loop eventually dies rather than circulating forever. It is also the mechanism `traceroute` exploits: by sending packets with TTL 1, 2, 3, ... it provokes each successive router on the path to announce itself via the ICMP error, revealing the route hop by hop.

How a packet moves hop by hop: switches vs. routers

Now we can assemble the L2/L3 picture into the actual per-hop behavior — the single most important mechanism in this chapter. A frame's journey alternates between two operations: switching (within a link/LAN) and routing (between networks).

```
flowchart LR
    H1["Host A  
IP 10.0.0.5  
MAC aa:aa"]
    SW["Switch  
(L2: MAC learning,  
no header rewrite)"]
    R["Router / gateway  
IP 10.0.0.1  
MAC 52:54:...  
(L3: decrement TTL,  
rewrite L2 MACs)"]
    NET(("Internet  
many router hops"))
    H2["Host B  
IP 203.0.113.10  
MAC bb:bb"]

    H1 -->|"dst MAC = router, dst IP = B"| SW
    SW -->|"forward by MAC, frame unchanged"| R
    R -->|"new dst MAC = next hop, dst IP = B, TTL-1"| NET
    NET -->|"final router: dst MAC = B"| H2
```

Trace it precisely. Host A wants to reach Host B on a different network. A builds a frame with:

- **L3 (unchanged end to end):** source IP = A (10.0.0.5), destination IP = B (203.0.113.10).
- **L2 (rewritten every hop):** source MAC = A's, destination MAC = **the router's**, because the router is the next hop on A's link.

The **switch** between A and the router looks only at the destination MAC. It sees the router's MAC, consults its forwarding table, and sends the frame out the router's port — *without modifying a single field*. A switch is transparent; it does not appear in `traceroute`, it does not decrement TTL, it does not touch IP addresses. It forwards frames by MAC and learns locations as it goes. This is why L2 is fast and cheap and why switching scales to enormous throughput: it is a table lookup and a copy.

The **router** is where the real work happens. It receives the frame, strips the Ethernet header, and examines the IP header. It sees destination IP = B, performs a longest-prefix match against its forwarding table to find the next hop toward B, and then:

1. **Decrements the TTL** and recomputes the IPv4 header checksum (dropping the packet if TTL hit zero).
2. **Builds a brand-new Ethernet frame:** source MAC = the router's outgoing-interface MAC, destination MAC = the *next* hop's MAC (found via that interface's ARP table). The IP header — source and destination IP — is untouched.
3. Forwards the new frame out the appropriate interface.

Repeat this at every router across the Internet: the L3 addresses (A and B) are constant end to end, and the L2 addresses are rewritten hop by hop. The MAC addresses are *link-local*; they mean nothing beyond a single link. This is the crisp division of labor: **IP addressing gets the packet from network to network; MAC addressing gets the frame from device to device on one link.** Internalize this and NAT, overlay networks, load-balancer behavior, and half of your future `tcpdump` sessions become legible.

	Switch (L2)	Router (L3)
Reads	Destination MAC	Destination IP
Forwarding logic	Exact-match MAC table	Longest-prefix match
Rewrites headers?	No	Yes — new L2 MACs, TTL-1
Broadcast domain	Extends it	Terminates it
Visible to traceroute?	No	Yes
Learns topology by	MAC learning from traffic	Routing protocols / config

MTU, fragmentation, and Path MTU Discovery

Recall that each L2 link has an MTU, and a path may cross links with *different* MTUs — a host on a 9000-byte jumbo-frame data-center link, a

VPN or tunnel that adds encapsulation overhead and effectively lowers the MTU to, say, 1400 bytes, and a plain 1500-byte segment somewhere in between. What happens when a packet is too big for the next link?

Historically, IPv4 allowed **fragmentation**: a router facing a too-large packet with the "Don't Fragment" (DF) bit clear could split it into fragments, each an independent IP packet with offset fields, to be reassembled at the *destination* (never at intermediate routers). Fragmentation is now considered harmful and largely avoided. It is fragile — loss of any single fragment forces retransmission of the whole original packet — it burdens the destination with reassembly state, it interacts badly with firewalls and load balancers that need L4 headers present only in the first fragment, and it opens denial-of-service and evasion vectors. **IPv6 removes router fragmentation entirely**: routers never fragment, and a too-large packet is simply dropped with an ICMPv6 "Packet Too Big" message returned to the source.

The modern approach is **Path MTU Discovery** (PMTUD, RFC 1191 for IPv4, RFC 8201 for IPv6). The sender sets the DF bit on every packet and starts by assuming the local MTU. If a router along the path faces a link with a smaller MTU, it drops the packet and returns an ICMP "Fragmentation Needed" (IPv4, type 3 code 4) or "Packet Too Big" (ICMPv6) message *stating the next-hop MTU*. The sender lowers its assumed path MTU accordingly (TCP recomputes its effective MSS) and retransmits. Over a couple of iterations the sender converges on the smallest MTU along the path and sends packets that fit everywhere without fragmentation.

PMTUD's Achilles' heel is that it **depends on ICMP being delivered back to the sender**. A misconfigured firewall or security group that blindly drops all ICMP breaks it: the sender keeps sending too-large packets, the constricting router keeps silently dropping them (because the source never lowers its estimate), and the connection **black-holes** — the handshake succeeds (small packets), but the first large data segment vanishes and the connection hangs. This **PMTUD black hole** is a genuinely common and maddening production failure: small requests work, large responses hang, and it looks like an application bug until you notice it correlates with payload size. The mitigations are to never blanket-drop ICMP "too big" messages, and TCP's "MSS clamping" (and RFC 4821 Packetization Layer PMTUD, which probes with data segments

instead of relying on ICMP). This is a recurring theme in Chapter 12 (Debugging and Observing Networks).

Layer 4: ports, TCP vs. UDP

The transport layer adds two things the network layer lacks: **multiplexing** (via ports) and, optionally, **reliable ordered delivery**. IP delivers a packet to a *host*; it has no notion of *which process* should receive it. The 16-bit source and destination **port** numbers in the L4 header provide that. A listening server binds a well-known port (443 for HTTPS, 53 for DNS); a client picks an *ephemeral* port from a local range (Linux default roughly 32768–60999). The tuple that uniquely identifies a connection — the key the kernel demultiplexes on — is the **four-tuple**: (*source IP, source port, destination IP, destination port*). Two connections from the same client to the same server differ only in the client's ephemeral port, and that alone keeps their byte streams separate. This four-tuple is also why a single client can open at most ~28,000 concurrent connections to one (server IP, port) before ephemeral-port exhaustion — a limit that bites connection-heavy services and one reason `SO_REUSEPORT`, multiple destination IPs, and connection pooling exist.

The two dominant L4 protocols embody a fundamental trade-off:

- **TCP** is connection-oriented and reliable. It guarantees that bytes arrive in order, exactly once, with no gaps, by numbering every byte, acknowledging receipt, retransmitting losses, and pacing itself with flow control (respecting the receiver's buffer) and congestion control (respecting the network's capacity). You pay for this with the handshake RTT, per-connection state, and **head-of-line blocking** — a single lost segment stalls delivery of everything behind it until it is retransmitted, even if later bytes already arrived. Chapter 3 is the deep dive.
- **UDP** is connectionless and unreliable: it adds ports and a checksum to IP and nothing else. No handshake, no ordering, no retransmission, no congestion control. It is a thin veneer over IP's best-effort datagram service. That minimalism is a feature for latency-sensitive or broadcast/multicast workloads (DNS, real-time media, telemetry) and, crucially, it is the substrate on which **QUIC** builds a *better* reliable transport in user space — getting TCP's reliability and TLS's security while dodging TCP's head-of-line

blocking and kernel ossification. Chapter 4 (UDP and QUIC) covers both.

The choice of L4 is one of the highest-leverage decisions in a networked system, and it is almost always TCP or a TCP-like protocol for backend RPC — but understanding *why* requires understanding what TCP is buying you, which is the whole of Chapter 3.

The performance vocabulary: bandwidth, latency, throughput, goodput

Engineers routinely conflate terms that mean very different things, and the confusion leads to wrong diagnoses. Precise definitions:

- **Bandwidth** is *capacity*: the maximum bits per second a link can carry — a property of the medium and hardware (a 10 Gbit/s NIC, a 1 Gbit/s link). It is a ceiling, not a measurement of what you are achieving.
- **Latency** is *delay*: the time for one bit to travel from source to destination. Its round-trip form, **RTT** (Round-Trip Time), is what you measure with `ping` and what TCP's timers and windows revolve around.
- **Throughput** is the *actual* rate of bits delivered over the path — always $\leq$ bandwidth, and often far below it, because of protocol overhead, congestion, loss, and the window/RTT limit described below.
- **Goodput** is throughput counting only *useful application payload* — excluding headers, retransmissions, handshakes, and ACKs. Goodput is what your users actually get; the gap between throughput and goodput is pure overhead.

The relationship people get wrong most often: **bandwidth and latency are independent, and adding bandwidth does not reduce latency**. A wider pipe does not make bits arrive sooner; it lets *more* bits be in flight at once. Latency is floored by physics — the speed of light in fiber is roughly 200,000 km/s (about two-thirds of c), so a one-way trip across, say, 8,000 km of fiber is at minimum ~ 40 ms, ~ 80 ms round trip, *before* any queuing or processing. No amount of bandwidth changes that. This is why a 1 GB transfer between continents can be slow on a "fast" link, and why moving computation closer to data (edge, regional replicas) —

reducing *latency* — often matters more than provisioning more *bandwidth*.

The four components of delay

The end-to-end latency of one packet across one hop decomposes into four additive components. Understanding which one dominates tells you what to fix.

```
flowchart LR
    A["Processing delay  
router inspects header,  
routing lookup  
(microseconds)"]
    B["Queuing delay  
packet waits in buffer  
behind others  
(0 to milliseconds+,  
load-dependent)"]
    C["Serialization delay  
push all bits onto wire  
= packet size / bandwidth  
(bandwidth-dependent)"]
    D["Propagation delay  
bits travel the distance  
= distance / signal speed  
(distance-dependent)"]
    A --> B --> C --> D
```

- **Processing delay** — time for a node to examine a packet's headers, verify the checksum, and make a forwarding decision (the longest-prefix lookup). On modern hardware this is microseconds, usually negligible except in software routers or deep packet inspection.
- **Queuing delay** — time a packet waits in a buffer behind other packets before it can be transmitted. This is the *variable* component: near zero on an idle link, unbounded as a link saturates. Queuing delay is the source of most jitter and of the pathology called **bufferbloat**, where oversized buffers hold packets for tens or hundreds of milliseconds under load instead of dropping them, wrecking latency without improving throughput. Queuing delay is where load-dependent tail latency comes from.
- **Serialization (transmission) delay** — time to clock all of a packet's bits onto the wire, equal to *packet size / bandwidth*. A 1500-byte (12,000-bit) packet on a 1 Gbit/s link takes 12 μ s to serialize; on a 10

Mbit/s link, 1.2 ms. This is the *only* delay component that higher bandwidth actually reduces.

- **Propagation delay** — time for a bit to physically traverse the distance, equal to *distance / signal speed*. Governed by the speed of light in the medium and utterly indifferent to bandwidth. Dominant on long-haul links.

Summed over every hop, these four give end-to-end latency. The diagnostic value is knowing which dominates: a saturated link is queuing delay (fix congestion or add capacity), a long-haul link is propagation delay (move closer or accept it), a slow link with big packets is serialization delay (raise bandwidth or shrink packets).

Bandwidth-delay product: why throughput $\neq$ bandwidth

Here is the single most important quantitative relationship in transport performance. A reliable protocol like TCP can only have so much data "in flight" — sent but not yet acknowledged — because it must be able to retransmit anything unacknowledged, and because the receiver's window bounds it. The maximum useful amount of in-flight data is the **bandwidth-delay product (BDP)**:

$$\text{BDP (bytes)} = \text{bandwidth (bytes/sec)} \times \text{RTT (sec)}$$

The BDP is the "volume" of the pipe — how many bytes fit in the path at once. To *fill* the pipe and achieve throughput equal to bandwidth, the sender's in-flight window must be at least the BDP. If the window is smaller, throughput is capped at *window / RTT*, regardless of how much bandwidth is available.

A worked example makes the stakes clear. On a 1 Gbit/s path (125 MB/s) with a 1 ms RTT (intra-data-center), the BDP is $125 \text{ MB/s} \times 0.001 \text{ s} \approx \mathbf{125 \text{ KB}}$ — a modest window fills it. But on the *same* 1 Gbit/s bandwidth with an 80 ms RTT (cross-continent), the BDP is $125 \text{ MB/s} \times 0.08 \text{ s} \approx \mathbf{10 \text{ MB}}$. To saturate that link a single TCP connection needs a 10 MB in-flight window. If the OS default socket buffer or the receiver's advertised window is, say, 256 KB, throughput is capped at $256 \text{ KB} / 0.08 \text{ s} \approx \mathbf{3.2 \text{ MB/s}}$ — about 2.5% of the available gigabit. This is why "long fat networks" (high bandwidth $\times$ high latency) require **window scaling** (TCP's option to advertise windows beyond 64 KB) and generous, autotuned socket buffers, and it is why a single TCP stream often cannot

fill a fast, distant link while many parallel streams can. Every discussion of TCP tuning, of why you parallelize large transfers, and of why RTT (not bandwidth) is the throughput lever on long paths, comes back to the BDP. Chapter 3 develops the congestion-control dynamics behind it.

Where the kernel stack fits

Every layer transition in this chapter, on both hosts, happens *inside the kernel* — and Volume 2, Chapter 10 (The Linux Network Stack) is the mechanical account of that. When your application calls `write()` on a socket, control crosses the syscall boundary; the kernel copies the payload into a socket send buffer, the TCP layer segments it and manages sequence numbers and the congestion window, the IP layer looks up the route and fills the header, the netfilter/conntrack hooks run, the qdisc layer paces transmission, and the NIC driver DMAs the frame onto the wire — often with TCP Segmentation Offload doing the actual per-segment carving in hardware. On receive, an interrupt (moderated by NAPI polling) pulls frames off the NIC, softirq context walks them up through IP and TCP, reorders and reassembles the byte stream in the socket receive buffer, and wakes the process blocked in `read()` or reported ready by `epoll`.

The point of naming this here is placement. This chapter's "layers" are not abstract diagrams; each is concrete kernel code with concrete queues, buffers, and counters, and every one is a place where packets can be *dropped* — the SYN queue, the accept queue, the socket receive buffer, the qdisc, the NIC ring. When you tune `net.core.rmem_max` or `net.ipv4.tcp_rmem`, you are sizing the window that governs the BDP relationship above. When you see `netstat -s` or `ss` counters incrementing, you are watching decapsulation fail at a specific layer. The end-to-end mental model of this chapter and the per-host mechanics of Volume 2, Chapter 10 are two views of the same machine; keep both loaded.

The distributed-systems lens

Everything above described *one* request between *two* hosts. Real backend systems shatter that tidy picture, and the consequences define large-scale networking.

One user request fans out into dozens of internal hops. When a user hits a single public endpoint, the edge load balancer forwards to an API gateway, which calls an auth service, which calls a session store, which the gateway then routes to an orders service, which reads from a database primary and two caches, emits an event to a message broker, and calls a pricing service that itself calls three more. A single external request routinely becomes tens of internal RPCs across many services and teams. **Each of those internal hops traverses the full stack twice** — down through the caller's kernel and up through the callee's — and in a service mesh, *four* times per logical hop, because the traffic passes through a sidecar proxy on the way out of the caller's pod and another on the way into the callee's (Chapter 10 covers this). The stack you traced once is executed hundreds of times per user action.

Per-hop latency compounds, and BDP governs each hop. If each internal RPC adds even 1 ms of median latency, a fan-out depth of ten adds 10 ms to the median — and tail latency is far worse, because a request that touches 20 services waits on the *slowest* of 20 samples from each service's latency distribution. A p99 dependency, hit twenty times, becomes the *common* case for the overall request. This is why tail latency, not average latency, is the metric that matters in distributed systems, and why hedging, timeouts, and load balancing (Chapters 9 and 11) exist to tame it. And on every hop that crosses regions, the BDP relationship dictates whether a single connection can move the required data in the time budget — cross-region replication and backups live or die by window sizing.

Layering tells you where an outage lives, and that is most of triage. When something breaks at 3 a.m., the fastest path to resolution is localizing the failure to a layer, because each layer has its own tools and its own owners:

- Requests hang before any response, `ping` works, but `curl` to the service times out → likely L4 (accept-queue overflow, connection limits, a firewall dropping SYNs).
- `ping` by IP works but by hostname fails → L7/naming: DNS (Chapter 5).
- TLS handshake fails with a certificate error → L6/PKI (Chapter 6).
- Small requests succeed, large ones hang → the PMTUD black hole (L3/ICMP).

- HTTP 200s but wrong or slow bodies → L7 application logic, or an L7 proxy misrouting.
- Whole subnet unreachable, `ip neigh` shows `FAILED` → L2/L3 local connectivity.

The layered model is not pedagogy you outgrow; it is the coordinate system you localize incidents in. "Which layer?" is the first question a competent responder asks, and the OSI/TCP-IP numbers are how the answer is communicated across a room of engineers who each own a different band of the stack.

The abstraction is leaky by design, and senior engineers read through it. Your ORM's `db.Query()` hides a TCP connection from a pool, which hides a four-tuple, which hides a route through a VPC, which hides a physical path with a real BDP and real queuing delay. When that query is slow, the cause can live at any layer, and the engineers who resolve these incidents quickly are the ones who can descend from application symptom to the specific layer and tool that reveals the truth. That descent — from `client.Get()` to bits on a wire and back — is the skill this chapter exists to seed, and the rest of Volume 3 exists to sharpen.

Key takeaways

- The OSI 7-layer model is shared vocabulary; the real stack is the 4/5-layer TCP/IP model, where TLS occupies OSI's presentation role but is implemented inside the application over TCP. Use layer numbers to localize problems precisely.
- Encapsulation nests headers (HTTP → TCP segment → IP packet → Ethernet frame → bits); decapsulation strips them in reverse. Each layer interprets only its own header, and every header is overhead — brutal for small packets.
- A cold `https` request pays DNS + TCP handshake (1 RTT) + TLS handshake (1 RTT in TLS 1.3) before the first HTTP byte — the entire case for connection reuse, keep-alive, session resumption, and edge termination.
- Switches (L2) forward by MAC with exact-match learning and rewrite nothing; routers (L3) forward by longest-prefix match, decrement TTL, and rewrite the L2 MACs every hop. IP addresses are constant end to end; MAC addresses are link-local and change at each router.

- MTU mismatches are handled by Path MTU Discovery, which depends on ICMP; blanket-dropping ICMP causes the PMTUD black hole — small packets pass, large ones vanish, connections hang.
- Bandwidth (capacity) and latency (delay) are independent; more bandwidth never reduces latency. End-to-end delay = processing + queuing + serialization + propagation; queuing is the variable, load-dependent, tail-latency-producing component.
- Throughput on a reliable transport is capped by $\text{window} / \text{RTT}$; to fill a pipe the in-flight window must reach the bandwidth-delay product ($\text{bandwidth} \times \text{RTT}$). Long fat networks demand window scaling and large buffers, or a single stream leaves the link mostly idle.
- In distributed systems one user request fans out into dozens of internal hops, each traversing the full stack twice (four times through a mesh). Per-hop latency compounds into tail latency, BDP governs each cross-region hop, and localizing an outage to a layer is most of triage.

Further reading

- **RFC 1122**, *Requirements for Internet Hosts — Communication Layers* (IETF, 1989) — the canonical description of the TCP/IP layering model. <https://www.rfc-editor.org/rfc/rfc1122>
- **RFC 826**, *An Ethernet Address Resolution Protocol* (IETF, 1982) — ARP, the L2/L3 glue. <https://www.rfc-editor.org/rfc/rfc826>
- **RFC 1191**, *Path MTU Discovery* (IETF, 1990), and **RFC 8201**, *Path MTU Discovery for IPv6* (IETF, 2017). <https://www.rfc-editor.org/rfc/rfc1191> and <https://www.rfc-editor.org/rfc/rfc8201>
- **RFC 4821**, *Packetization Layer Path MTU Discovery* (IETF, 2007) — the ICMP-independent PMTUD that avoids the black-hole failure. <https://www.rfc-editor.org/rfc/rfc4821>
- **RFC 9293**, *Transmission Control Protocol* (IETF, 2022) — the consolidated modern TCP specification. <https://www.rfc-editor.org/rfc/rfc9293>
- **RFC 8446**, *The Transport Layer Security (TLS) Protocol Version 1.3* (IETF, 2018). <https://www.rfc-editor.org/rfc/rfc8446>
- **ISO/IEC 7498-1:1994**, *Information technology — Open Systems Interconnection — Basic Reference Model* — the OSI model of record.

- Kurose, J. and Ross, K., *Computer Networking: A Top-Down Approach*, 8th ed. (Pearson, 2020) — the standard university text; its delay-components and BDP treatment aligns with this chapter.
- Peterson, L. and Davie, B., *Computer Networks: A Systems Approach*, 6th ed. — freely available online at <https://book.systemsapproach.org/>; strong on the systems view of the stack.
- Grigorik, I., *High Performance Browser Networking* (O'Reilly, 2013), free online at <https://hpbn.co/> — excellent practical treatment of latency, RTT, BDP, and the cost of the connection-setup handshakes.
- Beej's Guide to Network Programming, <https://beej.us/guide/bgnet/> — sockets, ports, and the four-tuple from the programmer's side.
- Volume 2, Chapter 10 — The Linux Network Stack — the per-host, in-kernel companion to this chapter's end-to-end view.

Chapter 2 — IP, Routing, Subnetting, and NAT

What this chapter covers. Chapter 1 (The Journey of a Packet) traced a datagram down one host's stack, across the wire, and up another's. This chapter zooms in on the layer that decides *where* a packet goes: the Internet Protocol. For a backend engineer, IP is not trivia to memorize for an interview — it is the substrate on which every cloud VPC, every Kubernetes cluster, every service mesh, and every global load balancer is built. When you allocate a `/16` for a new region and discover eighteen months later that it overlaps a peered VPC, that is an IP-layer mistake. When a node stops accepting traffic because its conntrack table filled, that is an IP-layer mistake. When a BGP route leak in some other company's network makes your API unreachable from half a continent, that is the IP layer's fragility surfacing in your on-call rotation. This chapter builds the mental model and the arithmetic you need so that these are decisions you make deliberately rather than accidents you clean up.

We work bottom to top: IPv4 addressing and the header, CIDR and the subnet math you must be able to do by hand, the reserved ranges that behave specially (loopback, link-local, the cloud metadata endpoint, RFC 1918 space), then routing — how a host or router chooses a next hop by

longest-prefix match, how routes are learned statically and dynamically, and how BGP glues the tens of thousands of autonomous systems that make up the Internet into one whole while remaining alarmingly easy to break. Then IPv6, which is not "IPv4 with more bits" but a different operational model. Then NAT: why it exists, the SNAT/PAT machinery, the connection-tracking state it depends on, why it breaks inbound connections, and how STUN/TURN claw some of that back. We finish by mapping every one of these primitives onto the cloud objects you actually click and Terraform — VPCs, subnets, route tables, security groups, internet and NAT gateways — and onto container networking, so the abstractions stop being magic.

Learning goals — after this chapter you should be able to:

- Do subnet arithmetic by hand: given a CIDR block, compute the network address, broadcast address, usable host count, and the split into equal or variable-length subnets, and do it fast enough to size a VPC in a design review.
- Recite and reason about the special ranges — RFC 1918 private space, `127/8`, `169.254/16` including the `169.254.169.254` metadata endpoint — and explain the security consequences of each.
- Explain longest-prefix match and read a real routing table, distinguishing connected, static, and dynamically learned routes and the role of the default route.
- Describe, at the mechanism level, what an autonomous system is, what BGP advertises, and precisely how route leaks and hijacks happen and propagate — hedging the incident details you are not certain of.
- Explain SNAT, PAT, and the conntrack table as state, and predict the failure modes: port exhaustion, table exhaustion, broken inbound reachability, and hairpinning.
- Map the whole model onto a cloud VPC and a container network namespace, so a VPC diagram and a `veth` pair read as the same primitives you learned here.

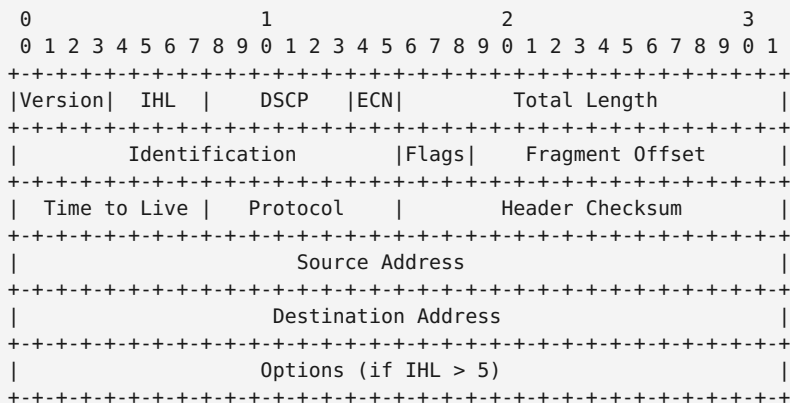
The IPv4 address and header

An IPv4 address is a 32-bit unsigned integer. The familiar dotted-quad notation — `10.20.30.40` — is a human convenience: each of the four octets is one byte, written in decimal, most significant first. `10.20.30.40`

is exactly the integer $(10 \ll 24) \mid (20 \ll 16) \mid (30 \ll 8) \mid 40 = 169090600$. Internalize this, because every piece of subnet math below is integer arithmetic dressed up in dotted-quad clothing, and the moment you think of an address as "four numbers" instead of "one number displayed in four groups" you will make off-by-one errors on network and broadcast addresses.

The address alone does not tell you which bits identify the *network* and which identify the *host within that network*. That split is carried separately — historically by an implied class (the deprecated Class A/B/C scheme), today by an explicit prefix length. We get to that in the next section. First, the header the address lives in.

The IPv4 header is a minimum of 20 bytes (five 32-bit words) and carries the fields that routers and endpoints actually act on. You rarely parse this by hand — the kernel does — but you must know what these fields *mean*, because they surface constantly in packet captures, firewall rules, and outage post-mortems.



The fields that matter operationally:

- **Version / IHL.** Version is 4. IHL (Internet Header Length) counts 32-bit words, so the common no-options header is IHL 5 = 20 bytes. Options push it higher and are rare on the modern Internet; middleboxes frequently drop packets carrying them.
- **DSCP / ECN.** These six plus two bits were once the "Type of Service" byte. DSCP carries Differentiated Services class markings for QoS; ECN (Explicit Congestion Notification) lets routers signal incipient

congestion without dropping, which matters enormously for modern congestion control (Chapter 3, TCP in Depth). Note that DSCP markings are almost always rewritten or zeroed at administrative boundaries, so they are meaningful within a data center, not across the public Internet.

- **Total Length.** The whole datagram in bytes, header plus payload, max 65,535. This, minus the header, is how the receiver knows where the payload ends.
- **Identification / Flags / Fragment Offset.** IPv4 fragmentation machinery. If a datagram exceeds a link's MTU and the Don't Fragment (DF) flag is clear, a router splits it; the three fields let the destination reassemble. Fragmentation is a reliability and security liability — it interacts badly with firewalls, load balancers, and PMTUD black holes — and modern practice leans hard on Path MTU Discovery with DF set so that an ICMP "fragmentation needed" is returned instead. Chapter 1 covered MTU; the operational takeaway is that fragmentation in your captures is usually a symptom, not a design.
- **TTL.** Decrement by one at every router; a packet hitting TTL 0 is dropped and an ICMP Time Exceeded is returned to the source. This is the entire basis of `traceroute` and a crude but effective loop-prevention mechanism. A packet arriving with an unexpectedly low TTL tells you it took more hops than you thought.
- **Protocol.** The Layer-4 payload type: 6 = TCP, 17 = UDP, 1 = ICMP, 132 = SCTP, 4 = IP-in-IP, 41 = IPv6-in-IPv4, 47 = GRE. This is the demux key the receiving host uses to hand the payload to the right protocol handler, and the field your firewall rules match on when you say "allow TCP."
- **Header Checksum.** Covers the header only, not the payload (L4 has its own checksum). Because TTL changes at every hop, every router must recompute this — one reason IPv6 drops it entirely.
- **Source / Destination Address.** The 32-bit addresses. Note that the *source* address is chosen by the sender and, on the open Internet, trivially forgeable — the root of spoofing and reflection/amplification DDoS. Ingress filtering (BCP 38 / RFC 2827) at the network edge is the defense, and its incomplete deployment is why amplification attacks still work.

CIDR and subnet arithmetic

Classless Inter-Domain Routing (CIDR, RFC 4632) replaced the rigid Class A/B/C system in the early 1990s. The idea is simple and total: an address block is an address plus a **prefix length**, written `address/length`. The prefix length is the number of leading bits that are fixed — the network portion — leaving the remaining `32 - length` bits to enumerate hosts.

`10.20.0.0/16` means: the first 16 bits (`10.20`) are the network; the last 16 bits vary, so the block spans `10.20.0.0` through `10.20.255.255` — 65,536 addresses. The **network mask** is the same information expressed as 32 bits: `/16` is `11111111 11111111 00000000 00000000 = 255.255.0.0`. Masks and prefix lengths are interchangeable; tools accept both.

The arithmetic you must be able to do without a calculator:

- **Number of addresses in a /n** is $2^{(32-n)}$. A `/24` is $2^8 = 256$. A `/22` is $2^{10} = 1024$. A `/28` is $2^4 = 16$.
- **Usable hosts** is that minus two: the all-zeros host is the **network address** (the identity of the subnet itself) and the all-ones host is the **broadcast address**. So a `/24` gives 254 usable hosts, a `/28` gives 14. The exceptions: a `/31` (RFC 3021) is used for point-to-point links and has *two* usable addresses with no broadcast, and a `/32` is a single host route. Clouds bite here too — AWS reserves *five* addresses per subnet (network, VPC router, DNS, a future-use address, and broadcast), so an AWS `/24` yields 251 usable, not 254.
- **Network address** of any address is the address ANDed with the mask. `10.20.30.40/24`: AND with `255.255.255.0` zeroes the last octet, giving network `10.20.30.0`.
- **Broadcast address** is the network address ORed with the inverted mask (the host bits all set): `10.20.30.255`.
- **Which subnet an address belongs to** is exactly the network-address computation. This is the operation a router performs, at line rate, on every packet.

Worked example. You are handed `172.18.64.0/20` for a Kubernetes cluster's pod network and must confirm its extent. `/20` fixes 20 bits; the varying bits are the low 12, i.e. the low 4 bits of the third octet plus the whole fourth octet. $2^{12} = 4096$ addresses. The third octet ranges over multiples of 16 starting at 64: the block is `172.18.64.0` through

172.18.79.255 . Network 172.18.64.0 , broadcast 172.18.79.255 , 4094 usable. If someone else has 172.18.72.0/22 , it falls *inside* your block (172.18.72.0 - 172.18.75.255) — that overlap is the kind of thing that silently breaks routing when two VPCs are peered.

Subnetting: splitting a block

Subnetting is borrowing host bits to make more, smaller networks. Take 10.0.0.0/24 and split it into four subnets: you need 2 bits ($2^2 = 4$), so you extend the prefix from /24 to /26 . Each /26 has $2^6 = 64$ addresses, 62 usable. The four are:

Subnet	Range	Network	Broadcast	Usable
10.0.0.0/26	.0 - .63	10.0.0.0	10.0.0.63	.1 - .62
10.0.0.64/26	.64 - .127	10.0.0.64	10.0.0.127	.65 - .126
10.0.0.128/26	.128 - .191	10.0.0.128	10.0.0.191	.129 - .190
10.0.0.192/26	.192 - .255	10.0.0.192	10.0.0.255	.193 - .254

The following diagram shows how one prefix length nests inside another — the mental model for all subnetting is a binary tree where each additional prefix bit splits a block in half.

```
flowchart TD
    A["10.0.0.0/24 - 256 addrs"]
    A --> B["10.0.0.0/25 - 128"]
    A --> C["10.0.0.128/25 - 128"]
    B --> D["10.0.0.0/26 - 64"]
    B --> E["10.0.0.64/26 - 64"]
    C --> F["10.0.0.128/26 - 64"]
    C --> G["10.0.0.192/26 - 64"]
    D --> H["10.0.0.0/27 - 32"]
    D --> I["10.0.0.32/27 - 32"]
```

Variable-length subnet masking (VLSM) drops the requirement that all subnets be equal. Suppose from 10.0.0.0/24 you need one subnet for ~100 hosts, one for ~50, and two for point-to-point links. Size each to the smallest prefix that fits, largest first, to avoid fragmentation:

- 100 hosts needs /25 (126 usable): 10.0.0.0/25 .
- 50 hosts needs /26 (62 usable): 10.0.0.128/26 .

- Each point-to-point link needs `/30` (2 usable) — or `/31` if the gear supports RFC 3021: `10.0.0.192/30` and `10.0.0.196/30`.

Allocating largest-first keeps the remaining space contiguous and summarizable. Allocating smallest-first tends to strand unusable gaps — the address-space equivalent of memory fragmentation.

Route summarization (aggregation) is subnetting in reverse: adjacent blocks that share a prefix collapse into one advertisement. `10.0.0.0/24` and `10.0.1.0/24` are adjacent and together fill `10.0.0.0/23`, so a router can advertise the single `/23` instead of two `/24`s. This is why hierarchical, contiguous allocation matters at scale: it is the difference between a global routing table with a handful of aggregate routes per region and one bloated with thousands of unaggregatable specifics. CIDR's original purpose in 1993 was precisely to slow the growth of the global BGP table through aggregation; the table has grown anyway (roughly 900,000+ IPv4 routes as of the mid-2020s — treat that figure as approximate and always climbing), but far slower than classful routing would have allowed.

Special and reserved ranges

Several ranges do not behave like ordinary global unicast addresses. A senior engineer must know these cold, because most of them have security consequences.

RFC 1918 private space. Three ranges are reserved for private internets and are never routed on the public Internet:

- `10.0.0.0/8` — 16,777,216 addresses. The go-to for large cloud VPCs and corporate networks.
- `172.16.0.0/12` — `172.16.0.0 - 172.31.255.255`, 1,048,576 addresses. The one people get wrong: it is `/12`, not `/16`, so it is the sixteen `/16`s from `172.16` to `172.31`, *not* the whole `172.x`. Docker's default bridge lives here (`172.17.0.0/16`).
- `192.168.0.0/16` — 65,536 addresses. Home and small-office networks; too small for most cloud fleets.

These are the pools you carve VPCs from. The catch, revisited in the cloud section, is that private does not mean *isolated*: the moment two RFC 1918 networks are joined — VPC peering, a VPN, a Transit Gateway, a corporate acquisition — overlapping ranges collide and routing

becomes ambiguous. Disciplined global CIDR planning is the only prevention.

Loopback `127.0.0.0/8`. The entire `/8` loops back to the local host; `127.0.0.1` is the conventional address but anything in `127/8` works. Packets to it never hit a wire. Note the whole `/8` is reserved — a common surprise when someone tries to use `127.0.0.2` as a real address.

Link-local `169.254.0.0/16` (RFC 3927). Auto-assigned when no DHCP is available; valid only on the local link and never routed. Two special-interest members:

- `169.254.169.254` — the **cloud instance metadata service (IMDS)** endpoint on AWS, GCP, Azure, and others. A workload queries it to discover its own identity, and critically to obtain temporary IAM/role credentials. This single address is one of the most important attack surfaces in cloud security: a Server-Side Request Forgery (SSRF) bug that can be coerced into fetching `http://169.254.169.254/...` can exfiltrate the instance's cloud credentials. The 2019 Capital One breach hinged on exactly this — an SSRF reaching IMDS to steal role credentials. AWS's mitigation is **IMDSv2**, which requires a session token obtained via a PUT (with a hop-limit-restricted TTL so it cannot be proxied through the app), turning a naive GET-based SSRF from a credential theft into a dead end. Enforce IMDSv2 and set the metadata hop limit to 1.
- `169.254.169.123` (AWS time sync) and similar are further link-local service endpoints; the pattern generalizes.

Other reserved ranges you will encounter: `0.0.0.0/8` (this network / unspecified; `0.0.0.0` as a bind address means "all local interfaces," as a route means "default"), `100.64.0.0/10` (RFC 6598 carrier-grade NAT space, also used by some clouds and Tailscale to avoid RFC 1918 collisions), `224.0.0.0/4` (multicast), and `255.255.255.255` (limited broadcast). Documentation ranges (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`, RFC 5737) exist so examples never accidentally name a real host.

Routing and longest-prefix match

A host or router forwards a packet by consulting its **routing table**: an ordered set of destination prefixes, each with a next hop (a gateway address and/or an outgoing interface). The decision rule is **longest-**

prefix match (LPM): among all routes whose prefix contains the destination address, the one with the *longest* prefix wins. More specific beats less specific, always. The default route `0.0.0.0/0` matches everything and therefore, being the shortest possible prefix, loses to every other matching route — it is the fallback of last resort, the gateway you send anything you don't have a more specific route for.

Here is a real Linux routing table:

```
$ ip route show
default via 10.0.0.1 dev eth0 proto dhcp metric 100
10.0.0.0/24 dev eth0 proto kernel scope link src 10.0.0.37
10.0.5.0/24 via 10.0.0.9 dev eth0 proto static
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
```

Reading it:

- `10.0.0.0/24 dev eth0 ... scope link` is a **connected route** — the kernel installed it automatically when `eth0` got its address; hosts on this subnet are reachable directly over the link, no gateway needed. `src 10.0.0.37` is this host's address.
- `10.0.5.0/24 via 10.0.0.9` is a **static route** — someone (or a configuration management tool) added it; traffic to that subnet goes to the router at `10.0.0.9`, which is itself on the connected `10.0.0.0/24`.
- `172.17.0.0/16 dev docker0` is Docker's connected route for its bridge.
- `default via 10.0.0.1` catches everything else.

Now trace a packet to `10.0.5.20`. It matches both `default (/0)` and `10.0.5.0/24 (/24)`; LPM picks the `/24`, next hop `10.0.0.9`. A packet to `93.184.216.34` matches only `default`, so it goes to `10.0.0.1`. A packet to `10.0.0.37` matches the connected `/24` — the host talks to itself over loopback, but that is the socket layer's doing, not this table's.

```

flowchart TD
    P["Packet dst = 10.0.5.20"] --> M{"Find all prefixes  
containing dst"}
    M --> R1["0.0.0.0/0 → via 10.0.0.1 (len 0)"]
    M --> R2["10.0.5.0/24 → via 10.0.0.9 (len 24)"]
    R1 --> C{"Longest prefix wins"}
    R2 --> C
    C --> W["Choose /24: next hop 10.0.0.9 on eth0"]
    W --> ARP["Resolve 10.0.0.9 via ARP,  
send frame to its MAC"]

```

Note the final step: the next hop is an *IP* address, but the frame is sent to a *MAC* address. The host uses ARP (Chapter 1) to resolve the next hop's IP to its link-layer address, then sends the L2 frame there while the IP destination in the header stays the final destination. This IP-stays-fixed, MAC-changes-per-hop distinction is the single most important thing to hold in your head about forwarding.

Static vs dynamic routing

Static routes are hand-configured. They are perfect for small, stable topologies and for the cloud, where the control plane (not a routing protocol) programs the route tables. They do not react to failure: if the next hop dies, a static route keeps pointing at the corpse until something rewrites it.

Dynamic routing protocols let routers discover topology and reconverge around failures automatically. Two families matter conceptually:

- **Interior Gateway Protocols (IGPs)** run *within* one administrative domain (an autonomous system). **OSPF** (Open Shortest Path First) and **IS-IS** are the dominant link-state IGPs. Each router floods "link-state advertisements" describing its own links; every router thus builds an identical map of the whole domain and runs Dijkstra's shortest-path algorithm over it to compute next hops. Convergence is fast and loop-free by construction. IS-IS is favored in large ISP cores (it runs directly over L2, scales well, and carries both IPv4 and IPv6 in one instance); OSPF is ubiquitous in enterprise. The mechanism to remember: link-state = every router knows the whole graph and computes its own shortest paths.
- **Exterior Gateway Protocol: BGP**, which glues autonomous systems together and is important enough to get its own section.

BGP: the Internet's glue and its fragility

The Internet is not one network; it is on the order of a hundred thousand allocated **autonomous systems (ASes)** — of which some seventy to eighty thousand are actively visible in the global routing table — networks under independent administrative control, each identified by an AS number (ASN, now up to 32-bit; e.g., Cloudflare is AS13335, Google AS15169). The **Border Gateway Protocol** (BGP-4, RFC 4271) is how ASes tell each other which IP prefixes they can reach.

BGP is a **path-vector** protocol. A BGP speaker advertises, to its neighbors (peers), a set of reachable prefixes, each annotated with the **AS_PATH** — the sequence of ASNs the route has traversed. When AS A advertises `203.0.113.0/24` to AS B, B re-advertises it to C with A's ASN prepended, so C sees `AS_PATH = B A`. Two things fall out of carrying the whole path:

1. **Loop prevention.** A router that sees its own ASN already in a received `AS_PATH` rejects the route — that is how BGP avoids counting-to-infinity without a global map.
2. **Policy, not shortest path.** Unlike an IGP, BGP does not pick the objectively shortest route. It picks according to *policy* and business relationships. The dominant factors, in the order the decision process consults them, start with **LOCAL_PREF** (an operator's explicit preference, used to prefer cheaper or customer routes), then shortest `AS_PATH`, then a cascade of tie-breakers. An AS prefers routes through its paying customers over peers over its own upstream providers, because money, not hop count, is the objective.

Concretely: origin AS64500 advertises `203.0.113.0/24` with `AS_PATH = 64500`; its transit provider AS64510 re-advertises it as `AS_PATH = 64510 64500`; your ISP AS64520 passes it on as `AS_PATH = 64520 64510 64500`, and your router selects among all such paths it hears using the policy-first decision process below.

BGP is astonishingly effective — it has scaled the Internet by three orders of magnitude — and astonishingly fragile, because at its core it is a system built on *trust*. When a neighbor advertises a prefix, classic BGP

has no built-in way to verify the neighbor is actually entitled to originate it. Two failure modes follow:

- **Route hijack.** An AS originates a prefix it does not own, and if its advertisement is more specific or more preferred, traffic for that prefix is drawn to it — blackholed or intercepted. The 2008 incident where Pakistan Telecom's attempt to block YouTube domestically leaked a more-specific `208.65.153.0/24` globally and took YouTube offline for much of the world is the textbook case. In 2018 a BGP hijack redirected traffic destined for Amazon Route 53 DNS to steal cryptocurrency from users of a wallet site. Treat exact figures in such incidents as approximate unless you check the specific post-mortem.
- **Route leak.** An AS re-advertises routes in a direction that violates the intended policy — for example, propagating routes learned from one provider to another provider, turning itself into unintended transit and pulling in traffic it cannot carry. This is usually a misconfiguration, not malice, and it causes large, sudden outages and latency spikes. Major leaks have repeatedly degraded reachability for large chunks of the Internet for tens of minutes at a time.

The defenses, which you should recognize even if you never configure a router: **RPKI** (Resource Public Key Infrastructure) lets a prefix owner publish a cryptographically signed **Route Origin Authorization (ROA)** stating which ASN may originate the prefix, and networks that deploy **Route Origin Validation** drop advertisements that contradict it. RPKI adoption has climbed substantially in the 2020s but is not universal, and RPKI validates only the *origin*, not the whole path — path validation (BGPsec, ASPA) remains largely undeployed. Complementary practices are IRR-based prefix filtering and the **MANRS** norms (filtering, anti-spoofing, coordination). The distributed-systems point for a backend engineer: your service's global reachability depends on the correct behavior of networks you neither own nor can see, and BGP anycast — the mechanism by which one IP is advertised from many locations so users reach the nearest — is exactly what your CDN and global load balancer rely on (Chapter 10, Proxies, Reverse Proxies, Service Mesh, and CDNs). When people say "the Internet had an outage," it was very often BGP.

IPv6: not IPv4 with more bits

IPv6 uses **128-bit** addresses — 2^{128} of them, an amount so large that the scarcity economics driving NAT in IPv4 simply vanish. But treating IPv6 as "IPv4 with a bigger address field" leads to operational mistakes, because its addressing and autoconfiguration model is genuinely different.

Notation. Eight groups of four hex digits separated by colons: `2001:0db8:0000:0000:0000:ff00:0042:8329`. Two abbreviation rules compress it: leading zeros in a group may be dropped (`0db8` → `db8`, `0000` → `0`), and *one* run of all-zero groups may be replaced by `::` (only once, else the length is ambiguous). The example becomes `2001:db8::ff00:42:8329`. Loopback is `::1`; the unspecified address is `:::`. `2001:db8::/32` is the documentation prefix.

The /64 convention. In IPv6, a single subnet is essentially always a `/64`: the top 64 bits are the network prefix, the bottom 64 are the **interface identifier**. This is not a suggestion — **SLAAC** (Stateless Address Autoconfiguration) depends on it. A host learns the `/64` prefix from a router's **Router Advertisement** (ICMPv6 Neighbor Discovery replaces ARP and much of DHCP) and forms its own address by combining that prefix with an interface ID. The interface ID was historically derived from the MAC via EUI-64, but because that leaks the hardware address and enables tracking, hosts now use **privacy/stable-random** interface IDs (RFC 8981 / RFC 7217) by default. The consequence for backend operators: a host can have many IPv6 addresses at once (a link-local `fe80::/10`, one or more global addresses, a stable one, temporary ones that rotate), and source-address selection rules (RFC 6724) decide which it uses — a frequent source of "why did my traffic come from *that* address" confusion.

Address types to know: **global unicast** (`2000::/3`, the routable Internet), **link-local** (`fe80::/10`, mandatory on every interface, used for neighbor discovery and routing protocols, never routed off-link), **unique local** (`fc00::/7`, effectively RFC 1918 for IPv6, the private range for internal networks), and **multicast** (`ff00::/8` — IPv6 has no broadcast at all; "all-nodes" is a multicast group).

Dual-stack is the pragmatic reality: hosts run IPv4 and IPv6 simultaneously, and applications use **Happy Eyeballs** (RFC 8305) to race A and AAAA connections and use whichever completes first, so a broken

IPv6 path degrades to IPv4 instead of hanging. There is no header compatibility between the two — a v6 packet is not a v4 packet — so dual-stack means genuinely running both stacks, with all the config and firewall duplication that implies. For a fleet this doubles the address-planning and ACL surface, which is exactly why many internal networks stayed v4-with-NAT far longer than the IANA IPv4 exhaustion (the free pool ran out in 2011) would suggest they should have.

NAT: SNAT, PAT, and connection tracking

IPv4 has about 3.7 billion usable unicast addresses and the world has far more than that many devices. **Network Address Translation** is the load-bearing hack that reconciles the two: a whole private network hides behind one (or a few) public addresses, with a NAT device rewriting addresses and ports as packets cross the boundary.

The variant you actually run is **PAT** (Port Address Translation), also called **NAPT**, overload NAT, or on Linux **masquerade**. It multiplexes many internal hosts onto one public address by also rewriting *ports*. When `10.0.0.37:51000` sends to `93.184.216.34:443`, the NAT box rewrites the source to `203.0.113.5:62000` (its public address, a chosen source port), forwards it, and records the mapping. The reply, addressed to `203.0.113.5:62000`, is matched against that record, rewritten back to `10.0.0.37:51000`, and delivered inside. The four-tuple (and the protocol) is the key; the rewritten source port is what makes the reply unambiguously routable back to one internal flow.

```
flowchart LR
    H1["Host A 10.0.0.37:51000"] --> N
    H2["Host B 10.0.0.38:51000"] --> N
    N["NAT / PAT  
public 203.0.113.5"] --> I["Internet  
93.184.216.34:443"]
    N -. "conntrack table" .-> T["10.0.0.37:51000 ↔ 203.0.113.5:62000  
10.0.0.38:51000 ↔ 203.0.113.5:62001"]
```

Note that both internal hosts happened to pick source port 51000; PAT disambiguates them by assigning *different* public ports (62000 vs 62001), which is the whole trick.

This mapping is state. NAT is fundamentally a stateful operation, and on Linux that state lives in the **conntrack** (connection tracking) table of

the netfilter subsystem — the same machinery covered from the kernel side in Volume 2, Chapter 10 (The Linux Network Stack). Two hard operational limits fall out of NAT being stateful:

- **Table exhaustion.** The conntrack table has a finite size (`nf_conntrack_max`). Every active flow consumes an entry; a burst of connections — or a slow leak of entries that never time out — fills it, and once full the kernel drops new connections and logs `nf_conntrack: table full, dropping packet`. This is a classic, brutal outage mode: a node or NAT gateway that suddenly refuses *all new* traffic while existing flows continue, so it looks like a partial failure. In Kubernetes this bites regularly because kube-proxy's iptables mode relies heavily on conntrack; watch `nf_conntrack_count` against `nf_conntrack_max` and size accordingly.
- **Port exhaustion.** One public address has only ~64,000 ports (per protocol, per destination tuple). A NAT gateway serving many clients to a *single* popular destination can run out of source ports for that tuple and start refusing connections even though CPU and bandwidth are idle. Cloud managed NAT gateways surface this as an "ErrorPortAllocation" style metric; the fix is more public IPs on the gateway (more port space) or spreading destinations.

Here is Linux masquerade in practice — the config on a NAT/router host:

```
# Enable forwarding between interfaces
sysctl -w net.ipv4.ip_forward=1

# Masquerade traffic leaving the public interface (source NAT to eth0's
address)
iptables -t nat -A POSTROUTING -o eth0 -s 10.0.0.0/24 -j MASQUERADE

# Inspect live NAT state
conntrack -L -j 2>/dev/null | head
# tcp 6 431999 ESTABLISHED src=10.0.0.37 dst=93.184.216.34 sport=51000
dport=443 \
#      src=93.184.216.34 dst=203.0.113.5 sport=443 dport=62000
[ASSURED] ...

# Watch table pressure
sysctl net.netfilter.nf_conntrack_count net.netfilter.nf_conntrack_max
```

`MASQUERADE` is the source-address-picks-itself-from-the-outgoing-interface variant of SNAT, convenient when the public address is dynamic; plain `SNAT --to-source` is used when it is fixed and is slightly cheaper.

NAT breaks inbound — and how traversal claws it back

The defining asymmetry of NAT: a mapping is created by an *outbound* packet. There is no entry for an unsolicited *inbound* connection, so the NAT box has no idea which internal host to deliver it to and drops it. NAT thus silently provides a crude stateful firewall — inbound is denied by default — but it also *breaks* every protocol that needs a host behind NAT to accept connections: peer-to-peer, VoIP, WebRTC, some database replication topologies, FTP's data channel, and so on.

Workarounds, in ascending order of desperation:

- **Static port forwarding / DNAT.** Manually map an inbound public port to an internal host. Fine for a handful of servers; unmanageable at scale and impossible when the internal host is itself dynamic.
- **UPnP / NAT-PMP / PCP.** Protocols by which an internal host *asks* the NAT to open a mapping. Convenient, and a security liability if exposed carelessly.
- **STUN** (Session Traversal Utilities for NAT, RFC 8489). A host asks a public STUN server "what source address and port do you see me coming from?" The answer reveals the public-side mapping the NAT created, and if both peers learn each other's public mappings they can often send directly to them — **hole punching**. STUN is cheap (one small server, no media relayed) but fails against **symmetric NAT**, where the NAT assigns a *different* public port per destination, so the port a peer learns via STUN is not the port that peer's traffic will arrive on.
- **TURN** (Traversal Using Relays around NAT, RFC 8656). The fallback: when hole punching fails, both peers connect *outbound* to a public relay, which forwards media between them. This always works (both sides made outbound connections, which NAT allows) but costs a server that carries all the traffic — expensive at scale.
- **ICE** (Interactive Connectivity Establishment, RFC 8445) is the orchestration layer that gathers candidate addresses (host, STUN-derived server-reflexive, TURN relay), tries them in priority order, and picks the best working pair. WebRTC uses ICE/STUN/TURN as its whole connectivity story; when you build real-time media features, this is the machinery.

The deeper point: NAT is why the end-to-end principle eroded, why running a server "at home" is hard, and one of the strongest arguments

for IPv6, whose abundance removes the address-scarcity justification for NAT entirely (though NAT's accidental firewall property keeps some operators attached to it even on v6, via NPTv6 — a practice most engineers consider misguided).

The cloud realization: a VPC is IP networking

Everything above is not background for cloud networking — it is cloud networking, wearing different names. A cloud **Virtual Private Cloud (VPC)** is a private IP network you define by choosing a CIDR block. AWS, GCP, and Azure differ in detail but the primitives map cleanly:

Concept in this chapter	Cloud object
CIDR block you carve from RFC 1918	VPC CIDR (e.g., <code>10.42.0.0/16</code>)
Subnet (a smaller prefix)	Subnet, pinned to one Availability Zone
Routing table + LPM	Route table associated with subnets
Default route <code>0.0.0.0/0</code>	Route to an Internet Gateway or NAT Gateway
Stateful L3/L4 firewall	Security group (stateful) / NACL (stateless)
SNAT/PAT for egress	Managed NAT Gateway
BGP/anycast to the Internet	Internet Gateway + the provider's edge

```

flowchart TD
    subgraph VPC["VPC 10.42.0.0/16"]
        subgraph AZa["AZ-a"]
            PUB["Public subnet 10.42.0.0/24"]
            PRIV["Private subnet 10.42.10.0/24"]
        end
        NGW["NAT Gateway (in public subnet)"]
        RTPUB["Route table (public):  
0.0.0.0/0 → IGW"]
        RTPRIV["Route table (private):  
0.0.0.0/0 → NAT GW"]
    end
    IGW["Internet Gateway"]
    NET["Internet"]
    PUB --- RTPUB
    PRIV --- RTPRIV
    RTPUB --> IGW --> NET
    RTPRIV --> NGW --> RTPUB

```

The design that falls out of the model: a **public subnet** is one whose route table sends `0.0.0.0/0` to the Internet Gateway; instances there can have public IPs and be reached from outside. A **private subnet** sends its default route to a **NAT Gateway** (which itself lives in a public subnet), so instances can reach *out* — to pull images, call APIs — but nothing on the Internet can initiate a connection *in*. That is precisely the NAT-breaks-inbound property from the previous section, now sold as a security feature. Databases and internal services go in private subnets; only load balancers and bastions face the Internet.

The distributed-systems failure modes are the ones you now recognize:

- **CIDR planning across a fleet.** Give every region and every VPC a non-overlapping block from a master plan *before* you have thirty VPCs. Two VPCs with overlapping `10.0.0.0/16` ranges cannot be peered — the router cannot disambiguate a destination that exists in both. Fixing this after the fact means renumbering a live network, which is among the least pleasant tasks in infrastructure. Reserve generously (v4 address space inside a VPC is free; a `/16` per VPC costs nothing) but partition hierarchically so blocks summarize.
- **Subnet sizing and address exhaustion.** Kubernetes on the cloud is voracious with addresses — AWS's VPC CNI, for instance, assigns real VPC IPs to *Pods*, so a busy cluster can exhaust a subnet's addresses and fail to schedule pods with a cryptic ENI/IP error while CPU sits idle. Size pod subnets for peak pod count, not node count.

- **NAT Gateway as a shared chokepoint.** A managed NAT Gateway has finite port capacity per associated IP; a fleet of instances hammering one external dependency through one NAT Gateway can hit port-allocation errors — the cloud incarnation of the PAT port-exhaustion limit. The fix is the same: more IPs, or per-AZ NAT Gateways, or interface endpoints that bypass NAT for in-cloud service traffic.
- **Security groups are stateful; NACLs are not.** A security group that allows an outbound connection automatically permits the return traffic (it tracks state, like conntrack). A network ACL is stateless and evaluates each direction independently, so a NACL that forgets to allow the ephemeral-port return range silently breaks connections that "should" work — a classic subtle bug. Reach for the model, not folklore.

This is the cloud realization; Volume 12 (Cloud Infrastructure) develops VPC design, Transit Gateways, PrivateLink, and multi-region topologies in depth. Here the point is only that there is nothing new under the hood — it is subnets, route tables, LPM, and NAT.

Container networking: the same primitives, one host down

Container networking is IP networking compressed into a single host's kernel, and it reuses every primitive above. Volume 2, Chapters 9 (Namespaces and cgroups) and 10 (The Linux Network Stack) cover the kernel mechanics; here is the IP-layer view.

Each container gets its own **network namespace** — an isolated copy of the network stack with its own interfaces, routing table, and conntrack. A namespace is connected to the host with a **veth pair**: a virtual Ethernet cable with two ends, one inside the container's namespace (its `eth0`) and one in the host, plugged into a **bridge** (`docker0`, or a CNI-managed bridge). The bridge is a virtual L2 switch; all containers on it share a subnet (Docker's default `172.17.0.0/16`) and reach each other directly, exactly like hosts on a physical LAN. To reach the outside, the container's default route points at the bridge's address, and the host **masquerades** the container's private source address behind the host's real IP — the same `iptables ... MASQUERADE` rule from the NAT section. That is why, by default, a container can reach the Internet but is not reachable from it

without explicit port publishing (`-p`), which installs a DNAT rule. Container networking *is* subnets + routing + NAT on one box.

```
flowchart TD
    subgraph NS1["netns: container A"]
        A["eth0 172.17.0.2/16"]
    end
    subgraph NS2["netns: container B"]
        B["eth0 172.17.0.3/16"]
    end
    A ---|veth pair| BR
    B ---|veth pair| BR
    BR["bridge docker0 172.17.0.1/16"]
    BR -->|"default route + MASQUERADE"| HOST["host eth0 → real network"]
```

Multi-host container networking — where a pod on node 1 must reach a pod on node 2 with a routable, flat address space — is solved by **overlays** or by making the pod network natively routable. An overlay (VXLAN, Geneve) encapsulates each inter-node pod packet inside a UDP packet addressed node-to-node, so the underlying network only ever sees node IPs while pods enjoy a flat `/16`; the receiving node decapsulates and delivers to the right namespace. The alternative is to *route* the pod CIDRs directly — Calico can even run BGP between nodes so pod subnets are advertised as ordinary routes, bringing this chapter full circle: the same path-vector protocol that glues the Internet also glues a Kubernetes cluster. Book 6 (Cloud-Native Systems) and Volume 2, Chapter 10 develop CNI, kube-proxy, and overlays in depth; the load-balancing and service-routing layers built on top are Chapter 9 (Load Balancing) and Chapter 10 (Proxies, Mesh, CDNs) of this volume.

Distributed-systems lens

Pulling the threads together, here is why this chapter's arithmetic and mechanisms are operational concerns rather than academic ones for a senior backend engineer:

- **Address planning is capacity planning.** A CIDR allocation is a commitment you make once and renumber painfully. The overlap that blocks a VPC peering, the exhausted pod subnet that fails scheduling, the region you cannot summarize because its blocks are scattered — all are planning failures that compound with fleet size.

Treat the master CIDR plan like a schema: hierarchical, versioned, allocated with headroom, reviewed before growth.

- **NAT and conntrack are hard limits, not soft ones.** Port exhaustion and table exhaustion do not degrade gracefully; they cliff. A NAT Gateway that runs out of source ports or a node whose conntrack table fills refuses *new* work while old work continues, producing the confusing "half-broken" outages that eat hours of debugging. Monitor `nf_conntrack_count`, NAT Gateway port-allocation errors, and ephemeral-port pressure as first-class SLIs.
- **Your global reachability rides on BGP you do not control.** Anycast, the mechanism behind every serious CDN and global load balancer, is BGP advertising one prefix from many places. Its power and its fragility are the same coin: a route leak or hijack in a network you have never heard of can make your service unreachable for a region, and the mitigation (RPKI, filtering, provider diligence) is largely out of your hands. Design for it — multiple upstreams, monitored from multiple vantage points — rather than assuming the underlay is reliable.
- **The cloud abstractions leak into the primitives.** Every VPC console screen is subnets, route tables, LPM, and NAT with a friendlier UI. Engineers who understand the primitives debug the abstractions; engineers who only know the abstractions are helpless when a security group works but a NACL blocks the return traffic, or a private-subnet instance cannot reach an API because someone deleted the NAT Gateway route. The abstraction is a convenience, not a replacement for the model.

Key takeaways

- An IPv4 address is one 32-bit integer; CIDR splits it into a network prefix and a host suffix at an explicit bit boundary. Number of addresses in a `/n` is $2^{(32-n)}$; usable hosts is that minus two (network and broadcast), with `/31` and `/32` and cloud reservations as exceptions. Be able to compute network and broadcast addresses by hand.
- Memorize the special ranges: RFC 1918 (`10/8`, `172.16/12` — note the `/12`, `192.168/16`), loopback `127/8`, link-local `169.254/16` and especially the `169.254.169.254` metadata endpoint, whose SSRF

exposure caused real breaches and whose mitigation is IMDSv2 with a hop limit of 1.

- Forwarding is longest-prefix match against the routing table; the default route `0.0.0.0/0` is the least specific and loses to every more specific route. The next hop is an IP, but the frame goes to that next hop's MAC — IP destination stays fixed hop to hop, L2 address changes.
- IGP (OSPF, IS-IS, link-state) route within an AS by giving every router the whole graph; BGP is the path-vector protocol that glues ASes, choosing routes by policy and money rather than hop count, and its trust model makes route leaks and hijacks a real and recurring operational risk that RPKI only partially addresses.
- IPv6's 128 bits and /64 -plus-SLAAC model remove the scarcity that justified NAT; dual-stack and Happy Eyeballs are the pragmatic transition, at the cost of running two stacks.
- PAT/masquerade is stateful NAT; its state (conntrack) and its single-address port space are hard-limited resources whose exhaustion causes cliff-edge outages. NAT breaks inbound by design, which STUN/TURN/ICE work around for peer-to-peer and real-time media.
- A cloud VPC is this chapter with a UI: CIDRs, subnets, route tables, security groups, internet and NAT gateways. Container networking is the same primitives on one host — namespaces, veth, bridge, masquerade — with overlays or BGP for multi-host reachability.

Further reading

- RFC 791, *Internet Protocol* — the original IPv4 specification; still the authoritative header reference.
- RFC 4632, *Classless Inter-Domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan* — the definitive CIDR document.
- RFC 1918, *Address Allocation for Private Internets*, and RFC 6598, *IANA-Reserved IPv4 Prefix for Shared Address Space* (carrier-grade NAT / `100.64.0.0/10`).
- RFC 3927, *Dynamic Configuration of IPv4 Link-Local Addresses* (`169.254/16`), and RFC 3021, *Using 31-Bit Prefixes on IPv4 Point-to-Point Links*.

- RFC 4271, *A Border Gateway Protocol 4 (BGP-4)*, and RFC 7454, *BGP Operations and Security (BCP 194)* — the protocol and how to run it safely.
- RFC 6480, *An Infrastructure to Support Secure Internet Routing (RPKI)*, plus the MANRS initiative documentation at manrs.org for current routing-security norms.
- RFC 8200, *Internet Protocol, Version 6 (IPv6) Specification*; RFC 4862, *IPv6 Stateless Address Autoconfiguration*; RFC 8305, *Happy Eyeballs Version 2*.
- RFC 8489, *Session Traversal Utilities for NAT (STUN)*; RFC 8656, *Traversal Using Relays around NAT (TURN)*; RFC 8445, *Interactive Connectivity Establishment (ICE)*.
- RFC 2827 / BCP 38, *Network Ingress Filtering* — the anti-spoofing baseline everyone should deploy and too few do.
- The Linux `ip(8)`, `ip-route(8)`, `iptables(8)`, and `conntrack(8)` man pages, and the netfilter project documentation at netfilter.org, for the mechanisms behind the commands in this chapter.
- Cloudflare's engineering blog and the MANRS/BGP incident write-ups from RIPE NCC and Oracle Internet Intelligence are reliable sources for accurate post-mortems of real BGP hijacks and leaks — read the specific write-up before quoting figures.
- Cloud provider VPC documentation: the AWS *VPC User Guide*, GCP VPC docs, and Azure *Virtual Network* docs, which describe each provider's exact subnet reservations and gateway behaviors referenced above. See also Volume 12 (Cloud Infrastructure) for design depth and Book 6 (Cloud-Native Systems) for CNI and Kubernetes networking.

Chapter 3 — TCP in Depth: Handshake, State Machine, Flow and Congestion Control

What this chapter covers. Chapter 1 (The Journey of a Packet) followed a segment down one stack and up another; Chapter 2 (IP, Routing, Subnetting, and NAT) covered the datagram layer that delivers packets

best-effort — unreliable, unordered, size-limited. TCP is the layer that turns that lossy datagram service into the abstraction almost every backend service actually programs against: a reliable, ordered, bidirectional byte stream. In Volume 2, Chapter 10 (The Linux Network Stack) you saw the *kernel machinery* — sockets, `sk_buff`s, the `qdisc`, `NAPI`, the send and receive queues. This chapter is about the *protocol and its dynamics*: what TCP guarantees, the exact mechanisms by which it keeps those guarantees, and — most importantly for anyone operating services at scale — how its control loops behave under load, loss, and latency. TCP is not a black box you can ignore. It is the single most important determinant of your p99 RPC latency, the reason a healthy service falls over when a NAT table fills, and the thing that made HTTP/3 abandon it entirely.

We work from the invariants outward: sequence and acknowledgment numbers and how they encode a byte stream; the three-way handshake and four-way close; the full connection state machine and the operational trap that is `TIME_WAIT`; flow control (the receive window, window scaling, zero-window, silly window syndrome); then the heart of the chapter — congestion control, from Van Jacobson's slow start and AIMD through Reno, NewReno, CUBIC, and BBR — plus RTO estimation, SACK, ECN, and the small-packet pathologies (Nagle, delayed ACK, and their famous interaction bug). We close on head-of-line blocking, which is TCP's one irreducible weakness and the reason QUIC exists (Chapter 4), and on a distributed-systems lens: how these mechanisms show up as latency, as outages, and as tuning decisions in production.

Learning goals — after this chapter you should be able to:

- Explain precisely how sequence numbers, cumulative ACKs, and retransmission deliver a reliable ordered byte stream over an unreliable datagram layer.
- Draw the TCP state machine from memory and reason about every transition, including why `TIME_WAIT` lasts $2 \times \text{MSL}$ and why it accumulates on the connection *initiator*.
- Distinguish flow control (receiver-driven, the advertised window) from congestion control (network-driven, the congestion window), and describe slow start, congestion avoidance, fast retransmit, and fast recovery at the level of the variables involved.

- Contrast loss-based congestion control (Reno/NewReno, CUBIC) with model-based BBR, and say correctly when each wins and how BBR probes bandwidth and RTT.
- Diagnose the classic small-packet pathologies — Nagle interacting with delayed ACK, silly window syndrome — and choose `TCP_NODELAY`, `TCP_QUICKACK`, and buffer sizing deliberately.
- Size TCP for a given bandwidth-delay product and explain why slow start punishes short-lived connections, motivating pooling, keep-alive, and HTTP/2 multiplexing.

The guarantees, and the machinery that keeps them

TCP offers exactly three guarantees on top of IP, and it is worth being precise about each because engineers routinely assume a fourth that does not exist.

Reliable. Every byte you `write()` that TCP acknowledges will be delivered to the peer application, or the connection will be torn down and both ends will find out (via error or reset). TCP does not silently drop data. It achieves this with sequence numbers, positive cumulative acknowledgments, and retransmission of anything left unacknowledged past a timeout.

Ordered. Bytes are delivered to the receiving application in exactly the order the sender wrote them, regardless of the order the underlying packets arrived. IP can and does reorder packets — different packets can take different paths (Chapter 2), and even within a router multipath and queueing reorder them. TCP reassembles using the sequence space.

A byte stream. This is the guarantee people forget. TCP has *no concept of message boundaries*. If you `write()` 100 bytes then 200 bytes, the peer may `read()` 300 bytes at once, or 1 then 299, or any other split. The segmentation into packets is TCP's business and is driven by the MSS, Nagle, and the send window — not by your write boundaries. Every framing bug in every hand-rolled protocol ("I called `send` once so I'll `recv` once") is a failure to internalize this. Message framing is the application's job (length prefixes, delimiters), which is exactly what gRPC and HTTP/2 provide (Chapters 7 and 8).

What TCP does *not* give you: bounded latency (it will retransmit and stall arbitrarily), message atomicity, or any notion of "the peer received and

processed my request." A successful `write()` means the bytes are in your local send buffer, nothing more. A byte being ACKed means the peer's *kernel* received it, not that the peer *application* read it, and certainly not that it acted on it. End-to-end delivery semantics are an application concern (Chapter 11 — Network Reliability).

Sequence and acknowledgment numbers

Every byte in each direction of a TCP connection has a 32-bit sequence number. The number is *per byte*, not per segment, though the header carries only the sequence number of the first byte in the segment; the rest are implied by the payload length. The initial sequence number (ISN) is not zero — it is randomized at connection setup, both to avoid confusion with stale segments from a previous incarnation of the same four-tuple and to make off-path injection harder (an attacker who cannot see the connection must guess a 32-bit ISN). RFC 6528 specifies deriving the ISN from a clock plus a keyed hash of the four-tuple.

The acknowledgment number is the sequence number of the *next byte the receiver expects* — equivalently, one past the last contiguous byte received. This is the crucial design choice: TCP ACKs are **cumulative**. An ACK of 5001 means "I have every byte through 5000." It says nothing granular about bytes beyond a gap. If segments for bytes 1-1000 and 2001-3000 arrive but 1001-2000 is lost, the receiver keeps ACKing 1001 — it cannot cumulatively acknowledge the 3000 it is holding, because 1001-2000 is missing. Those repeated identical ACKs are *duplicate ACKs*, and they are the signal that drives fast retransmit (below). Cumulative ACKs make the protocol robust — a single later ACK covers all earlier data, so lost ACKs are usually harmless — but they are also why plain TCP retransmission is coarse, which is what SACK later fixes.

```
sequenceDiagram
    participant S as Sender
    participant R as Receiver
    Note over S,R: Bytes 1001..2000 are lost in flight
    S->>R: seq=1, len=1000 (bytes 1..1000)
    R->>S: ack=1001
    S->>R: seq=1001, len=1000 (LOST)
    S->>R: seq=2001, len=1000 (bytes 2001..3000)
    R->>S: ack=1001 (dup ACK, gap at 1001)
    S->>R: seq=3001, len=1000 (bytes 3001..4000)
    R->>S: ack=1001 (dup ACK)
    Note over S: 3 dup ACKs -> fast retransmit
    S->>R: seq=1001, len=1000 (retransmit)
    R->>S: ack=4001 (cumulative, all caught up)
```

Connection setup: the three-way handshake

A TCP connection is established with a three-segment exchange whose purpose is to synchronize the two independent sequence spaces (one per direction) and to confirm both ends are willing to talk. "SYN" is the flag that means "synchronize sequence numbers."

1. The client picks an ISN `x` and sends a **SYN** with `seq=x`. It moves to `SYN_SENT`.
2. The server, if listening, picks its own ISN `y`, and replies with **SYN-ACK**: `seq=y`, `ack=x+1` (acknowledging the client's SYN, which consumes one sequence number even though it carries no data). It moves to `SYN_RECEIVED`.
3. The client sends **ACK**: `ack=y+1`. Both ends are now `ESTABLISHED`.

The `+1` matters: SYN and FIN flags each occupy one sequence number, so they can themselves be reliably acknowledged and retransmitted. This is why the client's first data byte has sequence number `x+1`.

Two options are negotiated in the SYN and SYN-ACK and matter enormously for performance: **MSS** (maximum segment size — the largest payload a segment will carry, derived from the path MTU, typically 1460 bytes on a 1500-byte Ethernet MTU) and **window scale** (below). Options present only in the SYN/SYN-ACK — window scaling, SACK-permitted, timestamps — must be offered in the handshake or they are unavailable for the connection's lifetime.

The handshake costs one full round trip before any application data flows. On a WAN with 80 ms RTT, that is 80 ms of latency before the first

request byte is even sent — the single strongest argument for connection reuse. **TCP Fast Open** (RFC 7413) lets a client send data in the SYN on a repeat connection to a server it has a cookie for, saving that round trip, but deployment is limited by middleboxes that strip the option.

The server side has a subtlety that has caused real outages: the **SYN backlog**. Between the SYN-ACK and the final ACK, a connection is half-open and sits in a queue. A SYN flood fills this queue with connections that never complete, exhausting it and denying service. The mitigation is **SYN cookies**: instead of storing state for a half-open connection, the server encodes the necessary state into the ISN `y` itself (a hash of the four-tuple, MSS, and a slowly-changing secret), so it can reconstruct state when the final ACK arrives without having held a backlog slot. Linux enables this under pressure via `net.ipv4.tcp_syncookies=1`. The cost is that a few options cannot be preserved, so it is a degradation, not a free lunch.

Connection teardown: the four-way close

TCP connections are full-duplex and each direction is closed independently, which is why teardown takes four segments rather than three. Closing is really two half-closes.

1. The end that calls `close()` (or shuts down its send side) sends **FIN**. It moves to `FIN_WAIT_1`.
2. The peer ACKs the FIN. The closer moves to `FIN_WAIT_2`. The peer moves to `CLOSE_WAIT` — its receive side is done, but *it may still send data*. This half-open state can persist indefinitely; the peer application must notice the EOF (a zero-length `read`) and close its own side.
3. When the peer is done sending, it sends its own **FIN**. It moves to `LAST_ACK`.
4. The original closer ACKs that FIN and enters `TIME_WAIT`. After a fixed wait it transitions to `CLOSED`.

The ACK and the peer's FIN can be combined into one segment when the peer has nothing more to send, collapsing four segments into three in practice. A connection can also be aborted abruptly with a **RST** (reset), which discards any buffered data and skips the orderly teardown entirely; you see RSTs when an application crashes, when data arrives for a socket with no listener, or when `SO_LINGER` is set to zero.

```
sequenceDiagram
    participant A as Active closer
    participant B as Passive closer
    Note over A,B: ESTABLISHED
    A->>B: FIN (seq=u)
    Note over A: FIN_WAIT_1
    B->>A: ACK (ack=u+1)
    Note over B: CLOSE_WAIT
    Note over A: FIN_WAIT_2
    Note over B: app finishes, calls close()
    B->>A: FIN (seq=v)
    Note over B: LAST_ACK
    A->>B: ACK (ack=v+1)
    Note over A: TIME_WAIT (2*MSL)
    Note over B: CLOSED
    Note over A: CLOSED after timer
```

The state machine

The eleven states and their transitions are the canonical mental model. The active opener (client) walks down the left; the passive opener (server) down the right; both meet at `ESTABLISHED` and diverge again at close depending on who closes first.

```
stateDiagram-v2
    [*] --> CLOSED
    CLOSED --> LISTEN: passive open
    CLOSED --> SYN_SENT: active open, send SYN
    LISTEN --> SYN_RECEIVED: recv SYN, send SYN-ACK
    SYN_SENT --> SYN_RECEIVED: recv SYN, send SYN-ACK
    SYN_SENT --> ESTABLISHED: recv SYN-ACK, send ACK
    SYN_RECEIVED --> ESTABLISHED: recv ACK
    ESTABLISHED --> FIN_WAIT_1: close, send FIN
    ESTABLISHED --> CLOSE_WAIT: recv FIN, send ACK
    FIN_WAIT_1 --> FIN_WAIT_2: recv ACK of FIN
    FIN_WAIT_1 --> CLOSING: recv FIN, send ACK
    FIN_WAIT_2 --> TIME_WAIT: recv FIN, send ACK
    CLOSING --> TIME_WAIT: recv ACK of FIN
    CLOSE_WAIT --> LAST_ACK: close, send FIN
    LAST_ACK --> CLOSED: recv ACK of FIN
    TIME_WAIT --> CLOSED: 2*MSL timer expires
```

A few transitions repay study. `SYN_SENT` → `SYN_RECEIVED` is the *simultaneous open* case, where two peers SYN each other at once; rare but real in peer-to-peer designs. `CLOSING` handles *simultaneous close*, where both send FIN before either's arrives. `CLOSE_WAIT` is the state you

will actually debug: a large number of sockets stuck in `CLOSE_WAIT` almost always means an application bug — the peer closed, the kernel ACKed, but your code never called `close()` on its end, leaking file descriptors until the process hits `EMFILE`. `netstat -tan` or `ss -tan state close-wait` piling up is a smell that points at the application, not the network.

TIME_WAIT: purpose and the accumulation problem

`TIME_WAIT` is entered by whichever side closes *actively* (sends the first FIN). It lasts $2 \times \text{MSL}$ — twice the Maximum Segment Lifetime — which on Linux is a hardcoded 60 seconds (`TCP_TIMEWAIT_LEN`), i.e., an assumed MSL of 30 seconds. It exists for two reasons, and both are correctness reasons, not paranoia:

1. **To absorb the peer's retransmitted FIN.** If the final ACK is lost, the peer will retransmit its FIN. If our socket had already vanished, the retransmitted FIN would elicit a RST, and the peer would see its clean close reported as an error. Sitting in `TIME_WAIT` lets us re-ACK the FIN.
2. **To let stray segments from this connection die out before the four-tuple is reused.** A segment delayed in the network for up to one MSL could otherwise arrive during a *new* connection that reused the same (src IP, src port, dst IP, dst port) and be accepted as valid data — a data-corruption bug. Waiting $2 \times \text{MSL}$ guarantees every old segment has expired.

The trouble is scale. `TIME_WAIT` accumulates on the *active closer*, and in a typical client-server RPC pattern the client (or a proxy in front of a backend) is the active closer of huge numbers of short connections. Each `TIME_WAIT` socket pins one four-tuple for 60 seconds. Since the destination IP and port are usually fixed (your backend), and the source IP is fixed (the client), the only degree of freedom is the **ephemeral source port**, of which there are at most $\sim 28,000$ by default (`net.ipv4.ip_local_port_range`, commonly 32768-60999) and $\sim 64\text{K}$ absolute. A busy client opening and closing connections faster than its ephemeral range refills — roughly 28K per 60 s with the default range — to a single destination will run out of source ports and see `connect()` fail with `EADDRNOTAVAIL` — a real and common outage mode for API gateways, sidecar proxies, and load-testing rigs.

The correct fixes, in order of preference:

- **Do not open so many connections.** Reuse them: HTTP keep-alive, connection pools, HTTP/2 multiplexing (Chapter 7). This eliminates the churn rather than papering over it.
- **Make the server the active closer** where possible, moving the `TIME_WAIT` burden to the side that has more four-tuple diversity (many client IPs).
- `net.ipv4.tcp_tw_reuse=1` lets the kernel reuse a `TIME_WAIT` socket in the *outbound* direction when TCP timestamps (RFC 7323) make it safe to distinguish old segments. This is the safe knob.
- Do **not** reach for `tcp_tw_recycle`. It was removed from Linux in 4.12 precisely because it broke connections from clients behind NAT (it keyed timestamp checks per source IP and discarded segments from clients whose clocks appeared to go backwards — i.e., different hosts behind one NAT). Any old blog recommending it is dangerous.

And remember `conntrack` (Chapter 2): even if you dodge ephemeral-port exhaustion, the NAT connection-tracking table (`nf_conntrack_max`) has its own limit and its own timeouts for `TIME_WAIT`, and filling it drops new flows silently. TCP-level and netfilter-level state are two separate tables that can each fail independently.

Flow control: not overrunning the receiver

Flow control answers a purely local question: *is the receiver's buffer able to accept more data right now?* It is entirely separate from congestion control, which asks whether the *network* can carry more. Confusing the two is the most common conceptual error about TCP.

Every ACK carries a 16-bit **receive window** (`rwnd`): the number of bytes of buffer the receiver currently has free, starting at the ACK number. The sender must never have more unacknowledged data in flight than the smaller of `rwnd` (receiver's limit) and `cwnd` (the congestion window — the network's limit, next section). The effective send window is `min(rwnd, cwnd)`.

Window scaling

A 16-bit window maxes out at 65,535 bytes. On a fast, high-latency path that is nowhere near enough. The amount of data that must be in flight to

keep a pipe full is the **bandwidth-delay product** (BDP): $\text{bandwidth} \times \text{RTT}$. On a 10 Gbps link with 80 ms RTT, $\text{BDP} = 10\text{e}9 \text{ bits/s} \times 0.08 \text{ s} \div 8 \approx 100 \text{ MB}$. A 64 KB window on that path caps throughput at $64 \text{ KB} / 80 \text{ ms} \approx 800 \text{ KB/s}$ — under 0.1% of the link. **Window scaling** (RFC 7323) fixes this: a window-scale option, negotiated once in the SYN, is a left-shift applied to the advertised window, allowing windows up to roughly 1 GB. It must be offered in the handshake or it is unavailable for the connection. This is why a connection that starts before scaling is negotiated, or a middlebox that strips the option, can silently cap a fast path at absurdly low throughput — a nasty, invisible failure.

Zero-window and the persist timer

If the receiving application stops reading, the receiver's buffer fills and it advertises `rwnd = 0`, freezing the sender. When the application drains the buffer, the receiver sends a **window update**. But that window update is a bare ACK, and ACKs are not retransmitted — if it is lost, the sender waits forever for a window that has actually opened, and the receiver waits forever for data. TCP breaks this deadlock with the **persist timer**: the sender periodically sends a **zero-window probe** (one byte, or a bare segment) to force the receiver to re-emit its current window. A peer stuck at zero window for a long time is a classic sign of a stalled or CPU-starved consumer — the application is not reading fast enough, and TCP is faithfully back-pressuring all the way to the sender.

Silly window syndrome

If the receiver advertises tiny window increments as it drains a few bytes at a time, and the sender dutifully sends tiny segments to fill them, the connection degenerates into a storm of 40-byte-header packets carrying a handful of payload bytes — enormous overhead, terrible goodput. This is **silly window syndrome**. TCP avoids it from both sides: the receiver (per **Clark's solution**) does not advertise a larger window until it can offer a full MSS (or half the buffer); the sender (via **Nagle's algorithm**, below) coalesces small writes. Modern stacks implement both, so you rarely see raw SWS — but the sender-side coalescing is Nagle, which has its own famous pathology.

Congestion control: not overrunning the network

Flow control protects the receiver; congestion control protects the *network* — the routers and links between the endpoints, whose queues are shared by every flow and have no explicit way to tell any single sender to slow down. Before congestion control existed, the early Internet suffered *congestion collapse* (in October 1986, throughput between LBL and UC Berkeley — two IMP hops and about 400 yards apart — collapsed from 32 kbps to 40 bps, the measurement Van Jacobson cites in his 1988 paper): senders retransmitted aggressively into already-full queues, and throughput fell toward zero. Van Jacobson's 1988 work introduced the algorithms that saved it and that, in evolved form, still run today.

The core idea: the sender maintains a **congestion window** (`cwnd`), a second limit on in-flight data, and *infers* the network's capacity from feedback. Loss-based schemes treat a lost packet as the congestion signal. The sender never sends more than `min(cwnd, rwnd)` unacknowledged bytes. Everything below is about how `cwnd` moves.

Slow start

A new connection has no idea what the path can carry. Rather than blast at line rate, it starts with a small **initial congestion window** (`initcwnd`) — historically 1–4 MSS, raised by RFC 6928 (and Linux default since ~kernel 2.6.39) to **10 MSS** (~14.6 KB) — and grows *exponentially*: for every ACK received, `cwnd` increases by one MSS, which roughly doubles `cwnd` every RTT. "Slow" refers to starting small, not to the growth rate, which is the fastest growth TCP ever uses. Slow start continues until `cwnd` reaches the **slow-start threshold** (`ssthresh`) or a loss occurs.

The exponential ramp is why short-lived connections are slow. A cold connection with `initcwnd=10` can send ~14.6 KB in the first RTT, ~29 KB in the second, ~58 KB in the third. A 200 KB response over an 80 ms-RTT path spends several RTTs in slow start before it is allowed to fill the pipe — the transfer is *latency-bound by the congestion controller's ramp*, not by bandwidth. This is the deep reason connection reuse matters so much: a warm connection has already grown `cwnd` and skips the ramp. It is a primary motivation for keep-alive, pooling, and HTTP/2's single long-lived multiplexed connection (Chapters 7 and 8).

Congestion avoidance and AIMD

Once `cwnd` $\geq$ `ssthresh`, TCP switches from exponential to **linear** growth: `cwnd` increases by roughly one MSS *per RTT* (concretely, `cwnd += MSS*MSS/cwnd` per ACK). This is the "additive increase" of **AIMD** — Additive Increase, Multiplicative Decrease. On loss, `cwnd` is cut multiplicatively (halved, classically). The result is the characteristic **sawtooth**: probe upward gently, back off sharply on congestion, repeat. AIMD is not arbitrary — Chiu and Jain (1989) showed it is the control law that converges to a fair and efficient allocation when many flows share a bottleneck, which additive-increase/additive-decrease and other combinations do not.

```
flowchart LR
    A["Slow start: cwnd doubles per RTT"] -->|"cwnd reaches ssthresh"| B["Congestion avoidance: +1 MSS per RTT"]
    B -->|"3 dup ACKs"| C["Fast retransmit + fast recovery: cwnd = cwnd/2"]
    C --> B
    B -->|"RTO timeout"| D["cwnd = 1 MSS, ssthresh = cwnd/2"]
    D --> A
```

Plotted against time, `cwnd` traces a sawtooth: a slow linear climb during congestion avoidance, interrupted by a multiplicative halving on each loss event, then another climb from the reduced window — a probe-and-back-off cycle that repeats for the life of the connection.

Fast retransmit and fast recovery

Waiting for a retransmission *timeout* (RTO, hundreds of milliseconds to seconds) on every lost packet would be catastrophic for throughput. **Fast retransmit** shortcuts it: three duplicate ACKs (four identical ACKs) for the same sequence number are taken as strong evidence that the next segment was lost — not merely reordered — so the sender retransmits *immediately* without waiting for the timer. Why three? A single duplicate ACK can result from harmless reordering; requiring three trades a little latency for confidence.

Fast recovery governs what `cwnd` does afterward. Rather than collapsing to slow start (which is reserved for the more severe signal of an actual timeout), the sender halves `cwnd` and `ssthresh` and continues in congestion avoidance. The intuition: duplicate ACKs mean packets are still *arriving* at the receiver, so the pipe is not empty — a gentle cut is

appropriate. A full **RTO timeout**, by contrast, means nothing is getting through, so the sender resets `cwnd` to 1 (or `initcwnd`) and re-enters slow start. The distinction between the mild signal (dup ACKs → halve) and the severe signal (timeout → collapse) is central to how TCP behaves under different loss patterns.

Reno, NewReno, and the multiple-loss problem

Reno is the classic combination of slow start, congestion avoidance, fast retransmit, and fast recovery. Its weakness: with cumulative ACKs alone, it recovers cleanly from only *one* lost segment per window. If two segments in a window are lost, Reno retransmits the first, gets a partial ACK, and often has to fall back to a timeout for the second, tanking throughput. **NewReno** (RFC 6582) fixes this without any new protocol feature: it stays in fast recovery on *partial* ACKs (an ACK that advances but does not cover the whole window at recovery start), retransmitting the next hole immediately rather than exiting recovery. NewReno recovers from multiple losses in a window at one-per-RTT — better than Reno, still slow. The real fix for multiple losses is SACK.

SACK: selective acknowledgment

Selective ACK (RFC 2018) removes the cumulative-ACK straitjacket for loss recovery. With SACK negotiated in the handshake, a receiver can tell the sender not just "I have everything through X" but also "and I additionally have these specific non-contiguous blocks [Y1-Y2], [Z1-Z2]." The sender then knows *exactly* which segments are missing and retransmits precisely those, in one round trip, rather than probing hole by hole. SACK is standard on every modern stack (`net.ipv4.tcp_sack=1`) and, combined with the recovery logic in RFC 6675, is what makes TCP tolerable on paths with sporadic loss. **D-SACK** (RFC 2883) extends it to report *duplicate* segments the receiver got, letting the sender detect spurious retransmissions and un-shrink `cwnd` it cut needlessly.

CUBIC: the modern loss-based default

Standard Reno/NewReno AIMD grows `cwnd` linearly at one MSS per RTT. On a modern high-BDP path that is painfully slow: after a loss cuts a 100 MB window in half, refilling it at ~1500 bytes per 80 ms RTT takes thousands of round trips — minutes. **CUBIC** (Ha, Rhee, Xu, 2008; RFC 8312; the Linux default since kernel 2.6.19 in 2006) replaces the linear

increase with a *cubic function of the time since the last congestion event*. The window grows quickly right after a cut (concave region) toward the value where loss last occurred (`W_max`), plateaus gently around `W_max` (the inflection, where it probes cautiously), then grows quickly again (convex region) if no loss appears, aggressively searching for new capacity. Crucially, the growth is a function of *wall-clock time, not RTT count*, which makes CUBIC **RTT-fair**: flows with different RTTs sharing a bottleneck get more equal shares than Reno gives, where the short-RTT flow's faster ACK clock lets it grab a disproportionate share. CUBIC is what your Linux servers run by default today.

CUBIC is still fundamentally **loss-based**: it treats a dropped packet as the signal to back off. That assumption is its weakness. On paths with *non-congestive* loss — wireless, lossy long-haul links — CUBIC misreads random loss as congestion and needlessly throttles. And on paths with deep buffers (**bufferbloat**), loss-based control fills the buffer completely before it ever sees a drop, so it operates at maximum queueing delay by design — great throughput, terrible latency, which is why a large download over CUBIC can wreck the latency of every other flow sharing that oversized home-router or carrier buffer.

BBR: model-based congestion control

BBR (Bottleneck Bandwidth and Round-trip propagation time; Cardwell et al., Google, 2016) attacks the bufferbloat and random-loss problems by *not treating loss as the primary signal at all*. Instead of inferring capacity from drops, BBR continuously *estimates* two physical properties of the path and paces the sender to match them:

- **BtlBw** — the bottleneck bandwidth: the *max* recent delivery rate, measured as delivered bytes over time.
- **RTprop** — the round-trip propagation delay: the *minimum* RTT observed, reflecting pure propagation with no queueing.

The optimal operating point (Kleinrock's, 1979) is exactly `BtlBw × RTprop` bytes in flight — enough to fill the pipe but not the queue. BBR *paces* packets out at the estimated BtlBw rather than sending in ACK-clocked bursts, and periodically runs two probing phases: **ProbeBW**, where it briefly sends ~25% faster (a pacing gain of 1.25) to test whether more bandwidth is available, then drains the queue it just built (gain 0.75), then cruises; and **ProbeRTT** every ~10 seconds, where it drops the in-flight data to a minimum to re-measure the true, unqueued RTprop (since

a persistently full pipe would otherwise inflate the RTT estimate). Because BBR aims to keep the bottleneck buffer nearly *empty*, it achieves high throughput at low latency and is largely *immune to non-congestive loss* — a few random drops do not change its bandwidth estimate. On lossy WAN and high-BDP paths, BBR can dramatically outperform CUBIC; Google reported large throughput gains and latency reductions on YouTube and B4 after deploying it.

BBR is not free of controversy. The original **BBRv1** could be *unfair* to concurrent loss-based (CUBIC/Reno) flows on shallow-buffered links — because it ignores the loss signal those flows respect, it could crowd them out — and could itself build a standing queue in some regimes. **BBRv2** and the later **BBRv3** (the version being upstreamed and refined through the early 2020s) add explicit responses to loss and ECN to improve fairness and coexistence. The practical guidance: BBR is a strong choice for *your own* long-haul, high-BDP, or lossy WAN paths (backbone links between datacenters, CDN egress), where you control both ends or the mix; on shared/short-buffer links with lots of CUBIC neighbors, evaluate before flipping the switch. On Linux it is one line — `net.ipv4.tcp_congestion_control=bbbr` (with `fq` `qdisc` for BBR's pacing) — but it is a policy decision, not a default you set blindly.

Property	Reno/NewReno	CUBIC	BBR
Signal	loss (dup ACK / timeout)	loss	delivery rate + min RTT (model)
cwnd growth	linear (+1 MSS/ RTT)	cubic in time	paced to BtlBw × RTprop
High-BDP paths	poor (slow refill)	good	very good
Non-congestive loss	overreacts	overreacts	largely immune
Buffer occupancy	fills buffer	fills buffer	keeps buffer near empty
Latency under load	high (bufferbloat)	high (bufferbloat)	low
Fairness vs CUBIC	n/a (baseline)	RTT-fair-ish	v1 can be unfair; v2/ v3 better

RTT estimation and the retransmission timeout

The retransmission timeout (RTO) is how long the sender waits for an ACK before assuming loss and retransmitting. Set it too short and you retransmit segments that were merely delayed, wasting bandwidth and needlessly cutting `cwnd`; too long and you stall for a second on every real loss. TCP estimates it adaptively from measured RTTs (Jacobson/Karels, RFC 6298):

```
SRTT  = (1 - alpha) * SRTT  + alpha * RTT_sample    # smoothed RTT,
alpha = 1/8
RTTVAR = (1 - beta) * RTTVAR + beta * |SRTT - RTT_sample| #
variance, beta = 1/4
RTO    = SRTT + max(G, 4 * RTTVAR)                  # G = clock
granularity
```

The key insight is including the *variance* term. On a jittery path a fixed multiple of the mean is either too tight or too loose; scaling by $4 \times \text{RTTVAR}$ widens the timeout exactly when the path is unpredictable. RTO is floored (Linux ~ 200 ms `TCP_RTO_MIN`) and capped (~ 120 s).

Karn's algorithm solves a subtle sampling problem. When a segment is retransmitted and an ACK arrives, the sender cannot tell whether the ACK was for the *original* transmission or the *retransmission* — measuring RTT from the wrong send time corrupts SRTT. Karn's rule: **do not take an RTT sample from any retransmitted segment**. To still make progress when everything is being retransmitted, Karn pairs this with **exponential backoff**: each successive retransmission of the same segment doubles the RTO, so a persistently unreachable peer is probed with geometrically decreasing frequency rather than hammered. Once an unambiguous (non-retransmitted) segment is ACKed, normal RTT sampling resumes. Modern stacks that negotiate **TCP timestamps** (RFC 7323) sidestep the ambiguity entirely — the echoed timestamp identifies which transmission an ACK corresponds to — allowing RTT samples even from retransmissions.

The small-packet pathologies: Nagle, delayed ACK, and their bug

Two independent optimizations, each sensible alone, combine into one of the most notorious latency bugs in networking.

Nagle's algorithm (RFC 896) reduces the overhead of tiny segments (a 1-byte payload in a 40-byte-header packet is 98% waste — think a Telnet keystroke). Its rule: *while there is unacknowledged data outstanding, do not send a new small (sub-MSS) segment; buffer it until either a full MSS accumulates or the outstanding data is ACKed*. It coalesces small writes into fewer, fuller packets.

Delayed ACK (RFC 1122) reduces pure-ACK traffic: rather than ACK every segment immediately, the receiver waits up to ~40-200 ms (Linux ~40 ms) hoping to either piggyback the ACK on outbound data or accumulate a second segment to ACK both cumulatively.

The interaction: a sender using Nagle writes a small request and waits (correctly per Nagle) for the previous data to be ACKed before sending the tail of it. The receiver, using delayed ACK, is *sitting on the ACK* hoping for more data or a chance to piggyback — but it will not send data until it has the full request, which the sender will not send until it gets the ACK. Deadlock, broken only when the receiver's ~40 ms delayed-ACK timer fires. The result is a request that should take one RTT taking one RTT **plus ~40 ms**, on a fraction of requests, in a way that is maddening to diagnose because it is intermittent and load-dependent. This is the classic "why is my RPC sometimes 40 ms slower for no reason" bug, and it has bitten Redis clients, database drivers, and RPC frameworks repeatedly.

The fix for any latency-sensitive, request/response protocol is **TCP_NODELAY**, which disables Nagle so small segments go out immediately:

```
int one = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one));
```

Nearly every serious networking library sets **TCP_NODELAY** by default for exactly this reason — gRPC, most HTTP servers, Redis, Kafka clients. Nagle's coalescing is a benefit only for bulk-streaming protocols that do many tiny writes and do not care about per-message latency, which is rare in modern backends. The receiver-side counterpart is **TCP_QUICKACK** (Linux), which suppresses ACK delay — but note it is not sticky; it resets and must be re-applied, so **TCP_NODELAY** on the sender is the durable fix. The right mental model: **do not disable delayed ACK to work around Nagle; disable Nagle**. They are only pathological *together*.

Keepalives

A TCP connection with no traffic sends nothing on the wire — the protocol is happy to leave an idle connection open forever. That is a problem when the peer has crashed, been rebooted, or had its path silently torn down by a middlebox: your end still thinks the connection is `ESTABLISHED` and will discover otherwise only when it next tries to send. **TCP keepalive** (`SO_KEEPALIVE`) probes an idle connection: after `tcp_keepalive_time` (Linux default *7200 seconds* — two hours) of idleness, it sends a keepalive probe (a bare ACK for the byte before `snd_una`); if no response, it retries `tcp_keepalive_probes` times at `tcp_keepalive_intvl` intervals before declaring the connection dead.

The two-hour default is useless for detecting failures quickly and, worse, is far longer than the idle timeout of most NATs and cloud load balancers (often 60–350 seconds), which silently drop the flow's state so that your eventual keepalive or data hits a black hole or elicits a RST. In practice, backends either tune keepalive far lower (per-socket via `TCP_KEEPIDLE`, `TCP_KEEPINTVL`, `TCP_KEEPCNT`) or, better, implement **application-level heartbeats** with explicit timeouts, which also verify the *application* is alive and not just its kernel (gRPC's HTTP/2 PING frames, for example — Chapter 8). Kernel keepalive tells you the peer's TCP stack answered; an application ping tells you the peer can actually serve you.

Head-of-line blocking within a stream

TCP's ordering guarantee has a cost that no amount of tuning removes: **head-of-line (HOL) blocking**. Because bytes must be delivered *in order*, a single lost segment blocks delivery of *every* later byte that has already arrived, until the retransmission fills the gap. The receiver's kernel is holding perfectly good data in its reassembly buffer but cannot hand any of it to the application, because doing so would violate the ordered-byte-stream contract.

This is inherent to TCP's single-stream model and it becomes acute when you multiplex *independent* logical streams over one TCP connection. HTTP/2 (Chapter 7) does exactly this: many concurrent requests share one TCP connection to amortize handshakes and warm one `cwnd`. But if a single packet carrying bytes for request A is lost, TCP stalls the *entire* connection — every one of the multiplexed requests B, C, D is blocked behind A's retransmission, even though their bytes have arrived, because

TCP cannot see the streams; it sees one byte stream and one gap in it. HTTP/2 solved application-layer HOL blocking (the HTTP/1.1 problem of one slow response blocking the pipeline) only to expose it at the transport layer.

```
flowchart TB
    subgraph net["One TCP byte stream (HTTP/2 multiplexing)"]
        direction LR
        a1["A: seg1 OK"] --> a2["A: seg2 LOST"] --> b1["B: seg1 arrived"]
    end
    c1["C: seg1 arrived"]
    net --> block["TCP holds B and C in the reassembly buffer:  
cannot deliver past the gap at A-seg2"]
    block --> app["Application sees A, B, C all stalled  
until A-seg2 is retransmitted (1 RTT)"]
```

This is precisely why **QUIC** (Chapter 4) abandons TCP and runs its own reliable, ordered delivery *per stream* over UDP: a lost packet for stream A blocks only stream A; streams B and C are delivered independently because QUIC understands stream boundaries that TCP structurally cannot. HTTP/3 is HTTP over QUIC, and eliminating transport-layer HOL blocking on lossy paths (mobile, WAN) is its central motivation. TCP's greatest strength — a simple ordered byte stream — is also the source of its one unfixable weakness.

Tuning: buffers, BDP, and the initial window

Getting good throughput out of TCP is mostly about not starving the two windows that gate it.

Socket buffers must be at least the BDP. The send buffer (`SO_SNDBUF` / `net.ipv4.tcp_wmem`) bounds how much unacknowledged data the sender can hold; the receive buffer (`SO_RCVBUF` / `net.ipv4.tcp_rmem`) bounds `rwnd`. If either is smaller than $\text{bandwidth} \times \text{RTT}$, the connection cannot keep the pipe full regardless of congestion control. For the $10 \text{ Gbps} \times 80 \text{ ms}$ path ($\text{BDP} \approx 100 \text{ MB}$) you need buffers on that order. Linux does **autotuning** by default (`net.ipv4.tcp_moderate_rcvbuf=1`), growing the receive buffer toward the measured BDP up to the `tcp_rmem` maximum — so the usual failure mode is a `tcp_rmem / wmem` *ceiling* set too low for a fat long path, not the tuning logic itself. Inspect and raise:

```
sysctl net.ipv4.tcp_rmem net.ipv4.tcp_wmem # min default max, in
bytes
# e.g. net.ipv4.tcp_rmem = 4096 131072 6291456
sysctl -w net.ipv4.tcp_rmem="4096 131072 134217728" # raise the
ceiling for fat paths
```

Note that calling `setsockopt(SO_RCVBUF)` *disables* autotuning for that socket and pins the buffer — so setting a fixed value can *hurt* on paths where autotuning would have grown larger. Prefer raising the `sysctl` ceilings and letting autotuning work, unless you have a specific reason to pin.

The initial congestion window (`initcwnd`) determines how much the server can send in the first RTT before any ACK returns. Default 10 MSS (~14.6 KB) fits most small responses in one RTT; a larger `initcwnd` speeds the first flight of bigger responses but risks bursting into an unknown path. It is set per-route:

```
ip route change default via 10.0.0.1 dev eth0 initcwnd 20 initrwnd 20
ip route show # verify the initcwnd on the route
```

Inspect a live connection's actual state — `cwnd`, RTT estimate, retransmits, chosen congestion control — with `ss`, which is the single most useful TCP-debugging command:

```
ss -tino 'dport = :443'
# ESTAB 0 0 10.0.1.5:52344 93.184.216.34:443
# cubic wscale:7,7 rto:204 rtt:24.1/3.2 mss:1448 cwnd:32 ssthresh:24
# bytes_sent:1.2M bytes_acked:1.2M segs_out:900 retrans:0/3
delivery_rate:15.4Mbps
```

`retrans:0/3` (currently retransmitting 0, 3 total over the connection's life), `cwnd:32`, `rtt:24.1/3.2` (SRTT/RTTVAR), and `delivery_rate` together tell you whether a slow transfer is loss-bound, RTT-bound, window-bound, or fine. This is where TCP theory meets the on-call terminal.

Distributed-systems lens

TCP is not a background detail of your architecture; it is frequently the dominant term in your latency and the root cause of your stranger outages.

TCP dominates RPC latency and throughput. A synchronous RPC pays, at minimum, one RTT for the handshake (if the connection is cold), plus the transfer time gated by `cwnd` and `rwnd`. For the small request/response messages that make up most microservice traffic, the handshake and slow-start ramp *dwarf* the actual data transfer. Your service's p99 is often a TCP artifact — a retransmission timeout, a delayed-ACK stall, a cold connection's slow start — not your application code. Measuring the transport (`ss`, `tcpdump`, eBPF probes from Volume 2, Chapter 11) is often more productive than profiling the handler.

Slow start punishes short-lived connections, so reuse them. Every new connection starts cold: one RTT for the handshake and several more climbing out of `initcwnd`. A service that opens a fresh connection per request throws away the warmed `cwnd` every time and pays the ramp repeatedly. This is *the* argument for connection pooling, HTTP keep-alive, and HTTP/2's single long-lived multiplexed connection (Chapters 7 and 8): keep connections warm so `cwnd` stays large and the handshake is amortized across thousands of requests. The savings are not marginal; on a WAN they can be the difference between a 5 ms and a 200 ms call.

TIME_WAIT, ephemeral ports, and conntrack cause real outages. The three most common TCP-exhaustion outages all trace to connection churn: ephemeral source-port exhaustion on a client or proxy that opens too many short connections to one backend (60-second `TIME_WAIT` × ~28K ports), `nf_conntrack` table overflow on a NAT gateway or Kubernetes node (Chapter 2), and file-descriptor exhaustion from `CLOSE_WAIT` leaks in a buggy application. All three present as "new connections fail" while existing ones work, and all three are fixed at the *architecture* level — fewer, longer-lived connections — far more reliably than by tuning timeouts. When an API gateway falls over under load with `EADDRNOTAVAIL` in the logs, this is the family of causes.

BBR versus CUBIC matters on the WAN. Within a datacenter, RTTs are sub-millisecond, loss is rare, and congestion control barely matters — any algorithm fills the pipe in a couple of round trips. Across a WAN (region-to-region replication, CDN origin fetches, cross-cloud links) with tens of milliseconds of RTT and occasional non-congestive loss, the choice is decisive: CUBIC's loss-based backoff can leave a fat pipe half-empty, while BBR paces to the measured bandwidth-delay product and shrugs off random loss. Teams running large cross-region data movement (database replication, backup, analytics egress) increasingly run BBR with `fq`

precisely for this reason — but with the fairness caveats above, so it is a measured rollout, not a default.

HOL blocking drove HTTP/3. The transport-layer HOL blocking that HTTP/2 exposed is not a bug you can tune away; it is TCP's ordered-byte-stream contract doing exactly what it promises. On a clean datacenter path it is invisible. On a lossy mobile or long-haul path it is the difference between a page that renders progressively and one that freezes on a single dropped packet. That is why HTTP/3 moved to QUIC-over-UDP, replacing one shared TCP byte stream with many independent QUIC streams (Chapters 4 and 7). Understanding *why* — that TCP cannot see the streams multiplexed inside its own byte stream — is the whole point of this chapter's last section.

Key takeaways

- TCP turns IP's best-effort datagrams into a reliable, ordered *byte stream* — with no message boundaries. Framing is the application's job; assuming one `send` maps to one `recv` is a bug.
- Cumulative ACKs and per-byte sequence numbers give robustness but coarse loss recovery; SACK restores precision by naming the exact missing blocks.
- The three-way handshake costs a full RTT before data; the four-way close leaves `TIME_WAIT` on the *active closer* for $2 \times \text{MSL}$ (60 s on Linux), which exhausts ephemeral ports under connection churn. Fix it by reusing connections, not by dangerous knobs like the removed `tcp_tw_recycle`.
- Flow control (`rwnd` , receiver-driven, needs window scaling to fill a fat pipe) is entirely distinct from congestion control (`cwnd` , network-driven). The send window is `min(rwnd, cwnd)`.
- Slow start ramps `cwnd` exponentially from `initcwnd` (10 MSS); congestion avoidance grows it linearly (AIMD sawtooth). Fast retransmit/recovery handles single losses via 3 duplicate ACKs without collapsing to slow start; only an RTO timeout resets `cwnd` to 1.
- CUBIC (Linux default) is loss-based and RTT-fair but fills buffers and overreacts to random loss; BBR is model-based (paces to $\text{BtlBw} \times \text{RTTprop}$), excels on high-BDP lossy WANs, and keeps buffers near-empty — with fairness caveats that v2/v3 address.

- RTO derives from smoothed RTT plus $4\times$ variance; Karn's algorithm forbids sampling RTT from retransmitted segments and backs off exponentially.
- Nagle + delayed ACK deadlock adds ~ 40 ms to request/response latency; disable Nagle with `TCP_NODELAY` (do not fight it with disabled delayed ACK). Most modern libraries do this.
- TCP's ordered stream causes head-of-line blocking: one lost segment stalls all multiplexed streams above it — the unfixable weakness that motivated QUIC and HTTP/3.

Further reading

- Kevin R. Fall, W. Richard Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*, 2nd ed., Addison-Wesley, 2011 — the definitive treatment of the mechanisms in this chapter.
- RFC 9293 — *Transmission Control Protocol (TCP)* (2022), the current consolidated TCP specification obsoleting RFC 793. <https://www.rfc-editor.org/rfc/rfc9293>
- V. Jacobson, "Congestion Avoidance and Control," *SIGCOMM* 1988 — the origin of slow start and congestion avoidance. <https://ee.lbl.gov/papers/congavoid.pdf>
- D.-M. Chiu, R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," *Computer Networks and ISDN Systems*, 1989 — why AIMD.
- S. Ha, I. Rhee, L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," *ACM SIGOPS Operating Systems Review*, 2008; and RFC 8312 (informational). CUBIC's design and analysis.
- N. Cardwell et al., "BBR: Congestion-Based Congestion Control," *ACM Queue / Communications of the ACM*, 2016–2017. <https://queue.acm.org/detail.cfm?id=3022184>
- RFC 6298 — *Computing TCP's Retransmission Timer (SRTT/RTTVAR/RTO)*; RFC 2018 — *TCP Selective Acknowledgment Options*; RFC 6582 — *The NewReno Modification to TCP's Fast Recovery*.
- RFC 7323 — *TCP Extensions for High Performance* (window scaling and timestamps); RFC 6928 — *Increasing TCP's Initial Window* (initcwnd 10).

- RFC 896 — *Congestion Control in IP/TCP Internetworks* (Nagle's algorithm); RFC 1122 — *Requirements for Internet Hosts* (delayed ACK).
- RFC 7413 — *TCP Fast Open*; RFC 6528 — *Defending against Sequence Number Attacks* (ISN generation).
- The Linux kernel networking documentation on TCP sysctls (<https://www.kernel.org/doc/html/latest/networking/ip-sysctl.html>) and the `tcp(7)`, `ss(8)`, and `ip-route(8)` man pages — the operational reference for every knob above.

Chapter 4 — UDP and QUIC

What this chapter covers. Chapter 3 (TCP in Depth) spent its length defending a thesis: that TCP's reliability, ordering, and congestion control are worth their cost, and that most of the time you should let the kernel do this work for you. This chapter argues the two exceptions. The first is UDP — the transport that does almost nothing, and is chosen precisely *because* it does almost nothing, so that the application can supply exactly the delivery semantics it needs and not one byte more. The second, and the reason this chapter is long, is QUIC: a full reliable, multiplexed, encrypted transport built *on top of* UDP, in user space, that exists because TCP had become impossible to evolve. QUIC is not a niche protocol. It is the transport under HTTP/3, it carries a large and growing fraction of all web traffic to Google, Meta, Cloudflare, and the major CDNs, and it is the clearest working example in production networking of a design principle you will meet again and again: when a layer ossifies, you do not fix it in place — you move the function somewhere you still control. We will treat UDP as prologue and QUIC as the main event, and we will insist on mechanism throughout: not "QUIC has 0-RTT," but what packet, carrying what frame, protected by what key, makes 0-RTT possible and what it costs you in replay safety.

Learning goals — after this chapter you should be able to:

- State exactly what UDP guarantees (integrity of a delivered datagram, port multiplexing) and what it does not (delivery, ordering, deduplication, flow control, congestion control), and choose UDP correctly for DNS, real-time media, gaming, and custom protocols.

- Reason about UDP datagram sizing, IP fragmentation, and why a UDP-based protocol must do its own path-MTU discovery and its own congestion control to be a good network citizen.
- Explain the UDP amplification/reflection attack mechanism and the specific defenses (BCP 38, response-rate limiting, QUIC's anti-amplification limit).
- Explain, in mechanism, the four problems that made TCP inescapable — cross-stream head-of-line blocking, slow layered handshakes, middlebox ossification, and kernel-deployment latency — and how QUIC's design neutralizes each.
- Describe QUIC's integrated TLS 1.3 handshake, its stream model and per-stream reliability, connection IDs and connection migration, and user-space pluggable loss recovery and congestion control — and trace a 1-RTT and a 0-RTT connection setup.
- Compare QUIC against TCP+TLS honestly, including QUIC's real costs: UDP being blocked or deprioritized, and the CPU overhead of a user-space transport.
- Apply the distributed-systems lens: how 0-RTT and connection migration cut tail latency for mobile and edge clients, how per-stream reliability removes the HTTP/2-over-TCP HOL penalty, and why user-space transport mirrors kernel-bypass and shifts protocol evolution onto your own deploy cadence.

UDP: the transport that gets out of the way

The User Datagram Protocol, specified in RFC 768 in 1980 in barely three pages, is the minimal thing you can put on top of IP and still call a transport. It adds exactly two capabilities to raw IP. First, **multiplexing**: a 16-bit source port and 16-bit destination port let a host demultiplex an incoming datagram to the right socket, so many application endpoints can share one IP address. Second, an **optional integrity check**: a 16-bit checksum over the payload, the UDP header, and a pseudo-header drawn from the IP layer (source and destination addresses, protocol number, length). Over IPv4 the checksum may be disabled by sending zero; over IPv6 it is mandatory because IPv6 has no header checksum of its own. That is the entire contract. The header is eight bytes — source port, destination port, length, checksum, four 16-bit fields — against TCP's twenty-byte minimum.

Everything TCP does beyond addressing, UDP declines to do. There is no handshake: the first UDP datagram you send is the connection, such as it is, and there is no shared state to establish because there is no shared state at all. There is no sequence number, so datagrams can be reordered, duplicated by the network, or silently dropped, and the receiver has no way to know from UDP alone that anything is missing. There is no acknowledgment and no retransmission — a lost datagram is simply gone. There is no flow control, so a fast sender can overrun a slow receiver's socket buffer and the excess is dropped. And, most consequential for the health of the Internet as a whole, there is **no congestion control**: UDP will emit datagrams as fast as the application hands them over, with no built-in reaction to loss or delay that signals a congested path.

This is not a deficiency to be apologized for. It is the point. TCP's guarantees are not free — they cost a round trip to establish, they cost head-of-line blocking on loss, they cost the latency of retransmitting data the application may no longer want by the time it arrives. An application that does not need in-order reliable byte-stream semantics should not pay for them. UDP is the substrate you reach for when you intend to supply your own delivery semantics, tuned to the workload, at the application layer — including, as we will see, a complete reliable transport when that is what you actually want.

```
flowchart TB
    subgraph TCP["TCP – the kernel does the work"]
        T1["Connection-oriented: 3-way handshake before data"]
        T2["Reliable: sequence numbers, ACKs, retransmission"]
        T3["Ordered: byte stream delivered in send order"]
        T4["Flow control: receiver window"]
        T5["Congestion control: slow start, CUBIC/BBR"]
        T6["20+ byte header, per-connection state"]
    end
    subgraph UDP["UDP – the app does the work"]
        U1["Connectionless: first datagram is the traffic"]
        U2["Unreliable: no ACK, lost datagram is gone"]
        U3["Unordered: datagrams may arrive out of order or duplicated"]
        U4["No flow control: sender can overrun receiver"]
        U5["No congestion control: app must self-limit"]
        U6["8 byte header, no per-flow state in the transport"]
    end
```


When UDP is the right answer

Four workload families choose UDP, and the reasons differ enough to be worth separating.

Request/response where the request is a single small datagram and the application handles retries. DNS is the archetype (Chapter 5, DNS in Depth). A query and its answer each fit, classically, in a single sub-512-byte datagram; the resolver sets a timeout and re-queries on loss, often against a different server; and the amortized cost of a TCP handshake per lookup would be absurd for a protocol invoked dozens of times per page load. DNS falls back to TCP only when a response is too large for the negotiated UDP payload (EDNS0 buffer size) or when the truncation bit forces it. The pattern generalizes: any idempotent, self-contained, latency-sensitive request/response exchange where you would rather manage your own timeout and retry than pay for connection setup is a UDP candidate.

Real-time media, where stale data is worthless. In a voice or video call, a packet that arrives 300 ms late is not useful — the moment it describes has already been rendered as a concealment or a freeze, and re-sending it would only delay everything behind it. TCP's insistence on in-order delivery is actively harmful here: one lost packet stalls the stream until the retransmission arrives, injecting exactly the jitter the codec is trying to hide. RTP (RFC 3550) runs over UDP for precisely this reason and layers on its own sequence numbers and timestamps so the receiver can reorder, detect loss, and drive jitter buffers and concealment — reliability the application actually wants, without the reliability it does not. WebRTC media, SIP-signaled telephony, and live streaming ingest all sit here.

Interactive gaming and other tight control loops. A multiplayer game sends frequent small state updates — positions, inputs — where only the *latest* update matters. Reliable in-order delivery of stale positions is worse than useless; the game would rather drop an old update and apply the new one. Many game networking stacks build a thin reliability layer over UDP that is selective: reliable-ordered for a chat message or an inventory change, unreliable for the 20-per-second position stream, all multiplexed on one UDP flow. QUIC's stream model, which we reach shortly, is in part a generalization of exactly these hand-rolled layers.

Custom and infrastructure protocols that want IP-with-ports and nothing else. SNMP, NTP, syslog, DHCP, VXLAN and Geneve tunnel encapsulation, QUIC itself — all ride UDP because they want the multiplexing and the checksum and intend to supply (or deliberately omit) everything else themselves. Tunneling is a telling case: VXLAN wraps L2 frames in UDP not because it wants unreliability but because UDP's port field gives ECMP hashing something to spread flows across, and because UDP passes through middleboxes and NAT that would choke on a raw custom protocol number.

The obligations UDP hands you

Choosing UDP means inheriting three responsibilities the kernel was handling for you.

Sizing and fragmentation. UDP itself imposes no size limit below the 65,535-byte ceiling of its length field, but the moment your datagram exceeds the path MTU, the IP layer fragments it. IP fragmentation is a trap for a datagram protocol: if any single fragment is lost, the entire datagram is undeliverable and must be discarded, so a 4 KB UDP datagram over a 1500-byte path is three fragments any one of which loses you the whole thing — and fragments are disproportionately dropped by firewalls and stateless load balancers that cannot see the ports in a non-first fragment. The discipline is to keep datagrams within the path MTU and to do your own path-MTU discovery. This is why DNS historically capped UDP payloads near 512 bytes, why EDNS0 lets endpoints negotiate a larger but still bounded buffer, and why QUIC requires the datagrams carrying a client's first flight to be padded to at least 1,200 bytes (a conservative floor almost every path can carry unfragmented) and performs Datagram PLPMTUD (RFC 8899) to probe upward safely.

Being a good citizen: congestion control. This is the obligation engineers most often neglect and the one with the widest blast radius. TCP's congestion control is not there to help the individual connection; it is there to keep the shared network from collapsing when many flows contend. A UDP application that blasts at line rate ignoring loss is a free rider that starves every TCP flow sharing the bottleneck, and if enough traffic behaves this way the network can enter congestion collapse — the pathology that motivated congestion control in the first place. RFC 8085 (UDP Usage Guidelines, a BCP) is explicit: an application sending more

than trivial volume over UDP **must** implement congestion control, and should reuse a proven algorithm rather than invent one. "The app handles reliability" is only half the deal; the app must also handle *restraint*. QUIC takes this seriously — RFC 9002 specifies a full congestion controller — precisely because a transport that did not would be a menace.

Security: the amplification/reflection risk. UDP has no handshake, so a server replies to the source address in the datagram without ever confirming the sender is really there. Because the source address in an IP packet can be forged (spoofed), an attacker sends a small query with the *victim's* address as the source, and the server dutifully mails a large response to the victim. When the response is much larger than the request, the attacker has an amplifier: a few bytes of spoofed query become kilobytes aimed at the target, from a server that is not even compromised. DNS (especially with large DNSSEC responses), NTP's `monlist`, memcached exposed on UDP 11211, and SSDP have all been abused this way, memcached notoriously with amplification factors in the tens of thousands. The defenses are layered: network operators should deploy source-address validation (BCP 38 / RFC 2827) so spoofed packets never leave the origin network; services should rate-limit responses (DNS Response Rate Limiting); and protocols should design so that an unverified peer cannot elicit a large response. QUIC bakes the last of these into the specification, as we will see — a server must not send more than three times the bytes it has received from an as-yet-unvalidated client address. Keep this mechanism in mind; it is a recurring theme in why QUIC's handshake is shaped the way it is.

Why QUIC exists: TCP became ineluctable

Chapter 3 established TCP as a mature, well-tuned reliable transport. So why build a new one? Not because TCP's algorithms are bad — QUIC reuses most of them — but because TCP as a *deployed system* had reached a state where its remaining problems could not be fixed. Four forces combined into an argument for moving the transport out of the kernel and onto UDP.

Cross-stream head-of-line blocking. TCP delivers a single, strictly ordered byte stream. HTTP/2 (Chapter 7) multiplexes many logical request/response streams onto one TCP connection to avoid the connection-per-request cost of HTTP/1.1. But TCP knows nothing about

those streams; it sees one byte sequence. If a single TCP segment is lost, TCP holds *all* subsequently received bytes in its reassembly buffer until the retransmission arrives — because it must deliver in order — which means a packet loss affecting stream 5 also stalls streams 1 through 4, even though their bytes have already arrived. HTTP/2 solved application-layer HOL blocking (HTTP/1.1's one-at-a-time pipeline) only to inherit a worse *transport-layer* HOL blocking that gets more painful as loss rises and as you multiplex more streams onto the one connection. You cannot fix this above TCP; the ordering is TCP's, imposed on bytes it cannot tell apart.

Handshake latency from layering. A conventional HTTPS connection pays for two handshakes in sequence: TCP's three-way handshake to establish the connection (one RTT before any data), then the TLS handshake on top of it (one more RTT for TLS 1.3, two for TLS 1.2). The layers do not know about each other, so their round trips stack. For a client far from the server — a mobile user, an edge client hitting an origin — that is two or three RTTs of pure setup before the first byte of a response, and on a 150 ms path that is most of what the user perceives as "slow."

Middlebox ossification. TCP runs in the clear: sequence numbers, flags, options, and window are all visible on the wire. Over three decades, firewalls, NATs, load balancers, and "transparent" proxies came to inspect and depend on those fields, and worse, to *reject* what they do not recognize. TCP Fast Open, which would let data ride the SYN, is dropped or mangled by enough middleboxes to be unreliable in the wild. New TCP options are stripped. Even deploying a new congestion-control behavior can trip a middlebox that second-guesses the sender. The protocol calcified because its evolvable surfaces were exposed and something on the path started depending on their current values — the phenomenon called ossification. The lesson QUIC draws is blunt: **if it is visible, something will ossify it, so encrypt it.**

Kernel-deployment latency. TCP lives in the operating-system kernel. A change to TCP's loss recovery or congestion control is a kernel change, which propagates at the speed of OS upgrades across billions of devices and every intermediary — years, sometimes a decade, and on client devices largely outside any single operator's control. Even a demonstrably better algorithm cannot reach the traffic that would benefit

from it on any useful timescale. The transport had become un-iterable not only on the wire but in the deployment pipeline.

Put these together and the conclusion is structural. The fixes TCP needs are impossible *where TCP lives*. So QUIC moves the transport: onto UDP (which middleboxes forward as an opaque datagram), into user space (where an application ships a new version on its own release cadence), and behind encryption (so the wire surface cannot ossify). QUIC is, in one sentence, a reliable, ordered-per-stream, multiplexed, always-encrypted transport implemented in user space on top of UDP, standardized by the IETF as **RFC 9000** (the core transport), **RFC 9001** (TLS integration), **RFC 9002** (loss detection and congestion control), and **RFC 8999** (version-independent properties), all published in **May 2021**, after Google deployed and iterated its precursor "gQUIC" in Chrome and on its own servers through the mid-2010s.

```
flowchart TB
    subgraph classic["Classic HTTPS stack"]
        A1["HTTP/2"]
        A2["TLS 1.3 (in the app)"]
        A3["TCP (kernel)"]
        A4["IP"]
        A1 --> A2 --> A3 --> A4
    end
    subgraph quicstack["HTTP/3 stack"]
        B1["HTTP/3"]
        B2["QUIC transport (user space): streams, reliability, congestion control"]
        B3["TLS 1.3 handshake integrated into QUIC (RFC 9001)"]
        B4["UDP"]
        B5["IP"]
        B1 --> B2
        B2 -. "keys, handshake" .-> B3
        B2 --> B4 --> B5
    end
```

Inside QUIC

QUIC's cleverness is not any single feature but the way integration lets features reinforce each other. We take them one at a time and then show how they compose.

Integrated TLS 1.3: one handshake, not two

QUIC does not run TLS "on top of" itself the way HTTPS runs TLS on top of TCP. Instead it embeds the TLS 1.3 handshake *inside* the transport handshake and uses TLS's key schedule to protect QUIC's own packets. The full mechanics are Chapter 6 (TLS 1.3 and the Web PKI); here we need the shape. QUIC carries the TLS handshake messages — ClientHello, ServerHello, Certificate, Finished — inside special CRYPTO frames rather than in a TLS record layer, and it feeds TLS's derived secrets back to protect QUIC packets. There is no separate TLS record protocol; QUIC *is* the record protocol. The consequence is that the transport handshake and the cryptographic handshake are the same round trip. Where TCP+TLS 1.3 spends one RTT on TCP and then one RTT on TLS, QUIC establishes a connection *and* completes the TLS 1.3 key exchange in **one RTT** total, because the ClientHello rides in the very first flight of packets. There is no unencrypted QUIC connection; encryption is not layered on, it is constitutive.

Two further consequences matter. First, because there is no cleartext transport handshake, QUIC has no "plaintext SYN" for a middlebox to inspect and ossify — the anti-ossification property falls out of the integration for free. Second, QUIC uses **separate packet number spaces** for the Initial, Handshake, and Application (1-RTT) phases, each protected with keys of increasing strength as the handshake progresses, so that early packets sent under weak, publicly derivable "Initial" keys cannot be confused with, or downgrade, the fully protected application data.

0-RTT. When a client has talked to a server before, TLS 1.3 lets it cache a pre-shared key (a session resumption ticket). On the *next* connection, the client can encrypt application data with that key and send it in the very first flight — **0-RTT** — so the request reaches the server without waiting even one round trip for the handshake to complete. For a repeat visitor this is the difference between a request that starts immediately and one that waits 150 ms first. The catch, and it is a real one, is **replay**: 0-RTT data is not protected against an attacker capturing the first flight and re-sending it, because the anti-replay guarantees come from the handshake the 0-RTT data is trying to skip. So 0-RTT is safe only for *idempotent* operations — a GET, not a "transfer money" POST — and the specifications (RFC 9001, and RFC 8446 for TLS generally) require that applications gate what they permit in 0-RTT. Servers also enforce anti-

replay defenses, but the durable rule for the application engineer is: **never carry a non-idempotent request in 0-RTT.**

```
sequenceDiagram
    participant C as Client
    participant S as Server
    Note over C,S: TCP + TLS 1.3 (two stacked handshakes)
    C->>S: TCP SYN
    S->>C: TCP SYN-ACK
    C->>S: TCP ACK (1 RTT gone)
    C->>S: TLS ClientHello
    S->>C: TLS ServerHello, cert, Finished
    C->>S: TLS Finished (2 RTTs gone)
    C->>S: HTTP request (data at last)
    Note over C,S: QUIC (one integrated handshake)
    C->>S: Initial: ClientHello (+ 0-RTT data if resuming)
    S->>C: Initial + Handshake: ServerHello, cert, Finished
    C->>S: HTTP request (1 RTT, or 0 RTT on resumption)
```

Streams: reliability without cross-stream head-of-line blocking

This is the feature HTTP/3 was built to get. A QUIC connection carries many independent **streams**, each an ordered, reliable byte sequence, identified by a stream ID. Streams are cheap: they are created implicitly by sending data with a new ID, and QUIC distinguishes client- vs server-initiated and bidirectional vs unidirectional streams by the low bits of the ID. Application data travels in **STREAM frames**, each tagging its bytes with a stream ID and an offset within that stream. Crucially, **many frames for many streams are packed into one QUIC packet, which is one UDP datagram** — the multiplexing is inside QUIC, invisible to UDP and to the network.

Now the payoff. Reliability and ordering in QUIC are **per stream**, not per connection. QUIC acknowledges *packets* (via ACK frames that reference packet numbers) for loss detection, but it *delivers* stream data in order **within each stream independently**. If a packet carrying STREAM frames for stream 5 is lost, QUIC retransmits that stream data — but streams 1 through 4, whose bytes arrived in other packets, are delivered to the application immediately. There is no cross-stream blocking, because QUIC understands the stream boundaries that TCP could not see. This is the exact defect from the "why QUIC exists" section, resolved: HTTP/2's transport-layer HOL blocking existed because TCP imposed one order on all streams; HTTP/3 over QUIC eliminates it because QUIC imposes order only within each stream. Under loss — mobile networks,

congested paths, long fat pipes — the difference is large and grows with the loss rate and the number of concurrent streams.

```
flowchart TB
    subgraph h2["HTTP/2 over TCP – one lost segment stalls everyone"]
        direction LR
        L1["Stream 1 bytes"] --> B["TCP reassembly: strict single order"]
        L2["Stream 2 bytes"] --> B
        L3["Stream 3 (LOST segment)"] -. blocks .-> B
        B --> B0["App receives: NOTHING until stream 3 retransmit arrives"]
    end
    end
    subgraph h3["HTTP/3 over QUIC – loss is contained to its stream"]
        direction LR
        Q1["Stream 1 bytes"] --> D1["Deliver stream 1 now"]
        Q2["Stream 2 bytes"] --> D2["Deliver stream 2 now"]
        Q3["Stream 3 (LOST)"] -. only stream 3 waits .-> D3["Retransmit stream 3 only"]
    end
    end
```

QUIC also has flow control at two levels — **per-stream** limits (how much data may be outstanding on one stream) and a **connection-level** limit (aggregate across all streams) — advertised and raised with `MAX_STREAM_DATA` and `MAX_DATA` frames, plus `MAX_STREAMS` to bound how many streams a peer may open. This mirrors TCP's receive window but is finer-grained: a single stalled stream cannot consume the whole connection's buffer budget and starve the others.

One honest caveat: QUIC removes *transport* HOL blocking, not all HOL blocking. Within a single stream, order is still enforced, so a loss affecting one stream still stalls that stream. And HTTP/3's header compression (QPACK, RFC 9204) reintroduces a bounded, carefully designed form of cross-stream dependency that its authors worked hard to keep from recreating HOL blocking. The transport-level win is real and central; it is not a claim that ordering ever comes for free.

Connection IDs and connection migration

A TCP connection is identified by its four-tuple: source IP, source port, destination IP, destination port. Change any element and it is, definitionally, a different connection — which is why your phone dropping from Wi-Fi to cellular tears down every TCP connection, because the source IP changed. For a long-lived download or a video call on a mobile

device, this means re-handshaking (TCP + TLS again) on every network transition, precisely when the user is moving and latency is already bad.

QUIC breaks the identity free of the address. Each endpoint labels the connection with one or more **Connection IDs** (CIDs) that it chooses and asks the peer to use as the destination label on packets. A packet is matched to its connection by its CID, *not* by its four-tuple. So when the client's IP and port change — Wi-Fi to LTE, or a NAT rebinding after an idle period — the packets still carry the same Connection ID, and the server recognizes them as the same connection and continues without a new handshake. This is **connection migration**, and it is the feature that makes QUIC feel qualitatively different on mobile: a download survives the elevator, the video call survives walking out of Wi-Fi range.

Migration is designed carefully against abuse, and the design connects back to the UDP amplification problem. An attacker could otherwise migrate a connection to a victim's address to redirect a flood, so when a peer sees packets arriving from a new address it must perform **path validation** — send a PATH_CHALLENGE frame with an unpredictable payload and require a matching PATH_RESPONSE — before treating the new path as usable at full rate, and it applies the same anti-amplification limit (send no more than three times what has been received from the unvalidated path) until validation succeeds. Endpoints also rotate CIDs (issuing new ones with NEW_CONNECTION_ID frames and retiring old ones) so that a passive observer cannot trivially link a user's pre-migration and post-migration flows by a stable CID — a genuine, if partial, privacy improvement over the permanently correlatable TCP four-tuple.

User-space, pluggable loss recovery and congestion control

QUIC's loss detection and congestion control (RFC 9002) are conceptually close to modern TCP — the recommended default behaves like NewReno, and implementations routinely run CUBIC or BBR — but three design choices make them evolve on a different timescale.

First, QUIC **acknowledges packet numbers that never decrease and never repeat**. TCP's ACKs reference byte sequence numbers, and a retransmitted segment reuses its original sequence number, creating the classic *retransmission ambiguity*: when an ACK arrives, TCP cannot always tell whether it acknowledges the original transmission or the retransmit, which muddies RTT estimation and loss detection. QUIC

assigns every packet — including retransmissions of the same data — a fresh, monotonically increasing packet number, so an ACK is never ambiguous about which transmission it covers. The stream data is retransmitted (in new STREAM frames, in a new packet, with a new packet number), while the packet number cleanly identifies the transmission event. This gives much crisper RTT samples and loss inference than TCP can achieve.

Second, QUIC's ACK frames carry **many acknowledgment ranges** and an explicit **ack delay**, richer than TCP's cumulative ACK plus the bolt-on SACK option, and — being inside the encrypted payload — immune to the middleboxes that strip or rewrite TCP options.

Third, and most important structurally, all of this lives **in user-space library code that ships with the application**. When Google, Meta, or Cloudflare wants to deploy a new congestion controller or a loss-recovery tweak, they update their QUIC library and roll it out on their normal release cadence — days to weeks — to both ends they control, rather than waiting for kernel upgrades across the fleet and the Internet. Congestion control becomes a product feature you iterate, not a kernel constant you inherit. This is the deployment-velocity argument from earlier, made concrete.

Encrypted headers and the wire image

QUIC protects almost the entire packet, not just the payload. It defines a **long header** (used during the handshake, carrying the version, and the source and destination Connection IDs, all necessarily visible because a router or load balancer needs them to route Initial packets) and a **short header** (used after the handshake for 1-RTT data, exposing essentially only the destination Connection ID and a few bits). Even the packet numbers are encrypted, via **header protection** derived from the handshake keys. The **only** fields a passive observer can reliably read are the ones QUIC deliberately leaves in the clear because the network genuinely needs them: the Connection ID (so stateless load balancers can route packets of one connection to the same backend) and, optionally, a single **spin bit** that endpoints may toggle once per RTT so operators can passively estimate round-trip time. That spin bit is worth noticing: QUIC's designers gave the network *exactly one* deliberately exposed signal precisely so that operators would not have an excuse to inspect anything else, and so that whatever the network learns to depend on is a field the

protocol chose to expose and can reason about. The governing principle is stated in RFC 9000's companion documents as protecting the "wire image": minimize what is observable so that only what is intended to be extensible remains extensible, and ossification has nothing to grip.

QUIC versus TCP+TLS: an honest comparison

QUIC is not free lunch, and a senior engineer should be able to argue both sides.

Dimension	TCP + TLS 1.3	QUIC
Where implemented	Kernel (TCP) + userspace (TLS)	Entirely userspace, over UDP
Handshake to first byte	2 RTT (1 TCP + 1 TLS), 0-RTT possible via TFO (fragile)	1 RTT, 0-RTT on resumption
Multiplexing HOL blocking	Cross-stream: one loss stalls all H2 streams	Per-stream: loss contained to its stream
Survives IP/port change	No — four-tuple identity	Yes — Connection ID + migration
Header/metadata exposure	Seq, flags, options, window in cleartext	Almost all encrypted; CID + optional spin bit exposed
Congestion control evolution	Kernel cadence (years)	App release cadence (weeks)
Middlebox compatibility	Universally forwarded; TFO/new options ossified	UDP sometimes blocked/deprioritized
CPU cost per byte	Low; hardware offload (TSO/GRO, kTLS, NIC crypto) mature	Higher; per-packet userspace work, offload still maturing
Ecosystem tooling	Decades of <code>tcpdump</code> , <code>ss</code> , kernel counters	Newer; needs keylog to decrypt, less kernel visibility

Two of QUIC's costs deserve emphasis because they are the ones that bite in production.

UDP is sometimes blocked or deprioritized. Because UDP has historically carried mostly DNS and attack traffic, some enterprise firewalls and captive networks block or heavily rate-limit UDP on port 443, and some paths simply give UDP worse queuing than TCP. Real QUIC deployments therefore treat it as an *optimization with a fallback*: HTTP/3 is advertised via the HTTP `Alt-Svc` header (or the HTTPS DNS record), the browser tries QUIC, and if UDP fails or is too slow it falls back to HTTP/2 over TCP. You get QUIC's benefits where the path allows and lose nothing where it does not — but you now maintain and test *both* stacks.

A user-space transport costs CPU. In the kernel, TCP benefits from decades of offload: segmentation offload (TSO/GRO) lets the NIC and kernel amortize per-packet work across large buffers, and kTLS pushes record encryption toward hardware. QUIC, doing per-packet framing, encryption (including header protection), and ACK processing in user space, historically spent noticeably more CPU per byte than TCP+TLS. That gap is closing — UDP generic segmentation offload (GSO) and generic receive offload let QUIC stacks batch datagrams, and NIC-level QUIC offload is emerging — but at very high throughput on a busy server the CPU difference is a real line item, which is one reason large operators run QUIC at the edge (where per-connection latency wins dominate) while intra-datacenter east-west traffic often stays on TCP or plain HTTP/2, where RTTs are already sub-millisecond and the migration/0-RTT benefits do not apply.

QUIC's default deployment surface is **HTTP/3** (RFC 9114), which maps HTTP semantics onto QUIC streams — one bidirectional stream per request/response, unidirectional streams for control and QPACK — and is the subject of Chapter 7 (HTTP/1.1, HTTP/2, and HTTP/3). But QUIC is a general transport: it also carries DNS-over-QUIC (RFC 9250), it gains an unreliable-datagram service via the DATAGRAM extension (RFC 9221) — on which the MASQUE proxying protocols (CONNECT-UDP, RFC 9298) tunnel other traffic — and it is increasingly a substrate people reach for whenever they want "TCP but evolvable and encrypted." The DATAGRAM extension is a nice closing of the circle: QUIC, itself built on UDP to escape TCP, hands the application back an *unreliable* datagram service — UDP semantics — but now inside an authenticated, congestion-controlled, migratable connection.

The distributed-systems lens

Zoom out from the packet to the fleet, and QUIC's design choices line up with the structural concerns of large-scale backend systems in ways that are worth naming explicitly.

Tail latency at the edge and on mobile. For a global service fronted by a CDN (Chapter 10, Proxies, Reverse Proxies, Service Mesh, and CDNs), the dominant latency term for a client far from an origin is round trips, and the dominant *pain* is the tail — the p99 user on a bad mobile path. QUIC attacks exactly the tail. 0-RTT resumption removes a full RTT from every repeat connection, which for a returning mobile user on a 200 ms path is a visible chunk of page load. Connection migration removes the catastrophic re-handshake spike that a network transition imposes on TCP, converting a multi-hundred-millisecond stall into a seamless continuation. Per-stream reliability caps how bad a single loss can make things: on a 2% loss path with a dozen multiplexed streams, HTTP/2-over-TCP degrades sharply because every loss stalls every stream, while HTTP/3 degrades gracefully because loss is contained. These are not average-case wins — they are tail-case wins, and the tail is what SLOs and user perception are made of. This is why CDNs turned QUIC on first: they own the edge where far-from-origin, high-loss, mobile clients live, and that is where the design pays.

Per-stream reliability as the fix for a self-inflicted wound. It is worth being precise about what HTTP/3 buys, because it is easy to overclaim. HTTP/1.1 had application-layer HOL blocking (one request per connection at a time). HTTP/2 fixed that by multiplexing over one TCP connection — and thereby created a *new* transport-layer HOL blocking, because it concentrated many streams onto one ordered byte stream. HTTP/3 over QUIC is the move that finally resolves the tension HTTP/2 introduced: multiplex like H2, but over a transport that understands streams so a loss does not couple them. If you remember one causal chain from this chapter, make it that one — it explains why the industry did not simply stop at HTTP/2.

User-space transport is the same idea as kernel bypass. Volume 2, Chapter 10 (The Linux Network Stack, and kernel bypass with DPDK/AF_XDP) covered moving packet processing *out of the kernel* to escape its overhead and its release cadence for the highest-performance datapaths. QUIC is that same instinct applied not to raw throughput but

to *protocol evolvability*: put the transport where you can change it. The two are cousins. Kernel bypass moves the datapath to user space for speed; QUIC moves the transport to user space for agility. Both accept that the kernel, for all its virtues, is a shared, slow-moving resource, and both conclude that when you need to own the pace of change, you take the function into a component you deploy yourself. The general principle — **push a function to the layer whose deploy cadence you control** — recurs across the whole curriculum, from sidecars to feature flags to, here, the transport itself.

Evolution moves onto your release train — with its costs. Once congestion control and loss recovery are library code, improving them is a software-deploy problem, not an Internet-upgrade problem. That is a genuine and large win: Google could roll BBR experiments across its QUIC traffic and measure them in production in a way no one can do to kernel TCP at Internet scale. But there is a flip side the lens must include. You are now responsible for a transport you used to get from the OS: its correctness, its security patches, its interop testing against every other implementation, its CPU budget. And because QUIC needs a TCP fallback for the paths that block UDP, you operate *two* transports, not one, and must keep both correct and observant. The tooling is younger, too — you cannot read a QUIC connection with `tcpdump` the way you read TCP, because it is encrypted; debugging requires the endpoint to export TLS keying material (an `SSLKEYLOGFILE`) and tools like `qlog/qvis` that consume QUIC's structured logging. Observability, covered in Chapter 12 (Debugging and Observing Networks), is harder for an encrypted user-space transport than for cleartext kernel TCP, and that is part of the cost of the agility.

The UDP citizenship lesson generalizes. The very first obligation UDP handed us — implement congestion control or be a menace — is not a QUIC-specific footnote; it is the rule for any service that speaks a custom protocol over UDP inside your own infrastructure. Teams that build bespoke UDP-based telemetry firehoses, replication streams, or RPC layers routinely rediscover congestion collapse the hard way when the firehose and the production traffic share a link. If you find yourself reaching past TCP and past QUIC to raw UDP for an internal system, the question to answer before you ship is not "how do I retransmit?" but "how do I back off when the path is congested?" — and the honest answer is usually "reuse QUIC, or a library that already implements RFC 9002, rather than hand-roll it."

Key takeaways

- **UDP adds only ports and an optional checksum to IP.** No handshake, no reliability, no ordering, no flow control, no congestion control. That minimalism is the feature: you choose UDP to supply exactly the delivery semantics your workload needs — DNS, real-time media, gaming, tunnels, and custom protocols.
- **Choosing UDP means inheriting obligations.** Keep datagrams within the path MTU to avoid the IP-fragmentation trap; implement congestion control (RFC 8085 requires it) so you are not a free rider that causes collapse; and design against reflection/amplification, which spoofed source addresses make possible whenever a small request elicits a large response.
- **QUIC exists because TCP became inevolable**, on four fronts at once: cross-stream head-of-line blocking, stacked handshake latency, middlebox ossification of its cleartext wire image, and kernel-cadence deployment. The structural fix was to move the transport onto UDP, into user space, behind encryption.
- **QUIC integrates TLS 1.3 into the transport handshake**, achieving connection setup plus key exchange in one RTT, and 0-RTT on resumption — usable only for idempotent requests because 0-RTT data is replayable.
- **QUIC's streams give per-stream reliability and ordering**, so a lost packet stalls only its own stream. This is the core reason HTTP/3 over QUIC removes the transport HOL blocking that HTTP/2 over TCP suffers.
- **Connection IDs decouple a connection from its IP/port four-tuple**, enabling connection migration so QUIC connections survive Wi-Fi-to-cellular handoffs and NAT rebinding — a decisive mobile advantage — with path validation guarding against misuse.
- **QUIC's loss recovery and congestion control live in user space** and use monotonic packet numbers to kill retransmission ambiguity, so they evolve on a software-release cadence and can be iterated in production.
- **QUIC's costs are real:** UDP is sometimes blocked (so you keep a TCP/HTTP-2 fallback), a user-space transport spends more CPU per byte (offload is still maturing), and its encrypted wire image makes it harder to observe. Standardized as RFC 9000/9001/9002/8999 in

May 2021, it is the transport for HTTP/3 (RFC 9114) and increasingly a general-purpose evolvable transport.

Further reading

- RFC 768, *User Datagram Protocol* (1980) — three pages; the entire UDP contract.
- RFC 8085, *UDP Usage Guidelines* (BCP 145) — the authoritative statement of a UDP application's obligations, especially congestion control and datagram sizing.
- RFC 9000, *QUIC: A UDP-Based Multiplexed and Secure Transport* (May 2021) — the core transport specification; streams, connection IDs, flow control, migration, frames.
- RFC 9001, *Using TLS to Secure QUIC* — how the TLS 1.3 handshake is embedded and how QUIC packets are protected, including the packet-number spaces and 0-RTT.
- RFC 9002, *QUIC Loss Detection and Congestion Control* — the recommended loss recovery and congestion control, and the reasoning behind monotonic packet numbers.
- RFC 8999, *Version-Independent Properties of QUIC* — the small invariant set that must hold across QUIC versions, which is what the network is permitted to depend on.
- RFC 9114, *HTTP/3*, and RFC 9204, *QPACK: Field Compression for HTTP/3* — how HTTP maps onto QUIC streams and how header compression avoids re-introducing HOL blocking.
- RFC 9221, *An Unreliable Datagram Extension to QUIC*, and RFC 9250, *DNS over Dedicated QUIC Connections* — two illustrative non-HTTP uses of QUIC.
- RFC 8899, *Packetization Layer Path MTU Discovery for Datagram Transports* — the safe path-MTU probing QUIC (and other UDP protocols) uses instead of relying on ICMP.
- RFC 2827 / BCP 38, *Network Ingress Filtering* — source-address validation, the network-side defense against the spoofing that enables UDP amplification.
- Langley et al., *The QUIC Transport Protocol: Design and Internet-Scale Deployment*, SIGCOMM 2017 — Google's account of gQUIC's

motivation and its measured production results; the empirical case that became IETF QUIC.

- Robin Marx's *qlog* schema (an IETF standardization effort, `draft-ietf-quic-qlog-*`) and the *qvis* visualization toolset — the structured logging and visualization used to actually debug QUIC in practice; see also Chapter 12 (Debugging and Observing Networks) in this volume.
- Cloudflare's `quiche`, Google's Chromium QUIC, Meta's `mvfst`, and Microsoft's `msquic` — four production, open-source QUIC implementations whose source and design docs are the best way to see the RFCs realized in code.

Chapter 5 — DNS in Depth

What this chapter covers. Every request in your fleet begins with a name, not an address. Before a client opens the TCP connection Chapter 3 described, before IP routing (Chapter 2) can steer a packet toward a destination, before QUIC (Chapter 4) can complete a 0-RTT handshake, something must turn `api.internal.example.com` into an IP address — and that something is the Domain Name System. DNS is easy to dismiss as plumbing: a lookup, a cache, a config file you edit once. It is not plumbing. It is a globally distributed, hierarchical, eventually-consistent database whose consistency model, caching behavior, and failure modes directly determine how fast your service can fail over between regions, how a Kubernetes pod finds another service, how a request from Frankfurt reaches the nearest healthy datacenter, and — when it breaks — why your entire product is down even though every server is healthy. Backend engineers who treat DNS as opaque get burned by it repeatedly: by a client that cached a dead IP for hours, by a `ndots` setting that quadrupled their lookup latency, by a TTL that made a failover take twenty minutes instead of thirty seconds, by a single resolver outage that took the whole company offline. This chapter treats DNS as what it actually is — distributed systems infrastructure — and insists on mechanism throughout: not "DNS resolves names," but which resolver asks which server, in what order, over what transport, protected by what signature, cached for how long, and what precisely goes wrong at fleet scale.

Learning goals — after this chapter you should be able to:

- Describe the DNS namespace as a delegated tree (root, TLD, authoritative zones) and explain what delegation is at the record level (NS records and glue), not just as a concept.
- Trace a full resolution — stub resolver to recursive resolver to root to TLD to authoritative — and distinguish recursive from iterative queries and who does which.
- Choose the right record type for a job (A, AAAA, CNAME, MX, TXT, NS, SOA, SRV, PTR, CAA), and explain the CNAME-at-apex problem and the ALIAS/ANAME workaround managed providers use.
- Reason quantitatively about TTLs and caching — positive and negative (NXDOMAIN) caching — and the fundamental trade-off between failover agility and cache efficiency.
- Explain the DNS transport (UDP/53, TCP fallback, EDNS0, the 512-byte legacy limit) and why message size matters for DNSSEC and large responses.
- Explain cache poisoning and the Kaminsky attack, how DNSSEC's chain of trust provides origin authentication, and how DoT and DoH provide transport privacy — and the limits of each.
- Apply DNS to real backend problems: Kubernetes/CoreDNS service discovery and the `ndots` latency trap, DNS-based load balancing and health-checked failover, geo/latency routing via managed DNS, anycast DNS, and the client-side DNS-caching pitfalls (the JVM TTL gotcha).

What DNS actually is: a delegated, hierarchical, cached database

Strip away the folklore and DNS is a distributed key-value database. The keys are `(name, class, type)` tuples — `www.example.com`, class `IN` (Internet), type `A` — and the values are resource records. What makes it interesting is not the data model, which is trivial, but the way authority over the keyspace is partitioned and the way answers are cached. No single server holds the whole database; no single organization operates it. Instead the namespace is a tree, and authority over each subtree is *delegated* downward from its parent. The operator of `.com` does not know or store the address of your web server. They store a pointer — "for anything under `example.com`, go ask these name servers" — and nothing

more. This delegation is the entire architecture. It is why DNS scales to hundreds of millions of domains without a central bottleneck, why ICANN can run the root without knowing anything about your infrastructure, and why you can change your web server's IP without asking anyone's permission: you control your zone, and the delegation to it is stable.

The tree is read right-to-left. A fully qualified domain name like `api.eu.example.com.` — note the trailing dot, which denotes the root and which is usually implicit — is a path from the root through a series of labels: the root (the empty label after the final dot), the top-level domain `com`, the second-level domain `example`, and so on down to the leaf `api`. Each label is at most 63 octets; the whole name is at most 255 octets. Case is preserved but comparisons are case-insensitive for ASCII.

```
graph TD
    ROOT["Root zone (.)"]
    COM["com (TLD)"]
    ORG["org (TLD)"]
    UK["uk (ccTLD)"]
    EXAMPLE["example.com (your zone)"]
    IANA["iana.org"]
    COUK["co.uk"]
    WWW["www.example.com (A record)"]
    EU["eu.example.com (delegated sub-zone)"]
    API["api.eu.example.com"]

    ROOT -->|"NS delegation"| COM
    ROOT -->|"NS delegation"| ORG
    ROOT -->|"NS delegation"| UK
    COM -->|"NS delegation"| EXAMPLE
    ORG --> IANA
    UK --> COUK
    EXAMPLE --> WWW
    EXAMPLE -->|"NS delegation"| EU
    EU --> API
```

The unit of administrative control is the **zone**, not the domain. A zone is a contiguous portion of the tree under single authority, bounded below by delegations to child zones. The `example.com` zone contains `www.example.com` and `mail.example.com` directly, but if you delegate `eu.example.com` to a different set of name servers — perhaps run by a regional team — then `eu.example.com` becomes a separate zone with its own authority, and the parent zone holds only the delegation. This is a real operational lever: teams can own their subtrees independently, and a compromise or misconfiguration in one zone is contained.

Authority is not a single server. Every zone must have at least two **authoritative name servers** (in practice more, on diverse networks) that hold the definitive copy of the zone's records. One is the primary (historically "master"), which holds the editable copy; the others are secondaries ("slaves") that pull the zone via zone transfer. To the outside world they are equal — a resolver may ask any of them and must get the same authoritative answer. This replication is what makes DNS survive the loss of individual servers, and it is why a zone's NS set is deliberately spread across providers and network paths.

The resolution flow: who asks whom, and in what order

When your application calls `getaddrinfo("www.example.com", ...)`, an enormous amount of machinery engages, and understanding its division of labor is what separates engineers who can debug DNS from those who can only restart things. There are two kinds of resolver and two kinds of query, and confusing them is the most common conceptual error.

The **stub resolver** is the thin client library inside your process (glibc's resolver, `systemd-resolved`, Go's built-in resolver, the JVM's `InetAddress`). It does almost nothing itself. It reads `/etc/resolv.conf` (or its platform equivalent), finds the IP of a configured **recursive resolver**, and sends it a single **recursive query**: "give me the final answer for `www.example.com` A — do the work, I'll wait." The stub is not equipped to walk the tree; it delegates the whole job.

The **recursive resolver** (your ISP's resolver, `8.8.8.8`, `1.1.1.1`, your corporate resolver, or CoreDNS in a cluster) does the actual walking. If the answer is not in its cache, it performs a series of **iterative queries** against the authoritative hierarchy. "Iterative" means each authoritative server it asks does *not* do the work on its behalf; it returns either the answer or a referral — "I don't have that, but here are the name servers for the next level down, go ask them." The recursive resolver follows the referrals itself, starting at the root.

```
sequenceDiagram
    participant App as "App / stub resolver"
    participant R as "Recursive resolver (8.8.8.8)"
    participant Root as "Root server"
    participant TLD as "com TLD server"
    participant Auth as "example.com auth server"

    App->>R: "Recursive query: www.example.com A?"
    Note over R: "Cache miss"
    R->>Root: "Iterative: www.example.com A?"
    Root-->>R: "Referral: ask com NS (glue + addrs)"
    R->>TLD: "Iterative: www.example.com A?"
    TLD-->>R: "Referral: ask ns1.example.com (glue)"
    R->>Auth: "Iterative: www.example.com A?"
    Auth-->>R: "Authoritative answer: 93.184.216.34, TTL 300"
    R-->>App: "Answer: 93.184.216.34 (cached for 300s)"
    Note over R: "Caches answer + NS records along the way"
```

Two subtleties matter for real systems. First, the recursive resolver rarely starts from the root in practice, because it caches aggressively at every level. Having resolved anything under `.com` recently, it has the `.com` NS records cached and skips the root. Having resolved anything under `example.com`, it has your authoritative NS records cached and skips both root and TLD. In a busy resolver, the vast majority of queries are answered from cache with zero upstream traffic; the full five-hop walk is the cold-cache worst case. The root is protected by this caching plus anycast (more below) — it handles a firehose of queries precisely because most resolvers ask it rarely.

Second, **glue records**. When `.com` refers you to `ns1.example.com` as the authoritative server for `example.com`, you have a chicken-and-egg problem: to ask `ns1.example.com` you need its IP, but `ns1.example.com` lives inside the very zone you're trying to resolve. The `.com` zone solves this by including the A/AAAA records for `ns1.example.com` directly in the referral as **glue** — non-authoritative address hints supplied by the parent so resolution can proceed. This is why, when you register a domain whose name servers are within the domain itself, your registrar makes you register the name server host records ("host objects" / glue) separately.

The 13 named root servers (`a.root-servers.net` through `m.root-servers.net`) are not 13 machines. Each is an anycast constellation of hundreds of instances worldwide, all advertising the same IP, so a resolver's query to `198.41.0.4` is routed by BGP (Chapter 2) to the topologically nearest instance. We return to anycast at the end of the

chapter; for now, note that the root is a distributed system in exactly the sense this book cares about.

Record types: the vocabulary of the database

The value side of the DNS database is a resource record, and choosing the right type — and understanding a few sharp edges — is bread-and-butter backend work.

A and **AAAA** are the base case: **A** maps a name to a 32-bit IPv4 address, **AAAA** to a 128-bit IPv6 address. A single name may have several of each; returning multiple **A** records is the oldest and crudest form of load balancing (see below).

CNAME (canonical name) is an alias: it says "this name is really that name; go resolve *that* one and use its records." `www.example.com CNAME example.com` means a lookup for `www` follows the pointer to `example.com` and returns its **A** record. **CNAME**s are convenient — you point `www`, `app`, and `cdn` all at one canonical target and change the target's address in one place — but they carry a rule that trips up nearly everyone: **a name that has a CNAME may have no other records of any type**. The **CNAME** must be alone. This is not arbitrary; the **CNAME** means "everything about this name is defined elsewhere," so you cannot also attach an **MX** or **TXT** to it.

The consequence is the **CNAME-at-apex problem**, one of the most common real-world DNS snags. The apex (or "zone apex," or "naked domain") is the bare `example.com` with no subdomain. The apex *must* carry the zone's **SOA** and **NS** records — that is what makes it a zone. Because those records exist, the apex cannot also be a **CNAME**. So you cannot write `example.com CNAME my-load-balancer.aws.example.` — the standard forbids it. But you frequently *want* exactly that: your load balancer or CDN gives you a hostname, not a stable IP, and you want the naked domain to point at it. Managed DNS providers solve this with a non-standard record type — Route 53 calls it an **ALIAS**, others call it **ANAME** or "CNAME flattening" (Cloudflare). Mechanically, the provider's *authoritative server* does the **CNAME** resolution internally at query time and returns the final **A/AAAA** records to the client, so the wire response looks like a normal **A** record at the apex and the **CNAME-alone** rule is never violated on the wire. It works, but understand it is a provider feature, not a DNS record you can host on your own BIND server, and the

flattening is done by the authoritative server, adding a resolution step and coupling you to the provider's health of the target.

MX (mail exchanger) records list, with priority values, the hosts that accept mail for a domain; lower priority is preferred. **NS** records name the authoritative servers for a zone and appear both at the zone apex (authoritative) and in the parent as the delegation.

SOA (start of authority) is the one-per-zone record carrying the zone's administrative parameters: the primary name server, the responsible party's email (with `@` written as a dot), a **serial number** that secondaries compare to decide whether to re-transfer, and the timers — refresh, retry, expire, and the **negative-caching TTL** (the "minimum" field, repurposed by RFC 2308) that governs how long NXDOMAIN answers are cached.

TXT records hold arbitrary text and have become the universal extension point: SPF, DKIM, and DMARC email-authentication policies, ACME/Let's Encrypt domain-control challenges (`_acme-challenge.example.com`), and countless "prove you own this domain" verifications all ride on TXT.

SRV (service) records were DNS's first real answer to service discovery, and they matter because Kubernetes and Consul use them. An SRV record for `_service._proto.name` — e.g. `_sip._tcp.example.com` — returns priority, weight, **port**, and target host. Crucially SRV carries the port, so a client can discover both *where* and *on which port* a service runs, which plain A records cannot express.

PTR records provide **reverse DNS**: mapping an IP back to a name. They live in the special `in-addr.arpa` (IPv4) and `ip6.arpa` (IPv6) trees, with the address written backward — the PTR for `93.184.216.34` is `34.216.184.93.in-addr.arpa`. Reverse DNS matters operationally: mail servers reject senders whose IP has no matching forward-and-reverse (FCrDNS) record, and logs, traceroutes, and audit systems rely on it. Note the asymmetry — forward and reverse are separate zones, often under different authority (your cloud provider usually controls the reverse zone for their address space), so they can and do disagree.

CAA (certification authority authorization) records let a domain declare which certificate authorities are permitted to issue certs for it. `example.com CAA 0 issue "letsencrypt.org"` tells every compliant CA except Let's Encrypt to refuse issuance. CAA checking has been mandatory for public CAs since 2017, and it is a cheap, high-leverage

control against mis-issuance — worth setting for any domain you care about.

```
; A / AAAA
www.example.com.      300  IN  A       93.184.216.34
www.example.com.      300  IN  AAAA
2606:2800:220:1:248:1893:25c8:1946

; CNAME (alias) – note: cannot coexist with other records at this name
cdn.example.com.      300  IN  CNAME   d111abcdef8.cloudfront.net.

; MX (mail routing, lower priority preferred)
example.com.          3600 IN  MX      10 mail1.example.com.
example.com.          3600 IN  MX      20 mail2.example.com.

; NS + SOA at the apex
example.com.          86400 IN  NS      ns1.example.com.
example.com.          86400 IN  NS      ns2.example.com.
example.com.          3600 IN  SOA     ns1.example.com. hostmaster.ex
ample.com. (
                                2026080201 ; serial (YYYYMMDD
nn)
                                7200      ; refresh
                                3600      ; retry
                                1209600   ; expire (2 weeks)
                                300 )     ; negative-cache T
TL

; SRV (service discovery: priority weight port target)
_grpc._tcp.example.com. 60  IN  SRV      10 60 50051 apil.example.com.

; TXT (SPF, ACME challenge, verifications)
example.com.          3600 IN  TXT      "v=spf1
include:_spf.google.com ~all"

; CAA (restrict certificate issuance)
example.com.          3600 IN  CAA      0 issue "letsencrypt.org"

; PTR (reverse) lives in a separate zone
34.216.184.93.in-addr.arpa. 3600 IN  PTR  www.example.com.
```

Caching and TTL: the knob that trades agility for scale

DNS works at planetary scale for one reason: caching. Every record carries a **TTL** (time to live) in seconds, set by the zone's authoritative operator — that is, by *you* for your own records. When a recursive resolver receives an answer, it may serve that same answer from its

cache to any client for up to TTL seconds without asking again. Stub resolvers and even applications cache too. The TTL is the record owner's instruction to the entire world about how long a stale answer is acceptable, and choosing it is a genuine engineering decision, not a default to ignore.

The trade-off is direct and it is the single most important operational fact about DNS. A **low TTL** — say 60 seconds — means changes propagate fast: when you move traffic off a failed load balancer by rewriting the A record, the world converges within a minute. But it also means resolvers must re-query roughly every minute, multiplying load on your authoritative servers and adding a resolution round-trip to a larger fraction of client requests, hurting tail latency. A **high TTL** — say 86400, a day — means near-perfect cache efficiency and minimal authoritative load, but a change you make is invisible to already-cached resolvers for up to a day, and there is *no way to recall it*. This is the crux: **DNS has no cache invalidation**. You cannot push an update; you can only wait for TTLs to expire. A record you published with an 86400 TTL is a commitment to the network that you might serve that value for a day, and if that value is a dead server, that is a day of partial outage for the clients unlucky enough to hold the cache.

The practical pattern is **TTL as a failover budget**. Records that front infrastructure you may need to fail over fast — apex records, load-balancer targets, anything health-checked — get low TTLs (30–60 seconds is common for actively managed endpoints). Records that never change — MX for a stable mail provider, TXT verification records — get high TTLs to save queries. And before a *planned* migration, you lower the TTL well in advance (at least one old-TTL period ahead, so the old high TTL has drained from caches), make the change on a now-short TTL, then raise the TTL back afterward. Skipping the pre-lowering is a classic mistake: you drop the TTL to 60 and cut over immediately, but resolvers that cached the record an hour ago under the old 3600 TTL will keep serving the old value for up to another hour.

Negative caching is the counterpart most engineers forget. When a resolver asks for a name that does not exist, the authoritative server returns **NXDOMAIN**, and — per RFC 2308 — that *negative* answer is itself cached, for a duration governed by the smaller of the SOA's TTL and its minimum field. This is why creating a brand-new record sometimes appears not to work: if anything queried the name before it

existed, resolvers cached the NXDOMAIN, and the new record is invisible until that negative cache expires. Set your SOA negative-cache TTL deliberately (300 seconds is a sane default); a large value there turns "I just added the DNS record" into a half-hour of confusing 404s.

Reality is messier than the TTL contract suggests. Some resolvers clamp very low TTLs up to a floor to protect themselves; some clients cache far longer than any TTL (the JVM historically cached forever — see the failover section); and misbehaving stub resolvers and application code routinely ignore TTLs entirely. So while you *set* the TTL as your intent, you must design failover assuming a meaningful population of clients will hold a stale answer well past it. TTL is a floor on convergence time, never a ceiling.

The wire: UDP, TCP, EDNS0, and the 512-byte ghost

DNS runs on port 53, and its transport history explains a surprising amount of production behavior. The default transport is **UDP** (Chapter 4): a query is one datagram, the answer is one datagram, no handshake, no connection state — ideal for a request that is small and idempotent and happens billions of times per second. The stateless, no-handshake nature is exactly why UDP was chosen; the cost is that DNS must handle loss, reordering, and — critically — spoofing itself, which we return to under security.

The original DNS spec capped a UDP message at **512 bytes**. Anything larger set the **truncated (TC) bit** in the response, and the resolver was expected to retry the whole query over **TCP**, which has no such limit (a 2-byte length prefix allows up to 65535 bytes). TCP was long treated as the exception — used for zone transfers (AXFR/IXFR, which move an entire zone from primary to secondary and are obviously too big for a datagram) and for the rare oversized answer. The 512-byte limit was not about UDP's capabilities; it was a conservative floor chosen so that any IP path could carry a DNS datagram without fragmentation.

That floor became a problem the moment responses grew — many A records, SRV sets, and above all **DNSSEC signatures**, which are large. The fix is **EDNS0** (Extension Mechanisms for DNS, RFC 6891): a pseudo-record (OPT) in the query's additional section by which the client advertises a larger UDP buffer it is willing to receive — commonly 1232 or 4096 bytes — plus extra flags and option codes (EDNS0 is also how

DNS Cookies and client-subnet hints ride along). With EDNS0, a DNSSEC-signed answer of 1.5 KB can travel in one UDP datagram instead of forcing a TCP retry on every query. The 1232 value is a modern recommendation chosen to stay under common IPv6 MTUs and avoid IP fragmentation, because fragmented UDP responses are both a reliability hazard and a security one (fragments are easier to spoof and are dropped by many firewalls).

The operational takeaways: (1) your firewalls and load balancers **must permit DNS over TCP on 53**, not just UDP — blocking TCP/53 breaks large answers and DNSSEC in ways that are maddening to diagnose because small queries still work; (2) EDNS0 must be allowed through; and (3) the `+dnssec/+bufsize` behavior you see in `dig` is EDNS0 in action.

```
# Trace the full delegation walk, top-down, showing referrals at each
level
$ dig +trace www.example.com

# See exactly what a specific authoritative server returns (bypass
caches)
$ dig @ns1.example.com www.example.com A +norecurse

# Force TCP; confirm TCP/53 is actually reachable through your firewall
$ dig +tcp www.example.com

# Ask for DNSSEC records with a large EDNS0 buffer; shows the OPT
pseudo-record
$ dig +dnssec +bufsize=1232 example.com A

# Query a specific public resolver and show the TTL counting down on
repeat
$ dig @1.1.1.1 www.example.com +noall +answer
www.example.com. 242 IN A 93.184.216.34
```

DNS security: spoofing, DNSSEC, and encrypted transport

DNS was designed in a trusting era, and its default properties are alarming when you say them plainly: queries and answers travel **unencrypted** and **unauthenticated** over **UDP**, a protocol with no handshake. A recursive resolver accepts an answer as genuine if it arrives on the right UDP port, is addressed correctly, and matches the query's 16-bit **transaction ID** and the question. That is the entire

authenticity check. Anyone who can guess or observe those fields and reply before the real authoritative server does can inject a forged answer — **cache poisoning** — and the resolver will cache the lie and serve it to every downstream client for the TTL the attacker chooses.

For years this was considered hard because the attacker had to win a race and guess a 16-bit ID — a 1-in-65536 shot per attempt. In 2008 **Dan Kaminsky** demonstrated why that comfort was false. His insight was that an attacker need not race a single query: by querying the target resolver for many *nonexistent* names under a domain (`aaaa.example.com`, `aaab.example.com`, ...), the attacker gets many fresh races in quick succession, and crucially the forged reply can carry, in its authority/additional section, a poisoned NS or glue record redirecting the *entire domain* — not just one hostname — to attacker-controlled servers. With only 16 bits of transaction ID and a predictable source port, poisoning a domain went from "years" to "seconds." The emergency mitigation, deployed across the internet in 2008, was **source-port randomization**: the resolver picks a random UDP source port per query, so the attacker must now guess ~16 bits of ID plus ~16 bits of port, raising the work by orders of magnitude. That is a mitigation, not a cure — it makes poisoning expensive, not impossible, and it does nothing against an on-path attacker who can see the real values.

The real cure for authenticity is **DNSSEC** (DNS Security Extensions). DNSSEC does *not* encrypt anything and does not provide privacy; it provides **origin authentication and integrity** by having zone operators cryptographically sign their records. Each signed record set (RRset) gets an **RRSIG** (the signature), the zone publishes its public key as a **DNSKEY**, and — this is the part that makes it a real trust system rather than a pile of self-signed keys — the *parent* zone publishes a **DS** (delegation signer) record that is a hash of the child's key. So the `.com` zone signs a DS record attesting to `example.com`'s key; the root zone signs a DS record attesting to `.com`'s key; and the root's own key (the "trust anchor") is distributed out-of-band and baked into validating resolvers. A validating resolver can thus build an unbroken **chain of trust** from the root down to any signed record, verifying at each step that the parent vouches for the child's key and that the key signed the data.

```

graph TD
    ROOT["Root zone – trust anchor (KSK baked into resolvers)"]
    ROOTDS["Root signs DS for com"]
    COM["com zone DNSKEY"]
    COMDS["com signs DS for example.com"]
    EX["example.com DNSKEY"]
    RRSIG["example.com signs RRSIG over A record"]
    DATA["www.example.com A = 93.184.216.34 (validated)"]

    ROOT --> ROOTDS
    ROOTDS -->|"hash matches"| COM
    COM --> COMDS
    COMDS -->|"hash matches"| EX
    EX --> RRSIG
    RRSIG -->|"signature valid"| DATA

```

Two things about DNSSEC deserve engineering honesty. First, **adoption is partial and slow**. A large share of zones remain unsigned, and — worse — many recursive resolvers do not *validate* even when zones are signed, so an end user often gets no protection even for a signed domain unless their resolver enforces it. The reasons are operational: DNSSEC is fiddly to run (key rollovers, signing pipelines, the fact that a signing mistake causes a hard `SERVFAIL` outage rather than a soft degradation), the signatures inflate responses and lean on EDNS0/TCP, and there is a real denial-of-service surface. Second, DNSSEC authenticates the *data*, not the *channel* — it tells you the answer genuinely came from the zone owner and was not tampered with, but it does nothing to hide *which* names you are looking up from anyone watching the wire.

Privacy is the separate problem that **DoT** and **DoH** solve, and they solve only the stub-to-recursive hop. **DNS over TLS** (DoT, port 853) wraps the stub-resolver-to-recursive conversation in TLS: same DNS messages, encrypted channel, on a dedicated port that a network operator can see (and block) as DNS-shaped even without reading contents. **DNS over HTTPS** (DoH, port 443) tunnels DNS inside ordinary HTTPS requests, making the DNS traffic indistinguishable from web traffic and thus resistant to network-level blocking — which is exactly why it is both loved (user privacy, censorship resistance) and resented (it bypasses enterprise DNS-based security controls and split-horizon resolvers, and moves visibility to the DoH provider). Note the scope carefully: DoT/DoH encrypt only the client-to-recursive-resolver leg. The recursive resolver's iterative queries out to the authoritative hierarchy are historically still cleartext UDP; and neither DoT nor DoH *authenticates the data* — that is

still DNSSEC's job. The two are complementary: DNSSEC says "this answer is genuine," DoT/DoH say "no one on your network saw you ask." A backend engineer running an internal resolver should think about both axes independently.

A final security note that ties into cloud security elsewhere in this curriculum: the cloud metadata endpoint `169.254.169.254` (link-local, source of many SSRF-to-credential-theft incidents) is *not* a DNS matter — it is a fixed link-local IP, no name involved. But DNS has its own SSRF-adjacent trap: **DNS rebinding**. An attacker controls a domain, answers the victim browser's first lookup with a public IP (passing same-origin and firewall checks), then, after a short TTL, answers the *next* lookup for the same name with an *internal* IP such as `169.254.169.254` or a `10.x` service. The victim's code, having "resolved the same hostname," now sends requests to an internal target while believing it is talking to the original external host — the TTL-driven change of a name's meaning is the whole exploit. Defenses are DNS-aware: forbid resolving internal/RFC 1918/link-local addresses in outbound-fetch code, pin the resolved IP for the life of a request, and validate `Host` against an allowlist rather than trusting the name to keep meaning one thing.

Operational DNS: service discovery at fleet scale

Now the part that consumes most of a backend engineer's actual DNS time. In a modern fleet, DNS is not just how the outside world finds your website; it is the **service discovery** substrate that lets thousands of ephemeral service instances find each other. The genius, and the danger, is that it reuses the same protocol and the same caching semantics for a job — tracking fast-changing, short-lived endpoints — that the protocol was not originally designed for.

Kubernetes and CoreDNS

Inside a Kubernetes cluster, **CoreDNS** runs as the cluster DNS service and every pod is configured (via its `/etc/resolv.conf`, injected by the kubelet) to use it as the recursive resolver. When you create a Service named `payments` in namespace `checkout`, Kubernetes assigns it a stable virtual IP (the ClusterIP) and CoreDNS publishes an A record at `payments.checkout.svc.cluster.local`. Any pod resolving that name gets the ClusterIP, and kube-proxy (or an eBPF dataplane) load-balances connections across the current healthy pod endpoints. This is the whole

trick: pods come and go constantly, but the *name* and the ClusterIP are stable, so callers use a name and never track individual pod IPs.

Headless services are the important variant. A Service with `clusterIP: None` has no virtual IP; instead CoreDNS returns the A records of *all* the individual pod IPs behind it, and publishes per-pod names and SRV records. This is how stateful, membership-aware systems — Kafka, Cassandra, Elasticsearch, a Postgres cluster, anything where a client must address *specific* members rather than a random one — do discovery: they resolve the headless service and get the full, live set of peers with their ports via SRV. Here DNS is doing genuine service discovery, membership and all, and the low TTLs CoreDNS sets (a few seconds by default) are what make membership changes visible quickly.

```
graph LR
    POD["Client pod"]
    RESOLV["/etc/resolv.conf: nameserver = kube-dns ClusterIP, ndots:5, search checkout.svc.cluster.local svc.cluster.local cluster.local"]
    CORE["CoreDNS pods (Deployment)"]
    API["kube-apiserver (Services + EndpointSlices)"]
    SVC["ClusterIP Service payments"]
    EP["Pod endpoints via kube-proxy / eBPF"]

    POD --> RESOLV
    RESOLV -->|"query payments.checkout.svc.cluster.local"| CORE
    CORE -->|"watches"| API
    CORE -->|"A = ClusterIP"| POD
    POD -->|"connect to ClusterIP"| SVC
    SVC -->|"load-balanced"| EP
```

Now the trap that has burned nearly every Kubernetes shop: **ndots and the search-domain latency tax**. A pod's `/etc/resolv.conf` typically contains a `search` list of several suffixes (`checkout.svc.cluster.local`, `svc.cluster.local`, `cluster.local`, plus the node's) and the option `ndots:5`. The `ndots:N` setting means: if the name you are looking up contains *fewer than N dots*, treat it as unqualified and try it with each search suffix appended *first*, before trying it as an absolute name. The value `5` is high — it exists so that names like `payments.checkout.svc.cluster.local` (which has 4 dots) are still treated as relative and get the search suffixes tried. The catastrophic side effect: when your code resolves an *external* name like `api.stripe.com` (2 dots, fewer than 5), the stub resolver dutifully tries `api.stripe.com.checkout.svc.cluster.local`, then

`api.stripe.com.svc.cluster.local`, then `api.stripe.com.cluster.local`, then the node suffix — each a guaranteed NXDOMAIN round-trip to CoreDNS — *before* finally trying `api.stripe.com.` itself and succeeding. That is four wasted DNS round-trips (and with IPv4+IPv6 dual lookups, potentially eight) on the hot path of every external call, hammering CoreDNS and adding latency to everything.

The fixes are worth knowing because you will need them: append a **trailing dot** to make an external name **fully qualified** (`api.stripe.com.`), which tells the resolver to skip the search list entirely; or set a lower `ndots` (e.g. `ndots:1` or `2`) in the pod's `dnsConfig` for workloads that mostly call external names; or run a **node-local DNS cache** (NodeLocal DNSCache) so that even the wasted NXDOMAIN queries are answered locally at near-zero latency and CoreDNS is shielded from the load. Understanding *why* the extra queries happen — the interaction of `ndots` with the search list — is what lets you diagnose the "why is every external API call slow inside the cluster" ticket in minutes instead of days.

```
# Per-pod override: fewer search-suffix attempts for external-heavy
workloads
apiVersion: v1
kind: Pod
metadata:
  name: outbound-worker
spec:
  dnsConfig:
    options:
      - name: ndots
        value: "2"
  containers:
    - name: app
      image: registry.example.com/outbound-worker:1.4.2
```

Consul and DNS-based discovery outside Kubernetes

Consul offers the same idea for mixed VM/container fleets: it exposes a DNS interface (default on port 8600, usually fronted by the local resolver) where `web.service.consul` returns the healthy instances of the `web` service as A records, and `_web._tcp.service.consul` returns SRV records with ports. Consul's health checks gate what DNS returns — an instance failing its check drops out of the answer set — so DNS becomes a *health-aware* discovery layer. The pattern generalizes: DNS is a convenient

discovery API because *every* language already has a DNS client built in, which is precisely why service-mesh and orchestration systems keep choosing it over bespoke protocols despite its caching quirks.

DNS-based load balancing and failover

DNS is also the oldest and highest-in-the-stack place to spread and steer traffic, and the trade-offs are sharp.

The crudest form is **multiple A records / round-robin**: publish several A records for one name and let the resolver (and client) pick among them, rotating the order on each response. It requires nothing but DNS and spreads load roughly evenly across a handful of endpoints. Its limits are severe and you must respect them: DNS-level load balancing is **blind to server health and load** (a plain round-robin A set happily hands out the IP of a dead server until you edit the record and TTLs expire), the balancing is coarse (resolvers and clients cache and reuse one answer for many connections, and OS/JVM behavior around record ordering is inconsistent), and there is no session or weight control. Round-robin A records are fine for spreading load across a small, stable, healthy set; they are not a substitute for a real load balancer (Layer 4/7) and certainly not for automatic failover.

Managed DNS providers turn DNS into a genuine traffic-management layer by attaching **health checks** and **routing policies** to records. AWS **Route 53**, Google **Cloud DNS**, Azure DNS, NS1, Akamai, and others let you:

- **Health-checked failover**: the authoritative server probes your endpoints and stops returning the A record of an endpoint that fails health checks, returning a standby instead. Now DNS is failover — but its speed is bounded by TTL plus client cache behavior (below).
- **Weighted routing**: split traffic by configurable weights across endpoints — the mechanism behind gradual rollouts, blue/green, and canaries at the DNS layer.
- **Geolocation / geoproximity routing**: return different answers based on the resolver's (approximate) location — send European clients to the Frankfurt region, North Americans to Virginia.
- **Latency-based routing**: return the region with the lowest measured network latency to the resolver, which is not the same as geographically nearest.

```

graph TD
    C1["Client in Frankfurt"]
    C2["Client in Virginia"]
    MDNS["Managed authoritative DNS (Route 53 / Cloud DNS): geo +  
latency routing + health checks"]
    HC["Health checkers probe each region"]
    EU["eu-central-1 endpoint"]
    US["us-east-1 endpoint"]
    STANDBY["Failover standby"]

    C1 -->|"api.example.com A?"| MDNS
    C2 -->|"api.example.com A?"| MDNS
    HC -->|"healthy?"| EU
    HC -->|"healthy?"| US
    MDNS -->|"geo: EU -> Frankfurt IP"| EU
    MDNS -->|"geo: US -> Virginia IP"| US
    MDNS -->|"if region unhealthy"| STANDBY

```

A subtlety that undermines geo/latency routing: the authoritative server sees the **recursive resolver's** IP, not the end user's. A user in Paris using a resolver in London (or, worse, a big public resolver whose anycast node is elsewhere) gets routed as if they were where the resolver is. The partial fix is **EDNS Client Subnet** (ECS, RFC 7871), an EDNS0 option by which the recursive resolver forwards a truncated prefix of the client's IP to the authoritative server so it can route by the real client location. ECS trades a slice of user privacy for routing accuracy and is widely but not universally supported; it is why CDN geo-routing usually works despite the resolver-location problem, and it interacts with caching (answers must be cached per client-subnet, not per name).

The failover pitfall: clients that cache DNS forever

Here is the single most expensive DNS mistake in backend systems, and it is a client-side one. You set a 60-second TTL, you configure health-checked failover, you fail a region — and half your clients keep hammering the dead IP for *hours*. Why? Because the TTL is only an instruction, and some clients ignore it. The infamous case is the **JVM**: with a security manager installed, the default value of `networkaddress.cache.ttl` was historically **-1**, **meaning cache successful DNS lookups forever** — the process resolves a name once at startup and never looks again until it restarts. A Java service that resolved `db.internal.example.com` at boot will keep connecting to the original IP across any number of failovers, oblivious to your carefully

tuned 60-second TTL, until someone restarts the JVM. Newer JDKs default this to a bounded value (commonly 30 seconds) when no security manager is present, but you should never leave it to defaults across your fleet.

```
// Set explicitly on JVM startup; do NOT rely on defaults for failover-  
critical services.  
// -1 = cache forever (dangerous), 0 = never cache (hammers DNS), 30 =  
a sane bound.  
java.security.Security.setProperty("networkaddress.cache.ttl", "30");  
// Also bound the NEGATIVE cache, or a transient NXDOMAIN sticks for  
10s by default:  
java.security.Security.setProperty("networkaddress.cache.negative.ttl",  
"5");
```

The JVM is the famous offender but not the only one. Many HTTP client libraries and connection pools resolve a hostname once when a connection or pool is created and then reuse that connection indefinitely, so even a language whose resolver honors TTLs will keep using a dead IP as long as the pooled connection or the process lives. Go's standard-library resolver does no caching of its own — each lookup is fresh (subject to the OS resolver) — but `http.Transport` keeps and reuses established connections; a persistent connection to a now-dead IP survives a DNS change until the connection itself fails and is re-dialed. The general lesson: **DNS TTL bounds convergence only for clients that actually re-resolve.** For failover you care about, you must either (a) bound resolver/library DNS caching explicitly and short, (b) use connection pools that periodically recycle and re-resolve, and (c) not rely on DNS alone for fast failover — pair it with a load balancer or service mesh that fails over at the connection layer in seconds regardless of client DNS behavior. DNS failover is real, but its speed is set by the *slowest, stalest* client in your fleet, not by the TTL you published.

Anycast DNS and GSLB

Two techniques make DNS itself globally resilient and fast, and both connect to material elsewhere in this volume. **Anycast** (Chapter 2, and revisited in Chapter 10) advertises the *same* IP address from many locations via BGP, so a resolver's query to that IP is routed to the topologically nearest instance and, if an instance or its site fails, BGP reconverges and queries flow to the next-nearest instance automatically.

Every root server, every major public resolver (1.1.1.1, 8.8.8.8), and every serious managed authoritative DNS service runs anycast. It gives DNS low latency (you hit a nearby node) and DDoS resilience (attack traffic is spread across dozens of sites rather than concentrated), and it makes the loss of an entire datacenter invisible to clients. For your own authoritative servers, "use an anycast DNS provider" is usually the single highest-leverage availability decision you can make.

GSLB (global server load balancing) is the marriage of health-checked, policy-driven DNS (geo/latency/weighted) with anycast delivery — the Route 53 / Cloud DNS / Akamai / NS1 pattern above, operated as the front door for a multi-region service. GSLB is how a global product steers each user to the nearest *healthy* region: the anycast DNS layer answers quickly from a nearby node, the routing policy picks the right region for the client, and the health checks pull a failed region out of rotation. It is DNS operating as the top of the traffic-management stack, above the L4/L7 load balancers of Chapter 10, and it is the layer at which "the eu-central-1 region is down, send everyone to eu-west-1" actually gets expressed to the world.

Distributed-systems lens

Pull back and DNS reveals itself as one of the largest and most consequential distributed systems your software depends on, with every property this book cares about.

DNS is an eventually-consistent database with no invalidation, and its consistency model is the TTL. Every design decision flows from this. You do not push DNS changes; you publish them and wait for caches to age out. The TTL is the tunable that sets how eventual the consistency is, and it trades directly against load and latency: short TTLs mean fast convergence and heavy authoritative load; long TTLs mean cheap serving and slow, uncontrollable convergence. Treat every TTL as an availability SLO you are signing — "I may serve this stale for up to N seconds" — because that is exactly what it is.

DNS is the service-discovery backbone of the modern fleet. CoreDNS in Kubernetes, Consul across VMs, and headless services for stateful systems all lean on DNS because every runtime already speaks it. That ubiquity is a superpower and a coupling: your entire cluster's ability to find its own services runs through a handful of CoreDNS pods, and the

caching/ndots/search-domain behavior that is invisible when healthy becomes a fleet-wide latency multiplier when misconfigured.

DNS failure is uniquely catastrophic because it is upstream of everything. When DNS is down, healthy servers are unreachable because nothing can find them; monitoring can't resolve its own endpoints; the incident tooling you'd use to fix it may itself fail to resolve. Several of the largest publicly reported internet outages of the last decade have had DNS at or near the root cause — the recurring pattern is a resolver, authoritative service, or DNS-dependent control plane failing and taking down services that were themselves perfectly healthy. The precise details vary from incident to incident and are best read in the published postmortems rather than recalled from memory, but the structural lesson is robust and worth internalizing: **a single DNS dependency is a single point of failure for your entire product**, which is the argument for multiple diverse authoritative providers, anycast, node-local caching, and treating your resolvers as tier-0 infrastructure with the redundancy that implies.

DNS is where global traffic is steered. Anycast + GSLB is the mechanism by which planetary services route each request to the nearest healthy region before a single packet of application traffic is sent. It sits above the load balancers of Chapter 10 and is the coarsest, earliest, and often fastest-to-configure lever for regional failover and geographic routing — bounded, always, by the client-caching realities that make DNS failover only as fast as your stalest client.

Key takeaways

- DNS is a delegated, hierarchical, eventually-consistent database. Authority is partitioned by **delegation** (NS records + glue) into **zones**; no server holds the whole thing, which is why it scales and why you control your own records.
- Learn the two-resolver / two-query split cold: the **stub** sends one **recursive** query to a **recursive resolver**, which does **iterative** queries down the tree (root to TLD to authoritative), caching aggressively at every level.
- Know the record types and their edges: **CNAME cannot coexist with other records**, hence the **CNAME-at-apex problem** and the provider-specific **ALIAS/ANAME** flattening workaround. **SRV** carries

a port and underpins service discovery; **CAA** restricts cert issuance; **PTR** is reverse DNS in a separate zone.

- **TTL is the master knob**: low TTL = fast failover, high authoritative load; high TTL = cheap serving, slow and uninvalidatable convergence. Lower TTLs *before* planned changes. Don't forget **negative caching** of NXDOMAIN.
- Allow **DNS over TCP/53** and **EDNS0**, not just UDP — large answers and DNSSEC depend on them; the legacy 512-byte UDP limit is why.
- **DNSSEC** provides origin authentication via a signed chain of trust (DS in the parent, DNSKEY + RRSIG in the zone, root trust anchor); it does not encrypt. **DoT/DoH** encrypt the stub-to-resolver hop; they don't authenticate data. Use both axes deliberately, and defend against **DNS rebinding** in outbound-fetch code.
- In Kubernetes, understand **CoreDNS**, **headless services**, and above all the `ndots:5` + **search-domain** latency trap: fully-qualify external names, lower `ndots`, or run NodeLocal DNSCache.
- DNS load balancing (round-robin A) is health-blind; real failover/geo/latency routing comes from **health-checked managed DNS** (Route 53, Cloud DNS) plus **anycast** and **GSLB** — but its convergence is capped by client caching. Bound client DNS TTLs explicitly (the JVM `networkaddress.cache.ttl = -1` "cache forever" gotcha) and don't rely on DNS alone for fast failover.

Further reading

- **RFC 1034** and **RFC 1035** — *Domain Names: Concepts and Facilities / Implementation and Specification* (Mockapetris, 1987). The foundational DNS specifications; still the correct reference for the data model, message format, and resolution.
- **RFC 2308** — *Negative Caching of DNS Queries (DNS NCACHE)*. Defines NXDOMAIN negative caching and the SOA fields that govern it.
- **RFC 6891** — *Extension Mechanisms for DNS (EDNS(0))*. The OPT pseudo-record, larger UDP buffers, and the option-code framework.
- **RFC 4033 / 4034 / 4035** — the DNSSEC specification set (introduction, resource records, protocol modifications). For the chain of trust, RRSIG/DNSKEY/DS semantics.

- **RFC 7858** — *DNS over TLS (DoT)*; **RFC 8484** — *DNS over HTTPS (DoH)*; **RFC 7871** — *EDNS Client Subnet*.
- **RFC 2782** — *A DNS RR for specifying the location of services (DNS SRV)*, and **RFC 8552/8553** for the underscore-prefix service-label conventions.
- Kaminsky, Dan — the 2008 DNS cache-poisoning work; see the CERT advisory **VU#800113** and the contemporaneous write-ups for the mechanism and the source-port-randomization mitigation.
- **Kubernetes documentation** — *DNS for Services and Pods*, and the **CoreDNS** project docs (coredns.io). See also the NodeLocal DNSCache addon docs for the `ndots` /CoreDNS-load mitigation.
- **AWS Route 53 Developer Guide** — routing policies (weighted, latency, geolocation, failover), health checks, and ALIAS records; and **Google Cloud DNS** documentation for the equivalent features.
- Cricket Liu & Paul Albitz, *DNS and BIND* (O'Reilly) — the standard operational reference for running authoritative and recursive DNS in depth.
- Cross-references in this volume: Chapter 1 (Journey of a Packet) for where name resolution sits in a request's lifecycle, Chapter 2 (IP Routing and NAT) for anycast and BGP, Chapter 4 (UDP and QUIC) for the transport DNS rides on, and Chapter 10 for load balancing and GSLB. For Kubernetes service discovery in depth, see Volume 12 / Book 6 (Cloud-Native) on cluster DNS and CoreDNS.

Chapter 6 — TLS 1.3 and the Web PKI

What this chapter covers. Every previous chapter in this volume delivered bytes: Chapter 1 followed a packet down the stack, Chapters 2 and 3 gave us routable, reliable byte streams over IP and TCP, and Chapter 4 rebuilt reliability in user space with QUIC. None of them said anything about *who* you are talking to or whether anyone in the middle can read or alter the conversation. That is this chapter. Transport Layer Security is the layer that turns an authenticated-by-nobody, readable-by-everybody byte pipe into a channel that is confidential, integrity-protected, and — this is the hard part — *authenticated* against a real-world identity. The cryptographic primitives underneath (AEAD constructions, elliptic-curve Diffie-Hellman, signature schemes, hash

functions) were covered in Book 5, Chapter 1; this chapter deliberately does not re-derive them. Instead it treats TLS as a *protocol* and the Web PKI as a *system of trust*: how the TLS 1.3 handshake actually works on the wire, why version 1.3 is both simpler and safer than everything before it, how X.509 certificates and certificate authorities let a client decide that a public key really belongs to `api.example.com`, and how that whole apparatus is operated — automated, rotated, revoked, logged, and occasionally allowed to expire and take production down. TLS is the most important security protocol a backend engineer runs, and mutual TLS is the foundation of every zero-trust service mesh you will build, so we treat it at mechanism depth throughout.

Learning goals — after this chapter you should be able to:

- State precisely what TLS provides — confidentiality, integrity, and authentication — and what it does *not* provide, and locate it in the stack over TCP and embedded in QUIC (Chapter 4).
- Trace the TLS 1.3 1-RTT handshake message by message, explain how ephemeral ECDHE gives forward secrecy, and describe the key schedule (HKDF-Extract/Expand) at a working level.
- Explain what TLS 1.3 *removed* relative to TLS 1.2 — RSA key transport, CBC and RC4, static renegotiation, compression — and why each removal closed a class of real attacks.
- Reason about 0-RTT resumption honestly, including the replay-attack caveat and which requests are safe to send as early data.
- Distinguish server-only TLS from mutual TLS (mTLS), and explain why mTLS is the basis of service-to-service zero-trust authentication (Book 5, Chapter 4; Volume 9).
- Describe the Web PKI: X.509 certificates, the root/intermediate/leaf chain of trust, trust stores, and the full client validation algorithm (chain, validity, hostname/SAN, revocation).
- Explain SNI and Encrypted Client Hello, why certificate revocation is genuinely hard, and how OCSP stapling and Certificate Transparency (Book 5, Chapter 5) work around and around it.
- Operate TLS at fleet scale: ACME/Let's Encrypt issuance, cert-manager in Kubernetes, rotation, expiry outages, wildcard-versus-SAN trade-offs, and termination architecture.

What TLS provides, and where it sits

TLS delivers three properties over an insecure transport, and it is worth being pedantic about each because engineers routinely assume one when they have another.

Confidentiality means a passive observer on the path — a compromised router, a mirrored switch port, a tap on a submarine cable — cannot read the application data. In TLS 1.3 this is provided by an authenticated-encryption cipher (AEAD) keyed by material only the two endpoints possess. Modern TLS confidentiality also includes *forward secrecy*: an adversary who records the ciphertext today and later steals the server's long-term private key still cannot decrypt the session, because its keys were derived from ephemeral values discarded when the connection ended.

Integrity means an active attacker who can modify bytes in flight cannot alter the plaintext without detection. Every AEAD record carries an authentication tag; a single flipped bit fails the tag check and tears down the connection. Integrity is not a separate MAC bolted on afterward — in an AEAD construction encryption and authentication are one operation, part of why TLS 1.3 could delete the older, fragile MAC-then-encrypt constructions entirely.

Authentication is the property engineers most often get wrong. TLS authenticates the *server's identity* by having the server prove possession of the private key corresponding to a public key that a trusted certificate authority has vouched for, and by checking that the certificate names the host you intended to reach. It does *not*, by default, authenticate the client — that is mutual TLS, an opt-in. And it authenticates identity only as strongly as the weakest link in the certificate-issuance chain: TLS's cryptography can be flawless and you can still be talking to an attacker if a CA was tricked into issuing a certificate for your domain. That is why the PKI half of this chapter matters as much as the handshake.

TLS sits directly on top of a reliable byte-stream transport, which in practice means TCP (Chapter 3): the application (HTTP, gRPC, a database wire protocol) hands plaintext to the TLS layer, which frames it into *records*, encrypts each record, and writes the ciphertext to the socket. Because TLS assumes an in-order, reliable, lossless stream underneath, it cannot run directly on UDP — which is exactly why QUIC (Chapter 4) does not layer TLS records over a datagram but *embeds the TLS 1.3*

handshake and key schedule directly into the QUIC transport, using TLS to produce keys and QUIC's own packet protection to encrypt. Read "QUIC uses TLS 1.3" (RFC 9001) as: same handshake state machine, same key schedule, different record layer. Everything here about the handshake and the PKI applies to HTTP/3 as much as to HTTPS-over-TCP; only the framing differs.

One historical note that clears up decades of confusion: the protocol was called SSL through versions 2.0 and 3.0, then renamed and published as TLS 1.0 in 1999. SSL 2.0/3.0 are both broken and long disabled; "SSL certificate" and "SSL termination" are just vernacular for TLS. TLS 1.0 and 1.1 were formally deprecated in 2021 (RFC 8996). The live versions are TLS 1.2 (RFC 5246, 2008) and TLS 1.3 (RFC 8446, 2018), and the gap between them is the subject of the next several sections.

flowchart TB

```
App["Application data (HTTP, gRPC, DB protocol)"]
TLS["TLS record layer: AEAD-encrypted records"]
TCP["TCP: reliable, in-order byte stream (Ch 3)"]
IP["IP: routing, addressing (Ch 2)"]
App --> TLS --> TCP --> IP
App -. "HTTP/3 path" .-> QUIC["QUIC: TLS 1.3 handshake embedded, QUIC packet protection (Ch 4)"]
QUIC --> UDP["UDP (Ch 4)"] --> IP
```

The TLS 1.3 handshake in depth

The handshake is the main event. It must accomplish four things before a single byte of application data flows: agree on a version and cipher, perform an authenticated key exchange so both sides share secret keys no eavesdropper can derive, authenticate the server (and optionally the client), and confirm both parties computed the same keys over the same unmodified set of messages. TLS 1.3 does all four in **one round trip** for a fresh connection — halving the setup latency of TLS 1.2's two-round-trip handshake, which is why 1.3 adoption was driven as hard by latency-obsessed frontends as by security teams.

The 1-RTT full handshake

Here is the full handshake, message by message. The client speaks first.

The **ClientHello** carries: the highest supported protocol version (encoded, for middlebox-compatibility, in a `supported_versions` extension

rather than the legacy version field, frozen at 1.2); a random 32-byte nonce; a list of acceptable `cipher_suites`; and — the crucial 1.3 change — a `key_share` extension holding one or more *ephemeral Diffie-Hellman public keys*, one per named group the client will use (say an X25519 and a secp256r1 key). The client does not wait to be told which group to use; it guesses the server will accept one and speculatively sends the public keys up front. That speculation is what buys the round trip. The ClientHello also carries the target hostname in the `server_name` (SNI) extension and, for HTTP, an ALPN extension advertising `h2`, `http/1.1`, and so on.

The server picks a cipher suite and a named group from the client's `key_share` entries and generates its *own* ephemeral key pair in that group. It now has everything it needs for the shared secret: its ephemeral private key and the client's ephemeral public key. It replies with a **ServerHello** carrying the selected suite, its own nonce, and its `key_share`. From the moment the ServerHello is sent, both sides can compute the ECDHE shared secret and derive the *handshake traffic keys*, so **everything the server sends after the ServerHello is already encrypted** — a defining property of TLS 1.3 and a sharp break from 1.2, where the server's certificate flew in cleartext.

Under that handshake encryption, the server sends, in one flight:

- **EncryptedExtensions** — negotiated parameters that need not be in the clear (the ALPN result and so on).
- **Certificate** — the server's X.509 chain: the leaf for the hostname plus the intermediates needed to build a path to a trusted root. (The root is *not* sent; the client must already have it.)
- **CertificateVerify** — a signature, made with the private key for the leaf's public key, over a transcript hash of every handshake message so far. This is the step that authenticates the server: it proves the server holds the private key, and binds that proof to *this* handshake so a signature cannot be replayed.
- **Finished** — an HMAC over the entire transcript, keyed from the handshake keys. It proves the server computed the same keys over the same messages — a downgrade and tamper check covering the whole exchange, including the plaintext ClientHello.

The client verifies the certificate chain (the PKI algorithm of the next sections), the CertificateVerify signature, and the server's Finished. If all

pass it sends its own **Finished** — preceded, in mutual TLS, by its own Certificate and CertificateVerify. That single client flight completes the handshake, and the client can begin sending application data *immediately after* its Finished, in the same flight, so useful data leaves after one round trip.

```
sequenceDiagram
    participant C as Client
    participant S as Server
    Note over C,S: 1-RTT full handshake (TLS 1.3, RFC 8446)
    C->>S: ClientHello + key_share + supported_versions + SNI + ALPN
    Note over S: pick suite and group, gen ephemeral key, derive
    handshake keys
    S->>C: ServerHello + key_share
    Note over C,S: both derive handshake traffic keys here
    S->>C: "{EncryptedExtensions}"
    S->>C: "{Certificate + CertificateVerify}"
    S->>C: "{Finished}"
    Note over C: verify chain, hostname, signature, Finished
    C->>S: "{Certificate + CertificateVerify} (mTLS only)"
    C->>S: "{Finished}"
    C->>S: "{Application data}"
    Note over C,S: braces = encrypted under handshake or application keys
```

Ephemeral ECDHE and forward secrecy

The reason the shared secret is safe against a future key compromise is that the Diffie-Hellman keys are *ephemeral*: generated fresh for this handshake and destroyed when it ends. Both sides compute the same shared secret from the standard Diffie-Hellman operation over their ephemeral key pairs, but neither ephemeral private key ever touches the wire, and the certificate's long-term private key is used *only to sign the transcript*, never to encrypt or transport key material.

This separation is the whole game. In TLS 1.2's RSA key-transport mode (now removed), the client encrypted a pre-master secret to the server's long-term RSA public key; anyone who recorded that handshake and later obtained the RSA private key — via a breach, a subpoena, a Heartbleed-class memory disclosure — could decrypt the entire session retroactively. There was no forward secrecy. TLS 1.3 mandates ephemeral (EC)DHE for every handshake, so the long-term key can only ever *impersonate the server going forward*, never *decrypt the past*. Named groups in practice are X25519 (fast, side-channel-friendly) and the NIST curves secp256r1/secp384r1; finite-field DHE groups exist but are rarely used.

The key schedule and HKDF

TLS 1.3 derives every key it needs from a single **key schedule** built on HKDF, the HMAC-based key-derivation function (RFC 5869), covered mechanically in Book 5, Chapter 1. You do not need to implement it, but you should understand its shape because it explains several security properties. HKDF has two operations: `HKDF-Extract`, which takes a salt and some input keying material and produces a fixed-length pseudorandom secret, and `HKDF-Expand`, which takes a secret and a context label and produces as many bytes of output key as you ask for.

The schedule chains three inputs through a sequence of Extract/Expand steps: a **PSK** (present only for resumption; zero otherwise), the **ECDHE shared secret**, and a final `0` input. Along the way it derives, via `HKDF-Expand-Label`, purpose-specific secrets — the client and server *handshake traffic secrets* (used the moment the `ServerHello` lands, to encrypt the certificate flight), the *application traffic secrets* (for data after Finished), an *exporter secret* (for channel binding), and a *resumption master secret* (for future PSK resumption). Because every secret is bound by its label and by the running transcript hash to the exact messages exchanged, and each traffic secret can be ratcheted forward with `key_update`, compromise of one key does not trivially yield the others. The takeaway: TLS 1.3 has *one* clean derivation tree with clear separation between handshake-protecting and data-protecting keys, replacing TLS 1.2's ad-hoc PRF and blurrier key separation.

What TLS 1.3 removed, and why it is safer

The most important design decision in TLS 1.3 was subtractive. A decade of attacks against TLS 1.2 — BEAST, CRIME, Lucky 13, POODLE, FREAK, Logjam, ROBOT, Sweet32 — nearly all exploited a *legacy option or a negotiable weakness*, not the strong modern configuration. TLS 1.2 was secure if you configured it perfectly and catastrophic otherwise, and negotiation itself was an attack surface. TLS 1.3's response was to delete the options.

Removed in TLS 1.3	The attack class it closed
RSA key transport (static key exchange)	No forward secrecy; retroactive decryption after key compromise; ROBOT (Bleichenbacher oracle)

Removed in TLS 1.3	The attack class it closed
CBC-mode ciphers with MAC-then-encrypt	Padding-oracle attacks: Lucky 13, POODLE (over the SSLv3 fallback)
RC4 stream cipher	Biased-keystream statistical recovery
Static/anonymous DH, export-grade ciphers	FREAK, Logjam (downgrade to breakable 512-bit key exchange)
TLS-level compression	CRIME, BREACH-style compression side channels
Renegotiation	Renegotiation-based request injection and DoS
Custom/weak DH groups	Logjam (precomputation against common weak groups)
MD5/SHA-1 signature hashes in the handshake	Collision-based forgery

What remains is a small, opinionated set. Key exchange is **only** ephemeral (EC)DHE. Bulk encryption is **only** AEAD. Signatures use modern schemes (ECDSA, RSA-PSS, EdDSA) over strong hashes. Renegotiation is gone, replaced by the cleaner `key_update` and post-handshake authentication messages. Compression is gone. The consequence is that a TLS 1.3 handshake has dramatically fewer knobs to misconfigure and dramatically fewer downgrade paths to exploit. A telling detail: TLS 1.3 deliberately injects specific *downgrade-detection sentinels* into the server random so that a 1.3-capable client and server cannot be tricked by an active attacker into silently negotiating 1.2. The design assumes the network is hostile and that negotiation itself must be authenticated — which the final Finished-message transcript check enforces end to end.

Cipher suites in TLS 1.3

TLS 1.2 cipher suite names were a mouthful because a single suite named *four* choices at once: key exchange, authentication, bulk cipher, and MAC (for example `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`). TLS 1.3 decouples these. Key exchange and the signature algorithm are negotiated separately (via `supported_groups`, `key_share`, and `signature_algorithms`), so a "cipher suite" in 1.3 names only the **AEAD**

algorithm and the hash for the key-derivation function. There are exactly five, and in practice you will see three:

TLS_AES_128_GCM_SHA256	# the mandatory-to-implement default
TLS_AES_256_GCM_SHA384	# larger AEAD key, larger KDF hash
TLS_CHACHA20_POLY1305_SHA256	# preferred where AES hardware is absent (mobile, some ARM)
TLS_AES_128_CCM_SHA256	# CCM mode; niche, common in constrained/IoT
TLS_AES_128_CCM_8_SHA256	# CCM with an 8-byte tag; IoT

Every one of these is an AEAD — AES-GCM, ChaCha20-Poly1305, or AES-CCM — with no separate MAC because the AEAD authenticates as it encrypts. AES-GCM is fastest on any CPU with AES-NI (essentially all server-class x86 and modern ARM); ChaCha20-Poly1305 runs in constant time in software without hardware support, so it is preferred on devices lacking AES acceleration (where a software AES-GCM implementation risks cache-timing side channels) and as an algorithmic-diversity hedge. A common server policy offers AES-GCM first but honors a client's ChaCha preference.

0-RTT resumption and its replay caveat

TLS 1.3 can go faster than one round trip for a *returning* client. During a prior connection the server can send the client one or more **NewSessionTicket** messages, each carrying an opaque ticket that encodes (or references) a pre-shared key derived from that session's resumption master secret. On a later connection the client includes the ticket in a `pre_shared_key` extension in its ClientHello, and both sides can resume using the PSK — skipping the certificate exchange and signature entirely, because the PSK already proves continuity of identity. This is 1-RTT resumption, and it is strictly a latency and CPU win.

0-RTT goes one step further and is where you must be careful. Along with the resumption PSK, the client may attach **early data** — actual application bytes — *in the very first flight, before the server has responded at all*. The bytes are encrypted under a key derived solely from the PSK. If the server accepts, the client has sent a request in zero round trips: the request rides along with the ClientHello, eliminating a visible fraction of load time for a client on a high-latency link.

The catch is fundamental and unfixable at the protocol level: **0-RTT early data is replayable**. Because the early data is not part of an interactive exchange — the server has contributed no fresh randomness yet when the client sends it — an attacker who captures the first flight can *replay the entire flight* to the same or another server in the cluster, and the early data will be accepted again. TLS 1.3 provides anti-replay mechanisms (single-use tickets, per-server strike registers of seen tickets, freshness windows using the ticket age), but these are best-effort and, crucially, hard to make reliable across a *fleet* of load-balanced servers that do not share a synchronous replay cache. RFC 8446 is explicit about this: the protocol cannot guarantee non-replay, so the *application* must only send 0-RTT data that is safe to execute more than once.

The operational rule is simple and strict: **only idempotent requests may go in 0-RTT**. An HTTP `GET` with no side effects is fine; a `POST` that charges a card, increments a counter, or sends an email is not. Well-behaved stacks enforce this — NGINX passes early data upstream only for safe methods under `ssl_early_data`, and deployments add an `Early-Data: 1` header so the origin can reject or re-drive a non-idempotent request. If you cannot reason confidently about idempotency across your whole request surface, leave 0-RTT off; the 1-RTT resumption path is already fast and carries none of this risk.

Server TLS versus mutual TLS

In the handshake above, only the server presented a certificate — "one-way" or server-authenticated TLS, the overwhelming common case. It fits the web: a browser needs to know it reached the real `bank.example.com`, but the bank authenticates the *user* by a separate mechanism (a password, a passkey) layered over the confidential channel, not by a client certificate.

Mutual TLS (mTLS) turns on client authentication at the TLS layer itself. The server includes a **CertificateRequest** in its flight; the client responds with its own Certificate and a CertificateVerify signature proving it holds the private key. Now *both* peers have cryptographically authenticated identities before any application data flows, each having validated the other's certificate against its own trust store. mTLS does not replace server TLS; it adds the symmetric half.

For backend engineers, mTLS is the cryptographic substrate of **zero-trust service-to-service authentication**. The traditional perimeter model ("we're all inside the VPC, so we trust each other") is exactly the assumption that lateral movement in a breach exploits. Zero trust flips it: every call is authenticated and encrypted regardless of network location, and mTLS is how a service proves "*I am the `payments` service*" to the `ledger` service on every connection, with no shared secret to leak and no topology assumption to violate. This is where mTLS connects to the identity systems of Book 5, Chapter 4 and Volume 9: in a SPIFFE/SPIRE deployment each workload receives a short-lived X.509 SVID whose SAN is a SPIFFE ID like `spiffe://prod.example.com/ns/payments/sa/api`, and services authenticate and authorize on that ID over mTLS. A service mesh (Chapter 10; Volume 6) automates the pattern via sidecar proxies so application code never touches a certificate. The point is architectural: **mTLS is the layer at which "which service is this?" gets a cryptographic, not a network, answer.**

```
sequenceDiagram
    participant A as "Service A (client)"
    participant B as "Service B (server)"
    Note over A,B: mutual TLS: both present certs from a shared CA / SPIFFE trust domain
    A->>B: ClientHello + key_share
    B->>A: ServerHello + key_share
    B->>A: "{CertificateRequest}"
    B->>A: "{Certificate: B leaf + intermediates}"
    B->>A: "{CertificateVerify + Finished}"
    Note over A: validate B chain + B SPIFFE ID, check authz policy
    A->>B: "{Certificate: A leaf + intermediates}"
    A->>B: "{CertificateVerify + Finished}"
    Note over B: validate A chain + A SPIFFE ID, check authz policy
    A->>B: "{Application data: authenticated both ways}"
```

Certificates and the Web PKI

The handshake proved the server holds a private key. The Web PKI is the answer to the far harder question: *why should the client believe the public key in that certificate belongs to the hostname it wanted?* The answer is a **chain of trust** rooted in a set of certificate authorities the client has decided, out of band, to trust.

X.509 certificates

A TLS certificate is an X.509 (v3) structure — DER-encoded ASN.1, usually shown to humans in PEM (base64) form. The fields that matter operationally:

- **Subject** — historically a Distinguished Name including a Common Name (CN). For hostname validation the CN is now **ignored** by browsers; identity lives in the SAN.
- **Subject Alternative Name (SAN)** — the identities the certificate is valid for: DNS names (`api.example.com` , `*.example.com`), IP addresses, or URIs (a SPIFFE ID). One certificate can cover many names; this is the field clients check against the hostname.
- **Issuer** — the DN of the signing CA. The leaf's issuer is an intermediate; the intermediate's is (eventually) a root.
- **Validity** — `notBefore` and `notAfter` . Invalid outside this window, full stop — the single field behind the most common self-inflicted TLS outage in the industry.
- **Public key** — the subject's public key and algorithm (RSA-2048/3072, or EC on P-256/P-384, increasingly Ed25519).
- **Basic Constraints** — `CA:TRUE` or `CA:FALSE` . Leaves are `CA:FALSE` ; only CA certificates may sign others, which stops a valid leaf from being abused to mint further certificates.
- **Key Usage / Extended Key Usage** — what the key may do (`digitalSignature` , `keyEncipherment` ; EKU `serverAuth` , `clientAuth`). A `serverAuth` cert cannot be used for client auth where EKU is enforced.
- **AIA / CRL Distribution Points / SCTs** — URLs for the issuer certificate and OCSP responder, revocation-list locations, and Signed Certificate Timestamps for Certificate Transparency.

You will read these constantly with `openssl` :

```
# Fetch and decode the leaf certificate a server presents for a given
SNI
openssl s_client -connect api.example.com:443 -servername api.example.c
om </dev/null 2>/dev/null \
    | openssl x509 -noout -text

# Just the fields you check most: subject, issuer, validity, SANs
openssl x509 -in leaf.pem -noout -subject -issuer -dates -ext subjectAl
tName
```

The chain of trust

Trust is delegated, not asserted directly. A **root CA** has a self-signed certificate whose public key is embedded in *trust stores* — the curated root lists shipped by operating systems (the Microsoft, Apple, and Linux `ca-certificates` stores) and browsers (Mozilla's is the de facto reference for Firefox and much of the Linux and language ecosystem; Chrome now ships its own Chrome Root Store). Root private keys are kept offline in HSMs and used rarely, because a compromised root is catastrophic and effectively unrevocable in the short term.

A root does not sign leaves directly. It signs one or more **intermediate CA** certificates whose keys live in the CA's online issuance infrastructure and do the day-to-day signing, and the intermediate signs your **leaf** (end-entity) certificate. So a real chain is leaf ← intermediate ← root: the leaf and intermediates travel in the handshake's Certificate message, the root must already be in the client's trust store. This two-tier structure exists precisely so a compromised intermediate can be revoked and replaced without touching the root that anchors billions of clients.

```
flowchart TD
    Root["Root CA – self-signed, in OS/browser trust store, offline HSM key"]
    Int["Intermediate CA – signed by Root, online issuance key"]
    Leaf["Leaf cert – CN/SAN api.example.com, CA:FALSE"]
    Root -->|signs| Int
    Int -->|signs| Leaf
    Store["Client trust store: set of trusted Root public keys"]
    Store -. "anchors trust in" .-> Root
    Handshake["Server sends: Leaf + Intermediate (NOT Root)"]
    Leaf -.-> Handshake
    Int -.-> Handshake
```

How a client validates a certificate

When the client receives the server's Certificate message, it runs a validation algorithm that is worth committing to memory because misconfigurations map directly onto its steps:

1. **Path building.** From the leaf, chain each certificate to its issuer until reaching a root in the local trust store. The server should send the intermediates; if it forgets (a classic misconfiguration), some clients recover via the AIA URL and some — notably many non-browser clients — do not, producing the maddening "works in my browser, fails in curl/Java" bug.
2. **Signature verification.** Verify each certificate's signature with its issuer's public key up to the trusted root. One broken signature invalidates the path.
3. **Validity window.** Every certificate must be within its `notBefore / notAfter` window *at the current time* — where clock skew and expiry outages bite.
4. **Constraints.** Every non-leaf must have `CA:TRUE`; path-length and name constraints must hold; EKU must permit `serverAuth`.
5. **Hostname / SAN matching.** The intended hostname (the SNI value, the URL host for HTTPS) must match a DNS name in the leaf's SAN, honoring the single-label wildcard rule (`*.example.com` matches `api.example.com` but *not* `example.com` or `a.b.example.com`). The legacy CN fallback is dead in modern clients.
6. **Revocation.** Check whether the certificate has been revoked since issuance — the weakest step in practice (next section).

Only if all six pass is the `CertificateVerify` signature checked against the now-trusted public key. The intuition: the PKI establishes that *this public key is authorized for this hostname*, and `CertificateVerify` establishes that *the peer holds the matching private key right now*. You need both.

SNI and Encrypted Client Hello

Because one IP address routinely serves thousands of TLS virtual hosts (any CDN, any shared hosting, any Kubernetes ingress), the server must know *which* certificate to present before it can send one — but the certificate is chosen by hostname, which is application-layer information. **Server Name Indication (SNI)** solves this by carrying the target

hostname as a cleartext extension in the ClientHello, so the server (or a TLS-terminating load balancer) selects the right certificate and backend. SNI is why virtual hosting over HTTPS works at all.

The cost is a privacy leak: SNI is in the clear, so a passive observer learns exactly which hostname you are visiting even though everything else is encrypted, and censors and corporate middleboxes routinely filter on it. **Encrypted Client Hello (ECH)** closes this gap. A service publishes an ECH public key in DNS (an `HTTPS / SVCB` record); the client encrypts the sensitive parts of its ClientHello — including the real SNI — to that key and sends an *outer* ClientHello bearing only a public, non-identifying "cover" name shared by all tenants of the fronting provider, which decrypts the inner hello and routes to the true backend. The observer sees only the cover name. ECH is deployed by Cloudflare and supported by recent Firefox and Chrome, but it depends on encrypted DNS (DoH/DoT, Chapter 5) to fetch the key without leaking the name there instead, and remains an evolving standard (`draft-ietf-tls-esni`) rather than a finished RFC — promising and partially deployed, not universal.

Revocation: the genuinely hard problem

Certificates have expiry dates, but sometimes a certificate must be killed *before* it expires — a private key was leaked, a server was decommissioned, a CA discovers it mis-issued. Revocation is the mechanism for that, and it is, honestly, the part of the Web PKI that works least well. The core difficulty was covered from the trust-model side in Book 5, Chapter 1; here is the protocol reality.

Certificate Revocation Lists (CRLs) are the original design: each CA publishes a signed list of the serial numbers it has revoked, and clients download and consult it. CRLs do not scale — lists for large CAs grow to megabytes, they are cached and thus stale, and downloading them on the hot path is intolerable, so in practice browsers largely stopped hard-checking them.

OCSP (Online Certificate Status Protocol) replaced the bulk list with a point query: the client asks the CA's responder "is serial N still valid?" and gets a signed, short-lived answer. This fixes the size problem and creates three others. **Latency**: an extra round trip to a third party on every new connection. **Privacy**: the responder — the CA — learns every site every user visits. And decisively, **soft-fail**: because responders have

outages, browsers treat a *failure to reach the responder* as "assume valid," so an attacker who can *block* the OCSP query defeats revocation entirely. A revocation check an attacker can suppress is not a revocation check.

OCSP stapling fixes the latency and privacy problems, though not soft-fail. Here the *server* periodically fetches a signed OCSP response for its own certificate and *staples* it into the TLS handshake (a `status_request` extension), so the client gets fresh revocation status with no extra round trip and no privacy leak to the CA. Stapling is a genuine improvement and widely deployed (`ssl_stapling on;` in NGINX), but the client still soft-fails if the staple is absent. The **Must-Staple** extension was meant to close that by telling clients to *hard*-fail without a staple, but it saw little adoption and browser support has waned.

The industry's actual answer to revocation's brokenness is threefold, and it drives modern practice. First, browsers ship **proprietary aggregated revocation sets** — Mozilla's CRLite and Chrome's CRLSets compress the whole Web PKI's revocation state into a pushed, locally-checked structure, sidestepping per-connection queries. Second, and most important for backend engineers, the CA/Browser Forum has been **driving certificate lifetimes steadily down** — from years, to 398 days, with a plan (ratified in 2025) to reach roughly 47-day maximum validity by 2029 — on the logic that a certificate that expires in weeks barely needs revocation. Third, short lifetimes are only tolerable if issuance is *automated* (ACME, below). The revocation story and the automation story are one: **make certificates cheap and short-lived, and revocation stops being the load-bearing control.**

Certificate Transparency

Revocation answers "this specific certificate is now bad." **Certificate Transparency (CT)** answers a systemic question: "did a CA — any CA — issue a certificate for my domain that I never asked for?" Before CT, a mis-issued or maliciously-issued certificate (a compromised or coerced CA signing `google.com` for an attacker) was essentially undetectable until someone stumbled on it in the wild. The 2011 DigiNotar breach — a compromised Dutch CA issuing fraudulent certificates used to intercept Gmail traffic in Iran — was the galvanizing incident.

CT, covered from the supply-chain-transparency angle in Book 5, Chapter 5, requires every issued certificate to be recorded in **public, append-only, cryptographically verifiable logs** built on Merkle trees. A CA submits the certificate to several independent logs; each returns a **Signed Certificate Timestamp (SCT)** promising to include it. Clients — Chrome and Apple most strictly — *require* a certificate to carry enough SCTs from qualifying logs or they reject it. The SCTs ride in the certificate itself (via a precertificate flow), in a TLS extension, or in a stapled OCSP response.

The property CT provides is not prevention but **detection**: because every usable certificate must be publicly logged, a domain owner (or an automated monitor, or services like `crt.sh`) can *see* every certificate ever issued for their names and alarm on any they did not authorize. It converts CA mis-issuance from an invisible catastrophe into a loud, auditable event, and has directly caused the distrust and removal of several CAs caught mis-issuing. The practical action for a backend engineer is to monitor CT logs for your own domains — a free, high-signal control that costs a webhook and catches both attacks and your own teams' shadow certificates.

ACME and Let's Encrypt

The transformation of the Web PKI over the last decade came from one idea: **issuance as an automated protocol**. Before 2015, getting a certificate meant a manual purchase, an out-of-band domain-control check, an emailed file, and a hand-installed key — painful enough that certificates were long-lived and HTTPS was the exception. **Let's Encrypt**, a free nonprofit CA launched in 2015, and the **ACME protocol** (Automatic Certificate Management Environment, RFC 8555, 2019) inverted this: a machine can prove control of a domain and obtain a certificate in seconds, for free, with no human involved. This is why the web went from minority to vast-majority HTTPS in a few years, and it is what makes short-lived certificates operationally viable.

ACME issuance works as an automated proof-of-control protocol between your **ACME client** (Certbot, `acme.sh`, Caddy's built-in client, cert-manager, Traefik) and the CA:

1. **Account.** The client generates an account key pair and registers with the CA.

2. **Order.** It requests a certificate for a set of identifiers (`api.example.com`, `*.example.com`).
3. **Challenge.** For each identifier the CA issues a challenge proving control of that name. The two common types:
 4. **HTTP-01:** the client serves a token at `http://api.example.com/.well-known/acme-challenge/<token>` and the CA fetches it from multiple network vantage points. Proves control of the web server; cannot issue wildcards.
 5. **DNS-01:** the client publishes a TXT record `_acme-challenge.api.example.com` derived from the token and account key. Proves control of DNS, which is what lets DNS-01 issue **wildcards**; requires programmatic access to your DNS provider's API.
6. **Validation.** The CA checks the challenge from multiple perspectives (added to harden against BGP-hijack-based domain-control fraud).
7. **Finalize.** The client submits a CSR with the certificate's public key; the CA issues, and the client downloads the leaf-plus-intermediate chain.
8. **Renew.** Long before expiry the client repeats the flow automatically. With Let's Encrypt's 90-day certificates, renewal is a cron job, not a calendar reminder — which is the whole point.

```
sequenceDiagram
    participant Cl as "ACME client (cert-manager / Certbot)"
    participant CA as "ACME CA (Let's Encrypt)"
    participant DNS as "DNS / web server for api.example.com"
    Cl->>CA: newOrder for api.example.com, *.example.com
    CA->>Cl: challenges (HTTP-01 and/or DNS-01) + tokens
    Cl->>DNS: publish TXT _acme-challenge (DNS-01) or token file (HTTP-01)
    Cl->>CA: challenge ready, please validate
    CA->>DNS: fetch token from multiple vantage points
    CA->>Cl: challenges valid
    Cl->>CA: finalize with CSR (public key)
    CA->>Cl: issued certificate: leaf + intermediate chain
    Note over Cl: install cert; schedule auto-renew well before 90-day expiry
```

cert-manager in Kubernetes

In Kubernetes the canonical ACME automation is **cert-manager**, which turns certificate lifecycle into declarative custom resources reconciled by a controller. You declare an `Issuer / ClusterIssuer` pointing at an ACME

endpoint and a solver, and a `Certificate` naming the DNS names and target `Secret`; the controller runs the ACME dance, stores the key and certificate in the `Secret`, and *renews automatically* before expiry — the same self-healing reconciliation model as the rest of Kubernetes.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform-oncall@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-account-key
    solvers:
      - dns01:
          route53:
            region: us-east-1      # IAM via IRSA; no static
credentials
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: api-example-com
  namespace: edge
spec:
  secretName: api-example-com-tls  # k8s Secret the Ingress/Gateway
  mounts:
    duration: 2160h                # 90d
    renewBefore: 720h              # start renewing 30d before expiry
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - api.example.com
    - "*.example.com"
```

The `renewBefore` window is not decoration — it is the buffer that turns a failed renewal into a paged alert with days of runway instead of an instant outage. We will insist on that margin in the operations section.

Operational TLS at fleet scale

The cryptography is the easy part. The failures that take down real backends are operational, and they cluster into a few well-known shapes.

The expired-certificate outage

The single most common self-inflicted TLS outage is a certificate that expired because nobody renewed it. It has taken down major payment networks, telecom providers, cloud consoles, and countless internal services; the pattern is always the same — a certificate provisioned manually years ago, an owner who changed teams, a monitoring gap, and a `notAfter` date that arrived on a weekend. The failure is total and instantaneous: at the stroke of expiry, every client's validity check fails and every connection is refused.

The defenses are unglamorous and non-negotiable:

- **Automate issuance and renewal** so no human is on the critical path — ACME/cert-manager, or an internal CA with the same automation. A certificate you renew by hand is a future outage with a scheduled date.
- **Monitor expiry independently of the renewal system** — an external probe that reads `notAfter` and alerts at, say, 21 and 7 days out. Monitor the *actually-served* certificate on every endpoint, not the one you believe you deployed; the gap between those two is where outages live.
- **Renew with a wide margin** (the `renewBefore` above), so a broken renewal is an alert with days of slack, not an outage.
- **Watch intermediates and roots, not just your leaf.** The 2021 global outage triggered by the expiry of Let's Encrypt's cross-signed DST Root CA X3 broke a long tail of older clients — a reminder that the *whole chain* has expiry dates, some outside your control.

```
# A dead-simple external expiry probe you can wire into monitoring
host=api.example.com
end=$(openssl s_client -connect "$host:443" -servername "$host" </dev/
null 2>/dev/null \
    | openssl x509 -noout -enddate | cut -d= -f2)
echo "$host expires: $end ($( (date -d "$end" +%s) - $(date +%s) ) / 86400 ) days")"
```

Wildcard versus SAN certificates

Two ways to cover many names, with a real trade-off. A **wildcard** (`*.example.com`) covers every single-label subdomain with one certificate and key — convenient, but it concentrates risk: one leaked key

compromises every subdomain, wildcards require DNS-01 (not HTTP-01), and they cannot span two levels (`*.example.com` does not cover `a.b.example.com`). A **multi-SAN** certificate lists each name explicitly — more precise and easier to scope and revoke per-name, but the list is public in CT logs and every name change means reissuing. At fleet scale the common pattern is per-service SAN certificates issued automatically (so any one key's blast radius is one service), with wildcards reserved for genuinely dynamic subdomain spaces. Automation erases the "reissue on every change" cost of SANs, tilting the modern trade-off toward narrowly-scoped, per-service certificates.

Termination architecture: edge, mesh, or end-to-end

Where TLS terminates is one of the more consequential design decisions in a backend system, and there is no single right answer — only trade-offs to make deliberately.

- **Edge termination at the load balancer / reverse proxy.** TLS is decrypted at the LB (an ALB, an NGINX/Envoy edge, a CDN). This centralizes certificate management, enables L7 routing and WAF inspection on cleartext, and offloads handshake CPU from application servers. The cost is a plaintext segment behind the LB — fine at the perimeter of a trusted network, unacceptable under zero trust if left plaintext.
- **End-to-end (re-encrypted) TLS.** The LB terminates for inspection and routing, then opens a *fresh* TLS connection to the backend, so no segment is ever plaintext. The common compliance-driven pattern (PCI, HIPAA): edge L7 features with no plaintext hop, at the cost of two handshakes and certificate management on the backends too.
- **Passthrough.** The LB does L4 forwarding only and the backend terminates TLS itself; the LB cannot route on L7. Used when the backend must own the certificate and no intermediary may see plaintext.
- **Mesh mTLS everywhere.** In a service mesh (Chapter 10; Volume 6), sidecar proxies terminate and originate mTLS for every workload, so every internal hop is mutually authenticated and encrypted with zero application-code involvement — the zero-trust end state, sane only because the mesh automates the lifecycle for thousands of short-lived workload identities.

Performance: handshakes, resumption, offload

A TLS 1.3 full handshake costs one network round trip plus asymmetric-crypto work (an ECDHE key agreement on both sides, a signature by the server, a signature verification and chain validation by the client). The round trip dominates latency; the asymmetric operations dominate server CPU under connection churn. Three levers matter at scale:

- **Session resumption (1-RTT PSK)** skips the certificate and signature entirely on reconnect — the biggest single win for services with many short connections, moving the server's cost from expensive signatures to cheap symmetric key derivation. In a fleet, tickets must be decryptable across servers, so the ticket-encryption key (STEK) is *shared and rotated*: rotate too slowly and a stolen key threatens the forward secrecy of resumed sessions; rotate too aggressively and you invalidate outstanding tickets — a real tuning knob.
- **Connection reuse and keep-alive** avoid the handshake altogether. HTTP/2 and HTTP/3 multiplex many requests over one long-lived connection, so the handshake amortizes over thousands of requests. The highest-leverage TLS performance fix is often "stop opening new connections."
- **Hardware and kernel offload.** AES-NI makes AES-GCM nearly free on the bulk path; kTLS lets the kernel do record encryption and enables `sendfile` zero-copy for encrypted static content; at very high scale, dedicated TLS hardware or SmartNIC offload moves handshakes off the application CPU.

The distributed-systems lens

Everything above compounds when you multiply it by thousands of services and millions of connections. Five threads matter for a senior backend engineer.

mTLS is the foundation of zero-trust service-to-service security. In a monolith, "who is calling this function?" is answered by the call stack; in a distributed system it must be answered on the wire, and mTLS is the only widely-deployed mechanism that answers it *cryptographically* on every call, independent of network topology. An authorization policy ("payments may call ledger, fraud may not") is only meaningful if the identities it names are cryptographically authenticated, and mTLS is

what replaces the network-position answer ("it came from inside the VPC") — the very assumption a breach's lateral movement exploits — with a key the attacker does not hold. This is the substrate under the service mesh (Chapter 10; Volume 6) and the workload-identity systems of Volume 9.

Certificate-lifecycle automation is what prevents expiry outages across thousands of services. A single manually-managed certificate is a manageable risk; ten thousand across hundreds of teams is a statistical certainty of outages. The only scalable answer is to make issuance a protocol and renewal a reconciliation loop — ACME/cert-manager at the edge, SPIFFE/SPIRE for workload identity (Book 5, Chapter 4), a mesh CA (Istio's istiod, Linkerd's identity) inside — so that *nobody's calendar* is on the critical path. The move to sub-50-day CA/Browser-Forum lifetimes is only survivable because of this automation.

Short-lived certificates plus Certificate Transparency strengthen the whole trust model. The supply-chain volume (Book 5, Chapters 1 and 5) argues that short-lived, auto-rotated credentials and public transparency logs beat long-lived secrets and private issuance. TLS is the largest production instance of that thesis: a 90-day (soon shorter) publicly-logged, auto-renewed certificate makes revocation nearly irrelevant and CA mis-issuance detectable. Treat CT-log monitoring of your own domains as a standard detection control, and prefer the shortest lifetime your automation comfortably supports.

Termination architecture is a first-class design decision. Where TLS terminates determines your plaintext blast radius, your L7 inspection and routing options, your certificate-management surface, and your handshake-CPU distribution. Edge termination is simplest, mesh-wide mTLS is the zero-trust ideal, and end-to-end re-encryption is the compliance-driven middle. Decide it explicitly per tier, and never let "plaintext behind the load balancer" persist by accident in a system that claims zero trust.

Handshake economics shape tail latency at fleet scale. TLS 1.3's 1-RTT handshake, PSK resumption, and connection reuse are not micro-optimizations when you serve billions of requests across high-latency mobile and edge clients — they move p99 measurably. 0-RTT can shave the last round trip for returning clients, but its replay caveat is a distributed-systems problem in miniature: non-replay is hard to guarantee precisely *because* your fleet is many machines that do not

share a synchronous replay cache. Enabling 0-RTT is a fleet-wide bet about request idempotency; make it deliberately or not at all.

Key takeaways

- **TLS provides confidentiality, integrity, and server authentication** over a reliable transport; it authenticates the *client* only in mTLS, and identity only as strongly as the weakest CA in the chain. It runs over TCP and is embedded (same handshake, different record layer) in QUIC/HTTP-3.
- **The TLS 1.3 handshake is 1-RTT**: the ClientHello speculatively carries ephemeral `key_share` values, so both sides derive keys after the ServerHello and *the certificate flight is already encrypted*. ECDHE gives forward secrecy; CertificateVerify authenticates the server; Finished authenticates the whole transcript.
- **TLS 1.3 is safer mainly by subtraction**: no RSA key transport, CBC/RC4, renegotiation, compression, or weak DH — deleting the options deleted BEAST, POODLE, Lucky 13, FREAK, Logjam, CRIME, and ROBOT. Bulk encryption is AEAD-only (AES-GCM, ChaCha20-Poly1305).
- **0-RTT early data is replayable and unfixable at the protocol level**; send only idempotent requests in it, or leave it off. 1-RTT PSK resumption keeps the latency win without the replay risk.
- **mTLS is the cryptographic basis of zero-trust service-to-service auth** — an authz policy's identity is only meaningful if mTLS authenticated it. It is the substrate of the service mesh and of SPIFFE/SPIRE workload identity (Book 5, Chapter 4; Volume 9).
- **The Web PKI is a delegated chain of trust**: root (offline, in trust stores) signs intermediate (online) signs leaf. Clients validate path, signatures, validity window, constraints, hostname/SAN, and revocation before trusting a key.
- **Revocation is the PKI's weakest link**: CRLs don't scale, OCSP soft-fails and leaks privacy, stapling helps but doesn't fix soft-fail. The real fix is short-lived automated certificates, and Certificate Transparency makes CA mis-issuance *detectable* even when it can't be prevented.
- **ACME automates issuance** (HTTP-01 for control-of-server, DNS-01 for control-of-DNS and wildcards); Let's Encrypt made it free and 90-

day; cert-manager makes it a Kubernetes reconciliation loop. Automation is what makes short lifetimes and expiry-free operation possible.

- **Operational TLS is where systems actually fail:** automate renewal with a wide margin, monitor the *served* certificate's expiry independently (including intermediates), choose termination architecture deliberately, and use resumption plus connection reuse to control handshake cost at scale.

Further reading

- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3* (August 2018) — the authoritative specification: the handshake state machine, the key schedule, 0-RTT, and Appendix E's security analysis. The single most important document behind this chapter.
- RFC 5246, *TLS 1.2* (2008), and RFC 8996, *Deprecating TLS 1.0 and TLS 1.1* (2021) — the version TLS 1.3 replaced and the formal retirement of its predecessors; useful to see what changed.
- RFC 5869, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, and the IANA TLS parameter registries — the KDF underneath the TLS 1.3 key schedule (mechanics in Book 5, Chapter 1) and the live list of cipher suites, groups, and extensions.
- RFC 5280, *Internet X.509 PKI Certificate and CRL Profile* — the definitive X.509 profile: certificate fields, path validation, name constraints, CRLs.
- RFC 6960, *OCSP*, and RFC 6066 (`status_request` , i.e. OCSP stapling) — the online revocation protocol and the stapling extension.
- RFC 6962, *Certificate Transparency* — the Merkle-tree log design, precertificates, and SCTs. See Book 5, Chapter 5 for the supply-chain-transparency framing.
- RFC 8555, *Automatic Certificate Management Environment (ACME)* — the issuance-automation protocol behind Let's Encrypt, Certbot, Caddy, and cert-manager.
- RFC 9001, *Using TLS to Secure QUIC* — how the TLS 1.3 handshake and key schedule are embedded in QUIC; read alongside Chapter 4 of this volume.

- `draft-ietf-tls-esni`, *TLS Encrypted Client Hello* — the evolving ECH specification; pair with the DNS `HTTPS / SVCB` records (RFC 9460) that carry ECH keys.
- The CA/Browser Forum *Baseline Requirements* and the 2025 ballot reducing maximum certificate validity toward roughly 47 days by 2029 (as of early 2026; track current BRs at <https://cabforum.org/baseline-requirements-documents/>) — the policy driving short-lived, automated certificates.
- Ivan Ristić, *Bulletproof TLS and PKI* (2nd ed.) — the standard practitioner reference for configuration, attacks, and operations; and the associated SSL Labs *SSL/TLS Deployment Best Practices* and server test.
- The SPIFFE and SPIRE specifications (spiffe.io) and the Istio and Linkerd security documentation — workload identity and mesh mTLS in practice; see Book 5, Chapter 4, Chapter 10 of this volume, and Volume 6.
- `cert-manager` documentation (cert-manager.io) — the reference for ACME automation in Kubernetes, including DNS-01 solvers and renewal tuning.

Chapter 7 — HTTP/1.1, HTTP/2, and HTTP/3

What this chapter covers. HTTP is the application protocol your fleet speaks more than any other. Your REST APIs, your gRPC services (Chapter 8), your health checks, your service mesh sidecars, your CDN edge (Chapter 10), your webhooks, your object storage clients — nearly all of it is HTTP, and most of it is HTTP you did not write by hand but whose behavior you are nonetheless responsible for when latency spikes or a load balancer distributes traffic unevenly. This chapter treats HTTP the way a backend engineer must: not as a request-and-a-JSON-body but as three distinct wire protocols — HTTP/1.1, HTTP/2, and HTTP/3 — sharing one *semantic* model, each solving a head-of-line-blocking problem the previous one left open, and each with concrete operational consequences for connection management, load balancing, and tail latency. We will insist on mechanism throughout. Not "HTTP/2 is faster," but *which* frame, on *which* stream, compressed by *which* header table,

and *why* one lost TCP segment still stalls all of it until HTTP/3 moves the stream state into QUIC (Chapter 4).

Learning goals — after this chapter you should be able to:

- Separate HTTP *semantics* (methods, status codes, header fields, caching, content negotiation — RFC 9110) from the *wire format* of a specific version (RFC 9112, 9113, 9114), and explain why the same API works unchanged across all three versions.
- Explain HTTP/1.1 persistent connections, chunked transfer-encoding, and why pipelining failed — the L7 head-of-line blocking that forced browsers into six connections per host.
- Describe HTTP/2's binary framing layer, stream multiplexing, HPACK header compression, flow control, and prioritization in mechanism, and state precisely why HTTP/2 still suffers *TCP-level* head-of-line blocking that no amount of L7 multiplexing can remove.
- Explain how HTTP/3 over QUIC gives each stream independent reliability, replaces HPACK with QPACK, and cuts connection-setup latency, and what it costs.
- Reason about connection pooling and keep-alive for HTTP/1.1 versus HTTP/2, and diagnose the HTTP/2-single-connection backend-pinning problem that makes L4 load balancing distribute traffic unevenly and forces L7 load balancing (Chapter 9).
- Apply the semantics that matter operationally: idempotency and safe retries (Chapter 11), caching headers, compression, range requests, the `Host` header and virtual hosting.

Semantics versus wire format: the split that RFC 9110 made explicit

The most important conceptual move you can make about HTTP is to stop thinking of it as one protocol. Since June 2022, the IETF has published HTTP as a deliberately layered set of documents, and the layering is the point. **RFC 9110, "HTTP Semantics,"** defines the things that are true of HTTP regardless of version: the request/response exchange, methods, status codes, the meaning of header fields, content negotiation, caching, conditional requests, range requests, authentication. **RFC 9111** defines caching. Then three separate documents define three *wire formats* that all carry those same semantics: **RFC 9112 (HTTP/1.1)** describes the

text-based message syntax; **RFC 9113 (HTTP/2)** describes the binary framing; **RFC 9114 (HTTP/3)** describes the mapping onto QUIC. RFC 7540 (the original HTTP/2) and RFC 7230-7235 (the older HTTP/1.1 set) are obsoleted by this reorganization.

This is not bureaucratic tidiness; it is why your architecture survives version churn. When your service exposes `GET /v2/orders/1234` returning `200 OK` with a JSON body and an `ETag`, those semantics — the method's safety and idempotency, the status code's meaning, the validator for conditional requests — are identical whether a client reaches you over HTTP/1.1, HTTP/2, or HTTP/3. A load balancer can terminate HTTP/3 from a mobile client at the edge and re-originate HTTP/1.1 to your backend, and the *message* is preserved even though the *bytes on the wire* are completely different — one is UTF-8 text with CRLF line endings, another is HPACK-compressed HEADERS frames, the third is QPACK inside QUIC STREAM frames. The semantic message is the contract; the wire format is an implementation detail of a hop. Internalize this and most "should we migrate to HTTP/2?" questions become answerable: you are changing transport efficiency, not application behavior.

A **message** in RFC 9110 terms is a set of header fields plus an optional body, in a request addressed to a *target resource* by method, or a response bearing a *status code*. The methods carry two properties that dominate distributed-systems correctness. A method is **safe** if it is read-only in intent (`GET`, `HEAD`, `OPTIONS`, `TRACE`); a client, cache, or prefetcher may issue it speculatively. A method is **idempotent** if issuing it N times has the same effect on server state as issuing it once (`GET`, `HEAD`, `OPTIONS`, `TRACE`, `PUT`, `DELETE` — but *not* `POST`, and not, in general, `PATCH`). This is a semantic property, invariant across versions, and it is precisely the property a retry layer depends on. We return to it under retries below; hold the thought that idempotency lives in RFC 9110, not in any wire format.

Status codes are grouped by class — 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error — and the class boundary is load-bearing for automated behavior. A generic client treats an unknown `287` like `200` and an unknown `499` like `400`, because the first digit is the contract. The distinction between 4xx (do not retry unchanged; you did something wrong) and 5xx / `502` / `503` / `504` (the server or an intermediary failed; a retry may succeed) is the backbone of every resilience library.

Header fields carry the metadata that makes HTTP more than a byte pipe: `Content-Type` and `Accept` for content negotiation, `Content-Encoding` and `Accept-Encoding` for compression, `Cache-Control` and `ETag` for caching, `Range` for partial transfers, `Host` for virtual hosting, `Authorization` for credentials. All of this is version-independent semantics. What changes across HTTP/1.1, HTTP/2, and HTTP/3 is only *how those fields and bodies are encoded and framed on the connection* — and that encoding is where every performance problem of the last twenty years lives.

HTTP/1.0 to HTTP/1.1: persistence, chunking, and the pipelining dead end

HTTP/1.0 established the text format we still recognize: a request line, header lines, a blank line, an optional body — all ASCII, CRLF-delimited, human-readable. Its fatal operational flaw was the connection model. By default each request/response used a fresh TCP connection, opened and then closed. On a network with meaningful round-trip time, this is ruinous: every request pays a full TCP three-way handshake (Chapter 3) before a single byte of HTTP flows, and if TLS is involved, the TLS handshake on top of that. For a page pulling dozens of resources, the connection setup cost dominated everything else.

HTTP/1.1 (1997, now RFC 9112) made **persistent connections** the default. The TCP connection stays open after a response completes and is reused for the next request. The `Connection: close` header opts out; `Connection: keep-alive` was the HTTP/1.0 opt-*in* to the same behavior. This one change amortizes the handshake across many requests and lets TCP's congestion window (Chapter 3) grow warm instead of restarting from the initial window on every request. It is the single most important performance property of HTTP/1.1 and the reason connection *pooling*, discussed later, exists at all.

Persistence created a framing problem. If the connection stays open, the receiver must know where one response ends and the next begins. With a known body size, `Content-Length` suffices. But servers often generate responses incrementally — streaming a large report, proxying an upstream — and cannot know the length up front. HTTP/1.1's answer is **chunked transfer encoding**: `Transfer-Encoding: chunked` tells the receiver the body arrives as a series of size-prefixed chunks, each a hex

length line followed by that many bytes, terminated by a zero-length chunk. This is what lets a server flush data as it produces it and still delimit the message on a reused connection.

```
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked

1c
{"status":"processing","id":
13
"a1b2c3d4e5f6a7b8"}
0
```

The ambiguity between `Content-Length` and `Transfer-Encoding`, and the disagreements between proxies about which one wins, is the mechanism behind **HTTP request smuggling** — a class of attack where a front-end and back-end parse the same byte stream into different request boundaries. RFC 9112 tightens the rules (a message with both headers must have `Content-Length` removed, and chunked must be the final encoding), but the vulnerability class persists in misconfigured proxy chains and is worth knowing exists.

Now the crucial failure. HTTP/1.1's persistent connection still handles **one request/response at a time, in order**. You send request A, and you cannot use that connection for request B until A's full response has come back. **HTTP pipelining** was the spec's attempt to fix this: a client may send A, B, C back-to-back without waiting, and the server must return their responses *in the same order*. In principle this fills the pipe. In practice it failed completely and is disabled in every mainstream browser. The reason is **head-of-line (HOL) blocking at layer 7**: because responses must come back in request order, a single slow response at the head of the line stalls every response queued behind it, even if those were ready first. Send a cheap `GET /style.css` behind an expensive `GET /report.pdf` and the CSS waits for the PDF. Buggy proxies that mis-ordered or dropped pipelined responses made it worse, and there was no safe way for a client to know whether an intermediary supported pipelining correctly. So the industry abandoned pipelining and reached for the only other lever available.

That lever was **parallel connections**. If one connection does one request at a time, open several. Browsers converged on a limit of roughly **six TCP connections per host** (the exact number varied by browser but six

became the de facto standard). Six connections means up to six requests genuinely in flight at once, at the cost of six TCP handshakes, six congestion windows that each start cold and compete with each other, six times the memory on the server, and — for pages with more than six resources — the same serialization problem shifted to whichever requests exceed the connection count. Sharding assets across multiple hostnames ("domain sharding") to multiply the six-per-host budget became a standard, ugly optimization. HTTP/1.1's concurrency story is fundamentally *you cannot multiplex on one connection, so you pay for several and hope six is enough*. Every subsequent HTTP version is, at its core, an attempt to get real multiplexing over a single connection.

```
flowchart LR
    subgraph H1["HTTP/1.1 – six connections per host"]
        C1["Client"]
        C1 -->|"conn 1: req A → resp A"| S1["Server"]
        C1 -->|"conn 2: req B → resp B"| S1
        C1 -->|"conn 3: req C → resp C"| S1
        C1 -.->|"conn 4-6 idle or reused"| S1
        C1 -->|"req G waits: no free conn (HOL at L7)"| S1
    end
    end
```

HTTP/2: real multiplexing over one connection

HTTP/2 (2015, RFC 7540, now RFC 9113) grew out of Google's SPDY experiment and keeps HTTP/1.1's semantics exactly while replacing the wire format wholesale. Where HTTP/1.1 is text, HTTP/2 is **binary**, and where HTTP/1.1 does one request at a time per connection, HTTP/2 multiplexes many concurrent requests over a **single TCP connection**. Almost everything about HTTP/2 follows from four mechanisms: the binary framing layer, streams, HPACK, and flow control.

The binary framing layer and streams

HTTP/2 slices the connection into **frames**. Every frame has a fixed 9-byte header — a 24-bit length, an 8-bit type, an 8-bit flags field, and a reserved bit followed by a 31-bit **stream identifier** (32 bits together) — followed by a type-specific payload. The frame types you care about are **HEADERS** (carries the HPACK-compressed request or response header block), **DATA** (carries body bytes), **SETTINGS** (negotiates connection parameters), **WINDOW_UPDATE** (flow control), **RST_STREAM** (abort one stream), **PRIORITY**, **PING**, **GOAWAY**, and **PUSH_PROMISE**.

A **stream** is an independent, bidirectional sequence of frames that all share one stream ID. Each request/response exchange gets its own stream. Client-initiated streams use odd IDs (1, 3, 5, ...), server-initiated streams use even IDs, and IDs only increase. Because every frame is tagged with its stream ID, the sender can **interleave** frames from many streams on the wire in any order, and the receiver reassembles each stream from the tagged frames. A `HEADERS` frame on stream 3 followed by a `DATA` frame on stream 5 followed by more `DATA` on stream 3 is perfectly legal. This is the multiplexing: dozens of requests share one connection, their frames interleaved, none blocking another *at the HTTP layer*. HTTP/1.1's L7 head-of-line blocking is gone — a slow response no longer stalls the ready ones, because their `DATA` frames simply interleave on the wire.

```
flowchart TB
    subgraph conn["Single TCP connection"]
        direction LR
        F1["HEADERS"]
        stream 1"] --> F2["HEADERS"]
        stream 3"] --> F3["DATA"]
        stream 1"] --> F4["DATA"]
        stream 3"] --> F5["DATA"]
        stream 1"] --> F6["HEADERS"]
        stream 5"]
    end
    conn --> R1["Stream 1 reassembled: GET /a"]
    conn --> R3["Stream 3 reassembled: GET /b"]
    conn --> R5["Stream 5 reassembled: GET /c"]
```

Concurrency is bounded by `SETTINGS_MAX_CONCURRENT_STREAMS`, which each peer advertises; 100 is a common default. This is what lets one HTTP/2 connection replace the browser's six HTTP/1.1 connections — and, in the backend world, lets a service mesh hold *one* long-lived connection between two services and run hundreds of concurrent RPCs over it. That property is the foundation of gRPC's efficiency (Chapter 8): gRPC is defined *on top of* HTTP/2 precisely so that many concurrent streaming RPCs share a single connection with no per-call handshake.

HPACK: header compression that had to be stateful and careful

HTTP headers are repetitive and verbose. Every request to your API likely repeats the same `user-agent`, `accept`, `authorization`, `cookie`,

and host information, often hundreds of bytes, on every one of hundreds of multiplexed requests. HTTP/1.1 sent all of it, uncompressed, every time. HTTP/2 introduces **HPACK (RFC 7541)**, a header compression scheme built specifically for this workload.

HPACK combines three things. First, a **static table** of 61 common header field entries (:method GET , :status 200 , accept-encoding gzip, deflate , and so on) that both peers know by index, so a common header becomes a single byte. Second, a **dynamic table**: as headers are sent, the encoder may insert field name/value pairs into a per-connection table that both sides maintain identically, so a repeated custom header (say a bearer token or a trace ID) is sent in full once and thereafter referenced by index. Third, **Huffman coding** of the literal strings that must be sent verbatim.

HPACK was deliberately designed *not* to use general-purpose compression like DEFLATE, and the reason is a security lesson worth carrying. SPDY originally compressed headers with DEFLATE, and that enabled **CRIME** (2012), an attack that recovered secret cookies by observing how compressed size changed when attacker-controlled request data matched the secret. HPACK's fixed-table-plus-Huffman design avoids the adaptive, cross-secret compression that CRIME exploited. This is a recurring theme: compression that mixes attacker-controlled and secret data in one context is dangerous, and HPACK is what careful looks like.

Flow control and prioritization

Because many streams share one TCP connection, HTTP/2 needs its own **flow control** so that one stream (or the connection as a whole) cannot exhaust a receiver's buffers and stall the others. Flow control is credit-based and applies to **DATA** frames only. Each receiver advertises a window per stream and a window for the whole connection; a sender may only send **DATA** up to the smaller remaining window, and the receiver replenishes credit by sending **WINDOW_UPDATE** frames as it consumes data. Misconfigured HTTP/2 flow-control windows are a real and subtle source of throughput problems: a default 65,535-byte window over a high-bandwidth-delay-product path throttles a single stream badly until the window is enlarged, exactly analogous to TCP's own window-scaling story from Chapter 3, but now duplicated at layer 7.

HTTP/2 also defined a **prioritization** scheme — a dependency tree with weights, letting a client say "load the CSS before the images." In practice this proved complex, inconsistently implemented, and often actively counterproductive; RFC 9113 deprecates the original priority signaling, and the ecosystem has moved to the simpler **RFC 9218 Extensible Prioritization Scheme** (the `priority` header field with `urgency` and `incremental` parameters), which HTTP/3 also uses. Treat original HTTP/2 priorities as a historical mistake.

Server push, and why it was deprecated

HTTP/2 included **server push**: via a `PUSH_PROMISE` frame, a server could proactively send a resource the client had not yet requested, on the theory that a server rendering `index.html` knows the client will next want `app.css` and could push it to save a round trip. In practice push was a persistent disappointment. Servers routinely pushed resources the client already had cached, wasting bandwidth; the interaction with the browser cache was hard to get right; and the latency benefit was marginal versus the simpler `103 Early Hints` / `preload-link` approach. Chrome removed HTTP/2 server push support in 2022; HTTP/3 (RFC 9114) does define a push mechanism, but it saw essentially no adoption. Server push is effectively dead; do not design around it. The lesson — that speculative pushing without knowing the client's cache state loses more than it wins — echoes the server-push-style deprecations elsewhere in the stack.

The wall HTTP/2 cannot climb: TCP head-of-line blocking

HTTP/2 eliminated head-of-line blocking *at the HTTP layer*. It could not eliminate it at the *transport* layer, and understanding exactly why is the single most important idea connecting this chapter to Chapters 3 and 4.

HTTP/2 multiplexes its streams over **one TCP connection**. TCP presents the application with a single, strictly **ordered byte stream**. TCP does not know anything about HTTP/2 streams — to TCP, the interleaved frames of streams 1, 3, and 5 are just one contiguous sequence of bytes with sequence numbers. TCP guarantees that it delivers those bytes to the application *in order* and *with no gaps*. That guarantee is the trap. If one TCP segment is lost — say the segment carrying bytes for stream 3's

DATA — TCP will *not* deliver any subsequent bytes to the application until that lost segment is retransmitted and arrives, **even though the later bytes belong to entirely different, healthy streams**. The kernel is holding stream 1's and stream 5's data in its receive buffer, complete and correct, but it cannot hand them up because doing so would violate in-order delivery of the byte stream, and it has no idea those bytes are logically independent.

So a single packet loss stalls *every* multiplexed HTTP/2 stream until one retransmission completes a round trip. On a clean datacenter link this is rare and cheap. On a lossy mobile or long-haul path it is devastating, and — this is the cruel part — it gets *worse* the more you multiplex, because more streams are riding the one connection that just stalled. HTTP/1.1's six separate connections actually isolated loss: a lost packet on connection 2 stalled only connection 2. HTTP/2 traded that isolation for multiplexing efficiency, and on lossy paths the trade can lose. This is **TCP-level head-of-line blocking**, and no change to HTTP/2 can fix it, because the problem is not in HTTP/2 at all — it is in the transport's single-ordered-stream abstraction that HTTP/2 is obligated to sit on top of.

```
sequenceDiagram
    participant N as Network
    participant K as TCP receive buffer
    participant A as HTTP/2 layer
    Note over N,A: Segment carrying stream 3 data is LOST
    N->>K: seg 1 (stream 1 DATA) ✓
    N->>K: seg 2 (stream 3 DATA) ✗ lost
    N->>K: seg 3 (stream 5 DATA) ✓ buffered
    N->>K: seg 4 (stream 1 DATA) ✓ buffered
    Note over K: In-order rule: cannot deliver seg 3,4
    until seg 2 arrives
        K-->>A: streams 1 and 5 STALLED (HOL)
        N->>K: retransmit seg 2 ✓ (one RTT later)
        K->>A: now deliver seg 2,3,4 in order
```

This is the exact problem QUIC was built to solve, and it is why HTTP/3 exists.

HTTP/3: HTTP over QUIC, with per-stream reliability

HTTP/3 (2022, RFC 9114) is HTTP semantics carried over **QUIC** (Chapter 4) instead of TCP+TLS. It keeps HTTP/2's conceptual model —

binary frames, multiplexed streams, header compression, request/response mapping — but relocates the stream machinery from the HTTP layer into the transport, and that relocation is the whole game.

QUIC runs over UDP and implements reliability, ordering, congestion control, and TLS 1.3 encryption in user space, but critically it makes **streams first-class in the transport**. QUIC knows about streams; TCP does not. QUIC guarantees reliable, ordered delivery **within each stream independently**, and imposes no ordering *between* streams. So when a UDP packet carrying stream 3's data is lost, QUIC retransmits it and stalls *only stream 3*. Streams 1 and 5, whose packets arrived, are delivered to the HTTP/3 layer immediately, because QUIC knows they are independent and there is no cross-stream in-order rule to violate. **Per-stream reliability eliminates the transport-level head-of-line blocking** that HTTP/2 could never escape. This is the single most important reason HTTP/3 exists, and everything else it offers is a bonus.

```
sequenceDiagram
    participant N as Network
    participant Q as QUIC (per-stream)
    participant A as HTTP/3 layer
    Note over N,A: Packet carrying stream 3 data is LOST
    N->>Q: pkt (stream 1 data) ✓
    N->>Q: pkt (stream 3 data) x lost
    N->>Q: pkt (stream 5 data) ✓
    Q->>A: deliver stream 1 immediately
    Q->>A: deliver stream 5 immediately
    Note over Q,A: only stream 3 waits
    N->>Q: retransmit stream 3 pkt ✓
    Q->>A: deliver stream 3
```

The other HTTP/3 improvements follow from QUIC's design, covered in depth in Chapter 4, so we summarize their HTTP relevance:

- **Faster connection setup.** QUIC integrates the transport and TLS 1.3 handshakes into one, so a new connection reaches encrypted data in a single round trip (1-RTT), and a resumed connection can send data in **0-RTT** with the first packet. TCP+TLS costs a TCP handshake *then* a TLS handshake — two round trips before HTTP/2 data flows on a fresh connection. For a mobile client with 100 ms RTT opening a connection to a CDN edge, this is a visible latency win on the first request.

- **Connection migration.** A QUIC connection is identified by a **connection ID**, not by the 4-tuple of IP addresses and ports. When a phone moves from Wi-Fi to cellular and its source IP changes, the QUIC connection survives — the client keeps using the same connection ID and the server recognizes it — where a TCP connection would have broken and forced a full reconnect. For long-lived HTTP/3 connections to mobile clients this removes a whole category of reconnection latency.
- **QPACK instead of HPACK.** HTTP/3 cannot use HPACK, and the reason is instructive. HPACK's dynamic table assumes headers arrive in the exact order they were encoded, because both sides mutate the table in lockstep — an assumption TCP's total ordering guarantees. But QUIC delivers streams out of order relative to each other, so a **HEADERS** block on stream 7 might arrive before the block on stream 3 that added the dynamic-table entry stream 7 references. **QPACK (RFC 9204)** solves this by carrying dynamic-table updates on a dedicated unidirectional stream and letting each header block declare the table state it depends on, blocking that one block if the referenced insertions have not yet arrived. It is HPACK adapted to a world without global ordering — the same design tension, resolved for a transport that no longer serializes everything.

HTTP/3 is negotiated not by ALPN-on-a-known-port (there is no prior TCP connection to run ALPN over) but via the **Alt-Svc header** or the **HTTPS DNS record (SVCB)**: a server reachable over HTTP/1.1 or HTTP/2 advertises `Alt-Svc: h3=":443"`, and the client, having connected over TCP first, upgrades subsequent connections to HTTP/3 over UDP. The costs are QUIC's costs from Chapter 4: UDP is sometimes blocked or deprioritized by middleboxes and firewalls, and a user-space transport burns more CPU per byte than the kernel's TCP, which matters at fleet scale.

The three versions side by side

Property	HTTP/1.1 (RFC 9112)	HTTP/2 (RFC 9113)	HTTP/3 (RFC 9114)
Wire format	Text, CRLF	Binary frames	Binary frames over QUIC

Property	HTTP/1.1 (RFC 9112)	HTTP/2 (RFC 9113)	HTTP/3 (RFC 9114)
Transport	TCP	TCP	QUIC over UDP
Encryption	TLS optional (separate)	TLS in practice (ALPN h2)	TLS 1.3 integral to QUIC
Multiplexing	No (1 req/conn at a time)	Yes, over 1 TCP connection	Yes, over 1 QUIC connection
L7 head-of-line blocking	Yes (pipelining)	No	No
Transport head-of-line blocking	Per-connection	Yes (TCP, all streams)	No (per-stream)
Header compression	None	HPACK (RFC 7541)	QPACK (RFC 9204)
Connection setup RTTs	TCP + TLS (~2-3)	TCP + TLS (~2-3)	1-RTT, 0-RTT on resume
Connection migration	No	No	Yes (connection ID)
Server push	No	Yes (deprecated/removed)	Not carried forward
Concurrency workaround	~6 connections/host	1 connection	1 connection

Practical backend concern: connection management and pooling

The version you speak changes your connection-management strategy completely, and getting it wrong wastes fleet resources or adds tail latency.

HTTP/1.1 needs a pool of connections. Because one HTTP/1.1 connection carries one request at a time, a client that wants concurrency must hold *many* connections open to each upstream. Every HTTP client library exposes this: Go's `http.Transport` has `MaxIdleConnsPerHost` (default 2, almost always too low for a busy service — raise it) and `MaxConnsPerHost`; a JVM connection pool has a max-per-route; a Python

`requests` session backed by `urllib3` has `pool_maxsize`. The pool exists to amortize TCP and TLS handshakes and keep warm congestion windows. Three failure modes dominate. First, **pool too small**: requests queue waiting for a free connection, adding latency invisible in server-side metrics because the request has not left the client yet. Second, **idle connections reaped by a middlebox**: a stateful firewall or L4 load balancer silently drops an idle TCP flow after some timeout, and the *next* request on that pooled connection fails with a connection reset because the client thought it was still open — this is the classic "works fine under load, fails after a quiet period" bug, fixed by setting the client's idle timeout *below* the network's and enabling TCP keep-alives. Third, **keep-alive timeout races**: the server closes an idle connection at the same instant the client sends a request on it, producing a sporadic reset; the fix is to make the client's idle timeout comfortably shorter than the server's, so the client always initiates the close.

HTTP/2 collapses the pool to one connection — and that is the gotcha. Because one HTTP/2 connection multiplexes up to `MAX_CONCURRENT_STREAMS` requests, a client typically opens *one* connection per upstream and runs everything over it. For microservices and gRPC this is a huge win: no per-call handshake, warm window, minimal memory. But it collides violently with **layer-4 (connection-level) load balancing**.

An L4 load balancer distributes *connections*, not requests. It picks a backend when the TCP connection is established and pins every byte of that connection to that backend. With HTTP/1.1 and a pool of, say, 20 connections, those 20 connections spread across 20 backends and traffic is roughly even. With HTTP/2 and its **single** connection, *all* of that client's requests — the entire multiplexed stream of them — ride one TCP connection and therefore land on **one** backend. The L4 load balancer, seeing one connection, has nothing to balance. Ten client instances each open one HTTP/2 connection and you get at most ten backends receiving traffic no matter how many backends exist, and if the balancer's hashing is uneven, some backends get hammered while others idle. Worse, a backend that just started (autoscaling up) receives no traffic because no *new connections* are being made — existing clients keep using their long-lived connections to the old backends. This is **HTTP/2 connection pinning**, and it is one of the most common production surprises when teams adopt gRPC.

```

flowchart TB
    subgraph bad["L4 load balancer + HTTP/2: pinning"]
        CA["Client A (1 H2 conn)"] --> LB4["L4 LB (per-connection)"]
        CB["Client B (1 H2 conn)"] --> LB4
        LB4 -->|"all A requests"| B1["Backend 1 (hot)"]
        LB4 -->|"all B requests"| B1
        LB4 -->|"no new conns"| B2["Backend 2 (idle)"]
        LB4 --> B3["Backend 3 (idle)"]
    end
    end
    subgraph good["L7 load balancer + HTTP/2: per-request"]
        CC["Client C (1 H2 conn)"] --> LB7["L7 LB (per-request/
stream)"]
        LB7 --> D1["Backend 1"]
        LB7 --> D2["Backend 2"]
        LB7 --> D3["Backend 3"]
    end
    end

```

There are three standard fixes, and you will meet all of them (see Chapter 9 on load balancing). First and best: **use an L7 (application-aware) load balancer** — Envoy, an Ingress controller, a service-mesh sidecar — that terminates HTTP/2, sees individual *streams/requests*, and load-balances each request across backends. This is the primary reason gRPC deployments require L7 proxies, not just an L4 TCP passthrough. Second: **client-side load balancing**, where the client resolves all backend endpoints and itself opens one HTTP/2 connection *per backend*, spreading requests across them — gRPC's `round_robin` policy over a `dns:///` or xDS resolver does exactly this. Third: bound connection lifetime with `MAX_CONNECTION_AGE` (gRPC server-side) so backends periodically send `GOAWAY` and clients re-resolve and reconnect, letting new backends enter rotation and preventing indefinite pinning. The `GOAWAY` frame is the graceful-drain primitive here: it tells the peer "finish your in-flight streams but open no new ones on this connection," which is exactly what you want during a rolling deploy so no request is cut off mid-flight.

For **HTTP/3** the connection model is HTTP/2's — one multiplexed connection — so the same L4 pinning logic applies, with the additional wrinkle that QUIC connections are keyed by connection ID, not 4-tuple, so an L4 balancer that hashes on the 4-tuple will misroute migrated connections unless it is QUIC-aware and routes on connection ID.

HTTP semantics that matter operationally

The version wars are about transport efficiency. The semantics below (RFC 9110/9111) are version-independent and are where your API's correctness and cacheability actually live.

Idempotency, retries, and the transport interaction

A resilience layer retries failed requests, and it may only *safely* retry **idempotent** ones (Chapter 11). `GET`, `PUT`, and `DELETE` are idempotent by definition; retrying them cannot corrupt state. `POST` is not idempotent — a retried `POST /payments` may charge a card twice — so a blind retry layer must either exclude `POST` or require an **idempotency key** (a client-generated unique header the server uses to deduplicate, the pattern every serious payments API uses). There is a subtle transport interaction: even an idempotent request is unsafe to retry if you cannot tell whether the server *processed* it before the connection broke. HTTP's method semantics let the *client* decide retry-safety, but the *ambiguity* of a mid-flight failure — did the `DELETE` land before the reset? — is what idempotency keys and careful `4xx / 5xx` handling exist to resolve. The `Retry-After` header (on `429` and `503`) is the server telling the client how long to back off; honoring it is the difference between a retry storm that deepens an outage and a controlled recovery.

Caching and conditional requests

`Cache-Control` is the primary caching directive: `max-age`, `no-store`, `no-cache` (revalidate before use, *not* "don't cache"), `private` versus `public`, `s-maxage` for shared caches like CDNs. **Validators** enable conditional requests: an `ETag` (an opaque version tag) or `Last-Modified` on a response lets a client later send `If-None-Match` / `If-Modified-Since`, and the server answers `304 Not Modified` with no body if the cached copy is still fresh — saving the payload but not the round trip. This machinery is what makes a CDN (Chapter 10) effective and is identical across all three HTTP versions; the transport carries the same `ETag`, whether as a text header or an HPACK/QPACK-compressed field.

Content negotiation and compression

`Accept`, `Accept-Language`, and `Accept-Encoding` let a client state preferences and the server select a representation, echoing its choice in `Content-Type`, `Content-Language`, and `Content-Encoding`, and declaring

Vary so caches key correctly on the negotiated dimension. Compression via Content-Encoding: gzip (or br for Brotli, or zstd) is negotiated this way and is orthogonal to HTTP/2's header compression — HPACK compresses headers, Content-Encoding compresses bodies. Be aware that body compression over TLS reintroduces the CRIME/BREACH risk when secret and attacker-controlled data share a compressed body; mitigations exist but the interaction is worth knowing.

Range requests, Host, and cookies

Range: bytes=0-1023 requests a partial representation; the server answers 206 Partial Content with a Content-Range, or 416 if the range is unsatisfiable. This underpins resumable downloads, video seeking, and parallel chunked fetches from object storage. The Host header (mandatory in HTTP/1.1; the :authority pseudo-header in HTTP/2 and HTTP/3) enables virtual hosting — many domains served from one IP, the server routing by Host — and is what lets an Ingress or reverse proxy route api.example.com and www.example.com on the same listener; it is also, via the :authority, how an L7 proxy decides where a request goes. Cookies (Set-Cookie / Cookie, RFC 6265) carry session state; their operational relevance here is that they are large, sent on every request, and thus a prime beneficiary of HPACK/QPACK dynamic-table compression — and a prime source of request bloat when they are not.

Distributed-systems lens

Zoom out to the fleet, and the through-line is that **HTTP version choice is a transport decision with architectural consequences, made safe by the semantics-versus-transport split.**

HTTP/2 multiplexing is why gRPC is efficient (Chapter 8). gRPC is not "HTTP with protobuf"; it is a protocol *defined on HTTP/2 streams*, one RPC per stream, using HTTP/2's multiplexing to run hundreds of concurrent and streaming calls over a single connection with no per-call handshake. Take away HTTP/2's binary framing and multiplexing and gRPC's efficiency evaporates. This is the clearest example of an application protocol being co-designed with the HTTP version beneath it.

The HTTP/2 pinning problem forces L7 load balancing (Chapter 9). The single long-lived connection that makes HTTP/2 efficient is exactly what defeats connection-level load balancing. At fleet scale this is not a

curiosity — it is the reason your gRPC mesh needs Envoy sidecars or client-side load balancing, the reason a naive "put gRPC behind an L4 TCP load balancer" produces one hot backend and an autoscaler that never scales because new pods get no connections. Every mature microservice platform has made this decision consciously.

Connection pooling strategy is a fleet-latency and resource lever.

Multiply a suboptimal pool setting by thousands of instances and the effect is enormous: pools too small add invisible client-side queuing latency to your p99; pools too large exhaust file descriptors and ephemeral ports and multiply server memory. The idle-timeout-versus-middlebox-timeout mismatch is one of the most common sources of intermittent 502s and connection resets in production, and it is purely a connection-management problem, not an application bug.

HTTP/3 and QUIC benefit edge and mobile clients most (Chapter 10).

In a datacenter with sub-millisecond RTT and near-zero loss, HTTP/3's advantages — no transport HOL blocking, faster setup, migration — are marginal, and QUIC's user-space CPU cost may not be worth it for service-to-service traffic; HTTP/2 over TCP is usually the right internal choice. HTTP/3's wins concentrate at the **edge**: mobile clients on lossy, changing networks talking to a CDN, where per-stream reliability and connection migration turn a jittery, reconnecting experience into a smooth one. This is why CDNs and consumer front doors adopted HTTP/3 aggressively while internal service meshes largely did not — different RTT and loss regimes justify different transports for the *same* semantics.

The semantics-versus-transport split is what makes all of this safe.

Because RFC 9110 semantics are invariant, an edge proxy can terminate HTTP/3 from a phone and speak HTTP/2 to a sidecar and HTTP/1.1 to a legacy backend, each hop choosing the transport that fits its network, and the *application message* is preserved end to end. Your service code, written against methods, status codes, and headers, does not know or care which wire format delivered it. That is the payoff of the layering: you get to change transports per hop, per network regime, per year, without touching the API contract.

Key takeaways

- **Semantics (RFC 9110) are separate from wire format (9112/9113/9114).** Methods, status codes, headers, caching,

idempotency are version-independent; the same API runs unchanged over HTTP/1.1, HTTP/2, and HTTP/3, and hops may use different transports.

- **HTTP/1.1 cannot multiplex on one connection.** Pipelining failed to L7 head-of-line blocking (in-order responses), so browsers use ~6 connections per host and backends use connection pools.
- **HTTP/2 multiplexes many streams over one TCP connection** via a binary framing layer, with HPACK header compression and per-stream/connection flow control. Server push and the original priority scheme are deprecated/removed.
- **HTTP/2 still suffers TCP-level head-of-line blocking:** one lost segment stalls *all* multiplexed streams, because TCP delivers a single ordered byte stream and cannot know the streams are independent. This is unfixable within HTTP/2.
- **HTTP/3 over QUIC gives each stream independent reliability**, eliminating transport HOL blocking, plus 1-RTT/0-RTT setup, connection migration, and QPACK (HPACK adapted to out-of-order streams). Its wins concentrate on lossy/mobile/edge paths.
- **HTTP/2's single connection collides with L4 load balancing** (connection pinning → hot backends, no traffic to new pods). Fix with L7 load balancing, client-side load balancing, or bounded connection age with `GOAWAY`.
- **Connection pooling and keep-alive timeouts are a fleet-scale lever:** too-small pools add invisible client latency; idle-timeout-versus-middlebox mismatches cause intermittent resets.
- **Only idempotent methods are safely retryable;** use idempotency keys for `POST`, honor `Retry-After`, and distinguish `4xx` (don't retry) from `5xx` (may retry).

Further reading

- RFC 9110 — *HTTP Semantics* (2022), the version-independent core. <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 9111 — *HTTP Caching* (2022). <https://www.rfc-editor.org/rfc/rfc9111>
- RFC 9112 — *HTTP/1.1* (2022, message syntax). <https://www.rfc-editor.org/rfc/rfc9112>

- RFC 9113 — *HTTP/2* (2022, obsoletes RFC 7540). <https://www.rfc-editor.org/rfc/rfc9113>
- RFC 9114 — *HTTP/3* (2022). <https://www.rfc-editor.org/rfc/rfc9114>
- RFC 7541 — *HPACK: Header Compression for HTTP/2*. <https://www.rfc-editor.org/rfc/rfc7541>
- RFC 9204 — *QPACK: Field Compression for HTTP/3*. <https://www.rfc-editor.org/rfc/rfc9204>
- RFC 9218 — *Extensible Prioritization Scheme for HTTP*. <https://www.rfc-editor.org/rfc/rfc9218>
- RFC 9000 — *QUIC: A UDP-Based Multiplexed and Secure Transport* (see also Chapter 4). <https://www.rfc-editor.org/rfc/rfc9000>
- RFC 6265 — *HTTP State Management Mechanism* (cookies); RFC 8446 — *TLS 1.3*.
- Ilya Grigorik, *High Performance Browser Networking*, O'Reilly, 2013 — free online at <https://hpbn.co>; the chapters on HTTP/1.x, HTTP/2, and connection management remain an excellent mechanism-level treatment.
- Daniel Stenberg, "HTTP/3 Explained" — <https://http3-explained.haxx.se> — an open, maintained explainer of HTTP/3 and QUIC by the curl author and community.
- Envoy Proxy documentation on HTTP/2 upstream connection management and load balancing (<https://www.envoyproxy.io/docs>) and the gRPC blog post "gRPC Load Balancing" (<https://grpc.io/blog/grpc-load-balancing/>) — the authoritative operational treatment of the HTTP/2 pinning problem and its fixes.
- J. Kelsey, "Compression and Information Leakage of Plaintext," FSE 2002, and the CRIME/BREACH disclosures — the background for why HPACK avoids general-purpose header compression.

Chapter 8 — gRPC and RPC Framework Internals

What this chapter covers. When one of your services calls another, it almost never opens a raw socket and speaks a hand-rolled protocol. It calls a *stub* — a generated object with the same method signatures as the

remote service — and the stub does the unglamorous work of turning a function call into bytes, pushing those bytes across the network, and turning the reply back into a return value. That machinery is a Remote Procedure Call framework, and at scale the default one is gRPC over HTTP/2 carrying Protocol Buffers. This chapter is about what actually happens underneath that stub. We will take the RPC idea apart — its history, its recurring appeal, and the lie at its center (that a remote call resembles a local one). We will go deep on serialization, because the wire format *is* the performance story and the schema *is* the service contract: Protocol Buffers' tag-length-value encoding, varints, and the compatibility rules that let a hundred teams evolve a hundred services independently. Then gRPC proper: why it sits on HTTP/2 (Chapter 7), the four call shapes, metadata and status codes, the deadline propagation and cancellation that keep tail latency and cascading failures in check (Chapter 11), interceptors, channels, and the client-side and subchannel load balancing that resolve the HTTP/2 connection-pinning problem (Chapters 7 and 9). We will place gRPC against REST-over-JSON honestly, cover the browser gap and the newer frameworks (Connect, Twirp, tRPC), and treat schema evolution as the operational discipline it is (Volume 8).

Learning goals — after this chapter you should be able to:

- Explain the RPC model, its lineage (Sun RPC, DCE, CORBA, Thrift, gRPC), and why the "make-remote-look-local" abstraction is both the reason RPC keeps returning and the reason it keeps hurting people — grounded in the Fallacies of Distributed Computing.
- Read and reason about the Protocol Buffers wire format at the byte level: field tags, wire types, varint encoding, length-delimited fields, and the exact rules that make a change backward- and forward-compatible.
- Describe gRPC's mapping onto HTTP/2 — one stream per call, HEADERS/DATA/trailers, length-prefixed message framing — and why HTTP/2 multiplexing, streaming, and binary framing are what make gRPC efficient.
- Distinguish the four call types (unary, server-streaming, client-streaming, bidirectional) and choose correctly among them.
- Configure deadlines, propagation, cancellation, retries, and hedging, and explain why each is load-bearing for a healthy fleet.

- Solve gRPC load balancing: name resolution, client-side vs look-aside LB, and subchannel/L7 balancing as the fix for HTTP/2 backend pinning.
- Decide between gRPC and REST/JSON for a given surface, and know what gRPC-Web and Connect do about the browser.

The RPC model and why it keeps coming back

A Remote Procedure Call framework exists to make a call to another process — usually on another machine — look, in your source code, like a call to a local function. You write `user, err := client.GetUser(ctx, &GetUserRequest{Id: 42})` and something on the other side of a network runs `GetUser` and sends back a `User`. The framework generates a **client stub** that presents the local-looking signature, **marshals** (serializes) the arguments into a byte stream, ships them over a transport, and on the far side a **server skeleton** (or dispatcher) **unmarshals** them, invokes the real method, marshals the return value, and sends it back. The stub then unmarshals the response and returns it to your call site. The whole apparatus is a productivity argument: distributed systems are made of many small request/response interactions, and forcing every one of them to be hand-serialized and hand-dispatched would be intolerable.

The idea is old and has been reinvented repeatedly. The canonical academic statement is Birrell and Nelson's 1984 paper "Implementing Remote Procedure Calls," which introduced the stub/binding model that everything since has followed. Sun's ONC RPC (the machinery under NFS, standardized as RFC 1057 and later RFC 5531) gave the industry a widely deployed implementation with its own IDL, XDR (External Data Representation), for describing the wire encoding. DCE/RPC in the early 1990s generalized it; Microsoft's DCOM and the Open Group's CORBA pushed RPC into object-oriented, cross-language, cross-vendor territory with heavyweight IDLs and ORBs (Object Request Brokers). CORBA's IIOP was a genuine binary RPC protocol, but the ecosystem collapsed under its own complexity and vendor incompatibility. The 2000s brought a reaction toward text and simplicity — XML-RPC, then SOAP (still RPC, now drowning in XML and WS-* specifications), then the pendulum swung hard to REST, which is deliberately *not* RPC. Meanwhile, inside large companies, binary RPC never left: Facebook built and open-sourced **Thrift** (2007, later Apache Thrift) with its own IDL, serializer, and pluggable transports; Google built **Stubby** internally and, in 2015, open-

sourced a cleaned-up successor as **gRPC**. Twitter built **Finagle**. The names change; the stub/IDL/marshal skeleton does not.

Why does RPC keep returning? Because the alternative — every team inventing an ad-hoc HTTP-plus-JSON convention per endpoint — has real costs that a framework amortizes: no shared contract, no generated clients, hand-written (and inconsistent) serialization, no uniform deadline/retry/auth handling, and no schema to catch a breaking change before it reaches production. RPC frameworks package all of that. The mistake is to conclude that because the *ergonomics* are local, the *semantics* are local. They are not, and the gap between the two is where most distributed-systems pain lives.

The fallacy at the center: RPC is not a local call

A local function call cannot partially fail. It returns, it panics/throws, or the process dies — but it does not sit in an ambiguous state for 900 milliseconds and then maybe return, maybe not, while you have no way to know whether the callee ran. A remote call can do exactly that. The network between caller and callee is a first-class, misbehaving component, and pretending otherwise is the original sin of RPC. This is precisely the territory of the **Fallacies of Distributed Computing** — the list attributed to Peter Deutsch and colleagues at Sun in the mid-1990s (with the eighth added by James Gosling): *the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology doesn't change; there is one administrator; transport cost is zero; the network is homogeneous*. Every one of these is a way that a remote call differs from a local one, and every one of them is a way that a naive RPC abstraction lies to you.

The consequences are concrete. Because the network is unreliable, an RPC has *three* outcomes, not two: success, definite failure, and **unknown** — the request may have been fully processed while the response was lost. That third outcome is why idempotency and idempotency keys matter (Chapter 7) and why retries are dangerous for non-idempotent operations (Chapter 11). Because latency is not zero, a call that is trivial locally (a getter) becomes a network round trip, and a loop that makes one call per item becomes an $N+1$ latency disaster; RPC's local-looking syntax actively hides this, which is why chatty RPC interfaces are a recurring performance anti-pattern. Because the network is not secure, every hop needs authentication and usually encryption (TLS, Chapter 6), which a

local call never needed. A good RPC framework does not hide these realities — it gives you first-class tools to manage them: explicit deadlines, explicit cancellation, status codes that distinguish "I definitely failed" from "I timed out and don't know," and retry policies you configure rather than get by accident. The rest of this chapter is, in large part, about those tools.

Serialization and the IDL: the contract is the schema

Before a call can cross a network, its arguments must become bytes, and the two processes must agree on how to read those bytes back. That agreement is the **serialization format**, and the machine-readable description of the messages and services is the **Interface Definition Language**. The IDL is not a convenience; in a fleet of hundreds of services owned by dozens of teams, the IDL is the contract between producer and consumer. It is the thing that a build can check, that a linter can gate on for breaking changes, and that generates the client so consumers cannot accidentally send a malformed request. Getting the serialization and IDL right is therefore not a micro-optimization — it determines whether services can evolve independently (the central theme of Volume 8, APIs and contracts).

Protocol Buffers in depth

Protocol Buffers ("protobuf") is Google's schema-driven binary serialization format and the default payload for gRPC. You describe messages and services in a `.proto` file; a compiler (`protoc`, or language-specific plugins) generates types and (de)serialization code. Here is a realistic schema in proto3, the current dialect:

```

syntax = "proto3";

package acme.users.v1;

option go_package = "github.com/acme/apis/gen/users/v1;usersv1";

import "google/protobuf/timestamp.proto";

// A user record. Field numbers are the wire identity – never reuse
them.
message User {
    int64 id          = 1;
    string email      = 2;
    string display_name = 3;
    Status status     = 4;
    google.protobuf.Timestamp created_at = 5;

    repeated string roles = 6;    // 0..N values

    reserved 7,
8;          // retired fields; numbers must never be reused
    reserved "legacy_handle";    // retired field name

    enum Status {
        STATUS_UNSPECIFIED = 0;    // proto3 requires a zero default
        STATUS_ACTIVE      = 1;
        STATUS_SUSPENDED   = 2;
    }
}

message GetUserRequest { int64 id = 1; }
message GetUserResponse { User user = 1; }

service UserService {
    rpc GetUser(GetUserRequest) returns (GetUserResponse);
}

```

The thing to internalize is that the **field number**, not the field name, is what goes on the wire. The name `email` exists only in source; the byte stream identifies that field as number 2. This is the whole basis of protobuf's compatibility model, so it is worth understanding the encoding exactly.

The wire format: tag-length-value and varints

A serialized protobuf message is a flat concatenation of fields, each written as a **key (tag)** followed by a **value**. The key is a single varint that

packs two things: `key = (field_number << 3) | wire_type`. The low three bits are the **wire type**, which tells a parser how to read the value that follows; the remaining bits are the field number. There are six wire types in current protobuf, but the ones you meet constantly are:

Wire type	Number	Used for
VARINT	0	<code>int32</code> , <code>int64</code> , <code>uint32</code> , <code>uint64</code> , <code>bool</code> , <code>enum</code> , <code>sint*</code>
I64	1	<code>fixed64</code> , <code>sfixed64</code> , <code>double</code>
LEN	2	<code>string</code> , <code>bytes</code> , embedded messages, packed repeated fields
I32	5	<code>fixed32</code> , <code>sfixed32</code> , <code>float</code>

A **varint** (variable-length integer) is protobuf's core trick for making small numbers cheap. Each byte uses its low seven bits for data and its high bit as a "more bytes follow" continuation flag; bytes are little-endian in group order. So the value 1 is one byte (`0x01`); values up to 127 fit in one byte; 128-16383 in two; and so on. This is why field numbers 1-15 are precious: their tag fits in a single byte (the tag is itself a varint), so you assign 1-15 to the fields that appear most often. The number 150 encodes as two bytes: `0x96 0x01`. Working it through — 150 in binary is `1001 0110`; you take the low 7 bits (`001 0110` = `0x16`) and set the continuation bit ($\rightarrow$ `0x96`), then the next 7 bits (`0000001` = `0x01`) with the continuation bit clear ($\rightarrow$ `0x01`).

Putting a whole message on the wire: consider a message with `int64 id = 1` set to 150 and `string email = 2` set to `"a@b.co"`. The bytes are:

```
08 96 01          field 1, wire type 0 (varint): 0x08 = (1<<3)|0,
value 150 = 96 01
12 06 61 40 62 2e 63 6f  field 2, wire type 2 (LEN): 0x12 = (2<<3)|2,
length 6, "a@b.co"
```

Two properties fall out immediately and matter enormously in practice. First, **fields that are unset are simply absent** — there is no null marker, no key, nothing. A message with three unset fields is zero bytes. In proto3 this also means a scalar field set to its zero value (0, empty

string, false) is, by default, indistinguishable on the wire from an unset field; both are absent. (If you need to tell "explicitly zero" from "unset," you use the `optional` keyword, which brings back explicit presence tracking, or a wrapper message.) Second, **a parser that encounters an unknown field number can skip it**, because the wire type in the tag tells it exactly how many bytes to consume — a varint (read until the continuation bit clears), a fixed 4 or 8 bytes, or a length-prefixed blob whose length it just read. This skip-the-unknown property is the mechanical basis of forward compatibility: an old binary can parse a message written by a newer schema, ignoring fields it does not understand, and (crucially) re-serialize them intact if it preserves unknown fields, which most implementations do.

The diagram below shows the byte layout for one length-delimited and one varint field.

```
flowchart LR
    subgraph msg["Serialized message bytes"]
        direction LR
        A["tag 0x08  
(field 1, varint)"] --> B["0x96 0x01  
value 150"]
        B --> C["tag 0x12  
(field 2, LEN)"]
        C --> D["0x06  
length = 6"]
        D --> E["61 40 62 2e 63 6f  
"a@b.co""]
    end
    end
```

Compatibility rules: the discipline that lets services evolve

Because the wire identity is the field number and unknown fields are skippable, protobuf gives you a precise, enforceable set of rules for evolving a schema without breaking either older readers (backward compatibility) or newer readers reading older data (forward compatibility). Learn these as rules, because your CI should enforce them:

- **Never change a field's number**, and **never reuse a retired number** for a different meaning. Reusing a number means old serialized data (in a queue, a log, a database column) will be misinterpreted by new code as the new field, silently corrupting data.

When you delete a field, add its number (and ideally its name) to a `reserved` clause so the compiler forbids anyone from reusing it.

- **Adding a field is safe** as long as it gets a new number. Old readers skip it; new readers see it, or see its default if an old writer did not set it.
- **Renaming a field is free on the wire** (the name is not transmitted) but breaks source code and JSON mappings, so treat it as a source-level change, not a wire change.
- **Changing a field's type is usually breaking.** Some pairs are wire-compatible because they share a wire type (e.g., `int32`, `int64`, `uint32`, `uint64`, `bool`, and `enum` are all varints and interconvert with defined truncation semantics; `sint32`/`sint64` use zigzag and are *not* compatible with the plain int types). Most changes are not safe; assume breaking unless you have checked the spec.
- **Enums are open in proto3:** an unknown enum value is preserved as its integer, so adding enum values is backward-compatible — but code must handle the unknown/`UNSPECIFIED` case, which is why the zero value is conventionally `*_UNSPECIFIED`.

These rules are what let a producer team add a field on Monday and a dozen consumer teams pick it up whenever they redeploy, with no coordinated release. That decoupling is the entire economic argument for schema-driven RPC at scale, and tools like Buf (`buf breaking`) exist to fail a pull request that violates any of the above.

Protobuf versus JSON, and the other binary formats

JSON is the format you reach for when a human needs to read the payload or when the consumer is a browser or a third party with no build-time schema. Its costs relative to protobuf are real: it is **schemaless on the wire** (every message re-transmits field names as strings, so it is larger), it is **text** (numbers are ASCII, requiring parse/format, and there is no native binary type — bytes must be base64-encoded, inflating them ~33%), and it has **no built-in evolution contract** (nothing stops a producer from renaming a field and breaking consumers). Its virtues are equally real: it is human-readable, universally supported, debuggable with `curl` and `jq`, and needs no code generation. For a rough sense of scale, a typical structured record is often several times smaller in protobuf than in JSON and parses meaningfully faster, but treat any

specific multiplier as workload-dependent and measure your own payloads rather than trusting a headline number.

The other binary formats each occupy a niche:

Format	IDL/schema	Distinguishing property	Fits when
Protocol Buffers	required (<code>.proto</code>)	compact TLV, strong evolution rules, gRPC-native	internal service RPC, durable messages
Apache Thrift	required (<code>.thrift</code>)	serializer + transport + server all in one stack	polyglot RPC where you want the whole stack from one project
Apache Avro	schema (JSON) travels or is registered	schema resolution at read time; great for data files/streams	Kafka + Schema Registry, analytics/columnar pipelines
Cap'n Proto	required	zero-copy: in-memory layout <i>is</i> the wire layout, no parse step	ultra-low-latency, mmap'd data, RPC with promise pipelining
FlatBuffers	required	zero-copy random access without unpacking	games, mobile, reading one field of a huge message

The dividing line worth remembering: protobuf and Thrift **parse** into language objects (a copy and allocation step); Cap'n Proto and FlatBuffers are **zero-copy**, letting you read fields directly out of the received buffer with no deserialization pass, which trades a rigid, pointer-based layout and larger encoded size for eliminating parse cost and allocation. Avro is the odd one out: it is optimized for data-at-rest and streaming where a **schema registry** stores schemas out of band and readers resolve a writer's schema against their own — which is exactly what you want for Kafka topics (Volume 4/5 territory) and exactly the wrong shape for per-call RPC.

gRPC in depth

gRPC is Google's open-source RPC framework: protobuf messages (by default) carried over HTTP/2, with a well-specified mapping of calls, metadata, status, deadlines, and streaming onto HTTP/2's primitives. Understanding gRPC is largely understanding that mapping.

Why HTTP/2 is the substrate

Recall from Chapter 7 that HTTP/2 replaced HTTP/1.1's text protocol with a **binary framing layer** in which a single TCP connection carries many independent **streams**, each a bidirectional sequence of frames (HEADERS, DATA, and others), multiplexed so that one slow request does not block others at the application layer. gRPC maps cleanly onto exactly these features, and that mapping is why gRPC is efficient rather than merely convenient:

- **One RPC = one HTTP/2 stream.** The request is a `POST` whose `:path` is `/package.Service/Method`. Because HTTP/2 multiplexes streams over one connection, thousands of concurrent RPCs share a single TCP+TLS connection with no per-call connection setup — you pay the TLS handshake once and amortize it across the life of the channel.
- **Streaming is native.** HTTP/2 streams are already bidirectional frame sequences, so gRPC's streaming call types are not a bolt-on; a server-streaming call is simply multiple DATA frames flowing back on the one stream before it closes.
- **Binary framing and HPACK header compression** mean the request/response envelope is small and the headers (which in gRPC carry metadata) are compressed against a dynamic table rather than re-sent as text each time.
- **Flow control and prioritization** come from HTTP/2, giving gRPC per-stream backpressure for free — a slow consumer of a server stream naturally throttles the producer.

Inside a stream, gRPC frames each protobuf message with a tiny **5-byte length prefix**: one byte for a compression flag (0 = uncompressed, 1 = compressed with the message-encoding named in metadata) and a 4-byte big-endian length, followed by the serialized message. So a DATA frame's payload is `[1 byte flag][4 byte length][message bytes]`, possibly several such frames coalesced or one message split across frames. The

response's HTTP status is always `200 OK` if the *HTTP* layer succeeded; the *gRPC* status rides in trailers (a trailing HEADERS frame) as `grpc-status` and `grpc-message`. This trailer-based status is essential for streaming: the server can send a full stream of DATA frames and only at the very end, in trailers, report that the operation ultimately failed — something a single up-front status code could not express.

```
sequenceDiagram
    participant App as Client app
    participant Stub as Client stub
    participant H2 as HTTP/2 conn
    participant Srv as Server dispatch
    participant Impl as Method impl
    App->>Stub: GetUser(ctx, req)
    Note over Stub: marshal req to protobuf
    prepend 5-byte length prefix
    Stub->>H2: HEADERS (:path, deadline, metadata)
    Stub->>H2: DATA (length-prefixed message)
    H2->>Srv: new stream, headers + data
    Note over Srv: unmarshal, check deadline
    Srv->>Impl: GetUser(req)
    Impl-->>Srv: resp
    Srv->>H2: HEADERS (:status 200)
    Srv->>H2: DATA (length-prefixed resp)
    Srv->>H2: TRAILERS (grpc-status: 0)
    H2-->>Stub: headers, data, trailers
    Note over Stub: unmarshal resp,
    map grpc-status to error
    Stub-->>App: resp, nil
```

The four call types

gRPC defines four call shapes, distinguished by whether the client and server sides are single messages or streams. All four ride on a single HTTP/2 stream; the difference is how many DATA frames flow each way.

```

flowchart TB
    subgraph U["Unary"]
        u1["client: 1 req"] --> u2["server: 1 resp"]
    end
    subgraph SS["Server streaming"]
        s1["client: 1 req"] --> s2["server: N resp"]
    end
    subgraph CS["Client streaming"]
        c1["client: N req"] --> c2["server: 1 resp"]
    end
    subgraph BD["Bidirectional streaming"]
        b1["client: N req"] --> b2["server: M resp (interleaved)"]
    end
end

```

- **Unary** (`rpc GetUser(Req)` returns `(Resp)`) is the ordinary request/response and covers the vast majority of RPCs. Use it unless you have a concrete reason not to.
- **Server streaming** (`returns (stream Resp)`) sends one request and receives a sequence of responses — server-side pagination, tailing a log, subscribing to a feed, streaming a large result set so the client can start processing before it is complete.
- **Client streaming** (`(stream Req)`) sends a sequence of requests and gets one response — bulk upload, ingesting a metrics batch, aggregating many events into a single summary.
- **Bidirectional streaming** (`(stream Req) returns (stream Resp)`) has both sides send sequences over the one stream, interleaved in whatever order the application logic dictates — chat, live telemetry with acks, an interactive session. Because it is fully duplex, the two directions are independent: neither side must wait for the other, and either side closes its half when done.

Streaming is powerful but not free of hazards. A long-lived stream **pins itself to one backend** (its HTTP/2 stream cannot migrate mid-flight), which interacts badly with rebalancing and rolling deploys; a server draining for deployment must send HTTP/2 `GOAWAY` and let long streams finish or be cancelled. Streams also hold server-side resources (a goroutine, buffers) for their whole lifetime, so an unbounded number of idle streams is a resource leak. Prefer unary unless streaming buys you something specific.

Defining and implementing a service

The `.proto` declares streaming with the `stream` keyword on either side:

```
service Telemetry {  
  rpc GetSummary(SummaryRequest) returns (Summary);           //  
  unary  
    rpc Tail(TailRequest) returns (stream Event);             //  
  server streaming  
    rpc Ingest(stream Metric) returns (IngestAck);            //  
  client streaming  
    rpc Session(stream ClientMsg) returns (stream ServerMsg); //  
  bidirectional  
}
```

A Go server implementation of the unary and server-streaming methods shows where deadlines, context, and status codes surface:


```

func (s *server) GetSummary(ctx context.Context, req
*pb.SummaryRequest) (*pb.Summary, error) {
    if req.GetWindowSeconds() <= 0 {
        // Semantic errors use gRPC status codes, not HTTP codes.
        return nil, status.Errorf(codes.InvalidArgument, "window_second
s must be positive")
    }
    // ctx already carries the client's deadline; pass it downstream so
the
    // deadline propagates to every dependency call.
    sum, err := s.store.Summarize(ctx, req.GetWindowSeconds())
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            return nil, status.Error(codes.DeadlineExceeded,
"summarize timed out")
        }
        return nil, status.Errorf(codes.Internal, "summarize: %v", err)
    }
    return &pb.Summary{Count: sum.Count, P99Ms: sum.P99Ms}, nil
}

func (s *server) Tail(req *pb.TailRequest, stream pb.Telemetry_TailServ
er) error {
    events := s.bus.Subscribe(req.GetTopic())
    defer s.bus.Unsubscribe(events)
    for {
        select {
        case
<-stream.Context().Done():           // client cancelled or deadline hit
            return status.FromContextError(stream.Context().Err()).Err(
)
        case ev := <-events:
            if err := stream.Send(toProto(ev)); err != nil {
                return err // transport error;
            }
        }
    }
}

```

The client side of a server-streaming call loops on `Recv` until `io.EOF`:

```

stream, err := client.Tail(ctx, &pb.TailRequest{Topic: "orders"})
for {
    ev, err := stream.Recv()
    if err == io.EOF { break }           // clean end of stream
    if err != nil { return err }         // includes grpc-status
    from trailers
    handle(ev)
}

```

Metadata, status codes, and errors

Metadata is gRPC's name for key/value pairs carried in HTTP/2 headers (request-initial and response-trailing). It is where you put things that are not the message body: an auth token (`authorization: Bearer ...`), a request/trace ID for distributed tracing, tenant identifiers, API-version hints. Keys ending in `-bin` carry base64-encoded binary values; everything else is ASCII. Metadata is exactly the seam interceptors hook into. Do not smuggle large payloads through metadata — it rides in HPACK-compressed headers, not flow-controlled DATA frames, and bloats every call.

gRPC defines its own **status codes**, a closed set of ~17 values, *not* HTTP status codes, because HTTP status has the wrong vocabulary for RPC. `OK` (0) is success; the ones you will handle constantly are `INVALID_ARGUMENT` (client sent something malformed — do not retry), `NOT_FOUND`, `ALREADY_EXISTS`, `PERMISSION_DENIED` and `UNAUTHENTICATED`, `RESOURCE_EXHAUSTED` (rate/quota — back off), `FAILED_PRECONDITION` vs `ABORTED` (the latter signals a concurrency conflict the client may retry, e.g., a transaction abort), `UNIMPLEMENTED`, `INTERNAL`, `UNAVAILABLE` (transient — the canonical retryable code, e.g., server draining or connection lost), and `DEADLINE_EXCEEDED`. The distinction that matters operationally is **retryability**: `UNAVAILABLE` and often `ABORTED` and `RESOURCE_EXHAUSTED` are retryable; `INVALID_ARGUMENT`, `NOT_FOUND`, `PERMISSION_DENIED`, `UNIMPLEMENTED` are not — retrying them just wastes work and can amplify load. Rich, structured error details (beyond a code and string) travel via the `google.rpc.Status` message and the `grpc-status-details-bin` trailer, letting you attach typed payloads like `RetryInfo`, `QuotaFailure`, or `BadRequest.FieldViolation`.

Deadlines, propagation, and cancellation

This is the single most important operational feature of gRPC, and the one teams most often under-use. A gRPC call carries a **deadline** — an absolute point in time by which the call must complete — transmitted as the `grpc-timeout` request header. It is a *deadline*, not a per-hop *timeout*, and the distinction is the whole point: because it is absolute, it **propagates**. When service A calls B with a 300 ms deadline, and B must call C to answer, B passes the *same* deadline context to C. If 250 ms have already elapsed, C sees an effective ~50 ms remaining, not a fresh 300. Each hop subtracts the time already spent, so the entire call tree shares one budget and the work is abandoned everywhere the instant the budget is blown.

```
sequenceDiagram
    participant A as Service A
    participant B as Service B
    participant C as Service C
    A->>B: RPC (grpc-timeout: 300m) [t=0]
    Note over B: 250ms elapsed handling
    B->>C: RPC (grpc-timeout: 50m) [t=250]
    Note over C: work exceeds 50ms
    C-->>B: DEADLINE_EXCEEDED
    B-->>A: DEADLINE_EXCEEDED
    Note over A,C: whole tree abandons work
    no orphaned in-flight requests
```

Deadline propagation is a primary defense against **cascading failure** (Chapter 11). Without it, a slow leaf service causes callers to pile up waiting, each holding a goroutine/thread and a connection, until the caller itself exhausts resources and fails — the failure climbs the call graph. With propagated deadlines, work that cannot finish in the budget is cancelled *everywhere* promptly, freeing resources instead of stacking them. Cancellation is the mechanism: when a client cancels a call (explicitly, or because its own context was cancelled, or its deadline fired), gRPC sends an HTTP/2 `RST_STREAM`, and on the server the handler's `ctx.Done()` (Go) or equivalent fires so it can stop work and, critically, stop its own downstream calls. In Go this is automatic and free if — and only if — you thread the incoming `ctx` through every downstream call, which is why "pass ctx everywhere" is not a style nit but a correctness requirement. Setting a deadline is one line:

```
ctx, cancel := context.WithTimeout(ctx, 300*time.Millisecond)
defer cancel()
resp, err := client.GetSummary(ctx, req) // grpc-timeout: 300m goes
on the wire
```

A call with *no* deadline is a latent outage: it can hang indefinitely, holding resources through any downstream stall. Treat an unbounded gRPC call the way you treat a query with no `LIMIT`.

Interceptors: middleware for RPC

Interceptors are gRPC's middleware: functions that wrap every call to add cross-cutting behavior without touching business logic. On the client, an interceptor sees the outgoing call before it is sent and the result after; on the server, it wraps the handler. This is where authentication (parse and verify the token from metadata), authorization, request logging, metrics (count calls, record latency histograms, tag by `grpc-status`), distributed tracing (extract/inject the trace context in metadata — OpenTelemetry ships gRPC interceptors that do exactly this), rate limiting, panic recovery, and payload validation live. There are unary and streaming variants; you chain them.

```
func authUnary(ctx context.Context, req any, info
*grpc.UnaryServerInfo,
handler grpc.UnaryHandler) (any, error) {
    md, _ := metadata.FromIncomingContext(ctx)
    tok := firstOrEmpty(md.Get("authorization"))
    claims, err := verify(tok)
    if err != nil {
        return nil, status.Error(codes.Unauthenticated, "invalid
token")
    }
    ctx = context.WithValue(ctx, claimsKey{}, claims)
    return handler(ctx, req) // proceed to the actual method
}

srv := grpc.NewServer(
    grpc.ChainUnaryInterceptor(recoverUnary, tracingUnary,
metricsUnary, authUnary),
)
```

Interceptors are the reason a fleet can enforce "every service emits the same latency metric, propagates the same trace headers, and rejects

unauthenticated calls the same way" from a shared library rather than per-team reimplementations.

Channels, connection management, and load balancing

A gRPC **channel** is the client-side abstraction of a virtual connection to a *logical* service (e.g., `dns:///users.svc.cluster.local:8443`). Under the channel, gRPC maintains one or more **subchannels**, each a real HTTP/2 connection to one backend endpoint. The channel owns the name resolution → connectivity → load-balancing pipeline, and it is here that gRPC solves a problem Chapter 7 raised and Chapter 9 develops.

The problem is **HTTP/2 connection pinning**. HTTP/2 multiplexes everything over one long-lived TCP connection, so a naive L4 (connection-level) load balancer sees a client establish *one* connection and pins it to *one* backend for its lifetime. All of that client's thousands of RPCs then hammer a single backend; new backends brought up by autoscaling or a deploy receive *no* traffic because no new connections are being made. L4 load balancing and HTTP/2 are fundamentally mismatched for gRPC.

gRPC's answer is to move load balancing up to **L7, per-RPC, on the client**. The channel resolves the service name to a *set* of backend addresses (via a **name resolver** — DNS by default, but pluggable: `xds:///` for the xDS/service-mesh control plane, custom resolvers for Consul/etcd), opens a subchannel to each, and a **load-balancing policy** picks a subchannel *per RPC*. The default policies are `pick_first` (try addresses in order, stick to the first that connects — fine for a single stable backend) and `round_robin` (open subchannels to *all* resolved backends and rotate RPCs across them). With `round_robin`, because each RPC is independently assigned, adding a backend immediately starts receiving its share as soon as the resolver reports it — the pinning problem dissolves. This is configured with a service config, often delivered via the resolver:

```
conn, err := grpc.NewClient(
    "dns:///users.acme.svc.cluster.local:8443",
    grpc.WithDefaultServiceConfig(`{"loadBalancingConfig":
    [{"round_robin":{}}]}`),
    grpc.WithTransportCredentials(creds),
)
```

Client-side round-robin works well when the client can see every backend directly (typical in Kubernetes with a headless service so DNS returns all

pod IPs). It scales poorly when there are tens of thousands of clients and backends — every client holding a connection to every backend is $O(\text{clients} \times \text{backends})$ connections. The alternatives:

- **Look-aside (external) load balancing:** clients ask a dedicated LB service ("which backends should I use, and how should I weight them?") over a side channel (gRPC-LB historically, now the **xDS** APIs pioneered by Envoy). The data plane is still direct client→backend, but the *decision* is centralized and can use global load information. This is what a service mesh control plane (Istio, or any xDS server) provides, and gRPC has a native xDS client so a proxyless mesh can drive gRPC's LB directly.
- **Proxy (in-band) load balancing:** put an L7 proxy (Envoy, a mesh sidecar) between client and backends; the proxy terminates HTTP/2 and re-balances per stream. Simpler clients, an extra hop, and the proxy must itself be scaled and made HA.

```
flowchart TB
    R["Name resolver  
(DNS / xDS / Consul)"] -->|address list| CH
    subgraph CH["gRPC channel (client)"]
        LB["LB policy  
round_robin"]
        SC1["subchannel → backend 1"]
        SC2["subchannel → backend 2"]
        SC3["subchannel → backend 3"]
        LB --> SC1
        LB --> SC2
        LB --> SC3
    end
    SC1 --> B1["backend 1"]
    SC2 --> B2["backend 2"]
    SC3 --> B3["backend 3 (new pod)"]
    note["Per-RPC pick spreads load;  
new backend gets traffic at once"]
```

Chapter 9 treats load balancing in general; the gRPC-specific lesson is that **gRPC needs L7 or subchannel-aware load balancing** — you cannot just put a classic L4 balancer in front of it and expect even distribution.

Retries and hedging

gRPC supports declarative retry policy in the service config, so you do not hand-roll retry loops (and their bugs) at every call site. A retry policy names which status codes are retryable, a cap, and exponential backoff parameters:

```
{
  "methodConfig": [{
    "name": [{"service": "acme.users.v1.UserService"}],
    "retryPolicy": {
      "maxAttempts": 4,
      "initialBackoff": "0.1s",
      "maxBackoff": "1s",
      "backoffMultiplier": 2,
      "retryableStatusCodes": ["UNAVAILABLE"]
    }
  ]
}
```

Two safety rules are built into the design. First, only failures the server signals as *not yet processed* are transparently retried — gRPC will not blindly retry a call it might have executed — and you should still confine retries to idempotent methods or use idempotency keys (Chapter 7), because the "unknown" outcome always exists. Second, gRPC implements **retry throttling** (a token bucket per channel) so that when a backend is broadly failing, retries are automatically damped rather than doubling the load on an already-struggling service — the classic retry-storm that turns a blip into an outage (Chapter 11).

Hedging is the more aggressive cousin: instead of waiting for a failure, send the same request to multiple backends after short delays and take the first response, cancelling the rest. This attacks *tail latency* — the occasional slow backend — by racing a second attempt before the first has failed. It costs extra load (you may do the work more than once) and is therefore only safe for idempotent, side-effect-free reads, but for latency-critical read paths it is a powerful lever and, again, configured declaratively:

```
{"hedgingPolicy": {"maxAttempts": 3, "hedgingDelay": "0.05s"}}
```

Deadlines, propagation, retries with throttling, and hedging together are gRPC's toolkit for the Fallacies: they acknowledge that the network is

unreliable and slow, and give you bounded, load-aware, tail-tolerant ways to cope. Chapter 11 develops the reliability theory; gRPC is where a lot of it becomes concrete configuration.

gRPC versus REST/JSON, and the browser gap

The real question is rarely "is gRPC better than REST?" but "which surface is this?" The honest comparison:

Dimension	gRPC (+ protobuf, HTTP/2)	REST / JSON over HTTP
Payload	compact binary TLV	verbose text
Contract	enforced schema (<code>.proto</code>) with evolution rules	usually OpenAPI or nothing; weaker enforcement
Streaming	first-class (4 call types)	awkward (SSE, long-poll, WebSocket)
Codegen	strong, multi-language, idiomatic stubs	possible via OpenAPI, less uniform
Browser support	not direct (needs gRPC-Web/Connect)	native everywhere
Debuggability	needs <code>grpcurl</code> /reflection; not human-readable	<code>curl</code> + <code>jq</code> , readable in any tool
Caching (HTTP)	not via HTTP caches	rich HTTP caching (Chapter 7)
Best fit	internal service-to-service RPC at scale	public APIs, browser clients, third parties

Use gRPC for **internal, high-volume, latency-sensitive service-to-service** traffic where both ends are yours, you want generated clients and enforced contracts, and you benefit from streaming and deadline propagation. Use REST/JSON for **public APIs, browser-facing endpoints, webhooks, and third-party integrations** where ubiquity, HTTP caching, human-debuggability, and zero client tooling matter more than bytes on the wire. Many organizations run both: gRPC in the mesh, a JSON/REST gateway at the edge (`grpc-gateway` transcodes JSON↔gRPC from the same `.proto`, giving one contract two front doors).

The **browser gap** is a hard constraint, not a preference. A browser cannot make an arbitrary gRPC call, because JavaScript has no access to HTTP/2 trailers or raw frames — the Fetch/XHR APIs do not expose them, and gRPC's status lives in trailers. **gRPC-Web** is the workaround: a variant protocol (and a required proxy, historically Envoy's gRPC-Web filter) that encodes the trailers *into the response body* so a browser can read them, at the cost of losing client-side and bidirectional streaming (only unary and server-streaming are supported). **Connect** (from Buf) is the more modern answer: a family of libraries (ConnectRPC) that speak gRPC, gRPC-Web, *and* a simpler Connect protocol that is plain HTTP/1.1-or-2 with JSON *or* protobuf bodies, so the same handler is callable by a gRPC client, a gRPC-Web browser client, or literally `curl` — no proxy required. Connect has largely become the pragmatic choice when you need one service reachable by both meshed gRPC clients and browsers.

Other RPC frameworks worth situating: **Twirp** (Twitch) is deliberately minimal — protobuf messages over plain HTTP/1.1, unary-only, JSON or binary, no HTTP/2 dependency and no streaming — trading gRPC's features for radical simplicity and debuggability. **trRPC** is a TypeScript-only framework that shares types between a TS server and TS client *without codegen or a separate IDL*, inferring the contract from the server's types; it is superb for a single TypeScript monorepo and irrelevant to a polyglot fleet. The pattern across all of them is the same trade you now recognize: how much of gRPC's power (streaming, HTTP/2, strict schema) you are willing to give up for simplicity, browser reach, or single-language ergonomics.

Distributed-systems lens

At the scale this book assumes — hundreds of services, dozens of teams, many repositories, deploys all day — gRPC-over-HTTP/2-with-protobuf is the default internal RPC substrate for concrete, compounding reasons. The **binary protobuf payload and HTTP/2 multiplexing** mean a service can sustain very high RPC throughput on a small, stable pool of connections, so connection setup and serialization stop being the fleet's dominant cost. **Schema-enforced contracts** turn the IDL into the interface between teams: the `.proto` is checked in, code-reviewed, breaking-change-linted, and generates every client, so a producer can add a field and consumers adopt it on their own schedule — the independent evolution that Volume 8 treats as the core discipline of API

governance. This is not a nicety; it is what makes a large service graph changeable at all. Without an enforced, evolvable contract, every cross-team change becomes a lockstep release.

The operational features map one-to-one onto the failure modes of distributed systems. **Deadline propagation** is the fleet's shared clock for giving up: because deadlines are absolute and flow down the call tree, a single latency budget governs an entire request fan-out, and blown budgets free resources everywhere at once instead of pooling into the cascading failure that Chapter 11 anatomizes. **Retries with throttling and hedging** let you trade a little extra load for a lot of tail-latency and transient-failure tolerance, while the built-in retry budget prevents the retry-storm that converts a minor degradation into an outage. **Client-side and subchannel load balancing** are the answer to the HTTP/2 pinning problem (Chapters 7 and 9): because gRPC balances per-RPC over subchannels — directly with `round_robin`, or via xDS/look-aside for very large fanouts — new capacity gets traffic immediately and hot backends are avoided, which a classic L4 balancer in front of HTTP/2 could never achieve. And because the whole thing rests on TLS (Chapter 6) with mTLS common in a mesh, the "network is not secure" fallacy is handled at the transport where it belongs.

Two costs deserve to be kept in view rather than assumed away. **Serialization is not free**: protobuf marshalling and unmarshalling consume CPU, and for very hot paths with large messages it can become the bottleneck — which is when zero-copy formats (Cap'n Proto, FlatBuffers) or careful message design (avoid re-serializing giant nested messages you only pass through) start to matter. And **connection management shapes fleet efficiency**: long-lived HTTP/2 connections must be aged out and rebalanced (via `GOAWAY` and `max-connection-age`) so that a topology change is actually absorbed, and a design that opens $O(\text{clients} \times \text{backends})$ subchannels will drown a large deployment in connections and memory — the reason look-aside/xDS balancing exists. The framework gives you the right primitives; using them well is an ongoing engineering responsibility, not a default you get for importing a library.

Key takeaways

- **RPC makes a remote call look local but it is not**: it can partially fail (success, failure, or *unknown*), it is slow, and it is insecure. A

good RPC framework surfaces those realities (deadlines, status codes, cancellation) rather than hiding them; the Fallacies of Distributed Computing are the checklist of what "not local" means.

- **The schema is the contract.** Protobuf's wire identity is the *field number*, encoded as a varint tag with a wire type; unknown fields are skippable, which is the mechanical basis of forward/backward compatibility. Never reuse a field number, `reserve` retired ones, add fields freely, and gate breaking changes in CI (`buf breaking`).
- **gRPC is protobuf over HTTP/2:** one RPC per stream, 5-byte length-prefixed messages in DATA frames, status in trailers. HTTP/2 multiplexing, streaming, binary framing, and HPACK are exactly what make gRPC efficient.
- **Four call types** — unary (default), server-streaming, client-streaming, bidirectional. Prefer unary unless streaming buys something; long streams pin to a backend and hold resources.
- **Deadlines propagate.** An absolute deadline flows down the call tree, each hop spending from one shared budget; this is the primary structural defense against cascading failure. A call with no deadline is a latent outage. Thread `ctx` everywhere so cancellation reaches downstream calls.
- **gRPC needs L7/subchannel load balancing.** A classic L4 balancer pins HTTP/2 to one backend; `round_robin` over resolved subchannels, or look-aside/xDS balancing for large fanouts, spreads load per-RPC and lets new backends receive traffic immediately.
- **Retries and hedging are declarative and load-aware** (retry throttling prevents storms); confine retries to idempotent operations, and hedge only side-effect-free reads.
- **Choose the surface, not the winner:** gRPC for internal high-volume service-to-service RPC; REST/JSON for public, browser, and third-party APIs. The browser gap is real — use gRPC-Web or, better, Connect, which serves gRPC, gRPC-Web, and plain HTTP/JSON from one handler.

Further reading

- gRPC official documentation and concepts — <https://grpc.io/docs/> — including the "Core concepts" and "gRPC over HTTP/2" pages that specify the request/response framing, trailers, and status model.

- "gRPC over HTTP/2" protocol specification — <https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-HTTP2.md> — the authoritative mapping of calls, `grpc-timeout`, `grpc-status`, and length-prefixed messages.
- Protocol Buffers documentation: language guide (proto3) and **Encoding** reference — <https://protobuf.dev/programming-guides/encoding/> — the definitive description of varints, wire types, and tag-length-value; and the "Proto Best Practices" / compatibility guide.
- A. D. Birrell and B. J. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, 2(1), 1984 — the foundational stub/binding RPC paper.
- "Fallacies of Distributed Computing" — attributed to Peter Deutsch and colleagues at Sun Microsystems (with the eighth by James Gosling); Arnon Rotem-Gal-Oz's essay is a widely cited explanation. See also the Wikipedia summary for the canonical list.
- RFC 9113 — *HTTP/2* (2022) — the substrate; and Chapter 7 of this volume for framing, multiplexing, and the pinning problem.
- The gRPC blog, "gRPC Load Balancing" — <https://grpc.io/blog/grpc-load-balancing/> — client-side, look-aside, and proxy balancing, and why L4 fails for HTTP/2.
- gRPC retry and hedging design — gRFC A6 ("client retries") in the `grpc/proposal` repository, which specifies the retry and hedging policies and the retry-throttling token bucket — <https://github.com/grpc/proposal/blob/master/A6-client-retries.md>; the service-config schema itself lives in the `grpc/grpc` repository.
- Buf documentation — <https://buf.build/docs/> — schema management, `buf breaking` for compatibility enforcement, and the ConnectRPC framework (<https://connectrpc.com>) for the gRPC/gRPC-Web/HTTP-JSON triple protocol.
- Apache Thrift (<https://thrift.apache.org/>), Apache Avro (<https://avro.apache.org/>), Cap'n Proto (<https://capnproto.org/>), and FlatBuffers (<https://flatbuffers.dev>) official docs for the serialization-format comparison; Twirp (<https://github.com/twitchtv/twirp>) and tRPC (<https://trpc.io>) for the minimal-RPC and TypeScript-native alternatives.

- Sam Newman, *Building Microservices* (2nd ed., O'Reilly, 2021) — the inter-service communication chapters place RPC-versus-REST trade-offs in an organizational context.

Chapter 9 — Load Balancing: L4, L7, and Algorithms

What this chapter covers. A load balancer decides, for every connection or every request, which of your replicas does the work. That decision looks trivial — pick one of N — and it is where a surprising share of production incidents are born. We start at the transport layer: what an L4 balancer actually does to a packet, and the three forwarding modes — NAT, direct server return, tunneling — that set your throughput ceiling and constrain your topology. Then the application layer: what terminating the connection and parsing HTTP buys, and what it costs in CPU, latency, and failure-domain size. Between them sits the most consequential interaction in modern service networking: an L4 balancer distributes *connections*, HTTP/2 multiplexes many requests onto *one* connection (Chapter 7), so a naive L4 tier in front of a gRPC fleet (Chapter 8) pins each client to one backend and produces imbalance that no amount of capacity fixes. We then take the algorithms apart — round robin and its weighted form, least-connections, least-request, latency EWMA, random, and power-of-two-choices — along with the hashing family: consistent hashing (Volume 14, Chapter 5) and Maglev hashing (Google, NSDI 2016). Health checking gets a long treatment because bad health checks are a leading cause of *correlated* failure. Finally the topologies: software proxies, cloud balancers, DNS-based distribution (Chapter 5), BGP anycast with ECMP and XDP/eBPF forwarders (Volume 2, Chapter 10), and the client-side model with no middle proxy at all.

Learning goals — after this chapter you should be able to:

- Explain what an L4 balancer rewrites and what it does not; choose between NAT, DSR, and IP/IP/GRE tunneling on return-path bandwidth, L2 adjacency, and MTU; and say what terminating at L7 buys and costs instead.

- Diagnose and fix HTTP/2 and gRPC connection pinning behind an L4 tier, using L7 proxies, client-side/subchannel balancing, or bounded connection lifetimes.
- Reason about load-balancing algorithms from first principles: why round robin fails under heterogeneous request cost, why global least-loaded herds, and why power-of-two-choices with an EWMA cost estimate is the modern default for meshes.
- Configure consistent hashing and Maglev hashing, and state their disruption and balance properties.
- Design health checking that degrades safely: active vs passive detection, hysteresis, panic thresholds, slow start, and health endpoints that do not couple the fleet to one dependency.
- Drain connections correctly during a deploy so that a rollout does not show up as errors.
- Choose an ingress topology — cloud LB, anycast + ECMP + Maglev/Katran, DNS, or client-side — and explain its failure modes.

Why load balance at all

There are four distinct reasons, and conflating them leads to bad designs.

Horizontal scale. A single process has a ceiling — cores, memory bandwidth, file descriptors, connection table size, GC behavior — and beyond it you add replicas that something must feed. This is the reason people usually name and the least interesting one: it says nothing about *how* work should be spread.

Failure tolerance. Replicas die: kernel panics, OOM kills, bad deploys, hypervisor migrations, a rack losing power. The balancer notices and stops sending work to the corpse. This carries the most operational subtlety, because the detection mechanism — health checking — is itself a distributed system with false positives, false negatives, and the ability to take down a healthy fleet.

Deployability. Rolling deploys, blue/green, and canaries (Volume 11, Chapter 9) all reduce to "shift traffic away from these instances, replace them, shift traffic back." Without a traffic control point you cannot deploy without dropping requests.

Utilization. Two backends at 30% and 90% cost the same as two at 60% and 60%, but the busiest replica in the second arrangement has four

times the headroom before it starts timing out. Tail latency in a fanout service is governed by the *worst* replica you touch: if each subrequest independently has a 1% chance of landing in a backend's slow tail, a fanout of 50 pushes roughly 40% of parent requests over that threshold and a fanout of 100 about 63%. That is Dean and Barroso's "tail at scale" arithmetic: balancing quality translates directly into p99 and into capacity you must buy.

A useful reframing for the rest of the chapter: a load balancer is not a box, it is the **traffic-distribution control plane of a fleet** — a view of which endpoints exist (service discovery), a view of which are healthy (health checking), and a policy for choosing among them (the algorithm). Where those three live — appliance, proxy, sidecar, or client library — is a topology choice largely independent of the algorithms themselves.

L4: balancing at the transport layer

An L4 load balancer forwards TCP connections and UDP flows without understanding the bytes they carry. Its unit of decision is the **flow**, keyed by the 5-tuple: source IP, source port, destination IP, destination port, protocol. The first packet of a flow (a TCP SYN, or the first datagram of a UDP flow) triggers a backend selection; every subsequent packet must go to the same backend, because that backend holds the TCP state machine (Chapter 3). An L4 balancer therefore needs a **connection tracking table** mapping flows to backends, a **deterministic hash** that yields the same answer for the same 5-tuple, or — in the best designs — both.

Because it never parses a payload, an L4 balancer is protocol-agnostic (TCP, UDP, QUIC, the PostgreSQL wire protocol, Redis, SMTP), does not need your TLS private keys, and costs very little per packet: a hash, a table lookup, a header rewrite. Millions of packets per second per host is routine for a kernel-bypass or XDP-based implementation.

The three forwarding modes

How the packet actually reaches the backend, and how the reply reaches the client, is the single most important L4 design decision.

NAT mode (masquerading). The balancer rewrites the destination IP (and possibly port) from the virtual IP (VIP) to the real server. For the reply to be un-NATed correctly it *must* traverse the balancer again — either because the balancer is the backend's default gateway, or because

the balancer also rewrote the source IP (SNAT), making itself the apparent client. NAT mode is simple and works across L3 boundaries, but it puts both directions through the balancer, and since HTTP responses dwarf requests the balancer's bandwidth becomes the fleet's bandwidth. SNAT additionally destroys the client IP (recover it via `X-Forwarded-For` or the PROXY protocol) and burns a source-port keypace: at most 65,535 concurrent flows per (SNAT source IP, backend IP, backend port), of which Linux's default ephemeral range (`net.ipv4.ip_local_port_range`, 32768–60999) offers roughly 28,000. That is the classic ephemeral-port-exhaustion failure.

Direct server return (DSR / direct routing). The balancer rewrites only the destination MAC address and puts the frame back on the wire; the IP header is untouched. The backend owns the VIP on a loopback interface with ARP suppressed, and replies to the client *directly*, bypassing the balancer — asymmetric routing by design. The balancer sees only inbound packets, which for a response-heavy workload is a small fraction of the byte volume, so a small tier can front an enormous fleet. The costs: balancer and backends must share an L2 segment (usually impossible in cloud VPCs); ports cannot be rewritten; and the balancer never sees responses, so passive health detection is impossible.

Tunneling (IPIP, GRE, or GUE over UDP). The balancer encapsulates the packet in an outer IP header addressed to the backend, which decapsulates, processes the inner packet, and replies directly to the client — DSR semantics without the L2 adjacency requirement. This is what hyperscale L4 balancers use: Maglev encapsulates with GRE, Meta's Katran with IPIP, GitHub's GLB and Cloudflare's Unimog with GUE (Generic UDP Encapsulation), whose UDP source port carries entropy so intermediate routers' ECMP hashing still spreads tunneled traffic. The price is MTU: over IPv4, IPIP costs 20 bytes of outer header, GRE 24 (20 + a 4-byte minimum GRE header), and GUE at least 32 (20 + 8 UDP + 4 GUE), with more for optional GUE fields. You must lower the backend's advertised MSS accordingly or accept fragmentation and PMTUD pain (Chapter 2).


```

flowchart LR
    subgraph L4["L4 balancer: decision per connection"]
        C1["Client A"]
        TCP_conn_1["TCP conn 1"] --> LB4["L4 LB"]
        LB4 -->|"all packets of conn 1"| B1["Backend 1"]
        C2["Client B"]
        TCP_conn_2["TCP conn 2"] --> LB4
        LB4 -->|"all packets of conn 2"| B2["Backend 2"]
        B1 -->|"DSR: reply direct to client"| C1
    end
    subgraph L7["L7 proxy: decision per request"]
        C3["Client C"]
        one_TCP_TLS_conn["one TCP/TLS conn"] --> LB7["L7 proxy"]
        LB7 -->|"GET /cart to pool cart"| B3["cart-svc"]
        LB7 -->|"POST /pay to pool pay"| B4["pay-svc"]
        LB7 -->|"next GET /cart, same pool"| B5["cart-svc replica"]
    end

```

L4 in practice: IPVS

Linux's in-kernel L4 balancer, IPVS, is what `kube-proxy` uses in IPVS mode and what most on-prem L4 tiers are built from. It exposes forwarding mode and scheduler as orthogonal knobs:

```

# VIP 203.0.113.10:443, Maglev hashing scheduler (kernel >= 4.18)
ipvsadm -A -t 203.0.113.10:443 -s mh

# Add real servers in direct-routing (DSR) mode: -g gatewaying, -i
# ipip, -m masquerade/NAT
ipvsadm -a -t 203.0.113.10:443 -r 10.0.1.11:443 -g -w 100
ipvsadm -a -t 203.0.113.10:443 -r 10.0.1.12:443 -g -w 100

ipvsadm -Ln --stats

```

On each DSR backend the VIP must exist but stay invisible to ARP:

```

ip addr add 203.0.113.10/32 dev lo
sysctl -w net.ipv4.conf.all.arp_ignore=1      # answer ARP only for
addresses on the target iface
sysctl -w net.ipv4.conf.all.arp_announce=2    # use the best local
address as ARP source

```

Get `arp_ignore`/`arp_announce` wrong and multiple hosts answer ARP for the VIP; the switch's MAC table flaps and traffic goes to a random member. This is the canonical DSR bring-up bug.

L7: balancing at the application layer

An L7 balancer is a **terminating proxy**. It completes the TCP and TLS handshakes with the client (Chapter 6), parses the application protocol — HTTP/1.1, HTTP/2, HTTP/3, gRPC, sometimes Redis or Postgres — and opens or reuses its *own* connections to backends. There are two connections, not one, and the proxy is a full participant in both. That unlocks everything an L4 balancer cannot do:

- **Per-request routing** by path, method, `Host`, header, cookie, or gRPC service and method (`/checkout.v1.Checkout/PlaceOrder`) — which makes the L7 tier the natural home for API gateways, header-based canary splits, and per-tenant routing.
- **Per-request load balancing** — the property that saves you under HTTP/2 (next section).
- **Retries, timeouts, hedging, and circuit breaking** (Chapter 11) applied uniformly, in one place, without touching application code.
- **Rewriting**: paths, headers, `X-Forwarded-For` and `X-Request-Id`, trace context (Volume 11, Chapter 4), stripping internal headers on the way out.
- **Protocol translation**: HTTP/1.1 to HTTP/2 backends, HTTP/3 at the edge to HTTP/1.1 internally, gRPC-Web to gRPC, WebSocket upgrades.
- **Rich observability**: per-route status distributions, upstream latency histograms, per-backend success rates — the raw material for passive health detection.

The costs are equally structural. TLS termination and HTTP parsing cost CPU (asymmetric handshake crypto being the expensive part; use session resumption and keepalive aggressively). The proxy adds a hop with its own queueing — typically sub-millisecond for a well-tuned Envoy or HAProxy on the same network, but the tail matters and buffering policy matters more. It holds state (connection pools, buffers, HTTP/2 stream tables), so it has its own memory ceiling. And because it understands your protocol, it can be broken by your protocol: request smuggling, header size limits, misparsed chunked encodings.

Dimension	L4	L7
Decision unit	Flow / connection	Request (or HTTP/2 stream)
Protocol awareness	None	Full (HTTP, gRPC, sometimes DB protocols)
TLS keys required	No (pass-through)	Yes (unless SNI-based pass-through routing)
Cost per unit work	Very low (per packet)	Moderate (parse, buffer, re-emit)
Client IP preservation	Native (DSR/tunnel) or lost (SNAT)	Lost at IP level; carried in <code>X-Forwarded-For</code> /PROXY protocol
Retries	Connection-level only: redispach a failed connect	Native, per request
Passive health signals	None (no response visibility)	Yes (status codes, latencies, resets)
HTTP/2 behavior	Pins whole connection to one backend	Balances every stream independently
Typical scale unit	Millions of pps per node	Tens of thousands of RPS per core-ish

A large fleet usually runs both: an L4 tier for raw ingress scale, DDoS absorption, and VIP distribution, and an L7 tier behind it for routing and per-request policy. That is not redundancy; they solve different problems.

The HTTP/2 pinning problem

This is the most common load-balancing bug in modern service-to-service architectures, and it is invisible until you look at per-backend request rates.

Recall from Chapter 7 that HTTP/2 multiplexes many concurrent requests as **streams** over a single long-lived TCP connection, and that RFC 9113 §9.1 tells clients they "SHOULD NOT open more than one HTTP/2 connection to a given host and port pair." gRPC (Chapter 8) inherits this:

a `grpc.ClientConn` maintains a persistent HTTP/2 connection and issues every RPC as a stream on it.

Now place an L4 balancer in front. It picks a backend once, at connection establishment, and every stream on that connection — thousands of RPCs per second, for hours — lands on that one backend. The consequences compound:

- **Load is distributed per client, not per request.** With 10 clients and 20 backends, at most 10 backends receive traffic, no matter how much capacity you provision.
- **New backends receive nothing.** Scale up during an incident and the new replicas sit empty, because no client has a reason to open a new connection. The autoscaler adds pods, the metrics do not move, and the on-call engineer concludes autoscaling is broken.
- **Imbalance is sticky.** A client assigned to a degraded backend stays there until the connection breaks, and request-cost heterogeneity across clients (a batch job vs. a UI service) projects straight onto backends.
- **Deploys are lumpy.** Draining a backend forces all its clients to reconnect at once.

There are three real fixes, and one non-fix.

Fix 1 — terminate HTTP/2 at an L7 proxy. Envoy, NGINX, HAProxy, and cloud ALBs parse HTTP/2 and make a fresh upstream selection per stream, multiplexing your streams across their own backend connection pool. This is the standard answer for north-south ingress and for east-west traffic in a sidecar mesh (Chapter 10), where the "proxy" is a localhost hop.

Fix 2 — client-side / subchannel balancing. Let the client resolve *all* backend addresses and maintain a subchannel (one HTTP/2 connection) to each, choosing per RPC. In gRPC this is a one-line service-config change:

```
conn, err := grpc.NewClient(
    "dns:///checkout.svc.cluster.local:8080",
    grpc.WithDefaultServiceConfig(`{"loadBalancingConfig":
[{"round_robin":{}}]}`),
    grpc.WithTransportCredentials(insecure.NewCredentials()),
)
```

The default policy is `pick_first`, which connects to one resolved address and stays there — exactly the pinned behavior, without a middlebox to blame. `round_robin` (or an xDS-supplied policy such as `weighted_round_robin`) opens a subchannel per endpoint and chooses per RPC. The address set is only as fresh as the resolver: `grpc-go`'s DNS resolver enforces a minimum re-resolution interval (30 seconds by default, adjustable via `dns.SetMinResolutionInterval`) and re-resolves on connection failure, which is why headless Services (Chapter 5) — which return one A/AAAA record per ready pod — are the usual Kubernetes pairing.

Fix 3 — bound connection lifetime. Even with L7 in the path, connections that live forever prevent rebalancing after a scale-up. Servers should periodically retire connections with a graceful `GOAWAY`, forcing clients to re-resolve and redistribute:

```
s := grpc.NewServer(grpc.KeepaliveParams(keepalive.ServerParameters{
    MaxConnectionAge:      30 * time.Minute, // send GOAWAY after this
    MaxConnectionAgeGrace: 5 * time.Minute, // then hard-close
}))
```

`gRPC` jitters `MaxConnectionAge` so that a fleet of clients does not reconnect in a synchronized wave. Envoy's equivalent is `max_connection_duration` on the HTTP connection manager; NGINX has `keepalive_time`.

The non-fix: "just add more replicas." Capacity does not redistribute a pinned connection.

```

flowchart TB
    subgraph P["Problem: L4 in front of HTTP/2"]
        GC1["gRPC client 1"] -->|"1 TCP conn, 5k streams/s"| L4B["L4 LB  
picks backend at SYN"]
        GC2["gRPC client 2"] -->|"1 TCP conn"| L4B
        L4B --> PB1["backend 1  
10k rps"]
        L4B --> PB2["backend 2  
0 rps"]
        L4B --> PB3["backend 3 - just scaled up  
0 rps"]
    end
    end
    subgraph F1["Fix A: L7 proxy, per-stream choice"]
        GC3["gRPC client"] -->|"1 conn"| E["Envoy / ALB  
terminates HTTP/2"]
        E -->|"per RPC"| FB1["backend 1"]
        E -->|"per RPC"| FB2["backend 2"]
        E -->|"per RPC"| FB3["backend 3"]
    end
    end
    subgraph F2["Fix B: client-side subchannels"]
        GC4["gRPC client  
round_robin over subchannels"] --> SB1["backend 1"]
        GC4 --> SB2["backend 2"]
        GC4 --> SB3["backend 3"]
        DISC["service discovery  
DNS / xDS"] -->|"endpoint list"| GC4
    end
    end

```

The same argument applies to any long-lived multiplexed transport: HTTP/3 over QUIC (Chapter 4), WebSocket fanout, database connection pools, and Kafka clients all pin at L4.

Algorithms

Everything above is mechanism. This section is policy: given a set of healthy endpoints, which one gets this request?

Round robin

Rotate through the endpoint list. It is $O(1)$, needs no state beyond an index, and produces perfectly equal *request counts*. That is exactly its weakness: equal request counts are the goal only when every request costs the same and every backend has the same capacity. Neither holds in practice. A `GET /users/me` and a `POST /reports/generate` can differ by three orders of magnitude in service time; a fleet running on mixed instance types or noisy-neighbor hypervisors can easily differ by $2\times$ in

throughput. Round robin will happily keep feeding a backend stuck in a stop-the-world GC pause, because it counts requests sent, not work outstanding.

It is also subtly bad in a mesh where M clients round-robin independently over N backends: the rotations are uncorrelated, so arrivals at each backend look closer to Poisson than to smooth, and queueing variance is higher than the "perfectly fair" framing suggests. The SRE book's datacenter chapter accordingly treats round robin as a baseline to improve on, not a target.

Weighted round robin

Attach a weight to each endpoint and give it a proportional share. The naive implementation — emit w consecutive picks for weight w — produces bursts, so production implementations smooth the sequence. NGINX uses *smooth weighted round robin*, maintaining a current-weight accumulator per peer and selecting the maximum, which yields interleaved sequences like `A B A C A B A` rather than `A A A A B B C`. Envoy uses an **earliest-deadline-first (EDF) scheduler**: each host sits in a priority queue with a deadline of $1/\text{weight}$, and the earliest is popped and re-inserted with its deadline advanced. EDF generalizes cleanly to *dynamic* weights, which is what slow start and utilization-based weighting need.

Where do weights come from? Static configuration (instance size), locality (prefer same-zone endpoints to avoid cross-AZ latency and charges), canary percentages, or — most interestingly — backend-reported load. gRPC and Envoy both support **ORCA** (Open Request Cost Aggregation), where the backend attaches utilization metrics (CPU, application-defined QPS or memory pressure) to responses or to out-of-band reports and the client's `weighted_round_robin` policy converts them into weights. It is the most direct way to make weights track real backend utilization on heterogeneous hardware.

Least connections and least request

Track outstanding work per backend and send to the minimum. HAProxy's `leastconn` counts open connections; Envoy's `LEAST_REQUEST` counts active requests, which is the right unit for multiplexed protocols where one connection carries many in-flight requests.

Outstanding-request count is a good load proxy because it is *self-correcting*: a backend that slows down accumulates in-flight requests and is automatically avoided, without anyone measuring its latency. It handles heterogeneous request cost far better than round robin.

But global least-loaded has a well-known pathology. Whenever the load view is shared and slightly stale, every chooser sees the same minimum and sends to it simultaneously; that backend becomes the *most* loaded and the herd swings to the next victim, so the system oscillates instead of converging. A single balancer process avoids this by updating the counter synchronously on dispatch — but a mesh with hundreds of independent clients has no such counter, and each client sees only its own in-flight requests, a partial and biased view.

Random, and the power of two choices

Pure random assignment needs no state and is trivially parallel; its problem is variance. The classic balls-into-bins result: throwing n balls into n bins uniformly at random gives a maximum load of roughly $\ln n / \ln \ln n$ — a meaningful multiple of the average even for modest n . Some backend always gets unlucky, and under fanout that backend is on the critical path of most requests.

Now pick **two** endpoints uniformly at random, compare only those two, and send to the less loaded. This is the "power of two choices" result of Azar, Broder, Karlin, and Upfal ("Balanced Allocations", STOC 1994) and of Mitzenmacher's thesis and subsequent paper: the maximum load drops from $\Theta(\ln n / \ln \ln n)$ to $\ln \ln n / \ln 2 + \Theta(1)$. That is an *exponential* improvement from one extra sample, and taking $d = 3$ or more buys only a further constant factor. Two samples is the knee of the curve.

P2C is the right answer for distributed balancing for three reasons that matter more than the asymptotics:

1. **No global state.** Each chooser needs load information for exactly the two endpoints it sampled — in practice, its own local estimate of those two. No cluster-wide counter, no gossip, no consistency problem.
2. **No herding.** Because the candidate pair is redrawn per decision, two clients deciding at the same instant usually consider different

pairs, and no single endpoint can attract every simultaneous chooser the way a shared "least loaded" view does.

3. **It degrades gracefully with stale data.** Even if load estimates are seconds old, picking the better of two random candidates still avoids the worst backends most of the time.

```
flowchart LR
  R["Incoming request"] --> S["Sample 2 endpoints  
uniformly at random"]
  S --> A["Candidate A  
in-flight: 12  
EWMA latency: 8 ms  
cost = 8 x 13 = 104"]
  S --> B["Candidate B  
in-flight: 3  
EWMA latency: 5 ms  
cost = 5 x 4 = 20"]
  A --> C["lower cost wins  
cost = EWMA x in-flight plus 1"]
  B --> C
  C --> D["send to B"]
  D --> E["Backend B"]
  N["Ignored this round:  
the other N-2 endpoints"] -.-> S
```

P2C with EWMA cost (peak-EWMA)

The refinement that makes P2C excellent rather than merely good is *what* you compare. Counting in-flight requests treats all requests as equal. Instead, maintain a per-endpoint exponentially-weighted moving average of observed latency and score each candidate as roughly $\text{EWMA_latency} \times (\text{outstanding_requests} + 1)$ — an estimate of the queueing delay a new request would meet there. This is Finagle's "peak EWMA" balancer and Linkerd's default, and it is why a sidecar notices a backend entering a GC pause within milliseconds: latency spikes, the EWMA rises, and the endpoint loses nearly every comparison until it recovers. Two implementation details matter. Decay the EWMA on *time*, not on request count, or an idle endpoint keeps a stale score forever. And treat a zero-observation endpoint optimistically enough to be sampled but not so optimistically that it is flooded — which is exactly what slow start (below) is for.

Configuring P2C is usually a single knob. Envoy:

```
lb_policy: LEAST_REQUEST
least_request_lb_config:
  choice_count: 2          # P2C; this is the default
  active_request_bias:     # exponent applied to active requests when
                           # hosts have unequal weights
  default_value: 1.0
  runtime_key: cluster.checkout.active_request_bias
```

With equal host weights Envoy's `LEAST_REQUEST` is P2C on active request count. With unequal weights it falls back to the EDF scheduler using a dynamic weight of $\text{weight} / (\text{active_requests} + 1)^{\text{active_request_bias}}$, so a bias of 0 disables load-awareness and higher values make it more aggressive.

NGINX (open source, `random` since 1.15.1) and HAProxy both expose P2C directly:

```
upstream checkout {
  zone checkout 64k;          # share group state across workers;
  without it each worker counts alone
  random two least_conn;      # sample 2, pick the one with fewer
connections (nginx >= 1.15.1)
  server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
  server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
  keepalive 64;              # upstream keepalive pool; essential
  to avoid per-request handshakes
}
```

```
backend checkout
  balance random(2)           # two draws, least-loaded of the two
  option redispatch
  server c1 10.0.1.11:8080 check maxconn 200
  server c2 10.0.1.12:8080 check maxconn 200
```

Hashing: consistent hashing and Maglev

Everything so far distributes work *evenly*. Sometimes you want it distributed *stably*: the same key must reach the same backend, so its local cache is warm, or a session lives in one process, or per-key ordering is preserved.

Naive $\text{hash}(\text{key}) \bmod N$ is catastrophic under membership change: adding one backend to a fleet of 20 remaps roughly 95% of keys. **Consistent hashing** (Karger et al., STOC 1997; Volume 14, Chapter 5 for the full treatment) places keys and backends on a ring and assigns

each key to the next backend clockwise, so a membership change moves only about $1/N$ of keys. Acceptable balance requires many **virtual nodes** per backend — typically hundreds — because the variance of a few random ring positions is large; Envoy's `RING_HASH` exposes this as `minimum_ring_size` (default 1024).

Maglev hashing (Eisenbud et al., *Maglev: A Fast and Reliable Software Network Load Balancer*, NSDI 2016) instead precomputes a fixed lookup table of size `M` (a prime, e.g. 65537) whose entries name backends, and looks up `table[hash(key) mod M]`. Each backend derives a preference permutation over slots from two hashes of its name, and backends take turns claiming their most-preferred free slot until the table is full. The result is *near-perfect* balance — every backend gets M/N entries, ± 1 — with minimal, though not strictly optimal, disruption on membership change. Lookup is a single array index, which is why it suits a packet-rate datapath.

```
def maglev_populate(backends, M=65537):

    """Maglev table generation (NSDI 2016, section 3.4). Returns a list of
    length M
        whose entries are indices into `backends`. `h1` and `h2` are two
    independent
        hash functions over the backend's name; M must be prime and M >>
    N."""
    N = len(backends)
    offset = [h1(b) % M for b in backends]
    skip   = [h2(b) % (M - 1) + 1 for b in backends] # coprime with
    prime M
    nxt     = [0] * N
    entry   = [-1] * M
    filled = 0
    while True:
        for i in range(N):
            c = (offset[i] + nxt[i] * skip[i]) % M # backend i's
            next preference
            while entry[c] != -1:                  # skip slots
                already taken
                nxt[i] += 1
                c = (offset[i] + nxt[i] * skip[i]) % M
            entry[c] = i
            nxt[i] += 1
            filled += 1
            if filled == M:
                return entry
```

Both are one line of config in Envoy, combined with a route-level hash policy that names the key:

```
# Cluster
- name: session_cache
  connect_timeout: 0.25s
  lb_policy: MAGLEV
  maglev_lb_config:
    table_size: 65537
  common_lb_config:
    consistent_hashing_lb_config:
      hash_balance_factor: 125 # bounded loads: no host may exceed
      1.25x the mean
```

```
# Route: choose the hash key
route:
  cluster: session_cache
  hash_policy:
    - header:
        header_name: x-user-id
        # alternatives: cookie (with ttl, to have Envoy mint one),
        connection_properties (source IP),
        # query_parameter, filter_state
```

That `hash_balance_factor` is **consistent hashing with bounded loads** (Mirrokni, Thorup, and Zadimoghaddam; arXiv 2016, later SODA 2018), and it fixes hashing's fundamental weakness: a hot key or a skewed key distribution overloads one backend and pure hashing has no escape valve. With a bound of $c \times \text{mean}$, an overloaded target spills the key to the next host on the ring, preserving most affinity while capping the damage. Envoy leaves the factor unset (unbounded) by default, accepts a minimum of 100, and documents 120–200 as the useful range; it applies to `RING_HASH` and `MAGLEV` alike. Use it whenever your key space might be skewed — that is, almost always. (Kubernetes note: `kube-proxy` in IPVS mode can be configured with the `mh` scheduler, which is how you get consistent-hash behavior for ClusterIP Services without a userspace proxy.)

Health checking

The algorithm chooses among *healthy* endpoints. Deciding which endpoints are healthy is where load balancers cause outages.

Active health checks

The balancer periodically probes each backend out-of-band. Layers matter:

- **L3** (ICMP echo): the host is up. Nearly useless.
- **L4** (TCP connect): something is listening. Also nearly useless — a process stuck in a GC pause or deadlocked on a lock still accepts connections, because the kernel completes the handshake and queues the socket in the accept backlog.
- **L7** (HTTP GET, gRPC health check): executes application code and observes a status. The only kind worth relying on. For gRPC use the standard `grpc.health.v1.Health` service — `Check` for polling, `Watch` for streaming — which Envoy and Kubernetes both drive natively.

```
backend checkout
  balance random(2)
  option httpchk
  http-check send meth GET uri /healthz ver HTTP/1.1 hdr Host checkou
t.internal
  http-check expect status 200
  # inter: normal interval; fastinter: when transitioning;
downinter: while down
  # rise/fall: hysteresis; slowstart: ramp weight from 0 over 60s aft
er coming up
  default-server inter 3s fastinter 500ms downinter 5s rise 2 fall 3
slowstart 60s maxconn 200
  server c1 10.0.1.11:8080 check
  server c2 10.0.1.12:8080 check
```

Two design constants deserve attention. **Hysteresis** (`rise/fall`, or Envoy's `healthy_threshold/unhealthy_threshold`) prevents flapping: one dropped probe on a lossy link should not eject a healthy backend, and one successful probe should not readmit a crashing one. Asymmetric thresholds are correct — quick to eject, slow to readmit — because sending traffic to a dead backend costs more than briefly under-using a live one. **Jitter** matters at scale: a 200-node balancer tier probing 5,000 backends once a second is a million probes per second, and unjittered intervals synchronize into spikes. Hence large fleets aggregate health through a discovery or control plane instead of probing point-to-point.

Passive health checks and outlier detection

Active probes ask "are you well?" on a synthetic path. Passive detection watches *real* traffic and ejects endpoints failing actual requests, catching what probes miss: a backend that serves `/healthz` fine but 500s on one route, a partial dependency failure, a bad canary.

Envoy's `outlier_detection` is the reference implementation:

```
outlier_detection:
  # consecutive-error detector
  consecutive_5xx: 5
  enforcing_consecutive_5xx: 100           # percent of the time the
  ejection is actually enforced
  consecutive_gateway_failure: 5
  # statistical detector: eject hosts whose success rate is an outlier
  vs the cluster mean
  success_rate_minimum_hosts: 5
  # need this many hosts before the stat is meaningful
  success_rate_request_volume: 100        # per host, per interval
  success_rate_stddev_factor: 1900       # divided by 1000 => 1.9
  standard deviations
  interval: 10s
  base_ejection_time: 30s
  # multiplied by the number of consecutive ejections
  max_ejection_percent: 10                # never eject more than 10%
  of the cluster
```

Note the two safety properties. Ejection time grows with repeated ejections, so a chronically bad host is removed for longer while a transiently bad one returns quickly. And `max_ejection_percent` caps the blast radius — because the single most dangerous behavior a health system can have is *ejecting everything*.

The panic threshold, or: how to survive being wrong

Suppose your backends all check a shared database in `/healthz`, and the database has a 30-second blip. Every backend fails its probe simultaneously. A naive balancer removes every endpoint and serves 100% errors — turning a degraded dependency into a total outage, even though the backends could still have served cached reads.

Envoy's answer is the **panic threshold**: if the fraction of healthy hosts in a cluster falls below a configured percentage (default 50%), Envoy *disregards health status entirely* and load balances across all hosts, healthy or not.

```
common_lb_config:
  healthy_panic_threshold:
    value: 50.0
# below 50% healthy, ignore health and use the whole pool
```

The reasoning: when most of your fleet looks unhealthy, the likeliest explanation is that the health signal is wrong or the failure is global — and either way, concentrating all traffic on the remaining "healthy" minority will immediately overload and kill it. This is the *fail-static* principle: **when the health data becomes implausible, stop trusting it**. AWS's Elastic Load Balancing implements the same idea as *fail-open*: if a target group has no healthy targets, the load balancer routes to all registered targets regardless of health, and newer target-group health settings let you raise the threshold at which fail-open kicks in.

The backend-side discipline is to keep health endpoints **shallow**. A liveness check answers "is this process able to serve?", not "is every dependency reachable?" If the database is down, every replica returning 503 destroys your ability to serve cached content or a useful error. Separate the concerns: liveness (restart me — in Kubernetes this literally kills the container), readiness (route to me), startup (still warming). Dependency checks belong at most in readiness, never in liveness.

Second: health checks must be served *under overload*. A backend that queues its probe behind 10,000 real requests times out, is ejected, and dumps its load on peers — which then do the same. That is a textbook cascading failure (Chapter 11; Volume 11, Chapter 10), triggered by a health check sharing a queue with production traffic. Serve health on a separate listener or thread, on a path load shedding never touches.

Slow start / warm-up

A newly-started backend is slow: cold JIT profiles, empty caches, unwarmed connection pools. Send it a full share immediately and it responds slowly, accumulates a queue, possibly fails health checks, is ejected, and repeats — and because it looks *idle* to a load-aware algorithm, least-request and P2C actively favor it, making things worse. This is the "new pod gets stampeded and crash-loops" pattern.

Slow start ramps the endpoint's effective weight from near zero to full over a window:

```

lb_policy: LEAST_REQUEST
least_request_lb_config:
  choice_count: 2
  slow_start_config:
    slow_start_window: 60s
    aggression:           # weight = w * max(min_weight_percent,
t_factor^(1/aggression))
    default_value: 2.0     # t_factor = elapsed/window; >1 ramps
faster early, 1.0 is linear
    runtime_key: cluster.checkout.slow_start_aggression
    min_weight_percent: 10 # floor, so a warming host is never
starved entirely; 10 is the default

```

HAProxy spells it `slowstart 60s` on the server line; NGINX Plus has `slow_start=60s`. If your balancer has no slow-start support, approximate it with a readiness probe that does not pass until the process has self-warmed (pre-populate caches, run a few synthetic requests, force JIT compilation of hot paths).

```

stateDiagram-v2
    [*] --> Starting
    Starting --> Warming: readiness passes
    Warming --> Serving: slow-start window elapsed
    Serving --> Probation: outlier detection trips
    Probation --> Serving: ejection time elapsed and probes pass
    Probation --> Ejected: repeat offender so ejection time grows
    Ejected --> Warming: active health check meets rise threshold
    Serving --> Draining: SIGTERM or deregistration
    Draining --> [*]: in-flight requests done or drain timeout
    Serving --> PanicMode: healthy fraction below threshold
    PanicMode --> Serving: health recovers

```

Session affinity

Affinity makes repeated requests from the same client reach the same backend. Three mechanisms, one recommendation.

Cookie-based affinity (L7). The balancer inserts a cookie naming the chosen backend and honors it on subsequent requests — the most precise mechanism, surviving NAT, mobile IP changes, and intermediate proxies.


```
backend app
  balance random(2)
  cookie SRVID insert indirect nocache httponly secure
  server a1 10.0.1.11:8080 check cookie a1
  server a2 10.0.1.12:8080 check cookie a2
```

`insert` adds the cookie, `indirect` strips it before the request reaches the server, and `nocache` stops a shared cache from storing a response whose `Set-Cookie` would then be served to other users — an omission that has caused real cross-user session leaks. AWS ALB offers both a balancer-managed cookie (`AWSALB`) and application-cookie stickiness.

IP hash / source affinity (L3/L4). Hash the client address. It needs no protocol awareness, but it is badly behaved: carrier-grade NAT and corporate egress collapse thousands of users onto one address (creating a hot backend), mobile clients change address mid-session, and you get no per-user control. NGINX's `ip_hash` — which for IPv4 hashes only the first three octets, so a whole /24 lands together — is the classic example; use it only when you have no L7 option.

Consistent hash on a key. Hash a header, cookie, or path segment with ring/Maglev hashing and bounded loads. This beats IP hash because you choose the key, and it is the right tool for *cache affinity* — routing all requests for `object_id` to the replica most likely to have it cached. It is a performance optimization, not a correctness mechanism, and should be built so that a wrong answer merely costs a cache miss.

The recommendation: prefer stateless backends. Affinity for *correctness* — because session state lives only in one process's memory — shows up later as uneven load no algorithm can fix, deploys that log every user out, and a backend failure that is a data-loss event rather than a retry. Put session state in a shared store (Redis, a signed cookie, a JWT) and keep affinity as a locality optimization you can disable at any time.

Connection draining and graceful shutdown

A rolling deploy replaces every instance in your fleet. If that shows up as a spike of 502s, your draining is broken. The correct sequence behind an L7 balancer:

1. **Announce departure before dying.** The instance starts failing readiness checks (or is deregistered from discovery) *while continuing*

to serve. This is the critical inversion: fail the probe first, keep working second.

2. **Wait for the balancer to notice** — at least one probe interval times the unhealthy threshold, plus discovery propagation. In Kubernetes, endpoint removal is eventually consistent across every kubelet and proxy, so a pod may receive traffic for seconds after `SIGTERM`.
3. **Stop accepting new work, finish in-flight work**. For HTTP/1.1, respond with `Connection: close`. For HTTP/2, send `GOAWAY` — Envoy and gRPC servers implement the two-stage graceful shutdown that RFC 9113 §6.8 describes: a first `GOAWAY` with `NO_ERROR` and a last-stream-ID of $2^{31} - 1$ as a "stop sending me new streams" hint, then after a drain interval a second `GOAWAY` naming the highest stream the server will actually finish.
4. **Hard-close after a drain timeout** so a hung request cannot block the deploy forever.

In Kubernetes the race in step 2 is the usual bug, and the standard mitigation is a `preStop` hook that delays `SIGTERM` long enough for endpoint removal to propagate:

```
spec:
  terminationGracePeriodSeconds: 60      # must exceed preStop +
  longest expected request
  containers:
    - name: api
      lifecycle:
        preStop:
          exec:
            command: ["/bin/sh", "-c", "sleep 15"]
      readinessProbe:
        httpGet: { path: /readyz, port: 8080 }
        periodSeconds: 2
        failureThreshold: 2
```

The matching balancer-side setting is the deregistration/drain timeout: AWS ALB and NLB use `deregistration_delay.timeout_seconds`, 300 seconds by default — usually far too long for a fast rollout, and worth tuning toward your p999 request duration plus margin. Envoy exposes `POST /healthcheck/fail` on its admin interface so an upstream tier drains it, plus `drain_time_s` and `drain_strategy` for its own shutdown. An L4 tier has no request boundary to drain on, so draining there means

stopping new flows from hashing to the backend while existing conntrack entries live out their timeout.

Topologies

Software proxies

HAProxy is a mature, very fast L4/L7 proxy with a multi-threaded event loop and excellent observability — the default choice for a dedicated LB tier with file-based configuration. **NGINX** balances competently, but its open source build omits active health checks, `slow_start`, and `sticky` cookies, which require NGINX Plus. **Envoy** is the proxy designed for dynamic environments: clusters, endpoints, routes, listeners, and secrets are all pushed at runtime over the **xDS** APIs from a control plane, which is what makes service meshes possible (Chapter 10). Its balancing feature set — locality-aware routing, subsetting, outlier detection, panic thresholds, retry budgets — is the most complete in open source, which is why this chapter uses it for most examples.

Cloud load balancers

AWS **NLB** is L4: flow-hash based, preserves the client IP for instance-registered targets, and supports TCP/UDP/TLS listeners. AWS **ALB** is L7: it parses HTTP/1.1, HTTP/2, and gRPC, routes on host, path, header, method, query string, and source IP, and therefore balances HTTP/2 per request rather than per connection. Google's global external Application Load Balancer is architecturally different: one **anycast** IP announced worldwide, connections terminated at the nearest Google Front End, and forwarded over Google's backbone to backends chosen by capacity and proximity — collapsing geo-routing and load balancing into one system.

DNS-based distribution

Returning multiple or weighted A/AAAA records is the cheapest possible "load balancer" and the one with the least control (Chapter 5). Its problems are structural: TTLs are advisory and resolvers cache aggressively; client libraries vary wildly in whether they try all returned addresses or only the first; there is no health awareness unless the provider does its own checking (Route 53 health checks with failover records); and traffic cannot shift faster than caches expire. DNS is a good

tool for coarse geographic distribution and a bad one for per-request balancing or fast failover.

Anycast, ECMP, and hyperscale L4

The hyperscale ingress pattern composes three mechanisms:

1. **BGP anycast** — the same VIP is announced from many sites. Internet routing delivers each client to (approximately) the topologically nearest site. Draining a site is a BGP withdrawal. See RFC 4786 for the operational guidance.
2. **ECMP** — inside the site, the routers see multiple next-hops for the VIP (one per balancer node, which announces the VIP over BGP from the host itself) and hash each packet's 5-tuple across them. This spreads traffic across the balancer tier with no balancer in front of the balancers.
3. **A consistent-hashing L4 forwarder** — Maglev, Katran, GLB, Unimog — on each balancer node, which maps the 5-tuple to a backend and encapsulates.

The composition is what makes it work. ECMP's hash is *stateless per router*, so when a balancer node joins or leaves, routers rehash and existing flows may land on a node that does not hold their conntrack entry. If that node chose backends by a plain hash over a changing set, the flow would break. Because every node runs the *same deterministic consistent hash over the same backend set*, a rerouted packet reaches the same backend anyway, and connection tracking becomes an optimization rather than a requirement. GitHub's GLB went further, encoding a *second* candidate backend in the GUE header so that a proxy which does not recognize a flow forwards it to the alternate instead of resetting it — surviving simultaneous balancer and backend membership changes.

Katran (Meta) implements this datapath as an **XDP/eBPF** program attached at the driver level, so packets are classified, hashed, and encapsulated before the kernel allocates an `sk_buff` — a large saving at multi-million-pps rates (Volume 2, Chapter 10; eBPF tooling in Volume 2, Chapter 11). Maglev instead used a userspace kernel-bypass forwarder sharing a packet pool with the NIC, with lock-free rings between its steering and processing threads.

```

flowchart TB
    CL["Clients worldwide"] -->|"anycast VIP 203.0.113.10"|
    INET["Internet / BGP"]
    INET -->|"nearest site wins"| S2["Site: Ashburn  
same VIP, same design"]
    INET -->|"nearest site wins"| S1["Site: Frankfurt"]
    S1 --> RTR["Site routers"]
    RTR -->|"ECMP hash on 5-tuple"|
    RTR --> M1["LB node 1  
XDP or bypass forwarder  
Maglev hash table"]
    RTR --> M2["LB node 2  
same table, same result"]
    RTR --> M3["LB node 3"]
    M1 -->|"IPIP or GUE encaps"| BE1["Backend / L7 proxy"]
    M2 -->|"encap"| BE1
    M2 -->|"encap"| BE2["Backend / L7 proxy"]
    M3 -->|"encap"| BE3["Backend / L7 proxy"]
    BE1 -->|"DSR: response direct to client"| CL
    BE2 -->|"DSR"| CL
    BE3 -->|"DSR"| CL

```

Client-side load balancing

The final topology removes the middlebox: the client resolves the endpoint set from service discovery and picks an endpoint itself. gRPC does this natively (name resolver plus LB policy, optionally driven by an xDS control plane); a service mesh achieves the same effect with a sidecar proxy on localhost, which is client-side balancing with the policy implemented out-of-process (Chapter 10).

What you gain: one fewer network hop and queue per request; no shared balancer to saturate or to become a correlated failure domain; and the client's own latency observations feeding the algorithm directly, which is what makes P2C+EWMA possible.

What you pay:

- **A fat client per language.** Discovery, health, retries, and balancing must be implemented and kept consistent in Go, Java, Python, Node, and Rust — the single biggest organizational argument for a sidecar mesh instead.
- **Connection count.** M clients $\times$ N backends. At $M = 2,000$ and $N = 500$ that is a million TCP connections, each with kernel memory, TLS state, and health-check overhead.

- **Discovery load.** Every client watches every endpoint set it uses.

The fix for the last two is **subsetting**: each client connects to a deterministically chosen subset of, say, 20–50 backends rather than all of them. The SRE book's datacenter chapter gives a deterministic subsetting algorithm that leaves backends with close to equal client counts; Envoy and gRPC's xDS integration expose equivalent controls. Smaller subsets mean fewer connections and worse balance, and below roughly 20 endpoints you start losing P2C's benefit and making single-backend failures visible to specific clients.

Distributed-systems lens

The balancer is a control plane, and its inputs are the risk.

Endpoint discovery, health state, and policy are three data flows with independent staleness, and a correct algorithm on a stale endpoint set produces a confidently wrong answer. Check the inputs before the algorithm: is the endpoint list current, is health state plausible, is the running config the one you think it is (Envoy's `/config_dump` and `/clusters admin endpoints`, `haproxy -c -f`, `aws elbv2 describe-target-health`)?

L4-vs-L7 and proxy-vs-client-side are latency, cost, and blast-radius decisions. Each L7 hop adds a queue and a parse; each shared proxy tier adds a correlated failure domain; each client-side implementation adds language surface area. Meshes won east-west traffic because a localhost sidecar gives L7 policy with client-side topology, at the cost of one process per pod. L4 tiers persist at the edge because nothing else forwards millions of packets per second per host.

P2C plus EWMA is the modern mesh default precisely because it needs no global state. With hundreds of independent choosers, any algorithm requiring a consistent global view of load is either stale (and herds) or expensive (and becomes a coordination bottleneck). P2C's guarantee holds under purely local information: *sampling two and comparing beats both global optimization and naive randomness, and it scales without coordination.*

Health checking is the mechanism by which load balancers cause correlated failure. Eject aggressively and a transient blip becomes a capacity crisis; eject conservatively and you serve errors from dead backends. The asymmetry to internalize: *ejecting too little is bounded,*

ejecting too much is not. Hence `max_ejection_percent`, panic thresholds, and shallow health endpoints. Combine bad health checks with retries (Chapter 11) and you get the full cascade — ejections shrink capacity, survivors overload, retries multiply load, more backends are ejected. Retry budgets, circuit breakers, and ejection caps all do the same job: bound the amplification factor of a feedback loop.

Consistent hashing turns a fleet's caches into one cache. With N replicas and random balancing, every replica caches the full hot set and your effective cache is one replica's memory. Hash on the cache key and each replica caches $1/N$ of the key space, making the effective cache N times larger. This is why cache tiers, CDN mid-tiers (Chapter 10), and shard-aware clients all hash — always with bounded loads, because real key distributions are Zipfian.

Anycast + ECMP + Maglev is the hyperscale ingress pattern, and its elegance is that every layer is stateless and independently scalable: BGP handles geography, ECMP the balancer tier, deterministic hashing the backend choice, encapsulation the return path. No layer needs to know what the others decided, and any layer can lose members without breaking flows.

Key takeaways

- **L4 balances flows, L7 balances requests.** That sentence predicts most behavioral differences between them, including the HTTP/2 pinning problem.
- **Choose the forwarding mode deliberately.** NAT is simple but routes responses through the balancer; DSR is fast but demands L2 adjacency and blinds the balancer to responses; IP/IP/GRE/GUE tunneling gives DSR semantics across L3 at an MTU cost.
- **HTTP/2 and gRPC break naive L4 balancing.** Fix it with an L7 proxy, client-side/subchannel balancing (`round_robin` or xDS policies, not the default `pick_first`), and bounded connection lifetimes so scale-ups actually receive traffic.
- **Round robin optimizes the wrong quantity** when request cost or backend capacity varies. Outstanding-work metrics self-correct; global least-loaded herds; **power-of-two-choices gets nearly all the benefit with none of the coordination**, and P2C scored by

EWMA latency × outstanding is the strongest general-purpose default.

- **Hash when you need locality, not balance** — with bounded loads, so a hot key cannot melt one backend. Maglev gives near-perfect balance and O(1) lookup; ring hashing gives minimal disruption.
- **Health checks must fail safe.** Shallow liveness, dependency checks only in readiness, hysteresis and jitter on probes, `max_ejection_percent` caps, and a panic threshold that disregards health data once it becomes implausible.
- **Slow-start new backends**, or load-aware algorithms will stampede the coldest replica you have.
- **Affinity is an optimization, not an architecture.** Stateless backends plus a shared session store beat stickiness on every axis except the first day of implementation.
- **Draining is a sequence, not a setting:** fail readiness while still serving, wait for propagation, refuse new work, finish in-flight, hard-close on a timeout.
- **Topology follows traffic direction.** Anycast + ECMP + a consistent-hashing XDP forwarder at the edge; L7 proxies for routing and policy; client-side or sidecar balancing with subsetting for east-west RPC.

Further reading

- D. E. Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," *USENIX NSDI 2016* — <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud> — ECMP plus consistent hashing plus kernel bypass, including the table-population algorithm reproduced above.
- Y. Azar, A. Z. Broder, A. R. Karlin, E. Upfal, "Balanced Allocations," *STOC 1994*; and M. Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing," *IEEE Transactions on Parallel and Distributed Systems* 12(10), 2001 (from his 1996 Berkeley thesis) — the theory behind P2C.
- D. Karger et al., "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web," *STOC 1997*; and V. Mirrokni, M. Thorup, M. Zadimoghaddam, "Consistent Hashing with Bounded Loads," arXiv <https://arxiv.org/>

abs/1608.01350 (2016), later *SODA 2018*, with the Google Research blog post of the same name on the Vimeo deployment. Volume 14, Chapter 5 covers both, plus rendezvous hashing.

- J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM* 56(2), February 2013 — why fanout turns a rare slow backend into a common slow request, and the hedging techniques that follow (Chapter 11).
- *Site Reliability Engineering* (Beyer, Jones, Petoff, Murphy; O'Reilly, 2016), Chapters 19 and 20 <https://sre.google/sre-book/load-balancing-datacenter/> — deterministic subsetting, weighted round robin from backend-reported utilization, and why simple least-loaded misbehaves.
- Envoy documentation: "Supported load balancers" https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers and "Outlier detection" https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier, plus the panic-threshold and slow-start pages of the same architecture overview.
- HAProxy configuration manual <https://docs.haproxy.org/> (`balance` , `hash-type` , `http-check` , `cookie` , `slowstart`) and NGINX's `ngx_http_upstream_module` https://nginx.org/en/docs/http/ngx_http_upstream_module.html (`least_conn` , `ip_hash` , `hash ... consistent` , `random` two [`least_conn`]).
- gRPC blog, "gRPC Load Balancing" <https://grpc.io/blog/grpc-load-balancing/> — proxy, client-side, and look-aside models; plus the load-balancing and service-config gRFCs in <https://github.com/grpc/proposal>, including ORCA-driven weighted round robin.
- Production L4 balancers: Katran <https://github.com/facebookincubator/katran> (Meta, XDP/eBPF, IP/IP), open-sourced in 2018; GitHub's "GLB: GitHub's open source load balancer" (August 2018) <https://github.blog/engineering/infrastructure/glb-director-open-source-load-balancer/> and the `github/glb-director` repository; and Cloudflare's "Unimog — Cloudflare's edge load balancer" (September 2020) <https://blog.cloudflare.com/unimog-cloudflares-edge-load-balancer/>.
- Linkerd's load-balancing reference <https://linkerd.io/docs/reference/load-balancing/> and Finagle's client documentation <https://twitter.github.io/finagle/guide/Clients.html> for P2C and peak-EWMA.

- RFC 4786 (BCP 126), *Operation of Anycast Services* (December 2006) <https://www.rfc-editor.org/rfc/rfc4786>; and RFC 9113 <https://www.rfc-editor.org/rfc/rfc9113>, §6.8 (GOAWAY and graceful shutdown) and §9.1 (connection reuse).
- Linux Virtual Server / IPVS documentation and `ipvsadm(8)`, including the `mh` (Maglev hashing) scheduler added in Linux 4.18 and exposed by `ipvsadm v1.30`; Volume 2, Chapter 10 covers where these sit in the kernel.
- AWS documentation for Application and Network Load Balancers — target groups, health check parameters, `deregistration_delay.timeout_seconds`, stickiness — and Google Cloud's external Application Load Balancer overview for the anycast/GFE architecture.

Load balancing rewards precision. The algorithms are simple enough to implement in an afternoon; the interactions are where systems fail — a multiplexed protocol meeting a per-connection balancer, a dependency check inside a liveness probe, a cold replica meeting a load-aware algorithm, a retry policy meeting an aggressive ejector. Chapter 10 places these balancers into proxy, mesh, and CDN topologies; Chapter 11 supplies the timeout, retry, and hedging policies that decide whether a slow backend is a blip or an outage.

Chapter 10 — Proxies, Reverse Proxies, Service Mesh, and CDNs

What this chapter covers. Almost nothing in a modern backend talks directly to anything else. Between a browser and your handler there is a CDN edge, an anycast load balancer, a TLS terminator, an API gateway, an ingress proxy, and — if you run a mesh — a sidecar on each end of every internal hop. Each of these is a *proxy*: a process that terminates one connection, makes a policy decision, and originates another. This chapter is about that layer. We start with the taxonomy people routinely muddle — forward proxies (client-side egress control) versus reverse proxies (server-side ingress) — and then treat the reverse proxy as what it has become: the universal front door where TLS termination (Chapter 6), HTTP routing (Chapter 7), load balancing (Chapter 9), buffering, timeouts, and observability are implemented once instead of a hundred

times. We go deep on Envoy, whose listener/filter/cluster model and xDS dynamic-configuration APIs are the reason the service mesh became buildable. Then the mesh itself: the problem it solves, the mechanism, what it buys you, and — honestly — what it costs, including the sidecar-less designs built to reduce that cost. Finally, CDNs: PoPs and anycast (Chapter 2), cache hierarchies and origin shielding, cache keys and Cache-Control semantics (Chapter 7), purging, edge compute, and TLS/QUIC termination at the edge (Chapters 4 and 6).

Learning goals — after this chapter you should be able to:

- Distinguish forward from reverse proxying precisely, and explain what each buys operationally.
- Configure a reverse proxy correctly for a production HTTP service: timeouts at every stage, buffering choices, connection reuse, client-IP preservation, and failure semantics.
- Describe Envoy's architecture — listeners, filter chains, HTTP filters, routes, clusters, endpoints — and explain the xDS protocol including ACK/NACK, versioning, ADS, and delta xDS.
- Explain what an API gateway adds over a plain reverse proxy, and where those concerns belong.
- Explain mechanically how a sidecar mesh intercepts traffic, how identity is issued and rotated, and how the control plane programs the data plane.
- Decide whether your organization needs a mesh, using latency, resource, and operational cost as inputs rather than fashion, and describe sidecar-less designs without overclaiming.
- Reason about CDN behavior: cache key composition, TTL and revalidation semantics, request collapsing, invalidation strategies, origin shield topology, and what edge compute is for.

One idea, two directions

A proxy is an intermediary that terminates a connection from one party and opens a connection to another on its behalf. That is the whole idea. Everything interesting follows from a single question: *on whose behalf does it act, and who chose it?*

A **forward proxy** acts for the client and is configured by the client (or imposed transparently by the network). In HTTP terms, an explicitly

configured forward proxy receives requests with an *absolute-form* request target — `GET http://example.com/api HTTP/1.1` — rather than the origin-form path a server sees, and for TLS it receives `CONNECT example.com:443 HTTP/1.1` and then blindly tunnels bytes. That `CONNECT` tunnel is why forward proxies see so little of encrypted traffic: unless they perform TLS interception with a private CA installed on every client (a "MITM proxy", used by corporate DLP stacks and by tools like `mitmproxy`), all they observe is the SNI in the ClientHello, the destination address, and byte counts. Encrypted Client Hello (ECH, Chapter 6) removes even the SNI where deployed.

Backend engineers meet forward proxies mostly as **egress control**. In a serious production network, workloads have no default route to the internet: outbound traffic goes through an egress proxy or NAT gateway that enforces a destination allowlist, logs every connection, and rate-limits per destination. The security value is containment — if an attacker achieves code execution in a pod, an egress allowlist is the difference between quiet exfiltration and a blocked connection with an alert — the network half of the containment story Volume 0 tells about compromised build and runtime environments. The operational value is one place that knows every external service your fleet depends on. `HTTP_PROXY`, `HTTPS_PROXY`, and `NO_PROXY` are the lingua franca; most HTTP clients honor them, which is both convenient and a trap, since `NO_PROXY` matching semantics differ subtly between runtimes: whether a leading dot is required for suffix matches, whether ports and CIDRs are honored, whether matching is case-sensitive. Verify per runtime rather than assuming your string means what you intended.

A **reverse proxy** acts for the server and is chosen by the server operator. Clients believe they are talking to the origin; they address `api.example.com` and the proxy answers. This inversion is what makes the reverse proxy the natural place for every cross-cutting concern: it is on the path of every request, it is operated by the platform team, and it can be changed without touching application code.

```

flowchart LR
    subgraph CN["Client network"]
        C1["Laptop or pod"]
        C2["Another workload"]
        FP["Forward proxy"]
    end
    egress allowlist, logging, cache"]
    end
    subgraph DC["Provider infrastructure"]
        RP["Reverse proxy"]
    end
    TLS termination, routing, WAF, LB"]
    S1["Service A"]
    S2["Service B"]
    end
    C1 -->|"explicit proxy config or CONNECT"| FP
    C2 --> FP
    FP -->|"allowlisted destinations only"| RP
    RP --> S1
    RP --> S2
    classDef pxy fill:#e8f0fe,stroke:#3b6fd4,stroke-width:1px
    class FP,RP pxy

```

The symmetry is worth internalizing: a forward proxy hides many clients behind one identity and enforces *what may be reached*; a reverse proxy hides many servers behind one name and enforces *what may be served*. In a mesh, one sidecar is simultaneously both.

The reverse proxy as universal front door

Once a reverse proxy sits in front of everything, the question is what it should do. The honest answer is: quite a lot, because each of these things is done badly when scattered across services.

TLS termination and origination. The proxy holds the certificates, negotiates TLS 1.3, selects ALPN (`h2` , `http/1.1` , and for QUIC `h3`), and pins the cipher policy fleet-wide. Rotating a certificate or disabling a protocol version becomes a config change instead of a fleet-wide redeploy. The proxy may then re-encrypt to the backend (mTLS in a mesh) or speak cleartext inside a trusted segment — decide that explicitly, not by accident.

Protocol translation. Clients speak HTTP/2 or HTTP/3 to the edge; backends often speak HTTP/1.1. The proxy demultiplexes streams into separate upstream requests, which is what makes HTTP/2's connection pinning problem tractable (Chapters 7 and 9). A well-configured proxy

also maintains long-lived keepalive pools upstream, sparing backends the connection churn that would otherwise arrive from the internet.

Routing. Host, path, header, method, query, and weight-based matching — where a monolith gets carved up: `/checkout/*` goes to the new service while everything else keeps hitting the legacy app, and neither client nor legacy app knows.

Buffering and flow control. Whether the proxy buffers a request body before contacting the upstream is one of the most consequential and least-understood settings on the box. Buffering protects slow backends from slow clients — the classic slowloris defense — by absorbing a trickled request and delivering it upstream in one burst, freeing an application worker that would otherwise sit blocked. It also breaks streaming uploads and adds latency and memory pressure. NGINX buffers request bodies by default (`proxy_request_buffering on`); Envoy streams by default, which is right for gRPC and long-lived streams but leaves slow-client defense to timeouts and connection limits.

Timeouts. A reverse proxy has at least four independent clocks: connect timeout to upstream, time to receive request headers from the client, time waiting for the upstream response, and an overall route timeout. Getting these wrong is a leading cause of resource exhaustion; getting them right is Chapter 11's material, but the proxy is where most of them are configured.

Header manipulation and client-IP preservation. The moment you add a proxy, `REMOTE_ADDR` at the application is the proxy's address. `X-Forwarded-For` is the de facto convention, `Forwarded` (RFC 7239) the standardized one, and both are *client-controlled* unless the proxy overwrites rather than appends them at the trust boundary. Get this wrong and your rate limiter and audit log become forgeable by anyone who sets a header. For L4 proxies that cannot rewrite HTTP, the PROXY protocol (v1 text, v2 binary) prepends the original source address to the TCP stream — supported by NGINX, Envoy, HAProxy, and every major cloud NLB.

Compression, caching, and observability. Content-encoding negotiation, a local response cache, and a uniform access log carrying latency, upstream selection, retry counts, and failure reason codes. That log is often the highest-fidelity telemetry in the system, because it sees every request — including the ones that never reached an application.

The field

Proxy	Config model	Strengths	Notable limits
NGINX	Static file, reload on change	Ubiquitous, fast, huge module ecosystem	Dynamic upstream API is a commercial feature; no native xDS
HAProxy	Static file plus Runtime API	Outstanding L4/L7 balancing, deep health checking	Idiosyncratic config language; less extensible in-process
Envoy	Static bootstrap plus xDS	Fully dynamic config, rich L7 filters, first-class gRPC	Heavier footprint; large configuration surface
Caddy	Caddyfile or JSON API	Automatic HTTPS via ACME, sane defaults	Smaller ecosystem for exotic L7 needs
Traefik	Dynamic providers (Kubernetes, labels, files)	Auto-discovery in container platforms	Fewer low-level knobs than Envoy or HAProxy

A concrete NGINX front door, with every timeout named:

```

upstream checkout {
    zone checkout 64k;
    least_conn;
    keepalive 64;
    server 10.0.3.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.3.12:8080 max_fails=3 fail_timeout=10s;
}

server {
    listen 443 ssl;
    http2 on;
    server_name api.example.com;

    ssl_certificate      /etc/ssl/api.example.com.crt;
    ssl_certificate_key  /etc/ssl/api.example.com.key;
    ssl_protocols        TLSv1.2 TLSv1.3;

    client_header_timeout 5s;
    client_body_timeout   10s;

    location /checkout/ {
        proxy_pass          http://checkout;
        proxy_http_version  1.1;
        proxy_set_header    Connection "";           # enable upstream
    keepalive
        proxy_set_header    Host $host;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;

        proxy_connect_timeout 2s;
        proxy_send_timeout    10s;
        proxy_read_timeout    10s;

        proxy_next_upstream   error timeout http_502 http_503;
        proxy_next_upstream_tries 2;
        proxy_request_buffering off;                 # stream uploads
    through
    }
}

```

Two details matter. `proxy_set_header Connection ""` is required for upstream keepalive to work at all, because NGINX otherwise sends `Connection: close` upstream and the connection is torn down after each request; it must be paired with `proxy_http_version 1.1`, since the default upstream version is 1.0. And `proxy_next_upstream` is a *retry* policy hiding in plain sight. NGINX is conservative here by default: once a request has been sent upstream, it will not be replayed to another server if the method is non-idempotent (POST, LOCK, PATCH) unless you add the

`non_idempotent` parameter. Do not add it casually — that parameter is how `http_503` on a POST becomes a duplicate order (Chapter 11). Other proxies are less cautious: Envoy's `retry_on` applies to any method, so an Envoy retry policy needs the same scrutiny without a safety net. The HAProxy equivalent, which by default retries only connection-establishment failures:

```
defaults
    mode http
    timeout connect 2s
    timeout client 30s
    timeout server 30s
    option httplog
    option redispatch
    retries 2

frontend fe_api
    bind :443 ssl crt /etc/haproxy/certs/api.pem alpn h2,http/1.1
    http-request set-header X-Forwarded-Proto https
    http-request del-header X-Forwarded-For          # drop client-
supplied value
    option forwardfor                                # then set it
ourselves
    acl is_checkout path_beg /checkout/
    use_backend be_checkout if is_checkout
    default_backend be_web

backend be_checkout
    balance leastconn
    option httpchk GET /healthz
    http-check expect status 200
    server c1 10.0.3.11:8080 check inter 2s fall 3 rise 2 maxconn 200
    server c2 10.0.3.12:8080 check inter 2s fall 3 rise 2 maxconn 200
```

The `del-header` followed by `option forwardfor` is the trust-boundary pattern: destroy whatever the client claimed, then assert the truth.

Envoy in depth: the programmable proxy

Envoy, built at Lyft and open-sourced in 2016 (a CNCF graduated project since 2018), matters here for two reasons: its configuration model is a clean decomposition of what a proxy actually does, and it was designed from the start to be configured by a remote control plane rather than a file. The second point is load-bearing — without it there is no mesh.

The object model

- **Listener** — a bound address and port plus a chain of **network filters**.
- **Filter chain** — an ordered list of L4 filters selected by a *filter chain match* (SNI, transport protocol, source/destination IP, ALPN), terminating in something that either proxies bytes (`tcp_proxy`) or parses HTTP (`http_connection_manager` , universally "HCM").
- **HTTP filters** — inside HCM, an ordered chain of L7 filters: JWT authentication, external authorization, rate limiting, CORS, fault injection, compression, WASM or Lua extensions, and finally the terminal `router` filter. Ordering is semantic; the router must be last.
- **Route configuration** — virtual hosts matched by `:authority` , each with routes matched by path/header/query, each naming a cluster (or a weighted set), a timeout, a retry policy, header mutations, and hedging.
- **Cluster** — a logical upstream: a discovery type (STATIC, STRICT_DNS, LOGICAL_DNS, EDS, ORIGINAL_DST), a load-balancing policy, circuit-breaker thresholds, outlier detection, health checks, and a transport socket for upstream TLS.
- **Endpoints** — a cluster's host:port members with locality, weight, and health status. In a mesh these arrive by EDS and change constantly.

A static bootstrap that terminates TLS, routes, and gets its endpoints dynamically:

```

node:
  id: edge-proxy-1
  cluster: edge

static_resources:
  listeners:
    - name: ingress_https
      address:
        socket_address: { address: 0.0.0.0, port_value: 8443 }
      filter_chains:
        - transport_socket:
            name: envoy.transport_sockets.tls
            typed_config:
              "@type": type.googleapis.com/
envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
              common_tls_context:
                alpn_protocols: ["h2", "http/1.1"]
                tls_certificates:
                  - certificate_chain: { filename: "/etc/envoy/certs/
api.crt" }
                    private_key:      { filename: "/etc/envoy/certs/
api.key" }
              filters:
                - name: envoy.filters.network.http_connection_manager
                  typed_config:
                    "@type": type.googleapis.com/
envoy.extensions.filters.network.http_connection_manager.v3.HttpConnect
ionManager
                    stat_prefix: ingress_http
                    use_remote_address: true
                    xff_num_trusted_hops: 1
                    request_timeout: 15s
                    route_config:
                      name: local_route
                      virtual_hosts:
                        - name: api
                          domains: ["api.example.com"]
                          routes:
                            - match: { prefix: "/checkout/" }
                              route:
                                cluster: checkout
                                timeout: 10s
                                retry_policy:
                                  retry_on: "5xx,reset,connect-failure"
                                  num_retries: 2
                                  per_try_timeout: 3s
                    http_filters:
                      - name: envoy.filters.http.router
                        typed_config:
                          "@type": type.googleapis.com/

```

```
envoy.extensions.filters.http.router.v3.Router
```

```
clusters:
```

```
- name: checkout
  connect_timeout: 2s
  lb_policy: LEAST_REQUEST
  type: EDS
  eds_cluster_config:
    eds_config:
      resource_api_version: V3
      api_config_source:
        api_type: GRPC
        transport_api_version: V3
        grpc_services:
          - envoy_grpc: { cluster_name: xds_cluster }
```

```
circuit_breakers:
```

```
  thresholds:
    - priority: DEFAULT
      max_connections: 1024
      max_pending_requests: 128
      max_requests: 1024
      max_retries: 32
```

```
outlier_detection:
```

```
  consecutive_5xx: 5
  interval: 10s
  base_ejection_time: 30s
  max_ejection_percent: 50
```

```
- name: xds_cluster
  connect_timeout: 1s
  type: STRICT_DNS
  typed_extension_protocol_options:
    envoy.extensions.upstreams.http.v3.HttpProtocolOptions:
      "@type": type.googleapis.com/
```

```
envoy.extensions.upstreams.http.v3.HttpProtocolOptions
```

```
  explicit_http_config:
    http2_protocol_options: {}
```

```
load_assignment:
```

```
  cluster_name: xds_cluster
  endpoints:
```

```
    - lb_endpoints:
      - endpoint:
          address:
            socket_address: { address: control-plane.mesh.svc,
port_value: 15010 }
```

```
admin:
```

```
  address:
    socket_address: { address: 127.0.0.1, port_value: 15000 }
```

Note `use_remote_address: true` with `xff_num_trusted_hops: 1`: that tells Envoy how many trusted proxies sit in front of it, so it computes the real client address instead of trusting an arbitrary XFF chain.

xDS: configuration as an eventually-consistent subscription

The x Discovery Service protocol is a family of gRPC (or REST-polling) APIs by which a control plane streams configuration to a proxy. Each resource type has its own service:

API	Resource	Answers the question
LDS	Listener	What should I listen on, with which filter chains?
RDS	RouteConfiguration	For this HCM, how do I map requests to clusters?
CDS	Cluster	What upstream services exist and how do I talk to them?
EDS	ClusterLoadAssignment	Which endpoints are in this cluster right now?
SDS	Secret	What certificates and trust bundles should I use?
Others	VHDS, SRDS, ECDS, RTDS	On-demand virtual hosts, scoped routes, extensions, runtime flags

Mechanically, the proxy opens a bidirectional gRPC stream and sends a `DiscoveryRequest` naming a type URL (for example `type.googleapis.com/envoy.config.cluster.v3.Cluster`), the resource names it wants (empty means "all"), and the last `version_info` and `response_nonce` it accepted. The server replies with a `DiscoveryResponse` carrying a `version_info`, a `nonce`, and the resources. The proxy then **ACKs** by echoing that version and nonce, or **NACKs** by echoing the *previous* good version plus an `error_detail`. That is the single most useful xDS fact for debugging: a proxy rejecting bad config keeps running the last good config, so the failure appears as a NACK on the control plane rather than an outage — and "I pushed config and nothing happened" is nearly always a NACK nobody looked for.

Two refinements matter in practice. **ADS (Aggregated Discovery Service)** multiplexes all resource types onto one stream to one management server, which is the only way to get ordering guarantees. The resources are cross-referential — a route names a cluster, a cluster's endpoints arrive by EDS — and xDS is only eventually consistent, so a route that arrives before the cluster it names fails requests with a 503 and the `NC` ("no cluster found") response flag until the cluster shows up. Envoy's documented sequencing is therefore make-before-break: CDS, then EDS, then LDS, then RDS on the way in, and the reverse on the way out. Independent per-type streams, possibly to different management servers, cannot provide that ordering. **Delta (incremental) xDS** sends only changed resources and explicit removals instead of the full state of the world. In a mesh with tens of thousands of churning endpoints, state-of-the-world pushes dominate both control-plane and proxy CPU; delta is what keeps large meshes affordable.

This is why xDS made the mesh possible. Every prior generation of proxy needed a config file and a reload to learn that an endpoint appeared or vanished; in a cluster where pods churn continuously, a reload-based proxy is either always stale or always reloading. xDS turns configuration into a *subscription to a stream of changes* — the correct model for an environment whose topology is itself a stream of changes. It is also deliberately a universal API: gRPC clients implement xDS directly, giving client-side balancing with no proxy at all (Chapter 9).

One runtime note: Envoy runs a small number of worker threads, each an independent event loop over its own connection set, so it scales with cores — but its per-instance memory is dominated by the *size of its configuration*, which makes config scoping a first-order mesh optimization.

API gateways: a reverse proxy with product concerns

An API gateway is a reverse proxy plus a set of concerns about the *API as a product* rather than about transport. The distinction is fuzzy — most gateways are Envoy or NGINX underneath — but the concerns are real:

- **Authentication and authorization at the edge.** Validating JWTs against a JWKS endpoint, introspecting opaque tokens, verifying mTLS client certs, mapping API keys to consumers. Terminating authentication here means backends can trust a verified identity

header — provided the gateway strips any client-supplied version of that header.

- **Rate limiting and quotas.** Per-consumer, per-plan, per-route limits, usually enforced against a shared counter service so limits are global rather than per-proxy-instance. Envoy's design is instructive: the local rate limit filter is fast and per-instance; the global one calls an external service over gRPC per request, trading latency for correctness.
- **Transformation and aggregation.** Rewriting between external and internal schemas, protocol bridging (JSON/HTTP to gRPC transcoding), and fan-out aggregation for clients that cannot make five round trips. Aggregation slides easily into business logic in the proxy; keep it mechanical and prefer a backend-for-frontend service once it grows.
- **Lifecycle concerns:** versioning, deprecation headers, developer portals, usage metering.

Token formats and authorization models belong to Volume 9 — Security, Authentication, and Cryptography (Chapter 5 on tokens and JWTs, Chapter 6 on OAuth 2.0 and OIDC, Chapter 7 on authorization models); the gateway is the enforcement point, not the policy source. Below, Envoy validates JWTs and calls an external authorizer, as HTTP filters ahead of the router:

```

http_filters:
- name: envoy.filters.http.jwt_authn
  typed_config:
    "@type": type.googleapis.com/
envoy.extensions.filters.http.jwt_authn.v3.JwtAuthentication
  providers:
    idp:
      issuer: https://idp.example.com/
      audiences: ["api.example.com"]
      remote_jwks:
        http_uri:
          uri: https://idp.example.com/.well-known/jwks.json
          cluster: idp_jwks
          timeout: 3s
        cache_duration: 600s
      forward_payload_header: x-jwt-claims
  rules:
  - match: { prefix: "/public/" }          # no requirement
  - match: { prefix: "/" }
    requires: { provider_name: idp }
- name: envoy.filters.http.ext_authz
  typed_config:
    "@type": type.googleapis.com/
envoy.extensions.filters.http.ext_authz.v3.ExtAuthz
  transport_api_version: V3
  grpc_service:
    envoy_grpc: { cluster_name: authz }
    timeout: 0.2s
  failure_mode_allow: false
- name: envoy.filters.http.router
  typed_config:
    "@type": type.googleapis.com/
envoy.extensions.filters.http.router.v3.Router

```

`failure_mode_allow` is the fail-open/fail-closed switch, and a genuine architectural decision: fail-closed makes an authorizer outage a total outage; fail-open makes it an authorization bypass. For most systems the answer is fail-closed with an aggressively cached decision layer — but say it out loud in a design review rather than letting a default decide.

The industry is converging on the Kubernetes **Gateway API** (`gateway.networking.k8s.io`) as the portable expression of this layer — `GatewayClass`, `Gateway`, `HTTPRoute`, `GRPCRoute` — with implementations including Envoy Gateway, Istio, Contour, NGINX, and cloud load balancers. It replaces the underspecified `Ingress` resource and its annotation sprawl, and its role-oriented split matches how platform teams actually divide responsibility.

The service mesh: the problem before the solution

Consider what a *correct* service-to-service call requires in a large fleet. It must discover healthy instances and balance across them per request, not per connection. It must apply and propagate a deadline. It must retry idempotent failures with jittered backoff and a retry budget, and must not retry anything else. It must shed load when the upstream is saturated and eject failing hosts. It must authenticate both ends cryptographically and authorize the call. It must emit rate, error, and latency metrics with consistent labels, and propagate trace context.

Now consider the pre-mesh way to get all that: a client library. Twitter's Finagle and Netflix's Ribbon/Hystrix/Eureka stack did exactly this, and did it well — on the JVM. Google's internal Stubby (gRPC's ancestor) did it across several languages, but only because a dedicated infrastructure team was funded to keep those implementations in agreement, which is precisely the resource most organizations do not have. The moment your fleet has Java, Go, Python, Node, and Rust services, you need five implementations that agree on retry semantics, load-balancing behavior, metric names, header formats, and certificate handling. They will not agree. Worse, changing any of them — tightening a default timeout, fixing a retry-storm bug, rotating a trust anchor — requires every team to bump a dependency and redeploy, which across hundreds of services is a quarter-long migration you will run repeatedly. The cross-cutting concern is not the hard part; its *uniform, independently upgradable rollout* is.

The mesh's proposition: move that logic out of the process into a proxy beside it, and control all such proxies from one place. The application makes a plain call to a service name; the proxy does the discovery, balancing, retrying, encrypting, and measuring.

Data plane and control plane

The **data plane** is the set of proxies on the request path. Classically there is one per workload instance — a *sidecar* container in the same pod, sharing its network namespace and therefore its loopback interface and IP. The sidecar is normally Envoy (Istio, Consul, most commercial meshes) or a purpose-built proxy (Linkerd's `linkerd2-proxy`, written in Rust and deliberately minimal — it does not chase Envoy's extensibility, which is how it stays small and fast).

Interception is the mechanical crux. The application is unmodified and does not know the proxy exists, so its traffic must be redirected. Istio installs iptables rules in the pod's network namespace at startup, either from an init container with `NET_ADMIN` or, preferably, from a node-level CNI plugin that avoids granting that capability to every workload. Abridged, the `nat` table looks like this:

```
# Inbound: everything arriving at the pod goes to the sidecar's inbound
port
-A PREROUTING -p tcp -j ISTIO_INBOUND
-A ISTIO_INBOUND -p tcp --dport 15020 -j RETURN      # agent, merged
metrics
-A ISTIO_INBOUND -p tcp --dport 15021 -j RETURN      # health probes
-A ISTIO_INBOUND -p tcp --dport 15090 -j RETURN      # Envoy Prometheus
telemetry
-A ISTIO_INBOUND -p tcp -j ISTIO_IN_REDIRECT
-A ISTIO_IN_REDIRECT -p tcp -j REDIRECT --to-ports 15006

# Outbound: everything the app sends goes to the sidecar's outbound
port
-A OUTPUT -p tcp -j ISTIO_OUTPUT
-A ISTIO_OUTPUT -m owner --uid-owner 1337 -j RETURN # traffic from the
proxy itself
-A ISTIO_OUTPUT -d 127.0.0.1/32 -j RETURN           # loopback stays
local
-A ISTIO_OUTPUT -j ISTIO_REDIRECT
-A ISTIO_REDIRECT -p tcp -j REDIRECT --to-ports 15001
```

The `--uid-owner 1337` exclusion prevents an infinite redirect loop: the proxy runs as that UID, so its own outbound connections are not re-captured. `REDIRECT` rewrites the destination to the local port while the kernel preserves the original, retrievable via `SO_ORIGINAL_DST` — that is how Envoy's `ORIGINAL_DST` listener learns where the application was actually going, and applies the right route and cluster.

The **control plane** computes configuration and pushes it via xDS. In Istio this is `istiod`, one binary that absorbed the older Pilot/Citadel/Galley split. It watches the Kubernetes API (Services, EndpointSlices, Pods) plus Istio's CRDs, translates that model into Envoy listeners, routes, clusters, and endpoints, and streams it to every sidecar over ADS. It is also the certificate authority: workloads request certificates over SDS on the same channel, and the agent beside each proxy handles key generation and rotation without ever writing a private key to disk. Linkerd's control plane is similar in intent (destination, identity, and policy controllers)

while pushing a much smaller configuration surface to a much smaller proxy.

```
flowchart TB
    subgraph CP["Control plane"]
        ISTIOD["istiod"]
    end
    xDS["xDS server plus CA"]
    K8S["Kubernetes API Services, EndpointSlices, CRDs"]
    K8S --> ISTIOD
    end
    subgraph PodA["Pod A"]
        APPA["App A"]
        SCA["Sidecar proxy A"]
        APPA -->|"loopback, plaintext"| SCA
    end
    subgraph PodB["Pod B"]
        SCB["Sidecar proxy B"]
        APPB["App B"]
        SCB -->|"loopback, plaintext"| APPB
    end
    end
    ISTIOD -->|"xDS config plus SDS certs"| SCA
    ISTIOD -->|"xDS config plus SDS certs"| SCB
    SCA ==>|"mTLS, SPIFFE identities, L7 routing"| SCB
    classDef cp fill:#fff3e0,stroke:#c77800
    classDef dp fill:#e8f0fe,stroke:#3b6fd4
    class ISTIOD,K8S cp
    class SCA,SCB dp
```

The dotted lines are the control path, the doubled line the data path, and the separation is the whole point: a control-plane outage stops *changes* from propagating but should not stop traffic, because each proxy keeps serving its last known-good configuration. Verify that property with a game day rather than assuming it.

What the mesh actually gives you

Identity and mutual TLS

This is the strongest single argument for a mesh. Each workload receives a short-lived X.509 certificate whose SAN is a **SPIFFE ID** URI — for example `spiffe://cluster.local/ns/payments/sa/checkout` — derived from its Kubernetes namespace and service account. That is a *workload* identity, not a network identity: it does not depend on ephemeral IP addresses and it survives rescheduling. SPIFFE specifies the ID format

and the X.509-SVID document, SPIRE is the reference implementation of an issuer, and Istio's built-in CA issues SPIFFE-compatible SVIDs directly (Volume 9, Chapter 10 — Zero Trust and Service-to-Service Auth — covers workload identity in depth). Certificates are issued at workload startup, are short-lived — Istio's default workload certificate lifetime is on the order of a day, rotated well before expiry — and are delivered over SDS into proxy memory rather than mounted as secrets.

With that in place, mTLS between every pair of workloads is a policy toggle rather than a project, and authorization becomes identity-based rather than IP-based:

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: payments
spec:
  mtls:
    mode: STRICT          # reject plaintext; PERMISSIVE accepts both
    during migration
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: checkout-callers
  namespace: payments
spec:
  selector:
    matchLabels: { app: checkout }
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/storefront/sa/web"]
      to:
    - operation:
        methods: ["POST"]
        paths: ["/v1/checkout"]
```

PERMISSIVE mode is the migration mechanism: it lets a service accept both plaintext and mTLS while callers are converted, and forgetting to move to **STRICT** afterwards is the most common way organizations end up with a mesh that provides *encryption in principle* and plaintext in practice. Audit it.

Traffic management

Per-request L7 load balancing (Chapter 9) comes free, including the fix for HTTP/2 and gRPC connection pinning, since the sidecar balances individual streams across endpoints. Retries, timeouts, and outlier-detection-based circuit breaking (Chapter 11) become declarative, and traffic splitting enables canary and blue/green releases without a deployment-tool hack (Volume 11):

```

apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: checkout
spec:
  hosts: ["checkout.payments.svc.cluster.local"]
  http:
  - match:
    - headers:
        x-canary: { exact: "true" }
    route:
    - destination: { host: checkout, subset: v2 }
  - route:
    - destination: { host: checkout, subset: v1 }
      weight: 95
    - destination: { host: checkout, subset: v2 }
      weight: 5
  timeout: 3s
  retries:
    attempts: 2
    perTryTimeout: 1s
    retryOn: connect-failure,refused-stream,unavailable
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: checkout
spec:
  host: checkout
  trafficPolicy:
    connectionPool:
      http: { http2MaxRequests: 1000, maxRequestsPerConnection: 0 }
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
  subsets:
  - name: v1
    labels: { version: v1 }
  - name: v2
    labels: { version: v2 }

```

Note the header-based override *before* the weighted split: that ordering gives deterministic canary testing (send `x-canary: true`, always hit v2) alongside probabilistic rollout. The Gateway API's GAMMA initiative is standardizing an equivalent expression of east-west routing.

Telemetry

Every sidecar emits the same metrics with the same labels for every service regardless of language: request and error counts by response code, duration histograms, TCP bytes, connection counts — all dimensioned by source and destination workload, namespace, and service. That uniformity is worth more than any individual metric, because it makes fleet-wide SLO dashboards and a service dependency graph derivable rather than hand-built (Volume 11). One caveat, stated plainly because it is routinely misunderstood: **a mesh cannot give you distributed tracing for free**. A sidecar can generate and forward spans, but it cannot know that inbound request X caused outbound request Y unless the application copies trace context headers (`traceparent` , or the B3 family) from the request it handles onto the requests it makes. Without in-process propagation you get disconnected single-hop spans, not traces.

The costs, honestly

Latency. Every meshed call traverses two extra proxies — the caller's sidecar and the callee's — each adding kernel crossings, a userspace parse and policy evaluation, and TLS work. Vendor benchmarks generally report per-hop overhead in the sub-millisecond to low-single-digit-millisecond range at moderate load; treat every published number as approximate and specific to a version and workload, and measure your own. The structure is not approximate, though: the tax is per hop, so a chain of seven services makes six calls and pays it twelve times, and it lands hardest on services whose own p50 is a few hundred microseconds.

Resources. A sidecar's memory is dominated by its configuration — the clusters, routes, and endpoints it knows about — and its CPU scales with request rate and TLS work. In a cluster with thousands of pods and no scoping, every sidecar holding config for every service is a real, avoidable expense; Istio's `Sidecar` resource and delta xDS both cut it substantially. Multiply the residual per-pod cost by pod count before you sign off.

Operational complexity. The mesh is a distributed system you now operate in addition to your own. Version skew between control plane and data plane must be managed (both Istio and Linkerd publish supported skew windows and canary-upgrade procedures). Certificate expiry in the mesh CA is a fleet-wide outage class that did not previously exist. And

container lifecycle interacts badly with sidecars: an application container that starts before its sidecar is ready sees connection failures, and a `Job` whose sidecar never exits never completes. Kubernetes' native sidecar support — init containers with `restartPolicy: Always`, which start before and terminate after regular containers — addresses this properly; it went beta and on by default in Kubernetes 1.29 and has since stabilized. On older versions you live with the workarounds (`holdApplicationUntilProxyStarts` and lifecycle-hook shutdown hacks).

Debuggability. A mesh inserts hops engineers cannot see with the tools they know. A `503` may originate at the application, the callee sidecar, or the caller sidecar, and only the proxy's response flags distinguish them: `UF` upstream connection failure, `UO` upstream overflow (circuit breaker tripped), `NR` no route configured, `URX` retry limit exceeded, `UAEX` external authorization denied. Teach these to on-call, and make these commands muscle memory:

```
# Which proxies are in sync with the control plane?
istioctl proxy-status

# What does this specific proxy actually believe about routes and
endpoints?
istioctl proxy-config routes    deploy/checkout.payments -o json
istioctl proxy-config endpoints deploy/checkout.payments --cluster \
    "outbound|8080|v2|inventory.catalog.svc.cluster.local"

# Straight from the proxy's admin interface
kubectl exec deploy/checkout -c istio-proxy -- \
    curl -s localhost:15000/clusters | grep health_flags
kubectl exec deploy/checkout -c istio-proxy -- \
    curl -s localhost:15000/config_dump > dump.json
```

So do you need one? A defensible rule: adopt a mesh when per-language libraries have become a coordination problem (dozens of services across three or more languages), when encryption in transit and identity-based authorization between internal workloads is a hard requirement, and when a platform team can own it as a product. Skip it when a handful of services share one language — a good library, or gRPC's built-in xDS support, delivers most of the value far cheaper — when your latency budget cannot absorb two extra hops, or when nobody is funded to operate it. "We might need it later" is not a reason; incremental adoption works, and mTLS plus telemetry in one namespace before any traffic policy is the right sequencing when you do adopt.

Sidecar-less designs: ambient and eBPF

The sidecar tax — one proxy per pod, its memory, its CPU, its lifecycle coupling — motivated designs that keep the mesh's guarantees while moving the proxy out of the pod. This area is evolving; the descriptions below reflect the projects' documented architecture, and specific performance and maturity claims should be verified against current releases.

Istio ambient mode splits the data plane in two. A per-node `ztunnel` (a Rust proxy running as a `DaemonSet`) handles L4 for all pods on that node: it holds each local workload's identity, establishes mutual TLS, and carries traffic inside **HBONE** — an HTTP/2 `CONNECT` tunnel over TLS on port 15008 — to the `ztunnel` on the destination node. Handling only L4 and mTLS keeps it far smaller than an Envoy sidecar, and it is shared across the node. Workloads needing L7 features — HTTP routing, header-based canaries, request-level authorization, retries — are additionally routed through a **waypoint proxy**, a real Envoy deployed per namespace or per service and configured via the Gateway API with an `istio-waypoint` gateway class. The trade is explicit: you pay for L7 only where you use it, at the cost of an extra hop when you do. Ambient mode was declared production-ready in Istio 1.24 (late 2024).

Cilium approaches it from the kernel. Its datapath is eBPF programs attached at the socket, tc, and XDP layers, implementing service load balancing, network policy, and observability without iptables — and for same-node pods without traversing the full network stack at all, since socket-level load balancing rewrites the destination at `connect()` time and eliminates per-packet NAT. Encryption between nodes is transparent WireGuard or IPsec rather than per-connection TLS, which is a *different trust model*: it secures the node-to-node path rather than authenticating individual workloads end to end. Cilium's service mesh adds a per-node Envoy for L7, and its mutual authentication feature integrates SPIFFE/SPIRE identities.

```

flowchart LR
    subgraph SIDE["Sidecar mesh"]
        direction TB
        SA["App A"] --> SPA["Envoy sidecar A"]
        SPA ==>|mTLS| SPB["Envoy sidecar B"]
        SPB --> SB["App B"]
    end
    subgraph AMB["Ambient or node-level mesh"]
        direction TB
        AA["App A"] --> ZA["ztunnel on node 1"]
        L4 plus mTLS
        ZA ==>|HBONE tunnel| WP["Waypoint proxy"]
        L7, only if needed
        WP ==> ZB["ztunnel on node 2"]
        ZB --> AB["App B"]
    end
end

```

The honest summary: sidecar-less designs move cost rather than deleting it, and they change the failure domain. A crashing sidecar affects one pod; a crashing node proxy affects every pod on that node, and a node proxy upgrade touches everything scheduled there. In exchange you get much lower aggregate overhead, no per-pod lifecycle coupling, and enrollment without restarting workloads. Both models are defensible; choose with your blast-radius tolerance in mind.

CDNs: pushing the work outward

A CDN is a globally distributed reverse-proxy fleet operated by someone else. The mechanism is the same one this chapter has been describing, applied at planetary scale with caching as the first-class concern.

Getting to the nearest PoP

A CDN operates points of presence (PoPs) in many metros, each a cluster of proxy servers and caches. Two mechanisms steer a client to one. **Anycast** (Chapter 2) announces the same IP prefix from every PoP via BGP, so the internet's own routing delivers each client to a topologically near PoP — no client logic, instant failover when a PoP withdraws its announcement, and inherent DDoS dispersion. Its weaknesses are that BGP optimizes for AS-path length and local policy rather than latency — so the topologically near PoP is not always the fastest one — and that a routing change can move a client to a different PoP mid-connection. For short HTTP transactions that is a non-issue. For long-lived QUIC

connections it is worse than it looks: QUIC connection IDs let a connection survive a *client-side* path change such as NAT rebinding, and CDNs encode server routing hints in the connection ID so packets reach the right machine within a PoP, but neither helps if packets arrive at a PoP that holds none of the connection state — the server replies with a stateless reset and the client starts over. **DNS-based steering** resolves a hostname to different addresses per client, using resolver location or ECS (EDNS Client Subnet, Chapter 5) as a hint; it offers finer policy control at the cost of TTL-bound agility and resolver-location error. Most large CDNs use both.

The cache hierarchy and origin shield

Within a PoP, a request is hashed — usually with consistent hashing on the cache key — to one of several cache servers, so each object lives on a predictable subset of machines rather than being replicated across all of them. On a miss, the naive design fetches from origin; with hundreds of PoPs, a cold object or a purge then produces hundreds of simultaneous origin requests for the same resource.

Two mechanisms fix it. **Request collapsing** — Varnish documentation calls the same behavior request coalescing — turns concurrent misses for the same key within one cache into a single upstream fetch, with the other requests waiting on its result. It is the most important stampede defense a cache has, and it is per-cache-node, which is exactly why a hierarchy is needed to collapse across PoPs. **Origin shield**, CloudFront's name for it (Fastly calls the feature shielding; Akamai's equivalent is tiered distribution), designates a PoP, usually near your origin, as a mid-tier cache that all edge PoPs fetch through, so the origin sees at most one request per object per shield rather than one per PoP. It substantially cuts origin load and egress for cacheable workloads, at the cost of an extra hop on genuine misses.

```

sequenceDiagram
    participant U as User
    participant E as CDN edge PoP
    participant SH as Origin shield PoP
    participant GW as API gateway
    participant SC as Sidecar of frontend
    participant FE as Frontend service
    participant BE as Backend service

    U->>E: GET /product/42 over QUIC or TLS 1.3
    Note over E: Compute cache key, lookup
    alt Cache hit and fresh
        E-->>U: 200 with Age and Cache-Status hit
    else Miss or stale
        E->>SH: Conditional GET with If-None-Match
        alt Shield hit
            SH-->>E: 200 or 304
        else Shield miss
            SH->>GW: GET over TLS to origin
            GW->>GW: Validate JWT, rate limit, route
            GW->>SC: Forward to mesh ingress
            SC->>FE: Plaintext over loopback
            FE->>BE: mTLS through sidecars, per-request LB
            BE-->>FE: 200
            FE-->>SC: 200 with Cache-Control
            SC-->>GW: 200
            GW-->>SH: 200
            SH-->>E: 200 stored per TTL
        end
    end
    E-->>U: 200 with Cache-Status miss
end

```

Cache keys, TTLs, and the semantics that matter

The **cache key** determines what counts as "the same object". By default it is roughly scheme + host + path + query string, and every part of that is a hazard. Marketing query parameters (`utm_source` and friends) fragment the cache into thousands of identical copies unless stripped; a key that *omits* a parameter your origin varies on serves the wrong content to everybody. The `Vary` response header extends the key by named request headers — `Vary: Accept-Encoding` is correct and necessary, `Vary: User-Agent` is cache-destroying given that header's cardinality, and `Vary: Cookie` with a per-user session cookie yields a hit rate near zero. Modern CDNs let you write the key explicitly; do so deliberately.

Freshness is governed by HTTP caching semantics (RFC 9111, Chapter 7), and the split between browser and CDN is what most people get wrong:

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=60, s-maxage=600, stale-while-
revalidate=30, stale-if-error=86400
CDN-Cache-Control: max-age=3600
Surrogate-Key: product-42 catalog
ETag: "9f2c1a-1c7"
Vary: Accept-Encoding
```

`max-age` binds private caches (the browser); `s-maxage` overrides it for shared caches; `CDN-Cache-Control` (a targeted cache-control field, RFC 9213) overrides both for the CDN specifically, letting you hold an object for an hour at the edge while browsers recheck every minute. `stale-while-revalidate` (RFC 5861) lets a cache serve a stale copy immediately and refresh asynchronously, decoupling user-visible latency from origin latency. `stale-if-error` permits serving stale content when the origin returns 5xx or is unreachable — one of the cheapest availability improvements available to any web system: your origin can be down and catalog pages still render. Distinguish `no-cache` (may store, must revalidate before reuse) from `no-store` (must not persist at all); using the latter where you meant the former throws away all revalidation benefit. Diagnostics are standardized too: `Cache-Status` (RFC 9211) reports hit/miss and remaining TTL per cache in the chain, e.g. `Cache-Status: ExampleCDN; hit; ttl=376`.

Invalidation

Three strategies, in increasing order of sophistication:

1. **Versioned URLs.** Put a content hash in the path (`/static/app.4f9c2a.js`) and cache immutably (`Cache-Control: public, max-age=31536000, immutable`). Never invalidate; publish a new URL. This should be the default for anything your bundler produces.
2. **Purge by URL.** Exact, and unusable at scale when one data change affects a hundred URLs.
3. **Purge by surrogate key / cache tag.** Tag responses with content identifiers on the way out and purge by tag when the entity changes. A product page, the category listing embedding it, and the API

response serving it all carry `product-42` ; one purge invalidates all three:

```
# Fastly: purge everything tagged product-42, service-wide
curl -X POST -H "Fastly-Key: $FASTLY_API_TOKEN" \
  "https://api.fastly.com/service/$SERVICE_ID/purge/product-42"

# "Soft" purge marks objects stale rather than removing them, so
# stale-while-revalidate and stale-if-error still protect the origin
curl -X POST -H "Fastly-Key: $FASTLY_API_TOKEN" \
  -H "Fastly-Soft-Purge: 1" \
  "https://api.fastly.com/service/$SERVICE_ID/purge/product-42"
```

Soft purge deserves emphasis: hard-purging a hot object across a global fleet converts a cache hit into a synchronized global miss — a self-inflicted stampede — whereas soft purge marks the object stale, so the first request revalidates while everyone else is briefly served the old copy. Cache-tag purging is offered by most major CDNs, sometimes only on higher plans.

Dynamic content, edge compute, and what the backend gets

Uncacheable responses still benefit from a CDN. Terminating TLS and TCP/QUIC at a PoP 10 ms away instead of 150 ms away removes most handshake cost from the critical path: the expensive round trips happen over the short leg, while the long leg to origin rides a pre-warmed, persistent connection the CDN already maintains. TLS 1.3 (Chapter 6) and QUIC (Chapter 4) amplify this — 1-RTT handshakes, 0-RTT resumption, no head-of-line blocking on lossy mobile paths. If you enable 0-RTT, understand the replay exposure: early data can be replayed, so it must be restricted to safe, idempotent requests (RFC 8470 defines the HTTP handling and the `425 Too Early` status).

Edge compute — Cloudflare Workers, Fastly Compute, AWS Lambda@Edge and CloudFront Functions, Akamai EdgeWorkers — runs your code in the PoP. The good use cases are those where running at the edge changes the network shape rather than merely relocating logic: cache-key rewriting, A/B bucketing and personalization that would otherwise make a page uncacheable, token validation before a request costs an origin hop, redirects, signed-URL checks, and response assembly from cached fragments. The bad use case is business logic that needs your database, which lives in one region: you have moved compute away from its data.

What the backend gets: **origin offload** (a well-tuned cache removes the large majority of requests from origin — directly a capacity argument), **latency reduction** for cached and uncached traffic alike, **DDoS absorption** because volumetric attacks land on anycast capacity rather than your origin, and a natural home for **WAF and bot management**. In exchange you accept a third party in the request path, a cache that serves wrong content if you misconfigure keys, and the discipline of **origin locking** — if your origin stays directly reachable, attackers will find it and bypass every edge control. Lock it with mTLS from the CDN, a shared secret header, or IP allowlists of the CDN's published ranges.

Caches are also attack surface. **Web cache poisoning** exploits *unkeyed inputs* — a header that influences the response but is not part of the cache key — to store an attacker-controlled response then served to everyone; James Kettle's 2018 PortSwigger research popularized the class and his later work on cache-key normalization extended it. **Web cache deception**, described by Omer Gil in 2017, works the other way: trick the origin into treating a private response as a static asset (`/account/settings.css`) so the cache stores it. One discipline prevents both: keys must include every input the response varies on, and nothing whose content depends on an authenticated session may be cached unless that session is part of the key.

The distributed-systems lens

The proxy layer is where the paved road is paved. In an organization with hundreds of services and dozens of teams, any policy you cannot enforce in shared infrastructure is a policy you have merely recommended. Mandating TLS 1.3, a maximum request timeout, a standard access log schema, or per-tenant rate limits by asking teams to update libraries is a multi-quarter negotiation with a long tail of non-compliance. Implementing it in the proxy layer is a config rollout with a canary and a rollback. That is an organizational property as much as a technical one, and it is the strongest argument this layer has.

The mesh relocates reliability primitives from N libraries to one data plane. Every pattern in Chapter 11 — timeouts, budgeted retries with jitter, circuit breaking, outlier ejection, load shedding — exists because networks fail partially. Implemented per-language they diverge and decay, and a retry bug in one library becomes a storm that takes down a shared dependency. Implemented in the data plane, semantics are

uniform and centrally upgradable. But the mesh cannot fix what it cannot see: it does not know which operations are idempotent, so a careless retry policy will duplicate a payment, and it cannot propagate trace context or deadlines *through* your code. The mesh handles transport; correctness at the application boundary remains yours.

Every proxy hop is latency and a failure domain. Count the hops in the CDN sequence diagram: edge, shield, gateway, ingress, caller sidecar, callee sidecar, application. Each adds latency — small individually, compounding across a deep call graph, and disproportionately visible at p99 where queuing at any one hop dominates — and each can fail, be misconfigured, exhaust a connection pool, or hold a stale certificate. The correct posture is neither avoidance nor faith: measure added latency at p99 rather than p50, design so control-plane unavailability degrades change-propagation rather than traffic, run game days that kill the control plane and the sidecars, and hold this layer to the same SLO scrutiny as any service you write.

CDNs move work to where the users are. Every request answered at an edge PoP is a request your origin never sees, a query never executed, and a long round trip never spent. Caching is the cheapest scaling mechanism in existence when the content model permits it — and making it permit that (stable cache keys, correct `Cache-Control`, tag-based invalidation, `stale-if-error` as an availability layer) is application design work, not a vendor setting. You cannot manage a cache you do not measure: treat hit ratio, `Age` distribution, origin offload percentage, and purge rate as first-class SLIs (Volume 11), and see Volume 12 for operating shared infrastructure of this kind at scale.

Key takeaways

- Forward proxies act for clients and enforce *what may be reached*; reverse proxies act for servers and enforce *what may be served*. Same machinery, opposite direction, different threat model.
- The reverse proxy is the correct home for TLS termination, protocol translation, routing, buffering, timeouts, client-IP normalization, and uniform access logging — the things that must be identical everywhere. Destroy client-supplied `X-Forwarded-For` / `Forwarded` at the trust boundary and set them yourself, or rate limiting and audit logs are forgeable.

- Envoy's model — listeners, filter chains, HTTP filters, routes, clusters, endpoints — plus xDS turned proxy configuration into a streaming subscription. ADS gives ordering, delta xDS gives scale, and a NACK leaves the last good config running (and your push silently ignored).
- A service mesh exists because cross-cutting concerns implemented in N language-specific libraries cannot be kept consistent or upgraded in lockstep; the sidecar plus control plane moves them into infrastructure.
- Automatic mTLS with SPIFFE workload identity is the mesh's strongest single benefit; per-request L7 balancing, declarative retries and circuit breaking, traffic splitting, and uniform golden signals follow. Trace *propagation* still requires application cooperation.
- The costs are per-hop latency, per-sidecar resources, operational complexity (version skew, CA expiry, container lifecycle), and harder debugging. Learn the proxy response flags and the config-inspection commands before the incident, not during it.
- Ambient and eBPF data planes cut the sidecar tax by moving proxies to the node and charging for L7 only where used — trading a smaller per-pod cost for a larger per-node blast radius.
- CDNs combine anycast reach, a cache hierarchy with origin shielding, and request collapsing to remove most traffic from your origin. Cache keys and `Cache-Control` are application design decisions; `stale-while-revalidate` and `stale-if-error` are availability features. Prefer versioned immutable URLs, use tag-based soft purge for the rest, and lock your origin so the edge cannot be bypassed.

Further reading

- Envoy documentation — <https://www.envoyproxy.io/docs/envoy/latest/> — the architecture overview and the xDS protocol reference at https://www.envoyproxy.io/docs/envoy/latest/api-docs/xds_protocol, which specifies ACK/NACK, versioning, ADS, and incremental xDS.
- Istio documentation — <https://istio.io/latest/docs/> — architecture, security (identity and mTLS), traffic management, and performance-and-scalability; plus <https://istio.io/latest/docs/ambient/> for ztunnel, HBONE, and waypoints.

- Linkerd documentation — <https://linkerd.io/docs/> — and the project's writing on why it built a purpose-specific Rust micro-proxy instead of adopting Envoy.
- SPIFFE and SPIRE — <https://spiffe.io/docs/latest/spiffe-about/overview/> — the SPIFFE ID and X.509-SVID specifications; see also Volume 9, Chapter 10 for zero trust and service-to-service authentication.
- Kubernetes Gateway API — <https://gateway-api.sigs.k8s.io/> — including the GAMMA initiative for expressing east-west (mesh) routing with the same resources.
- Cilium documentation — <https://docs.cilium.io/> — eBPF datapath, socket-level load balancing, transparent encryption, and the service-mesh and mutual-authentication features.
- RFC 9110 (*HTTP Semantics*) and RFC 9111 (*HTTP Caching*); RFC 5861 (`stale-while-revalidate` , `stale-if-error`), RFC 9211 (`Cache-Status`), and RFC 9213 (targeted cache-control fields such as `CDN-Cache-Control`).
- RFC 7239 (*Forwarded HTTP Extension*) and the PROXY protocol specification published by HAProxy Technologies — client-IP preservation at L7 and L4 respectively.
- RFC 8470 — *Using Early Data in HTTP* — replay considerations and `425 Too Early` for TLS 1.3 and QUIC 0-RTT at the edge.
- NGINX documentation (<https://nginx.org/en/docs/>) and the HAProxy configuration manual (<https://docs.haproxy.org/>) — authoritative references for the directives used above.
- Fastly and Cloudflare developer documentation on cache keys, surrogate keys/cache tags, soft purge, origin shielding, and edge compute — the clearest public writing on CDN cache mechanics.
- James Kettle, "Practical Web Cache Poisoning" (PortSwigger Research, 2018) and the follow-up "Web Cache Entanglement" (2020) — unkeyed inputs and cache-key normalization flaws. For the inverse bug class, Omer Gil's 2017 write-up of web cache deception.

- Chapters 2, 6, 7, 9, and 11 of this volume supply the anycast, TLS, HTTP-caching, balancing, and reliability primitives this chapter assembles.

Chapter 11 — Network Reliability: Timeouts, Retries, Backoff, and Hedging

What this chapter covers. Every chapter before this one built a mechanism: packets and routing, TCP and QUIC, DNS, TLS, HTTP, gRPC, load balancers, proxies and meshes. This chapter is about what you do when those mechanisms fail — which they do constantly, at a rate invisible in one service and unavoidable across a fleet. The subject is the small set of policies that decide whether a partial failure stays partial: timeouts and deadlines, retries and retry budgets, backoff and jitter, circuit breakers, hedged requests, load shedding, and graceful degradation. These are not independent knobs. Configured in isolation they interact catastrophically — the classic outage is a latency blip that naive retries amplify into a fleet-wide meltdown and then keep pinned long after the trigger is gone. Throughout we work from mechanism to real configuration for gRPC, Envoy, Go, and resilience4j, tied to gRPC deadlines (Chapter 8), outlier detection and balancing (Chapter 9), and the mesh as the place to implement all of it once (Chapter 10).

Learning goals — after this chapter you should be able to:

- Explain why an unbounded remote call is a resource leak, and use Little's Law to turn a timeout into a pool-sizing decision.
- Choose connect, TLS, per-attempt, and total timeouts from an observed latency distribution, and know which transport-layer timeouts you must also set.
- Implement deadline propagation, including clock-skew handling and deadline-aware queue eviction, and say why it dominates per-hop timeouts.
- Classify errors as retryable, non-retryable, or ambiguous, and state the idempotency precondition for each.

- Compute retry amplification for a call graph and design against it with budgets, a single-retry-layer rule, and circuit breaking.
- Implement exponential backoff with full, equal, and decorrelated jitter, and name the failure mode each addresses.
- Specify a circuit breaker, and distinguish it from Envoy-style concurrency limits and from outlier ejection.
- Decide when hedged requests are safe, size the hedge delay, and bound their cost.
- Build an admission-control path that sheds by queue delay and criticality, and assemble the set into one per-dependency policy.

The fallacies, and what they actually cost

The standard framing is the *fallacies of distributed computing*, a list assembled at Sun Microsystems over the first half of the 1990s: the earliest items are usually attributed to Bill Joy and Tom Lyon, the list as commonly quoted to L. Peter Deutsch, and the eighth — "the network is homogeneous" — to James Gosling; Arnon Rotem-Gal-Oz later wrote the standard explanatory treatment. The eight assumptions distributed programs make and that are all false: the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology doesn't change; there is one administrator; transport cost is zero; the network is homogeneous.

Quoting the list is easy; the useful move is converting each fallacy into the line of code it invalidates. "The network is reliable" invalidates any call site without an error path — and, more subtly, any error path that assumes the call *didn't happen*. "Latency is zero" invalidates any call without a bound, because "eventually" here means "possibly never, while holding a thread". "Topology doesn't change" invalidates cached DNS results and pinned connections (Chapters 5 and 9).

These are not rare events. In a fleet of a few hundred services doing tens of thousands of RPCs per second, a per-call failure probability of one in ten thousand is a continuous stream of failures, all day, forever. Reliability at the call site is not exception handling; it is the normal operating mode.

Timeouts: the load-bearing primitive

Everything else in this chapter depends on timeouts, so we start there.

An unbounded call is a resource leak

Consider a synchronous service with 200 request-handling threads, each making one call to a dependency that normally responds in 20 ms. By Little's Law — concurrency equals arrival rate times latency, $L = \lambda W$ — 200 threads at 20 ms occupancy sustain 10,000 requests per second. That capacity is a consequence of the *downstream* call's latency, not of your own code.

Now the dependency degrades to 2 seconds: no errors, no crashes, nothing a health check notices. The same 200 threads now sustain 100 requests per second, so from the outside your service is down — pool fully occupied, queue growing without bound. If the call has no timeout at all and the downstream has stopped responding entirely, the threads are gone permanently.

That is the whole argument, and it has three corollaries. First, **the timeout value is a capacity decision**: timeout T against a pool of N workers bounds worst-case throughput at N/T , so $T = 10s$ on a 200-thread pool declares that you will fall to 20 rps under stress while $T = 200ms$ sets the floor at 1,000. Sizing a timeout without knowing N is guessing. Second, **asynchronous runtimes change the constant, not the shape**: Go and Netty do not burn an OS thread per in-flight call, but connections and heap are finite and the pending queue still grows without bound — async raises the ceiling and hides the collapse behind memory growth rather than a thread dump. Third, **the caller's timeout is the only defense you control**: you cannot make the dependency fast, only decide how much of your capacity you will lend it.

The timeout ladder: connect, TLS, per-attempt, total

"A timeout" is not one thing. A single HTTP request over a fresh connection has at least these phases, each able to hang independently:

Phase	What hangs	Typical bound
DNS resolution	Resolver unreachable, UDP loss, slow recursive lookup (Ch. 5)	1–5 s, with a resolver-level retry

Phase	What hangs	Typical bound
TCP connect	SYN blackholed by a dead host or a stale NAT/firewall entry (Ch. 2, 3)	100–500 ms in-datacenter
TLS handshake	Slow OCSP fetch on the server, CPU-starved peer (Ch. 6)	200 ms – 2 s
Request write	Peer's receive window at zero, socket buffers full (Ch. 3)	Bounded by total
Response headers	Server accepted but is stuck in the handler	The interesting one
Response body	Slow streaming, stalled after headers	Bounded by total, or by a rate
Total / end-to-end	All of the above plus retries	The deadline

Two mistakes are near universal. Setting *only* a total timeout means a host that blackholes SYNs consumes the entire budget before you can try another. Setting *only* a per-attempt timeout means retries multiply latency: three attempts at 1 s each is a 3-second worst case your caller never agreed to.

A correctly configured Go HTTP client — `http.Client.Timeout` is the *total* bound including body read; the transport-level bounds are separate:

```

dialer := &net.Dialer{
    Timeout: 200 * time.Millisecond, // TCP connect, per address
    KeepAlive: 30 * time.Second,      // TCP keepalive probes on idle
    conns
}

transport := &http.Transport{
    DialContext: dialer.DialContext,
    TLSHandshakeTimeout: 400 * time.Millisecond,
    ResponseHeaderTimeout: 800 *
time.Millisecond, // server think-time bound
    ExpectContinueTimeout: 100 * time.Millisecond,
    IdleConnTimeout: 90 * time.Second,
    MaxIdleConnsPerHost: 64, // default is 2 – always raise it
    MaxConnsPerHost: 256, // a bulkhead: bounds concurrency per
dependency
    ForceAttemptHTTP2: true,
}

client := &http.Client{
    Transport: transport,
    Timeout: 1500 * time.Millisecond, // total, including body
    CheckRedirect: func(*http.Request, []*http.Request) error {
        return http.ErrUseLastResponse // redirects silently multiply
latency
    },
}

```

`MaxIdleConnsPerHost` defaulting to 2 is one of the most consequential defaults in the Go standard library: leave it and a high-throughput client churns connections, paying TCP and TLS handshakes on most requests (Chapter 6). `MaxConnsPerHost` doubles as a bulkhead. The server needs the mirror image, or a slow client does to you what a slow server does to your callers:

```

srv := &http.Server{
    Addr: ":8080",
    ReadHeaderTimeout: 2 * time.Second, // Slowloris defense
    ReadTimeout: 10 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout: 120 * time.Second, // must exceed LB idle
timeout, see Ch. 9
    MaxHeaderBytes: 1 << 16,
}

```

The `IdleTimeout` relationship matters: if the server's is shorter than the load balancer's, the LB occasionally hands a request to a connection the

server just closed — a race that surfaces as sporadic, unexplained 502s. Keep it longer than every proxy in front of you.

Timeouts the application layer cannot see

An application read timeout does not help if the kernel never gives up:

```
# How many times Linux retransmits unacknowledged data before giving up
# on an
# established connection. This is a retry count, not a duration: the
# effective
# timeout depends on the RT0 and its exponential growth.
sysctl net.ipv4.tcp_retries2 # default 15 -> roughly 13-30 minutes
per tcp(7)
```

Thirteen minutes is not a timeout; it is an outage. The per-socket override is `TCP_USER_TIMEOUT` (milliseconds), which bounds how long unacknowledged data may sit before the kernel tears the connection down — set it in Go via a `Control` function on the dialer, and combine it with keepalives so an *idle* connection to a vanished peer is also detected (Chapter 3). For HTTP/2 and gRPC the equivalent is keepalive pings — one broken multiplexed connection fails hundreds of RPCs at once:

```
conn, err := grpc.NewClient(target,
    grpc.WithKeepaliveParams(keepalive.ClientParameters{
        Time:                20 * time.Second, // ping when idle this
long
        Timeout:             5 *
time.Second, // ping must be answered within
        PermitWithoutStream: false,           // don't ping with no
active RPCs
    })),
)
```

Servers enforce a minimum ping interval (`keepalive.EnforcementPolicy`) and send `GOAWAY` carrying `ENHANCE_YOUR_CALM` and the debug data `too_many_pings` to clients that ping too aggressively. The `grpc-go` server default `MinTime` is five minutes, so the client above will be disconnected by a stock server. Configure the two ends as a pair, or you get mysterious resets under load.

Choosing the number

Do not guess, and do not copy from another service. Take it from the observed distribution of the dependency's latency *as seen by the client*, over a period including at least one peak. The heuristic: **set the per-attempt timeout between p99 and p99.9 of healthy latency, rounded up, then check the capacity implication.** A timeout is a claim about what "abnormal" means — set it at the median and you time out half your traffic; set it at $20\times$ p99 and you have effectively no timeout, because by the time it fires the pool is gone.

Then check the arithmetic in three directions. **Capacity floor:** with N slots and timeout T the throughput floor is N/T , and if that is below your traffic a full stall takes you down regardless. **Budget fit:** if your handler makes four sequential calls under a 300 ms p99 target, no single call gets 500 ms. **Retry fit:** three attempts at T plus backoff must fit inside the deadline you were given, or the retries are theater — the caller has already left.

A concrete example. A dependency shows $p50 = 8$ ms, $p99 = 45$ ms, $p99.9 = 120$ ms. A 150 ms per-attempt timeout cuts off roughly the slowest 0.1 % of healthy calls, which retries mostly recover, and a 400 ms budget allows two attempts plus jittered backoff. With 512 concurrent slots the throughput floor during a full stall is $512 / 0.15 \text{ s} \approx 3,400$ rps ; at 2,000 rps peak, a stalled dependency degrades this service without wedging it. Every number came from data.

Two caveats. Percentiles are not additive: a service that fans out to 20 backends and waits for all of them sees its p99 driven by the *slowest* of 20 draws — the "tail at scale" effect that motivates hedging below. And steady-state numbers are wrong during cold start, when JIT warmup and empty caches inflate latency for tens of seconds, so either warm up before taking traffic (Chapter 9's slow start) or your timeouts fire in a wave on every rollout.

Deadlines beat timeouts

A timeout is a per-hop duration. A **deadline** is a point in time that belongs to the whole operation, established once at the entry point and propagated to every hop. The difference sounds cosmetic; it is the single highest-leverage change most call chains can make.

Why per-hop timeouts multiply

In a chain $A \rightarrow B \rightarrow C \rightarrow D$, suppose each hop gets a 1-second timeout because 1 second "seems reasonable". Add two attempts at each hop and A's worst case becomes $2 \times (2 \times (2 \times 1 \text{ s})) = 8 \text{ s}$, because each layer's retries multiply the layer below — but A's own timeout is 1 second, so A gave up seven seconds before D finished. Everything B, C, and D did after that first second was waste: CPU spent, connections held, rows locked, producing a response nobody reads.

That waste is the mechanism of collapse. Every abandoned-but-still-running request occupies capacity a live request could use, so useful work falls, so more requests time out, so more work is abandoned-but-running — the positive feedback that lets an overloaded system sit at 100 % CPU while completing almost nothing. It is *congestion collapse* (Chapter 3), transplanted into the application tier.

```
flowchart TB
    subgraph naive["Per-hop timeouts - work continues after the caller has left"]
        A1["A, timeout 1s"] --> B1["B, timeout 1s"]
        B1 --> C1["C, timeout 1s"]
        C1 --> D1["D, timeout 1s"]
        A1 -->| "gives up at t=1s" | X1["A returns error"]
        D1 -->| "still executing at t=3s" | W1["Wasted CPU, held locks, held connections"]
    end

    subgraph deadline["Deadline propagation - everyone stops together"]
        A2["A sets deadline now plus 1s remaining 1000ms"] --> B2["B sees remaining 940ms"]
        B2 --> C2["C sees remaining 870ms"]
        C2 --> D2["D sees remaining 800ms"]
        D2 -->| "deadline reached" | X2["D cancels work, rolls back, returns DEADLINE_EXCEEDED"]
    end
```

The mechanism

gRPC implements this natively, and it is the model to copy even if you speak HTTP. The client sets a deadline; the library encodes the *remaining* time into the `grpc-timeout` header — a value plus a unit character, so `grpc-timeout: 800m` means 800 milliseconds. The server reads it, creates a context cancelled at that point, and every downstream call under that

context carries a freshly computed, smaller remaining time. On expiry the RPC fails with `DEADLINE_EXCEEDED` (status 4), and gRPC sends `RST_STREAM` so the server learns the client has gone (Chapters 7 and 8).

Two design details are worth stealing. First, **it propagates a relative duration, not an absolute timestamp**, which sidesteps clock skew: a D whose clock ran 300 ms fast would cancel work that still had budget, and machines routinely disagree by single-digit milliseconds under good NTP and by far more when NTP breaks (Volume 6, Chapter 2). Each hop converts the received duration against its own monotonic clock, so only that clock has to be sane; the cost is that transit time is charged to the caller, the conservative direction. Second, **cancellation is explicit and propagates**: expiry is not merely "the caller stopped listening" but a signal the callee acts on — abort the query, release the lock, skip the write.

In Go this is `context.Context` , and the discipline is mechanical:

```

func (s *Server) GetFeed(ctx context.Context, req *pb.GetFeedRequest)
(*pb.GetFeedResponse, error) {
    // ctx already carries the deadline the caller propagated.

    // Reserve a slice of the remaining budget for local work and the
    response
    // write, so we never return DEADLINE_EXCEEDED to our caller when
    we could
    // have returned a partial-but-useful answer.
    if dl, ok := ctx.Deadline(); ok {
        remaining := time.Until(dl)
        if remaining < 30*time.Millisecond {
            // Not enough budget left to be useful – fail fast instead
            of
            // consuming capacity on work that is already doomed.
            return nil, status.Error(codes.DeadlineExceeded, "insuffici
ent budget on arrival")
        }
    }

    // Sub-deadline for a non-critical enrichment call: it may use at
    most 40ms
    // of our budget, and its failure must not fail the request.
    enrichCtx, cancel := context.WithTimeout(ctx, 40*time.Millisecond)
    defer cancel()
    ads, err := s.ads.Fetch(enrichCtx, req.UserId)
    if err != nil {
        ads = nil // graceful degradation, see below
        s.metrics.Degraded.WithLabelValues("ads").Inc()
    }

    // The critical call inherits the full remaining budget.
    posts, err := s.posts.Query(ctx, req.UserId)
    if err != nil {
        return nil, err
    }
    return assemble(posts, ads), nil
}

```

Three rules follow:

1. **Never call `context.Background()` in a request path.** Every one in a handler is a severed deadline chain whose work outlives the request; linters enforce this (`contextcheck` , `noctx`). Deliberately detached background work is the exception and needs its own explicit timeout.
2. **Pass the context all the way to the syscall** — `db.QueryContext` , not `db.Query` . Cancellation there is best-effort: the driver stops waiting and, at most, sends an out-of-band cancel request the server

may ignore. Push the budget into the server too — for PostgreSQL, `SET LOCAL statement_timeout = 800` inside the transaction (milliseconds) — so the backend process and the snapshot it pins are released whether or not the cancel lands.

3. **Reserve budget.** A hop that spends its whole budget downstream has no time to serialize a response or run a fallback. Reserving a slice, and refusing work that arrives with too little budget, converts a guaranteed timeout into a cheap rejection.

Plain HTTP has no standardized deadline header, which is a real gap. Adopt a `grpc-timeout`-style relative header fleet-wide, or lean on the mesh — Envoy sets `x-envoy-expected-rq-timeout-ms` on upstream requests and honors `x-envoy-upstream-rq-timeout-ms` from clients (Chapter 10). What you must not have is a fleet where half the services propagate deadlines and half reset them; that is how a request A abandoned five seconds ago is still mutating rows in D.

Deadline-aware queueing

The second payoff is on the *server* side: propagated deadlines let you throw away work you know is worthless. A request that has sat in the accept queue for 900 ms with a 1-second deadline cannot complete, and serving it is strictly worse than dropping it. An overloaded server's first move, before dispatch, is therefore to evict expired items from the queue.

Retries

Retries are the most valuable and most dangerous tool in this chapter: valuable because most failures in a healthy system are transient and uncorrelated — a dropped packet, a host being drained, a GC pause, a reset from a proxy that just got new config — and dangerous because they add load precisely when load is the problem.

Idempotency is the precondition, and failure is ambiguous

Everyone knows to retry only idempotent operations. The subtlety is *why*: **a timeout tells you nothing about whether the operation executed.** The request may never have arrived, may have arrived and been rejected, may have executed fully with the response lost, or may still be executing — distinguishing these from the client is impossible in general, the

ambiguity that makes exactly-once delivery a fiction at the network layer (Volume 6, Chapter 9; Volume 10, Chapter 2).

So the practical taxonomy has three buckets, not two:

- **Definitely not executed** — retry is safe regardless of idempotency, and the signal comes from the transport: connection refused; an HTTP/2 `REFUSED_STREAM`, which RFC 9113 defines as the server not having processed the stream; any stream above the `last-stream-id` in a `GOAWAY` frame, the graceful-shutdown promise that those streams were not acted on. Envoy exposes these as the `connect-failure`, `reset`, and `refused-stream` retry conditions — `GOAWAY` semantics are a reliability feature, not housekeeping.
- **Definitely final** — 400, 403, 404, gRPC `INVALID_ARGUMENT` or `PERMISSION_DENIED`. The server reached a verdict; the second attempt gets the same one. A client that retries its own bugs triples its load for zero chance of success.
- **Ambiguous** — timeouts, 5xx after the request was written, resets mid-response. These require idempotency, which you build rather than hope for.

RFC 9110 defines GET, HEAD, PUT, DELETE, OPTIONS, and TRACE as idempotent and POST as not, so a generic client may auto-retry the former only. The application-level answer is an **idempotency key**: a unique key per logical operation, sent on every attempt and stored server-side with the result, so a duplicate returns the original outcome instead of re-executing (Volume 8, Chapter 6). With a key, POST becomes retryable; without one, retrying an authorization double-charges a customer.

A status taxonomy you can encode directly in a client:

Signal	Retry?	Notes
TCP connect refused / DNS failure	Yes	Not executed. Prefer a different host.
TLS handshake failure	Usually	Not executed; but a cert error is deterministic — do not retry.
HTTP/2 <code>REFUSED_STREAM</code> , <code>GOAWAY</code> above last-stream-id	Yes	Protocol guarantees not processed.

Signal	Retry?	Notes
Timeout before response headers	Only if idempotent	Ambiguous.
HTTP 408, 429, 502, 503, 504	Yes, with backoff	429/503 should carry <code>Retry-After</code> — honor it.
HTTP 500	Cautiously	Often a deterministic application bug; retrying multiplies load for nothing.
HTTP 400/401/403/404/409/422	No	Deterministic. Retrying is a client bug.
gRPC <code>UNAVAILABLE</code> (14)	Yes	The canonical retryable code.
gRPC <code>DEADLINE_EXCEEDED</code> (4)	Only if budget remains	Usually the budget is already gone.
gRPC <code>RESOURCE_EXHAUSTED</code> (8)	Only with backoff and a budget	The server is telling you it is overloaded.
gRPC <code>INTERNAL</code> (13), <code>INVALID_ARGUMENT</code> (3), <code>FAILED_PRECONDITION</code> (9)	No	Deterministic.

`RESOURCE_EXHAUSTED` and 429 mean "I am rejecting you deliberately", and retrying them quickly is what turns a working rate limiter into an outage: rejected load returns as new load. Honor `Retry-After`; gRPC's equivalent is the `RetryInfo` message in `google.rpc.Status` details, carrying a `retry_delay`. If you emit such rejections, send the hint; if you consume them, obey it.

Retry amplification

Now the arithmetic that ruins fleets. Consider a four-tier chain where each tier retries up to 3 times (one initial attempt plus two retries).

```

flowchart LR
    U["1 user request"] --> E["Edge / BFF  
up to 3 attempts"]
    E --> A["3 requests" | A["Service A  
up to 3 attempts"]]
    A --> B["9 requests" | B["Service B  
up to 3 attempts"]]
    B --> L["27 requests" | L["Leaf datastore  
27x offered load"]]
    L --> L2["more load, more timeouts,  
more retries" | L]

```

Three layers of 3 attempts is 27× amplification at the leaf; four layers is 81×. The crucial property is that **amplification is near 1× when everything is healthy and near maximum exactly when things are failing**, because retries only fire on failure. The leaf sees normal load right up until it starts to struggle, at which point offered load jumps by an order of magnitude or two. No capacity plan survives that.

Then the loop closes: the leaf, now at 27×, times out more requests, producing more retries. The system settles into a stable failed state where retry traffic alone exceeds capacity — a **retry storm** — and critically *it does not recover when the original trigger is removed*: restart the leaf and the standing wave knocks it over during warmup. The AWS EC2/EBS disruption in US-East in April 2011 is the best-documented self-sustaining example, with EBS nodes losing connectivity after a network change and re-mirroring aggressively until cluster capacity was exhausted and stayed exhausted; the DynamoDB disruption in the same region in September 2015 had a broadly similar shape.

And amplification is usually counted in requests per second when the more dangerous multiplication is in **concurrency**: a retry after a timeout means the client held a slot for the full timeout and then took another. Connection pools, not just CPUs, run out.

Mitigation 1: retry budgets

The fix is making the retry rate a *bounded fraction of the request rate* instead of a per-request multiplier. If retries never exceed 10 % of traffic, worst-case amplification is 1.1× no matter how bad things get, while still recovering every transient failure in a healthy system.

gRPC specifies this in gRFC A6 as **retry throttling**: a token bucket per server name, shared across all methods on the channel. `token_count`

starts at `maxTokens` and is capped there; every failed RPC decrements it by 1 and every success increments it by `tokenRatio`; when `token_count <= maxTokens/2` retries stop entirely. The bucket is fed by *successes*, so the retry allowance tracks how the dependency is actually doing: when a backend is broadly failing, tokens drain and retries stop within a fraction of a second, with no threshold to tune per incident.

A complete gRPC service config, shippable as a channel default:

```
{
  "methodConfig": [
    {
      "name": [{ "service": "payments.v1.Payments", "method": "Authorize" }],
      "timeout": "1.2s",
      "retryPolicy": {
        "maxAttempts": 3,
        "initialBackoff": "0.05s",
        "maxBackoff": "0.5s",
        "backoffMultiplier": 2,
        "retryableStatusCodes": ["UNAVAILABLE"]
      }
    },
    {
      "name": [{ "service": "payments.v1.Payments" }],
      "timeout": "2s",
      "retryPolicy": {
        "maxAttempts": 2,
        "initialBackoff": "0.1s",
        "maxBackoff": "1s",
        "backoffMultiplier": 2,
        "retryableStatusCodes": ["UNAVAILABLE"]
      }
    }
  ],
  "retryThrottling": {
    "maxTokens": 100,
    "tokenRatio": 0.1
  }
}
```

```
conn, err := grpc.NewClient(
  "dns:///payments.prod.svc.cluster.local:443",
  grpc.WithTransportCredentials(creds),
  grpc.WithDefaultServiceConfig(serviceConfigJSON),
  grpc.WithDefaultCallOptions(grpc.WaitForReady(false)),
)
```

Three details repay attention. `maxAttempts` counts the initial attempt, so `3` means two retries. gRPC's backoff is *fully jittered* by specification — each delay is drawn uniformly from `[0, current_backoff)`, after which `current_backoff` is multiplied by `backoffMultiplier` up to `maxBackoff` — so you get correct jitter for free, which is not true of most hand-rolled retry loops. And `WaitForReady(false)`, the default, makes an RPC issued while the channel is in `TRANSIENT_FAILURE` fail immediately rather than queue until reconnect; `true` suits only background work with generous deadlines. Note also that retrying `Authorize` at all presumes the method takes an idempotency key: gRPC stops retrying once response headers arrive, but `UNAVAILABLE` can still be returned after the server has done the work.

Envoy implements the same idea as a **retry budget** on the cluster, expressed as a percentage of active requests:

```
clusters:
- name: payments
  connect_timeout: 0.25s
  type: EDS
  eds_cluster_config: { eds_config: { ads: {} } }
  circuit_breakers:
    thresholds:
    - priority: DEFAULT
      max_connections: 1024
      max_pending_requests: 256    # bound the queue, not just the in-
flight set
      max_requests: 1024
      retry_budget:
        budget_percent: { value: 20.0 }    # retries <= 20% of active
requests
      min_retry_concurrency: 3            # ...but always allow a few
outlier_detection:                      # see Chapter 9
  consecutive_5xx: 5
  interval: 10s
  base_ejection_time: 30s
  max_ejection_percent: 30
```

```

routes:
- match: { prefix: "/payments.v1.Payments/" }
  route:
    cluster: payments
    timeout: 1.2s # total, across all attempts
    retry_policy:
      retry_on: "unavailable,connect-failure,refused-stream,reset"
      num_retries: 2
      per_try_timeout: 0.35s # attempts x per_try + backoff must
fit inside timeout
      retry_back_off:
        base_interval: 0.025s
        max_interval: 0.25s
      retry_host_predicate:
      - name: envoy.retry_host_predicates.previous_hosts
        typed_config:
          "@type": type.googleapis.com/
envoy.extensions.retry.host.previous_hosts.v3.PreviousHostsPredicate
        host_selection_retry_max_attempts: 3

```

Note the `previous_hosts` retry predicate: without it a retry may land on the very host that just failed. Combined with outlier detection from Chapter 9, it makes retries do what people imagine they do — route around a bad host. Envoy's retry backoff is fully jittered exponential, defaulting to a 25 ms base interval and a max interval of ten times that. One sharp edge: Envoy matches gRPC retry conditions against the status in *response headers*, so a failure delivered only in trailers does not trigger a retry.

A terminology trap: Envoy's `circuit_breakers` block is **not** the circuit-breaker state machine described later, but a set of hard concurrency limits — bulkheads — that reject immediately when exceeded. Different mechanisms, unfortunately shared name.

Mitigation 2: retry at exactly one layer

Even with budgets, retrying at every layer wastes budget and multiplies latency. The rule: **for any call-chain segment, exactly one layer owns retries**; everyone else propagates the failure. The strongest default is the layer closest to the failure that can change the outcome — the one that can pick a different backend, which is almost always the client-side load balancer or sidecar (Chapters 9 and 10), since it holds the endpoint list and the health signal. Higher layers just re-drive the same selection; lower layers cannot see alternatives.

Making it stick requires a signal, because a service that has exhausted its retries and returns `UNAVAILABLE` looks to its caller exactly like a transient failure worth retrying. Either **downgrade the status on exhaustion** to something the caller treats as non-retryable, or **propagate the attempt count** — Envoy can add `x-envoy-attempt-count`, and a server seeing an elevated count knows amplification is under way. Write it down as a fleet-wide contract, or retry behavior remains an emergent property of every layer's independent choices.

Backoff and jitter

Retrying immediately is close to useless: whatever transient condition caused the failure — a GC pause, a leader election, a saturated queue — has not cleared in the microsecond it takes to loop. Retrying immediately *and in lockstep with thousands of peers* is actively harmful.

Exponential backoff, and why it is not enough

The standard formula, capped so delays do not grow without bound, is `delay = min(cap, base * 2^attempt)`. It spaces attempts out and lets a struggling dependency recover, but it has a fatal property in a fleet: it is **deterministic**. If 10,000 clients observe a failure at time `t` — exactly what happens when a service restarts, a leader fails over, a deploy rolls, or a partition heals — all 10,000 retry at `t + base`, all fail, and all retry at `t + 3·base`. That is a synchronized standing wave that keeps the recovering dependency pinned: the **thundering herd**. Exponential growth alone does not break the synchronization — it only spreads the same bursts further apart, so the dependency alternates between spikes it cannot absorb and idle capacity it cannot use. Backoff without randomization changes *when* the herd arrives, not that it arrives together.

```
sequenceDiagram
    autonumber
    participant C1 as Client 1
    participant C2 as Client 2
    participant C3 as Client 3
    participant S as Recovering service

    Note over C1,S: No jitter – synchronized waves
    C1->>S: retry at t+100ms
    C2->>S: retry at t+100ms
    C3->>S: retry at t+100ms
    S-->>C1: overloaded, fail
    S-->>C2: overloaded, fail
    S-->>C3: overloaded, fail
    Note over S: burst of 3 concurrent,
    then idle, then burst of 3

    Note over C1,S: Full jitter – smeared arrivals
    C1->>S: retry at t+17ms
    S-->>C1: ok
    C2->>S: retry at t+63ms
    S-->>C2: ok
    C3->>S: retry at t+141ms
    S-->>C3: ok
    Note over S: steady arrival rate,
    each attempt sees free capacity
```

The jitter variants

The reference treatment is Marc Brooker's AWS Architecture Blog post *Exponential Backoff And Jitter* (2015), which simulates several strategies against a contended resource. The variants as that post defines them:

```

import random

BASE = 0.020 # 20 ms
CAP = 2.000 # 2 s

def no_jitter(attempt):
    return min(CAP, BASE * 2 ** attempt)

def full_jitter(attempt):
    return random.uniform(0, min(CAP, BASE * 2 ** attempt))

def equal_jitter(attempt):
    t = min(CAP, BASE * 2 ** attempt)
    return t / 2 + random.uniform(0, t / 2)

def decorrelated_jitter(prev_sleep):
    # Stateful: the next delay is drawn from [BASE, prev_sleep * 3]
    return min(CAP, random.uniform(BASE, prev_sleep * 3))

```

The differences are not arbitrary. **Full jitter** draws uniformly from `[0, backoff)`, maximizing spread; in the post's simulations it reduced both contention and total work relative to unjittered backoff. Its one drawback is that late in a sequence it can produce a very short delay. It is what gRPC and Envoy implement, and it is the right default. **Equal jitter** keeps half the delay deterministic, guaranteeing minimum spacing — useful when a too-early retry is genuinely expensive. **Decorrelated jitter** is stateful, deriving each delay from the previous one; it climbs faster in expectation, avoids full jitter's collapse toward zero, and the post found it competitive. The robust claim is qualitative — jitter reduces both contention and total work — so pick one and move on.

Jitter everything periodic, not just retries

Synchronization is not a retry problem; it is a *timing* problem, arising anywhere many processes share a schedule.

- **Cache expiry.** Keys written together with a 1-hour TTL expire together an hour later. Jitter the TTL (`ttl * uniform(0.9, 1.1)`) and coalesce concurrent misses per key — single-flight in the client, request collapsing in the CDN (Chapter 10).
- **Cron, health checks, and metric scrapes.** Everything at `0 * * * *` fires fleet-wide at once, as do Chapter 9's active checks from 500 proxies at a fixed interval; spread both by a hash of the hostname.

- **Reconnects after a proxy restart.** Dropping 20,000 connections means 20,000 simultaneous handshakes, whose CPU cost (Chapter 6) dwarfs steady state; gRPC's connection-backoff specification jitters exactly this.
- **Token and certificate refresh.** Credentials issued together expire together; refresh at a jittered fraction of the lifetime, not a fixed offset from expiry.

Circuit breakers

Backoff limits the rate of retries to one dependency. A circuit breaker goes further: when a dependency is clearly broken, stop calling it at all for a while. The purpose is symmetric. For the **caller**, failing fast preserves the thread pool, connection pool, and latency budget. For the **callee**, removing load is what permits recovery: a service pinned at 27× capacity by retries never gets the headroom to drain its queue, rebuild caches, and start succeeding.

The state machine

```
stateDiagram-v2
    [*] --> Closed
    Closed --> Closed: calls pass through, outcomes recorded
    Closed --> Open: failure or slow-call rate over threshold, minimum
    volume met
    Open --> Open: calls rejected immediately, no request sent
    Open --> HalfOpen: wait duration elapsed
    HalfOpen --> HalfOpen: bounded probes in flight, excess calls
    rejected
    HalfOpen --> Closed: probes succeed above the success threshold
    HalfOpen --> Open: a probe fails or is too slow
```

- **Closed** — normal. Calls pass through; outcomes feed a rolling window.
- **Open** — tripped. Calls are rejected immediately with a local error and no network I/O at all.
- **Half-open** — probing. After a wait duration a *bounded* number of trial calls are permitted; success closes the breaker, failure reopens it and restarts the wait. The bound is the point: returning straight to full traffic re-breaks a service that has just come back.

What to count, and over what window

A breaker that counts only errors misses the most common way a dependency kills you: not failing, but getting slow. Count **slow calls as failures** — resilience4j's slow-call rate threshold is the single most valuable setting in the library. Prefer a **time-based** window for anything with variable traffic: a 100-call window covers hours on a quiet endpoint and 50 ms on a busy one, so the breaker is alternately senile and hypersensitive. And require a **minimum call count** before the rate means anything, or two failed calls after startup give you a 100 % failure rate on a healthy dependency.

```
resilience4j.circuitbreaker:
  instances:
    payments:
      slidingWindowType: TIME_BASED
      slidingWindowSize: 60                # seconds
      minimumNumberOfCalls: 50             # don't judge on thin
  data
    failureRateThreshold: 50               # percent
    slowCallRateThreshold: 60              # percent
    slowCallDurationThreshold: 400ms       # "slow" == near our
  timeout
    waitDurationInOpenState: 10s
    permittedNumberOfCallsInHalfOpenState: 10
    automaticTransitionFromOpenToHalfOpenEnabled: true
    recordExceptions:
      - java.io.IOException
      - java.util.concurrent.TimeoutException
    ignoreExceptions:
      - com.example.ValidationException    # 4xx must not trip
  the breaker
```

`ignoreExceptions` is not a detail. If validation failures count toward the failure rate, one client sending malformed requests trips your breaker for every other client. Count only failures that indicate *the dependency* is unhealthy.

Scope, and the failure modes of breakers

Scope the breaker per (service, endpoint), so one broken endpoint does not cut off healthy ones. Per-*host* breakers are the wrong tool; that job belongs to outlier detection in the load balancer (Chapter 9). The two compose: ejection handles "one host is bad", the breaker handles "the dependency is bad".

Be honest about the failure modes. Breakers are **all-or-nothing**: an open breaker takes a service from degraded to hard-down for that dependency, so if your fallback is worse than a slow response you have made things worse — adaptive throttling, below, degrades continuously instead. They **flap** when thresholds sit near steady-state error rates, so use hysteresis and generous minimum call counts. Their state is **local but the decision is global**: with 1,000 replicas each sampling a few calls per second, individual breakers are noisy, which argues for the sidecar or shared client (Chapter 10). And a breaker is **no substitute for a timeout** — without timeouts there are no observed failures, only hangs.

Netflix's Hystrix popularized the pattern and has been in maintenance mode since roughly 2018, its README pointing users toward resilience4j; Envoy, Istio, and Linkerd provide mesh-layer equivalents. The pattern predates all of them — Michael Nygard's *Release It!* (2007) is canonical.

Hedged requests: buying tail latency

Everything so far is about failure. Hedging addresses a different problem: **the tail**. In a service that fans out, the slowest component sets user-visible latency, and "slow" happens for reasons that have nothing to do with your request — a GC pause, a compaction, a noisy neighbor, a queue you arrived behind. Jeff Dean and Luiz André Barroso's *The Tail at Scale* (CACM, February 2013) is the definitive treatment, and its essential observation is that at scale rare slowness is not rare: if one request in a hundred is slow and you wait for 100 backends, roughly two-thirds of user requests hit at least one slow backend. The technique: send to one replica; if nothing has arrived after roughly the p95 of expected latency, send the same request to a *different* replica; take whichever responds first and cancel the other.

```
sequenceDiagram
    autonumber
    participant C as Client
    participant R1 as Replica A
    participant R2 as Replica B

    C->>R1: request at t=0
    Note over R1: unlucky – stuck behind
    a compaction, will take 210ms
    Note over C: p95 of healthy latency is 15ms
    hedge delay set to 15ms
    C->>R2: hedge at t=15ms
    R2-->>C: response at t=21ms
    Note over C: return at t=21ms
    instead of t=210ms
    C-)R1: cancel – RST_STREAM / context cancel
    Note over C,R2: extra load is bounded by
    the fraction of calls exceeding p95,
    roughly 5 percent
```

The cost accounting is what makes hedging attractive: hedge only after the p95 and, by construction, only about 5 % of requests are ever hedged — while p99 and p99.9 improve sharply, because the request is now slow only if *both* replicas are slow. Dean and Barroso report a benchmark reading 1,000 keys spread across a hundred BigTable servers: hedging after 10 ms cut the 99.9th-percentile latency from roughly 1,800 ms to roughly 74 ms while sending about 2 % additional requests. Treat those figures as illustrative — and note the caveat implied by "both replicas": the gain collapses when slowness is *correlated* across replicas, as it is under fleet-wide overload or a shared-dependency stall.

The paper also describes **tied requests**: send to two replicas immediately, each tagged with the identity of the other, and have whichever server *dequeues* the request first cancel its counterpart. That removes the hedge delay entirely, at the cost of server-side support, and attacks queueing delay specifically.

When hedging is safe, and when it is a loaded gun

Hedging is retrying before failure, so every retry precondition applies, plus more:

1. **Idempotency is mandatory** — both requests may execute. Reads are the natural fit; mutations need idempotency keys and deduplication.

2. **You need capacity headroom.** Hedging adds load, so it must share the retry budget and disable itself when the dependency is unhealthy — otherwise the tail-latency tool becomes the amplifier. gRPC's retry throttling covers hedging too, which is the correct design.
3. **The hedge must go to a different replica** — Envoy's `previous_hosts` predicate again, plus the power-of-two-choices selection from Chapter 9.
4. **Cancellation must actually work**, or hedging costs a full 2×. gRPC and HTTP/2 make it cheap via `RST_STREAM`; the server must then honor `ctx.Done()` all the way into the database.
5. **Set the delay from data and adapt it.** A fixed delay that drifts below the current p95 hedges nearly everything and doubles your load — a self-inflicted retry storm.

gRPC supports hedging natively in the service config (hedging and retry are mutually exclusive per method):

```
{
  "methodConfig": [{
    "name": [{ "service": "search.v1.Index", "method": "Lookup" }],
    "timeout": "0.25s",
    "hedgingPolicy": {
      "maxAttempts": 2,
      "hedgingDelay": "0.015s",
      "nonFatalStatusCodes": ["UNAVAILABLE", "DEADLINE_EXCEEDED"]
    }
  ]},
  "retryThrottling": { "maxTokens": 100, "tokenRatio": 0.1 }
}
```

Envoy's `hedge_policy` offers `hedge_on_per_try_timeout`, which fires a hedged attempt when a per-try timeout expires while leaving the original in flight — a good approximation when the per-try timeout is already near p95.

Rolling your own in Go, with the refinement that a *fast failure* promotes the hedge immediately:

```

// Declared at package scope: Go does not permit type declarations that
// reference type parameters inside a generic function body.
type hedgeResult[T any] struct {
    val T
    err error
}

// Hedge issues attempt 0 immediately and attempt 1 after delay,
// returning the
// first success. do must be idempotent and must respect ctx
// cancellation.
func Hedge[T any](
    ctx context.Context,
    delay time.Duration,
    maxAttempts int,
    do func(ctx context.Context, attempt int) (T, error),
) (T, error) {
    ctx, cancel := context.WithCancel(ctx)
    defer cancel() // cancels every straggler on return

    results := make(chan hedgeResult[T], maxAttempts)

    launched := 0
    launch := func() {
        n := launched
        launched++
        go func() {
            v, err := do(ctx, n)
            results <- hedgeResult[T]{v, err}
        }()
    }

    launch()
    hedgeAt := time.After(delay)
    outstanding := 1
    var lastErr error
    var zero T

    for {
        select {
        case <-hedgeAt:
            hedgeAt = nil // fire once
            if launched < maxAttempts {
                launch()
                outstanding++
            }
        case r := <-results:
            outstanding--
            if r.err == nil {
                return r.val, nil
            }
        }
    }
}

```

```

    }
    lastErr = r.err
    if launched < maxAttempts {
        // Fast failure: don't wait out the hedge delay, retry
now.        hedgeAt = time.After(0)
            continue
    }
    if outstanding == 0 {
        return zero, lastErr
    }
    case <-ctx.Done():
        return zero, ctx.Err()
    }
}
}
}

```

The `defer cancel()` is the load-bearing line: it guarantees the losing attempt is cancelled the moment a winner returns. Omit it and you have built a request doubler.

Load shedding, admission control, and backpressure

Everything above is client-side. The server's obligation is simple to state: **never accept more work than you can complete within the deadline**. Accepting more is not generous; it is a lie that costs both parties.

The queue is the enemy

When arrival rate exceeds service rate a queue grows, and latency for everything in it grows with it. Past a point every queued request exceeds its deadline before dispatch, so the server spends all its work on answers that will be discarded: useful throughput goes to zero while CPU stays pinned. Hence unbounded queues — a default-constructed `LinkedBlockingQueue`, an accept backlog of 65535, an executor with an unbounded work channel — are a reliability anti-pattern (Volume 7, Chapter 11; Volume 10, Chapter 7).

Three shedding signals, in rough order of quality. **Queue delay** is the best: measure arrival-to-dispatch time and shed when it exceeds a target — the CoDel (Controlled Delay) algorithm from network queue management, applied to application queues. Ben Maurer's *Fail at Scale* (ACM Queue, 2015) describes CoDel-style management at Facebook with

a small target delay, plus switching from FIFO to **adaptive LIFO** under overload: when you are shedding anyway, the newest request is the better one to serve, because queued requests are near their deadline while a fresh one has its full budget. **Concurrency limits** come second: cap in-flight requests and reject beyond it, preferring adaptive limits (Netflix's `concurrency-limits`, whose TCP-Vegas- and gradient-style controllers infer the limit from observed latency versus its minimum) over static ones. **Utilization** thresholds are simplest and worst: laggy and confounded by noisy neighbors.

Prioritize, don't just reject

A user-facing read outranks a batch backfill, and a first attempt outranks a retry. Google's internal RPC system carries a **criticality** level on every request (`CRITICAL_PLUS` , `CRITICAL` , `SHEDDABLE_PLUS` , `SHEDDABLE`), propagated through the call tree the same way deadlines are, and sheds from the bottom up. The propagation is what matters: if a `SHEDDABLE` batch job's downstream calls arrive unlabeled, the backend cannot protect user traffic from them.

Deadline-aware eviction, queue-delay shedding, and criticality in one Go admission controller:

```

type Admission struct {
    sem          chan struct{} // concurrency limit
    targetDelay time.Duration // CoDel-style queue delay target
}

var ErrShed = errors.New("shed: server overloaded")

func (a *Admission) Admit(ctx context.Context, crit Criticality) (release func(), err error) {
    // 1. Drop work that is already doomed: no point queueing it.
    if dl, ok := ctx.Deadline(); ok && time.Until(dl) < a.targetDelay {
        return nil, ErrShed
    }

    enqueued := time.Now()

    // 2. Low-criticality work gets no queueing at all under
    contention.
    if crit >= Sheddable {
        select {
            case a.sem <- struct{}{}:
            default:
                return nil, ErrShed
        }
    } else {
        select {
            case a.sem <- struct{}{}:
            case <-ctx.Done():
                return nil, ctx.Err()
        }
    }

    // 3. If we waited too long, the queue is too deep – give the slot
    back.
    if waited := time.Since(enqueued); waited > a.targetDelay {
        <-a.sem
        return nil, ErrShed
    }
    return func() { <-a.sem }, nil
}

```

And the HTTP boundary, which must reject *cheaply* and tell the client what to do:

```

func Shedding(a *Admission, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        release, err := a.Admit(r.Context(), criticalityOf(r))
        if errors.Is(err, ErrShed) {
            // Retry-After tells a well-behaved client to back off;
without it,
            // rejected load returns immediately as new load.
            w.Header().Set("Retry-After", "1")
            http.Error(w, "overloaded", http.StatusServiceUnavailable)
            sheddedTotal.Inc()
            return
        }
        if err != nil {
            // The caller's context is already done; nothing will read
this.
            // 499 is nginx's non-standard "client closed request"
code, used
            // here only so the access log distinguishes it from a real
5xx.
            http.Error(w, "cancelled", 499)
            return
        }
        defer release()
        next.ServeHTTP(w, r)
    })
}

```

Two properties make this work. Rejection must be **cheap** — do it before deserialization, before auth, before any I/O — or shedding becomes its own overload. And it must be **honest**: return a status the client's policy treats as "back off", paired with `Retry-After` or gRPC `RetryInfo`.

Adaptive throttling: the client's half

A matching client-side technique from Google's SRE book deserves more adoption. Each client tracks, over a rolling window, `requests` (attempts made) and `accepts` (requests the backend actually served), then rejects new requests locally with probability:

```

p_reject = max(0, (requests - K * accepts) / (requests + 1))    # K is
typically 2

```

While the backend accepts most requests, `K * accepts` exceeds `requests` and `p_reject` is zero. As the backend starts rejecting, the ratio inverts and clients self-throttle *before* sending, so rejected load never

reaches the network. Unlike a circuit breaker this degrades continuously rather than snapping between all and nothing; $K = 2$ lets clients send up to twice what the backend is accepting, leaving room to discover recovery.

Graceful degradation, and the trouble with fallbacks

When a call fails after all of this you have three choices: fail the request, degrade, or fall back.

Degrade where the product allows. A feed without the personalization service renders in recency order; a product page without recommendations is still a product page. Two things make this real: the degraded path must be a first-class code path with its own metrics, so you can alert on degraded rate rather than only on error rate, and it must be exercised continuously — a path that runs once a quarter during an incident does not work (Volume 11, Chapter 8).

Fallbacks deserve suspicion. Amazon's Builders' Library article *Avoiding fallback in distributed systems* argues it bluntly: fallback logic is rarely exercised so it is rarely correct; it fires when the system is already stressed; and it is frequently *correlated* with the failure it is meant to handle — the fallback store is in the same AZ, the fallback path calls the same overloaded metadata service. Prefer making the primary path robust. If you must have a fallback, run it continuously rather than conditionally, so it is always warm and always tested.

Serving stale is usually the best degradation available. Data ten minutes old during an outage beats an error, and `stale-while-revalidate` and `stale-if-error` (RFC 5861) express that at the HTTP layer without writing code (Chapter 10).

Composing one coherent policy

These patterns interact, and configuring them independently is how you get a system that is individually reasonable and collectively suicidal. The decisions have a fixed order. The deadline comes first, set once at the entry point from the product SLO and partitioned across the hops below it; per-attempt timeouts follow from data and must fit inside that budget, retries included, because when the arithmetic does not fit you get fewer attempts and never a longer deadline. Only then do the protective

mechanisms attach: budgeted retries at exactly one layer, full jitter with server hints overriding your computation, a breaker composed with per-host ejection, hedging only where there is headroom, and shedding underneath all of it.

Written as a per-dependency specification — the artifact that should live next to the code and be reviewed like an API:

Parameter	Critical read	Non-critical enrichment	Write
Total budget (from caller)	400 ms	40 ms	1,200 ms
Connect timeout	100 ms	50 ms	100 ms
Per-attempt timeout	150 ms	40 ms	500 ms
Max attempts	3	1	2, idempotency key required
Retryable	UNAVAILABLE , connect-failure, refused-stream	none	same, plus timeout if key present
Backoff	25 ms base, full jitter, 250 ms cap	n/a	100 ms base, full jitter, 1 s cap
Retry budget	20 % / min 3 concurrent	n/a	10 % / min 3
Circuit breaker	50 % failure or 60 % slow over 60 s	none — fail open, degrade	50 % over 60 s
Hedging	after 60 ms, max 2	no	no
On failure	serve stale cache, else 503	omit the section	503, client must retry with key

Then make it observable, because an unobserved policy is a guess. The minimum, all labeled by dependency and outcome: attempts versus logical requests, so amplification is a graph rather than a theory; retry-budget exhaustion and breaker transitions, both alertable; deadline-exceeded counts split by *where* the deadline was set, since expiry on

arrival means an upstream hop is overspending; shed counts by criticality; and hedge rate alongside hedge win rate. Without per-dependency attribution, all you learn during an incident is that *something* is retrying.

Distributed-systems lens

These patterns are, more than anything else in this volume, the difference between a fleet that degrades and one that collapses.

Amplification, not the trigger, is what makes an outage large. The trigger is usually mundane; retry storms and thundering herds are the amplifier that converts a localized, brief problem into a fleet-wide, prolonged one. The signature is a system that does not recover when the trigger is removed. Such systems are **metastable**: a stable healthy state and a stable failed state, with enough of a shove moving them permanently from one to the other, usually requiring an operator to break the loop by hand — shed everything, restart cold, ramp slowly. Retry budgets, breakers, and shedding exist to prevent the second stable state from existing.

Deadline propagation is the highest-value cross-cutting change most fleets can make, because it converts wasted work into cancelled work; during an incident, capacity spent on requests whose callers have already given up can be most of the fleet's CPU. Its prerequisite is uniformity — one hop that resets the deadline breaks the chain below it — so it belongs in the shared framework rather than in each service.

Hedging and retries are one mechanism with two triggers — elapsed time versus observed failure — which is why they share a budget: they compete for the same headroom, and neither creates capacity. Paired with Chapter 9's power-of-two-choices, which reduces the chance of landing on a slow server in the first place, they are the whole tail-latency toolkit.

Implement it once, in the mesh or the shared library. Re-implemented per service in five languages you get five behaviors, several dangerous, and no way to change any of them mid-incident. In the sidecar or a shared client (Chapter 10) you get uniform semantics, uniform metrics, and the ability to push policy fleet-wide — dropping `num_retries` to zero everywhere in seconds is a standard lever for breaking a storm. The Chapter 10 caveat stands: the mesh cannot know

whether an operation is idempotent, so retry-safety remains an application-level assertion.

All of this is admission control on the whole system. Timeouts bound how much of your capacity a dependency may borrow; budgets bound how much extra load you may impose; shedding bounds how much work you accept. Each is a statement about finite resources under contention, which is why the shape recurs at every layer — TCP congestion control (Chapter 3), HTTP/2 flow control (Chapter 7), Kafka consumer backpressure (Volume 10, Chapter 7), database connection pools (Volume 5, Chapter 14).

Test them, or you do not have them. Every mechanism here is dormant in normal operation and decays silently. Fault injection — Envoy's fault filter, mesh-level delay and abort injection, chaos experiments (Volume 11, Chapter 8) — is how you discover that a breaker threshold was never reachable, or that the "1-second" timeout is really sixty because an SDK default overrode your config. And load tests must run *past* the knee of the curve: a system never pushed into shedding has an untested shedding path.

Key takeaways

- An unbounded call is a resource leak with a computable cost: with N slots and timeout T the throughput floor is N/T , so a timeout is a capacity decision, not a hygiene setting.
- Set the whole ladder — connect, TLS, response-header, total — plus transport bounds (`TCP_USER_TIMEOUT` , keepalives, HTTP/2 pings), deriving per-attempt values from observed p99-p99.9 client-side latency.
- Deadlines beat per-hop timeouts: propagate remaining *duration* to sidestep clock skew, cancel downstream on expiry, refuse work that arrives with too little budget, and evict expired work from queues before dispatch.
- Failure is ambiguous, so retries require idempotency. Retry transport-level "not processed" signals freely; retry ambiguous failures only with idempotency keys; never retry validation errors.
- Three retrying layers is $27\times$ amplification at the leaf, arriving exactly when capacity is shortest. Bound retries as a *fraction of traffic* (gRPC

`retryThrottling`, `Envoy retry_budget`), retry at exactly one layer, and route retries away from the failed host.

- Unjittered backoff builds synchronized waves that keep a recovering service pinned. Default to full jitter, and jitter cache TTLs, cron, health checks, and reconnects too.
- Circuit breakers count slow calls as failures over a time-based window with a minimum call count, exclude client errors, and bound half-open probes. They compose with, but do not replace, per-host outlier ejection.
- Hedge idempotent reads after roughly the p95, sharing the retry budget, targeting a different replica, and cancelling the loser.
- Servers must shed: bound every queue, evict expired work, shed on queue delay, prioritize by criticality, and reject cheaply with `Retry-After`. Client-side adaptive throttling stops rejected load before it is sent.
- Prefer degradation and stale data to fallbacks, which are rarely exercised and frequently correlated with the failure they are meant to absorb.
- Write the composed policy down per dependency, instrument it, and implement it once in the mesh or shared library so it can be changed fleet-wide mid-incident.

Further reading

- Arnon Rotem-Gal-Oz, *Fallacies of Distributed Computing Explained* — the standard write-up of the list.
- Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog, 2015 — <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/> — the definitions and simulations of full, equal, and decorrelated jitter used here.
- Amazon Builders' Library — <https://aws.amazon.com/builders-library/> — especially "Timeouts, retries, and backoff with jitter", "Using load shedding to avoid overload", and "Avoiding fallback in distributed systems".
- Jeffrey Dean and Luiz André Barroso, "The Tail at Scale", *Communications of the ACM*, Vol. 56 No. 2, February 2013 — <https://cacm.acm.org/research/the-tail-at-scale/> — hedged and tied requests.

- Betsy Beyer et al. (eds.), *Site Reliability Engineering*, O'Reilly, 2016 — <https://sre.google/sre-book/table-of-contents/> — Chapter 21 "Handling Overload" (criticality, adaptive throttling, the `requests / accepts` formula) and Chapter 22 "Addressing Cascading Failures".
- gRFC A6: *gRPC Retry Design* — <https://github.com/grpc/proposal/blob/master/A6-client-retries.md> — normative for `retryPolicy`, `hedgingPolicy`, and `retryThrottling`; and <https://grpc.io/docs/guides/deadlines/> for `grpc-timeout`.
- Envoy architecture documentation on routing and retries (https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/http/http_routing), circuit breaking (https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/circuit_breaking), and outlier detection (https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier).
- Ben Maurer, "Fail at Scale: Reliability in the face of rapid change", *ACM Queue*, 2015 — <https://queue.acm.org/detail.cfm?id=2839461> — controlled-delay queueing and adaptive LIFO; the underlying algorithm is Kathleen Nichols and Van Jacobson, "Controlling Queue Delay", *ACM Queue*, 2012.
- Michael T. Nygard, *Release It!*, 2nd edition, Pragmatic Bookshelf, 2018 — the origin of the circuit-breaker and bulkhead vocabulary. `resilience4j` (<https://resilience4j.readme.io/>) is the current reference implementation; for adaptive server-side limits, Netflix `concurrency-limits` (<https://github.com/Netflix/concurrency-limits>).
- RFC 9110 (*HTTP Semantics*) on idempotent methods and `Retry-After`; RFC 5861 (`stale-while-revalidate`, `stale-if-error`); RFC 9113 (*HTTP/2*) on `REFUSED_STREAM` and `GOAWAY`.
- AWS post-event summaries for the April 2011 EC2/EBS disruption and the September 2015 DynamoDB disruption in US-East — public write-ups of self-sustaining retry and re-mirroring storms.
- Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu, "Metastable Failures in Distributed Systems", HotOS 2021 — the formal framing of the "does not recover when the trigger is removed" failure mode.
- Within this volume: Chapters 3, 7, 8, 9, 10, and 12. Across volumes: Volume 6, Chapter 9 (idempotency and exactly-once), Volume 7, Chapter 11 (bulkheads, breakers, shedding), Volume 10, Chapter 7

(backpressure), and Volume 11, Chapters 1, 4, and 8 (SLOs, tracing, chaos).

Chapter 12 — Debugging and Observing Networks

What this chapter covers. This is the practitioner's capstone of the volume. Chapters 1 through 11 built a model of each layer in turn — the packet path, IP and NAT, TCP, UDP and QUIC, DNS, TLS, HTTP, gRPC, load balancing, proxies and mesh, and the reliability patterns that wrap remote calls. Production does not hand you a layer; it hands you "checkout p99 went from 40 ms to 4 s at 14:02" or "about 0.3% of gRPC calls fail with `UNAVAILABLE`, but only from the eu-west cluster." This chapter is the translation: a method for turning a symptom into a falsifiable hypothesis about a specific layer at a specific hop, the tools that test those hypotheses, a catalog of failure *signatures* worth memorizing, and the fleet-scale reality that you cannot SSH into a box and run `tcpdump` when the failure is one hop out of six across ten thousand pods. The through-line: capture is the last resort, not the first move.

Learning goals — after this chapter you should be able to:

- Run a layered triage — resolution, connectivity, transport, TLS, application — and bisect client versus network versus server, stating at each step what a result would *falsify*.
- Read `curl 's --write-out` timing breakdown as a protocol-phase oscilloscope, and interrogate DNS with `dig`, `resolvectl`, and `getent` while knowing why those paths differ from your application's.
- Explain the mechanism behind `ping`, `traceroute`, and `mtr`, and the ways ECMP, anycast, ICMP rate limiting, and asymmetric paths make their output misleading.
- Read `ss -ti` as a window into the per-connection TCP control block, and kernel and NIC counters (`nstat`, `ethtool -S`, `conntrack -S`) as proof or disproof of drop hypotheses without a capture.

- Write correct BPF filters, capture safely in production including inside container network namespaces, and decrypt TLS with `SSLKEYLOGFILE`.
- Recognize the classic signatures on sight: refused versus timeout, accept-queue overflow, PMTU blackholes, certificate and SNI failures, stale DNS, port and conntrack exhaustion, asymmetric routing, `ndots` amplification, retry storms, and HTTP/2 connection pinning.
- Design always-on telemetry that localizes the failing hop before anyone reaches for a capture.

Debugging is hypothesis testing, not tool trivia

The engineers who are fast at this are not the ones who have memorized the most flags. They keep an explicit list of candidate explanations and pick the next command by which candidate it eliminates. The method has three parts.

Work the layers, in dependency order. A request must resolve a name, route packets to an address, complete a transport handshake, complete a TLS handshake, and exchange application messages. Each stage depends on all the previous ones, so a failure at stage n makes every test at stage $n+1$ meaningless. Testing HTTP semantics while DNS returns a decommissioned address is not debugging; it is generating noise. Test in order, and stop at the first stage that misbehaves.

Bisect the path. A modern request crosses many hops: client → resolver → CDN edge → anycast L4 LB → L7 proxy → sidecar → server sidecar → server → database. The layered method tells you *which protocol stage* fails; bisection tells you *where*. Run the same test from progressively closer vantage points — your laptop, a pod in the same cluster, the node hosting the server, the server's own network namespace, `localhost` on the server. The first vantage point at which the failure disappears brackets the fault between it and the previous one. That turns "checkout is broken" into "checkout is broken from eu-west but not from the same-zone canary, and not from the node itself" — most of a root cause.

Change one variable. When a test differs from production in five ways — different source IP, different DNS path, no mesh sidecar, HTTP/1.1 instead of HTTP/2, no client certificate — a negative result proves nothing. `curl` from your laptop is not your Java service: different

resolver, different TLS library and trust store, different connection pool, different timeouts, no sidecar interception. Bring the test as close to the real client as you can afford — a debug container in the same pod shares the pod's network namespace, resolver configuration, and sidecar interception, the highest-fidelity vantage point Kubernetes offers — and when you cannot, list the differences before you interpret the result.

flowchart TD

```
S["Symptom: errors or latency on a specific call"] --> R{"Can you reproduce from a controlled client?"}
R -->|"no, intermittent or fleet-wide"| TEL["Start from always-on telemetry: RED metrics per hop, traces, LB and mesh access logs"]
TEL --> LOC["Localize the failing hop, then reproduce there"]
LOC --> D
R -->|yes| D{"Does the name resolve, and to the address you expect?"}
D -->|"NXDOMAIN, SERVFAIL, timeout, or stale or wrong address"| DNS["DNS - Chapter 5: resolver path, TTL and cache, ndots, search domains, EDNS and TCP fallback"]
D -->|ok| C{"Does a TCP connect to that address and port succeed?"}
C -->|"connection refused"| REF["Nothing is listening: wrong port, process down, wrong address family, or an RST-generating middlebox"]
C -->|"timeout, no response"| NET["Silent drop: firewall or security group, routing or asymmetric return path, conntrack, SYN backlog - Chapters 1, 2, 3"]
C -->|ok| T{"Does the TLS handshake complete and verify?"}
T -->|no| TLS["TLS - Chapter 6: expiry, chain, SNI, ALPN, version and cipher, client cert, clock skew"]
T -->|ok| H{"Does the application respond, correctly and in time?"}
H -->|"5xx, RST, GOAWAY, or gRPC UNAVAILABLE"| APP["Application and proxy layer - Chapters 7, 8, 9, 10: backend health, timeouts, circuit breaking, protocol errors"]
H -->|"slow, not failing"| LAT["Attribute latency: curl timing split, then per-hop spans - Volume 11 Chapter 4"]
H -->|"works here but not in production"| DIFF["Enumerate the differences: source identity, policy, sidecar, protocol version, pooling, payload size"]
```

"It's always DNS" — why the joke keeps being true

DNS sits on the critical path of every call while being the layer with the most independent caches, the loosest failure semantics, and the least visibility. Three structural reasons keep it at the top of the suspect list.

First, **caching is layered and none of it is yours**. An answer may be cached in the application process (JVM `networkaddress.cache.ttl`, a client library's own cache; Go's resolver does not cache at all), in a node-local cache (NodeLocal DNSCache, `nscd`, `systemd-resolved`), in the

cluster resolver (CoreDNS), and in an upstream recursive resolver — each with its own TTL accounting. Changing a record does not change behavior; it starts a countdown whose length you control only partly, and any layer that ignores or floors TTLs extends it. The resulting failure is usually latency or staleness rather than an error: a resolver answering in 5 seconds instead of 5 milliseconds produces "the service is slow," not "DNS is broken."

Second, **the transport is fragile in ways that hide**. Classic DNS is UDP-first; a response larger than the advertised EDNS(0) buffer comes back truncated with the TC bit set and the resolver retries over TCP. If a firewall permits UDP/53 and quietly drops TCP/53 — a common misconfiguration — resolution works until a zone grows a record or DNSSEC signatures push a response past the limit, and then it fails for a subset of names, intermittently. The symptom looks nothing like "DNS."

Third, **the tooling lies to you by default**: `dig` and your application take different resolver paths, as a later section explains, and when they disagree the difference is the bug.

The correct response to the cliché is not to assume DNS but to *falsify* it in under a minute, from the affected host at the affected moment, and move on.

Triage in sixty seconds: `curl` as an oscilloscope

Almost every HTTP-ish problem should start with `curl`, not because it is powerful but because a single invocation splits end-to-end latency into protocol phases — the fastest layer attribution available.

```
curl -sS -o /dev/null \  
-w \  
'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} \  
pre=%{time_pretransfer} ttfb=%{time_starttransfer} total=%{time_total} \  
code=%{http_code} ver=%{http_version} ip=%{remote_ip}:%{remote_port} \  
conns=%{num_connects} verify=%{ssl_verify_result}\n' \  
https://api.internal.example.com/v1/health
```

```
dns=0.004132 tcp=0.005871 tls=0.031004 pre=0.031120 ttfb=0.284517 \  
total=0.284790 code=200 ver=2 ip=10.42.7.19:443 conns=1 verify=0
```

Every `time_*` variable is **cumulative from the start of the request**, so the phases are *differences*, not the raw numbers — the single most

common misreading. Here: DNS 4.1 ms, TCP handshake 1.7 ms, TLS handshake 25.1 ms, request send ~0.1 ms, then **253 ms waiting for the first response byte**, with the body arriving in the following 0.3 ms. That is a server-side (or downstream-of-server) latency problem; no amount of network work will fix it. Had `tcp - dns` been 200 ms instead, you would be looking at handshake latency: an over-long path, a SYN retransmit (Linux's initial RTO is 1 s, so a lost SYN produces a suspiciously round ~1 s or ~3 s step), or a saturated middlebox.

Keep the format string in a file (`-w "@$HOME/curl-format.txt"`) so it is readable and reusable. curl 7.70.0 and later also accept `-w '%{json}'`, which dumps every write-out variable as a JSON object — ideal when scripting a probe loop and feeding the results to `jq`.

```
flowchart LR
    A["request start  
t = 0"] --> B["time_namelookup  
name resolved"]
    B --> C["time_connect  
TCP handshake complete"]
    C --> D["time_appconnect  
TLS handshake complete"]
    D --> E["time_pretransfer  
about to send request"]
    E --> F["time_starttransfer  
first response byte - TTFB"]
    F --> G["time_total  
last byte received"]
    B -.-> B1["large delta: resolver latency, ndots search fanout, UDP  
retry after 5 s timeout"]
    C -.-> C1["large delta: RTT or path length, SYN loss and RTO of about  
1 s, middlebox queueing"]
    D -.-> D1["large delta: extra round trips, OCSP or CRL fetch by the  
client, signature and key-exchange CPU cost, CPU starvation"]
    F -.-> F1["large delta: server think time, upstream dependency, proxy  
queueing, connection-pool wait at the peer"]
    G -.-> G1["large delta with small TTFB: bandwidth, cwnd growth,  
receiver window, loss during transfer"]
```

Three more `curl` habits worth building.

Pin the address to separate DNS from everything else. `--resolve` overrides resolution for one host/port pair while leaving `Host:`, SNI, and certificate verification intact:

```
curl -sS -o /dev/null -w '%{http_code} %{remote_ip}\n' \  
--resolve api.internal.example.com:443:10.42.7.31 \  
https://api.internal.example.com/v1/health
```

Loop that over every endpoint address and you have per-backend health with correct TLS semantics — the fastest way to find the one bad replica behind a name. `--connect-to HOST1:PORT1:HOST2:PORT2` does the same when you also need to redirect the port.

Pin the protocol. `--http1.1`, `--http2`, `--http2-prior-knowledge`, and — in curl builds with a QUIC-capable backend — `--http3`. If a request succeeds over HTTP/1.1 and fails over HTTP/2, you have implicated ALPN negotiation, an intermediary's HTTP/2 handling, header-size limits, or `GOAWAY` behavior rather than the application. If HTTP/3 fails while HTTP/2 works from the same host, suspect UDP blocked or rate-limited on the path (Chapter 4).

Read the exit code. curl's exit codes are a coarse but instant layer classifier: `6` could not resolve host, `7` failed to connect, `28` timed out, `35` TLS connect error, `52` empty reply, `56` failure receiving data (often a mid-stream reset), `60` peer certificate verification failure (this code absorbed the older `CURLE_SSL_CACERT` in curl 7.62.0), `92` an HTTP/2 stream error. `-v` gives the narrative and `--trace-time` timestamps every trace line, which is how you see *where* a stall sits without a capture; `--trace-ascii -` dumps the bytes.

DNS: ask precisely, and ask what the application asks

`dig` is the instrument, but use it deliberately.

```
# What does the resolver the application uses actually return, with
TTL?
dig +noall +answer api.internal.example.com

# Ask one specific server, bypassing the local resolver entirely
dig @10.96.0.10 api.internal.example.com A +stats

# Full delegation walk from the root, one label at a time
dig +trace api.internal.example.com

# What the zone actually says, straight from an authoritative server
(expect the aa flag)
dig @ns1.example.net api.internal.example.com +norecurse

# Does my recursive resolver already hold a cached answer? No aa flag,
and a TTL below the zone's
dig @10.96.0.10 api.internal.example.com +norecurse
```

Two nuances matter in incidents. First, **+trace deliberately bypasses your recursive resolver**: it starts at the root hints and follows delegations itself, which makes it excellent for proving the *authoritative* data is correct and useless for reproducing what your application sees. A **+trace** that looks perfect while users fail is the signature of a caching or resolver problem, not a zone problem. Second, the TTL in a cached answer counts *down*; two consecutive **dig** s against a recursive resolver showing 300 then 297 tell you the answer is cached and how long ago it was fetched. A TTL that never decreases means an authoritative answer, or a forwarder that rewrites TTLs.

To identify *which* instance of an anycast resolver answered, NSID (RFC 5001) is the tool:

```
dig +nsid @9.9.9.9 example.com | grep -A1 NSID
```

To reproduce the *application's* view, stop using **dig**. It speaks DNS on the wire; your process goes through the C library's name service switch, which consults **/etc/hosts**, possibly **nscd** or **systemd-resolved**, and applies **search** and **ndots** from **/etc/resolv.conf**:

```
getent hosts api.internal.example.com    # what glibc NSS returns,
hosts file included
cat /etc/resolv.conf
# nameserver, search, options ndots:N, timeout, attempts
resolvectl status                       # per-link DNS config under
systemd-resolved
resolvectl query api.internal.example.com # resolve through the stub
resolver, with cache info
resolvectl statistics                   # cache hits/misses,
transactions
resolvectl flush-caches                 # prove or disprove "it's a
stale cache"
```

Alpine images complicate this further: musl's resolver has historically differed from glibc's — notably the absence of TCP fallback for truncated responses (added only in musl 1.2.4, released in 2023) and querying the configured nameservers in parallel rather than in order. If a bug reproduces on Alpine and not on Debian with the same code, suspect the resolver implementation.

Finally, the `ndots:5` trap from Chapter 5: in Kubernetes, `example.com` (one dot, fewer than five) is tried against every search domain — `example.com.<ns>.svc.cluster.local`, `example.com.svc.cluster.local`, `example.com.cluster.local`, and any node domains — before being tried as-is. With A and AAAA for each, a single external lookup can become ten queries. The signature is a large `time_namelookup` for external names only, and a CoreDNS query rate several times your request rate. `dig` will not reproduce it, because it does not apply the search list the way an application does; `getent hosts` and a capture of UDP/53 will.

Connectivity and path: what `ping` and `traceroute` really tell you

`ping` sends ICMP Echo Requests and times the Echo Replies. It answers exactly one question: *does this address respond to ICMP right now, and with what round-trip time and loss?* It says nothing about service reachability, because firewalls, security groups, and load balancers treat ICMP and TCP by different rules — an anycast VIP that answers ping may have no healthy backend, and a perfectly healthy host may drop ICMP by policy. ICMP loss is also weak evidence of data-plane loss: routers handle ICMP directed at *themselves* on a slow control-plane path and rate-limit it hard.

Where `ping` earns its place is as a *baseline RTT* and as a **path MTU probe**:

```
# 1472 bytes of payload + 8 ICMP header + 20 IP header = 1500 bytes, DF
set
ping -M do -s 1472 -c 3 db.internal.example.com
# If this fails but -s 1400 succeeds, the path cannot carry 1500-byte
frames.
```

`tracert` exploits TTL: it sends probes with TTL=1, 2, 3, ...; each router that decrements the TTL to zero discards the packet and returns an ICMP Time Exceeded, revealing its address. The destination announces itself differently by probe type — a UDP probe to a high unused port elicits ICMP Port Unreachable, an ICMP Echo probe an Echo Reply, a TCP SYN probe a SYN/ACK or RST. Linux `tracert` defaults to UDP with destination ports starting at 33434 and incrementing per probe.

That default is a problem in modern networks, and understanding why is the difference between reading a traceroute correctly and being fooled by one:

- **ECMP hashes on the 5-tuple.** Because classic traceroute varies the destination port per probe, consecutive probes hash to *different* paths, so the output is a superposition of several. `paris-tracert` and `dublin-tracert` hold the flow identifier constant so you trace one path; `tracert -T -p 443` gets closer to the real flow, but its source port still varies.
- **ICMP generation is rate-limited and deprioritized.** A middle hop showing 40% loss while every subsequent hop shows 0% is not losing your traffic; it is declining to generate ICMP as fast as you are asking. Loss is meaningful only if it persists at that hop **and all hops after it**.
- **Anycast means "the hop" is not one machine.** Successive probes may be answered by different routers or PoPs (Chapters 2 and 10).
- **The return path is invisible.** Traceroute conflates the forward path with the return path of the ICMP errors, and asymmetric routing is normal.
- **MPLS and layer-2 clouds hide hops.** A "hop" that appears to add 30 ms may be a label-switched path crossing a continent.

`mtr` is traceroute run continuously with per-hop statistics, which distinguishes steady loss from a blip:

```
mtr --report --report-cycles 100 --tcp --port 443 --no-dns
api.internal.example.com
```

```
Start: 2026-08-13T09:14:02+0000
HOST: probe-a-7f9c
StDev
  1. |-- 10.42.0.1          0.0%  100    0.4   0.5   0.3
1.9   0.2
  2. |-- 10.0.30.1         0.0%  100    1.1   1.3   0.9
9.8   0.9
  3. |-- 100.65.4.18       38.0%  100    9.7   9.9   9.1
17.2  1.1
  4. |-- 100.65.9.2        0.0%  100   10.1  10.3   9.6
19.4  1.3
  5. |-- 10.42.7.19        0.0%  100   10.4  10.6   9.9
21.0  1.4
```

Hop 3's 38% "loss" is an artifact: hops 4 and 5 are clean, so nothing was dropped. Had hops 3, 4, and 5 all shown ~38%, that would be real loss entering at hop 3.

For plain port reachability, `nc` is the blunt instrument; `-w` is mandatory so it cannot hang:

```
nc -vz -w 3 db.internal.example.com 5432
# Connection to db.internal.example.com (10.42.9.4) 5432 port [tcp/
postgresql] succeeded!
```

A refusal returns immediately; a silent drop hangs until the timeout. That distinction is the most informative single bit in network debugging; the field guide below makes it precise.

For TLS reachability and certificate inspection, `openssl s_client` is the equivalent:

```
openssl s_client -connect api.internal.example.com:443 \
  -servername api.internal.example.com -alpn h2 -showcerts </dev/null
2>&1 | head -40
```



```

CONNECTED(00000003)
depth=2 C = US, O = Internal Root, CN = Internal Root CA X1
verify return:1
depth=1 C = US, O = Internal Root, CN = Internal Issuing CA 3
verify return:1
depth=0 CN = api.internal.example.com
verify return:1
---
ALPN protocol: h2
New, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256
Verify return code: 0 (ok)

```

`-servername` is not optional: omit it and a virtual-hosted server or shared L7 proxy hands you a default certificate, and you will "reproduce" a certificate bug that does not exist. The `Verify return code` is the punchline — `0 (ok)`, `10 (certificate has expired)`, `18 (self signed certificate)`, `19 (self signed certificate in certificate chain)`, `21 (unable to verify the first certificate)`. To check expiry in one line:

```

openssl s_client -connect api.internal.example.com:443 -servername
api.internal.example.com \
    </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates

```

Sockets: `ss` is the X-ray

`ss` reads socket state from the kernel via `netlink ( sock_diag )`, which is why it stays fast on a machine with 200,000 sockets and why it can expose per-connection TCP internals `netstat` never could.

```

ss -tanp state established '( dport = :5432 or sport = :5432 )'
ss -tlnp                    # listeners with owning process
ss -s                      # summary by state and family
ss -tan state time-wait | wc -l # TIME_WAIT population

```

On a **listening** socket, `ss` overloads the queue columns in a way you must know cold: `Recv-Q` is the number of established connections **waiting to be accepted**, and `Send-Q` is the **maximum accept-queue length**, `min(listen backlog, net.core.somaxconn)`. It is a direct read-out of the mechanism from Volume 2, Chapter 10:

```
$ ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN 129    128     0.0.0.0:8080      0.0.0.0:*          users:
(("app",pid=1041,fd=8))
```

`Recv-Q` at or above `Send-Q` means the application is not calling `accept()` fast enough, so the kernel silently drops the final ACK of new handshakes — invisible on the client except as a connection that stalls at 1 s, 3 s, 7 s (SYN-ACK retransmits) and then either succeeds or times out. On an **established** socket the same columns mean the ordinary thing: bytes queued in the receive buffer not yet read, and bytes queued for transmission not yet acknowledged. A persistently large `Recv-Q` on an established server socket says the application is not reading — a thread-pool or event-loop stall masquerading as a network problem.

The real power is `-i` (with `-m` for socket memory, `-e` for extended info):

```
ss -tim state established '( dport = :443 )'
```

```
ESTAB 0 0 10.42.7.19:51234 10.42.9.4:443
    skmem:(r0,rb131072,t0,tb262144,f4096,w0,o0,bl0,d17)
    cubic wscale:7,7 rto:236 rtt:33.412/2.107 ato:40 mss:1448
pmtu:1500 rcvmss:1448
    advmss:1448 cwnd:12 ssthresh:12 bytes_sent:918273
bytes_retrans:41940
    bytes_acked:876333 bytes_received:2044112 segs_out:1204
segs_in:1789
    data_segs_out:1191 send 4.2Mbps lastsnd:12 lastrcv:8 lastack:8
    pacing_rate 5.0Mbps delivery_rate 3.9Mbps app_limited busy:14231ms
    retrans:0/31 dsack_dups:2 reordering:6 rcv_rtt:34.1
rcv_space:65535 minrtt:31.9
```

Read it as the TCP control block from Chapter 3:

- `cubic` — the congestion control algorithm actually in use for this socket.
- `rtt:33.412/2.107` — smoothed RTT and its mean deviation (`rttvar`), in milliseconds; `minrtt` is the lowest RTT seen, the closest thing to the propagation floor. `rtt` much larger than `minrtt` means queueing somewhere (bufferbloat, an overloaded middlebox, or a busy receiver).
- `cwnd:12` with `ssthresh:12` — the congestion window has been cut and is in congestion avoidance. A `cwnd` stuck near the initial window

(10 segments on Linux) on a long-lived, high-throughput connection means repeated loss.

- `retrans:0/31` — currently outstanding / total retransmits on this socket. `bytes_retrans:41940` against `bytes_sent:918273` is a ~4.6% retransmission rate, which is severe.
- `send 4.2Mbps` is derived, roughly $\text{cwnd} \times \text{mss} \times 8 / \text{rtt}$: what the congestion window permits, not what happened. `delivery_rate` is what the kernel measured by rate sampling (the mechanism BBR is built on). `app_limited` is crucial — the connection was *not* limited by the network but by the application not supplying data, so do not diagnose bandwidth on an app-limited socket.
- `skmem:(...,d17)` — the `d` field is `sk_drops`: packets dropped on this socket, usually because the receive buffer was full. Non-zero `d` on a UDP socket is the canonical "my UDP receiver is too slow" evidence.
- `pmtu:1500` and `mss:1448` — the path MTU the stack believes in and the resulting MSS (1500 less 20 bytes of IPv4 header, 20 of TCP header, and 12 for the timestamp option). A low `pmtu` on a tunnelled path is normal; watching `pmtu` collapse mid-connection is the fingerprint of PMTU discovery reacting to an ICMP "fragmentation needed" or ICMPv6 "Packet Too Big."

`lsuf` answers the inverse question — which process owns a socket, and what else it has open — and catches file-descriptor exhaustion, which presents as connection failures that are not network failures at all:

```
lsuf -nP -iTCP -sTCP:ESTABLISHED -a -p 1041 | wc -l
cat /proc/1041/limits | grep 'open files'
ls /proc/1041/fd | wc -l
```

Kernel and NIC counters: proving drops without a capture

Counters are the cheapest evidence in the toolkit and most engineers under-use them. `nstat` reads the same SNMP-style counters as `netstat -s` but prints **deltas since its last invocation**, which makes causality obvious. It keeps history in a file (overridable via `NSTAT_HISTORY`); `-a` prints absolutes, `-z` includes zeroes, `-s` leaves history untouched.

```
nstat -n          # zero the baseline quietly
sleep 60
nstat             # deltas over the last 60 seconds
```

```
#kernel
TcpActiveOpens          18422          0.0
TcpPassiveOpens         19110          0.0
TcpAttemptFails         214           0.0
TcpEstabResets          902           0.0
TcpRetransSegs         3341          0.0
TcpOutRsts             1187          0.0
TcpExtListenOverflows   486           0.0
TcpExtListenDrops       486           0.0
TcpExtTCPSynRetrans     1904          0.0
TcpExtTCPTimeouts       621           0.0
TcpExtTCPBacklogDrop    33            0.0
TcpExtTCPZeroWindowDrop 4             0.0
```

The counters worth knowing by name, and what each implicates:

Counter	Meaning	Implicates
TcpExtListenOverflows / TcpExtListenDrops	accept queue full when a handshake completed	app not accepting fast enough; backlog/ somaxconn too small (Vol 2 Ch10)
TcpExtTCPReqQFullDrop / ... Droptail	SYN queue full	SYN flood or handshake rate above capacity; enable/ observe SYN cookies
TcpExtTCPSynRetrans	SYNs retransmitted	SYN or SYN-ACK loss: path drop, firewall, overloaded peer
TcpRetransSegs , TcpExtTCPLostRetransmit	data retransmission, retransmits themselves lost	genuine path loss or extreme congestion
TcpExtTCPTimeouts	RTO fired (not fast retransmit)	tail loss or heavy loss; expect ~200 ms+ latency spikes

Counter	Meaning	Implicates
TcpExtTCPBacklogDrop	socket backlog full while socket was locked	receiver CPU starvation
TcpExtTCPZeroWindowDrop	data dropped when receive window was zero	slow application reader
TcpExtPruneCalled , TcpExtRcvPruned	receive queue memory pressure	tcp_rmem too small or reader too slow
TcpExtTW , TcpExtPAWSEstab	TIME_WAIT churn, timestamp rejections	port reuse pressure, NAT with rewritten timestamps
TcpOutRsts	resets we sent	closing sockets with unread data, or connections to closed ports
UdpInErrors , UdpRcvbufErrors	UDP receive drops	receiver buffer too small / too slow (DNS, QUIC, syslog)
IpExtInNoRoutes	packets with no route	routing or policy-routing misconfiguration

Below TCP, the NIC and the softirq path have their own counters. Drops here never appear in TCP statistics as anything but loss:

```

ethtool -S eth0 | grep -Ei 'drop|err|miss|discard|fifo|no_buf'
#      rx_dropped: 0
#      rx_missed_errors: 21734      <- ring buffer overrun; NIC had
nowhere to put frames
#      tx_dropped: 0
ethtool -g eth0      # ring sizes; raising RX ring is the usual fix
for rx_missed_errors
ethtool -c eth0      # interrupt coalescing
ethtool -k eth0
# offloads: gro, gso, tso, lro (see the capture caveat below)

cat /proc/net/softnet_stat # col 1 processed, col 2 dropped (backlog
full), col 3 time_squeeze
ip -s link show eth0      # per-interface RX/TX errors, drops,
overruns

```

And for connection tracking (Chapter 2), an invisible capacity limit in Kubernetes and on NAT gateways:

```

conntrack -S
# per-CPU: found, invalid, insert_failed, drop, early_drop, ...
conntrack -C      # current entry count
sysctl net.netfilter.nf_conntrack_count net.netfilter.nf_conntrack_max
dmesg | grep -i 'nf_conntrack: table full'

```

Rising `insert_failed` is the signature of source-port collisions under heavy SNAT; rising `drop` with `nf_conntrack_count` pinned at `max` is table exhaustion. Both surface to applications as random connection timeouts under load, never as a clean error.

Packet capture: correct, safe, and last

Capture once counters and logs have narrowed the question to something only bytes on the wire can answer: who sent the RST, was the SYN ever transmitted, what did the peer's ClientHello offer, is the retransmission the same segment or a different one.

```

# Interface selection matters more than anything else.
tcpdump -D      # list capturable interfaces
tcpdump -i any -nn -c 200 -w /var/tmp/cap.pcap 'tcp port 5432 and host
10.42.9.4'

```

`-nn` disables name and port resolution (never let tcpdump do DNS during a DNS incident); `-i any` uses Linux "cooked" capture across all

interfaces, convenient but lossy on link-layer detail and prone to double-counting packets that traverse veth pairs. Modern libpcap defaults to a snap length of 262144 bytes, effectively full-packet; reduce it (`-s 128`) when you only need headers on a busy link.

BPF filter syntax is worth learning; the primitives compose:

```
# Only handshake and reset packets – tiny capture, huge signal
tcpdump -i eth0 -nn 'tcp[tcpflags] & (tcp-syn|tcp-rst|tcp-fin) != 0'

# ICMP "fragmentation needed" – the PMTU signal
tcpdump -i eth0 -nn 'icmp[icmptype] == 3 and icmp[icmpcode] == 4'

# IPv6 equivalent: Packet Too Big
tcpdump -i eth0 -nn 'icmp6 and ip6[40] == 2'

# DNS in both directions, while excluding the SSH session you are
typing into
tcpdump -i any -nn '(udp port 53 or tcp port 53) and not tcp port 22'

# VLAN-tagged traffic needs the vlan keyword before the rest of the
filter
tcpdump -i eth0 -nn 'vlan and host 10.42.9.4'
```

In production, capture to a bounded ring so you cannot fill the disk, and drop privileges:

```
tcpdump -i eth0 -nn -s 128 -W 10 -C 100 -Z tcpdump -w /var/tmp/
inc-4412.pcap 'port 443'
# 10 files of 100 MB each, rotating; runs as user "tcpdump" after
opening the socket
```

Two capture caveats that regularly produce false conclusions:

Offloads distort what you see. The `AF_PACKET` tap `tcpdump` uses sits on the kernel side of segmentation: with GRO/LRO on receive and TSO/GSO on transmit, you will see multi-kilobyte "TCP segments" that never existed on the wire, and your packet and retransmission counts will be wrong. When segment-level truth matters, temporarily disable the offloads (`ethtool -K eth0 gro off lro off gso off tso off`) or capture on the peer or a tap/SPAN port — noting the throughput and CPU cost before doing so on a hot machine.

Capture where the traffic actually is. In Kubernetes the pod's traffic is in the pod's network namespace, not the node's root namespace. Prefer an ephemeral debug container, which shares that namespace, so `tcpdump`

inside it sees the pod's traffic; otherwise enter the namespace from the node:

```
kubectrl debug -it pod/checkout-7f9c --image=nicolaka/netshoot --
profile=netadmin -- tcpdump -i eth0 -nn -c 100

# From the node, for a container whose PID you have:
pid=$(crictl inspect --output go-template --template '{{.info.pid}}' "$
CID")
nsenter -t "$pid" -n tcpdump -i eth0 -nn -c 100 'port 443'
```

Capture *both sides* whenever an "it never arrived" claim is in play. A packet present on the sender's capture and absent on the receiver's localizes the drop between them, which no single-side capture can do.

For analysis, Wireshark (or `tshark` for headless work) turns bytes into narrative. Three features pay for themselves:

- **Follow TCP/TLS/HTTP stream** to reconstruct one conversation out of thousands.
- **Expert Information** and the analysis filters:
`tcp.analysis.retransmission`, `tcp.analysis.fast_retransmission`,
`tcp.analysis.duplicate_ack`, `tcp.analysis.zero_window`,
`tcp.analysis.out_of_order`, `tcp.flags.reset == 1`.
- **TLS decryption via key logging.** TLS 1.3 forward secrecy means the server's private key cannot decrypt a capture; you need the ephemeral secrets. Most TLS libraries export them when `SSLKEYLOGFILE` is set (curl, Chrome, Firefox, Go's `tls.Config.KeyLogWriter`, NSS, OpenSSL-based apps that opt in). This works for QUIC too:

```
export SSLKEYLOGFILE=/var/tmp/keys.log
curl -sS --http2 -o /dev/null https://api.internal.example.com/v1/
health
tshark -r /var/tmp/cap.pcap -o "tls.keylog_file:/var/tmp/keys.log" -Y h
ttp2 -V | head -50
```

Treat the keylog file as a secret: it decrypts everything captured in that window.

A `tshark` one-liner for a quick per-stream latency scan:


```
tshark -r cap.pcap -q -z conv,tcp | head -20
tshark -r cap.pcap -Y 'tcp.analysis.retransmission' -T fields \
-e frame.time_relative -e ip.src -e ip.dst -e tcp.seq | head
```

Always-on: eBPF and flow-level network observability

Everything above is *reactive*: you must be present while the problem happens, on the right machine. eBPF (Volume 2, Chapter 11) changes the economics — attach to kernel tracepoints and kprobes at negligible overhead and leave it running. The BCC/libbpf tool set covers most network questions directly (on Debian and Ubuntu the BCC builds are suffixed, e.g. `tcplife-bpfcc`):

```
# Every TCP session that closes: endpoints, bytes, and lifetime
tcplife
# PID    COMM      LADDR      LPORT RADDR      RPORT TX_KB RX_KB
MS
# 1041   app        10.42.7.19  51234 10.42.9.4   5432   12   340
8.9
# 1041   app        10.42.7.19  51236 10.42.9.4   5432    0    0
3002.1

# Connect latency — separates "slow DNS" from "slow handshake" at the
syscall level
tcpconnlat
# PID    COMM    IP SADDR      DADDR      DPORT LAT(ms)
# 1041   app     4 10.42.7.19  10.42.9.4   5432   0.42
# 1041   app     4 10.42.7.19  10.42.9.7   5432 1002.11

# Every retransmission, with the socket state at the time
tcpretrans
# TIME      PID    IP LADDR:LPORT      T> RADDR:RPORT      STATE
# 09:21:04 0      4 10.42.7.19:51234   R> 10.42.9.4:5432
ESTABLISHED
# 09:21:05 0      4 10.42.7.19:51240   R> 10.42.9.4:5432
SYN_SENT
```

Those three answer, respectively: "which connections are short-lived or hung," "is connect latency bimodal with a ~ 1 s or ~ 3 s cluster" (SYN retransmission — handshake loss), and "who is retransmitting, in which state." `tcpconnect`, `tcpaccept`, and `tcpsynbl` (SYN backlog occupancy histogram) round out the set. Since Linux 5.17 the `skb:kfree_skb` tracepoint carries a drop *reason*, which turns "a packet disappeared" into a named cause:

```
bpfftrace -e 'tracepoint:skb:kfree_skb { @[args->reason] = count(); }'
```

At cluster scale, the same idea is productized. Cilium's Hubble records flow-level events from the eBPF datapath — including *verdicts* — so policy drops and connection failures are queryable without touching a node:

```
hubble observe --namespace checkout --verdict DROPPED --last 100
# Aug 13 09:22:01 checkout/api-7f9c:51234 -> payments/api-55d8:8443
policy-denied DROPPED (TCP Flags: SYN)
hubble observe --to-pod payments/api-55d8 --protocol tcp --follow
cilium monitor --type drop          # node-local view of datapath drops
with reasons
```

The value is not the tool but the property: a `policy-denied` verdict, recorded automatically, converts the most confusing symptom in Kubernetes — a connection that times out with no error anywhere — into a one-line answer.

```
flowchart TD
    subgraph APP["Application and HTTP"]
        A1["curl -v and -w timings"]
        A2["Access logs: LB, proxy, mesh sidecar"]
        A3["Distributed traces and spans"]
    end
    subgraph SEC["Names and TLS"]
        B1["dig, resolvectl, getent hosts"]
        B2["openssl s_client, SSLKEYLOGFILE decryption"]
    end
    subgraph SOCK["Sockets and TCP state"]
        C1["ss -tanp, ss -ti, ss -m"]
        C2["lsof, /proc/PID/fd and limits"]
        C3["eBPF: tcplife, tcpconnlat, tcpretrans, tcpsynbl"]
    end
    subgraph KERN["Kernel network stack"]
        D1["nstat and netstat -s counters"]
        D2["conntrack -S, nf_conntrack_count"]
        D3["/proc/net/softnet_stat, ip -s link"]
        D4["tcpdump and tshark on the nets"]
    end
    subgraph DRIVER["NIC, driver, and path"]
        E1["ethtool -S, -g, -c, -k"]
        E2["ping, traceroute, mtr, path MTU probes"]
        E3["Fabric and cloud VPC flow logs"]
    end
    APP --> SEC --> SOCK --> KERN --> DRIVER
```

A field guide to failure signatures

Most production network incidents are instances of a modest number of patterns; recognizing them by their *shape* is what makes senior engineers fast.

Connection refused versus connection timeout

This is the highest-information distinction in the discipline, so be precise about the mechanism. **Refused** means a TCP RST came back: a host received your SYN, had no listener on that port (or a firewall configured to reject rather than drop), and said so. Something is reachable and alive; your address is right and your port or process is wrong. Diagnosis is local: is the process up, is it bound to `0.0.0.0` or only `127.0.0.1`, is it bound to IPv6 only while the client resolved an A record, did the container port mapping change?

Timeout means silence: your SYN vanished, or the SYN-ACK vanished on the way back. Silence is the fingerprint of a *policy drop* or a *routing problem* — security groups, NetworkPolicy, iptables `DROP`, conntrack exhaustion, a blackholed route, an asymmetric return path. The timing is itself evidence: the client retransmits the SYN with exponential backoff (cumulatively about 1 s, 3 s, 7 s, 15 s on Linux, up to `tcp_syn_retries`, which defaults to 6), so connect latencies piling up at ~1 s and ~3 s are SYN loss, not a slow server.

A third case fools people: **connection succeeds, then resets mid-stream**, which is not a connectivity problem at all. Candidates: an idle timeout on a stateful device that dropped the flow's state and now rejects its packets; a proxy enforcing a request timeout; an application crashing or calling `close()` with unread data, which forces an RST rather than a FIN; or an aggressive `SO_LINGER`. Cloud NAT gateways and LBs commonly expire idle flows after a few minutes, many of them *silently*, so the first packet after the idle period is dropped or reset. The fix is keepalives shorter than the middlebox's idle timeout, at the TCP level (`TCP_KEEPIDLE`) or the application level (HTTP/2 PING, gRPC keepalive).

Intermittent timeouts under load: accept-queue overflow

Symptom: at low traffic everything is fine; above some request rate a small fraction of connections take ~1 s or ~3 s and some fail, while the application logs show no slow request — from its point of view those

connections were never accepted. Evidence: `ss -tlnp` shows `Recv-Q` at `Send-Q`, and `nstat` shows `TcpExtListenOverflows` and `TcpExtListenDrops` climbing in lockstep. Mechanism (Volume 2, Chapter 10): the handshake completed in the kernel, the accept queue was full, so the kernel dropped the client's final ACK. Fixes, in order of correctness: make the application accept faster (it is stalled, often on a lock, a GC pause, or a synchronous dependency); raise `net.core.somaxconn` and the backlog the runtime passes to `listen()`, since raising one without the other does nothing; add capacity. `net.ipv4.tcp_abort_on_overflow=1` converts silent drops into resets — worse for users, better for diagnosis, so use it as a temporary instrument.

Retransmissions and real packet loss

Retransmissions are normal in small quantities; the question is always *rate* and *kind*. Fast retransmit (triggered by duplicate ACKs and SACK) costs a fraction of an RTT and is barely visible. RTO-based retransmission (`TcpExtTCPTimeouts`) costs at least the RTO — Linux clamps the minimum at 200 ms — and is the mechanism behind the classic "p99 is 200 ms above p50 for no reason." Tail loss is especially punishing because no later packets exist to generate duplicate ACKs; the tail loss probe (RFC 8985) mitigates it. Localize loss by comparing `nstat` retransmit deltas at the two endpoints and looking for an `mtr` hop whose loss persists through the destination. Do not forget the boring causes: a NIC ring overrun (`rx_missed_errors`), a saturated link, a duplex mismatch.

The MTU/PMTU blackhole

The signature is unmistakable once you know it: **small requests succeed, large ones hang**. A health check returns instantly; a 4 KB POST or a large response body stalls and eventually times out. TLS handshakes may fail specifically at the certificate message, because that is the first large flight.

The mechanism: the sender emits a full-size segment with the IPv4 Don't Fragment bit set (Linux sets DF by default because it does Path MTU Discovery). Somewhere on the path — a VXLAN or GENEVE overlay, a WireGuard or IPsec tunnel, a PPPoE link — the effective MTU is smaller. That router drops the packet and returns ICMP Type 3 Code 4 ("fragmentation needed and DF set") carrying the next-hop MTU; IPv6 routers do not fragment at all, and the equivalent is ICMPv6 Type 2

("Packet Too Big"). If a firewall blocks that ICMP — a depressingly common "hardening" choice — the sender never learns and retransmits the same oversized segment forever. Small packets fit, so everything *looks* healthy.

Confirm and fix:

```
# Confirm the path MTU empirically
ping -M do -s 1472 -c 3 peer # 1500-byte total; fails on a 1450-byte
path
ping -M do -s 1422 -c 3 peer # 1450-byte total; succeeds

# Are the ICMP messages arriving at all?
tcpdump -i any -nn 'icmp[icmptype] == 3 and icmp[icmpcode] == 4'

# See what the stack believes per destination
ip route get 10.42.9.4
ss -ti dst 10.42.9.4 | grep -o 'pmtu:[0-9]*'
```

Mitigations, best first: **allow the ICMP** (Type 3 Code 4 and ICMPv6 Type 2 are part of IP, not an optional extra). **Clamp MSS** at the tunnel ingress: `iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu`. **Enable packetization-layer PMTUD** (`net.ipv4.tcp_mtu_probing=1` probes when a blackhole is suspected, `=2` always) — RFC 4821's approach of discovering MTU from TCP's own behavior rather than from ICMP; QUIC (Chapter 4) uses DPLPMTUD (RFC 8899) for the same reason. And set consistent MTUs on overlay interfaces rather than discovering the mismatch in production.

TLS handshake and certificate failures

The families are few and each has a distinct fingerprint:

- **Expired certificate** — sudden, total, simultaneous across all clients, at a round timestamp. Verify return code: 10. The subtle variant is a *root* expiring: IdenTrust's DST Root CA X3, the root that cross-signed Let's Encrypt's ISRG Root X1, expired on 30 September 2021, breaking clients with old trust stores or old path-building code (OpenSSL 1.0.2 was the notorious case) while up-to-date clients were unaffected. "Fails on *some* clients" is much harder to read than a uniform outage.
- **Incomplete chain** — the server does not send intermediates. Browsers often paper over this by fetching the issuer via the

certificate's AIA extension; most server-side clients (Go, Java, Python) do not. Signature: "works in my browser, fails from the service."
`openssl s_client -showcerts` shows a chain of length one; `Verify` return code: 21.

- **SNI mismatch** — the client sent no SNI, or the wrong name, and a shared L7 endpoint returned a default certificate or routed to the wrong virtual host. Always test with `-servername` or `--resolve`.
- **Version or cipher mismatch** — an old client against a TLS-1.2-minimum server, or a server requiring a curve or cipher the client lacks: alert `handshake_failure` (40) or `protocol_version` (70).
- **ALPN mismatch** — `no_application_protocol` (120): the client offered only `h2` and the server only `http/1.1`.
- **mTLS client-certificate problems** — expired workload identities, clock skew larger than the certificate's validity window (mesh certificates often live only hours, so minutes of skew are fatal), or an untrusted issuing CA after a root rotation. Check `timedatectl` before theorizing.

Under TLS 1.3 the certificate is encrypted, so a capture no longer shows the chain; the ClientHello (including SNI, unless ECH is in use) and the alert codes remain visible, and the alert alone usually tells you which family you are in.

Stale DNS and failover that did not fail over

Symptom: a database or service failover "completed," but a fraction of clients keep hitting the old address for minutes or hours. Mechanism: some caching layer ignored or floored the TTL. The classic offender is the JVM, which historically cached successful lookups **forever** when a security manager was installed (`networkaddress.cache.ttl=-1`); modern JDKs default to a finite value (commonly 30 seconds), but pin it explicitly anyway. Connection pools are the other half: a pooled connection is *never* re-resolved, so even a perfect TTL story leaves long-lived connections pinned to the dead address until they close. Evidence: `dig` on the affected host shows the new address while `ss -tanp` shows established connections to the old one. The fix is protocol-level, not DNS-level — bounded connection lifetime, health-check-driven eviction, and, for gRPC, name-resolution-aware load balancing (Chapters 8 and 9).

Ephemeral port and conntrack exhaustion

A client making many short-lived outbound connections to the *same* destination consumes 4-tuples. The ephemeral range (`net.ipv4.ip_local_port_range`, typically 32768-60999, about 28,000 ports) bounds concurrent connections to one destination address and port, and `TIME_WAIT` holds each tuple for 60 seconds after close on Linux. Signature: connection setup fails at a suspiciously consistent rate, `ss -s` shows tens of thousands of `TIME_WAIT`, and `nstat` shows `TcpExtTW` churn. Real fixes: connection reuse (keep-alive and a properly sized pool — the actual bug is almost always that the client library is not pooling), spreading across more destination addresses, widening `ip_local_port_range`, and `net.ipv4.tcp_tw_reuse=1` where timestamps are available. Do not go looking for `tcp_tw_recycle`: it was dangerous with NAT and was removed in Linux 4.12.

Behind a shared SNAT device — a cloud NAT gateway, or a node masquerading pod traffic — the same limit applies to the *aggregate* of all clients sharing that source address, one of the most common invisible ceilings in container platforms.

Asymmetric routing and stateful firewalls

If packets leave via path A and return via path B, and a stateful device sits on B without having seen the SYN on A, it drops the return traffic. Signature: connections work in one direction, or from one subnet but not another, or after a routing change that "shouldn't matter." Evidence: the server's capture shows the SYN arriving and the SYN-ACK being sent while the client's capture shows only its own SYNs — sent on one side, absent on the other, which is exactly what dual-sided capture is for. Related: Linux reverse-path filtering (`net.ipv4.conf.all.rp_filter`) itself drops packets arriving on an interface the routing table would not use to reply.

Retry storms and metastable failure

Signature: a brief backend slowdown becomes a total outage that does not recover once the original trigger is gone. Mechanism (Chapter 11): every layer retries and the multiplier compounds — a client retrying 3 times through a gateway retrying 3 times through a mesh sidecar retrying 3 times is up to 27 requests per user action. Once the backend is saturated, retries keep it saturated: a metastable failure sustained by its

own load. Evidence: server-side request rate several times the client-side rate; a spike in `URX` or `UO` response flags in mesh access logs; load that does not fall when upstream traffic does. Remediation is architectural — retry budgets rather than fixed counts, retries at one layer only, jittered exponential backoff, circuit breaking, load shedding.

HTTP/2 connection pinning imbalance

Signature: a fleet of identical backends where a few replicas run hot and the rest idle, and scaling out does not help. Mechanism (Chapters 7, 8, 9): HTTP/2 and gRPC multiplex many requests over one long-lived TCP connection, and an L4 load balancer balances *connections* — so once a client's connection lands on a backend, every request on it lands there too for the life of the connection. Add a client fleet that opens connections at startup and never rotates them and the assignment freezes, including across a scale-out, since new replicas receive no existing connections. Fixes: balance at L7, so a proxy distributes individual streams; force rebalancing with server-side connection age limits (gRPC's `MAX_CONNECTION_AGE` plus `MAX_CONNECTION_AGE_GRACE` sends `GOAWAY` and lets clients re-resolve); or use client-side, xDS-driven balancing so the client knows all endpoints.

flowchart LR

S1["Instant connection refused"] --> C1["Nothing listening: wrong port, bound to loopback or IPv6 only, process down, firewall set to reject"]

S2["Connect hangs, then fails at about 1s, 3s, or 7s"] --> C2["SYN or SYN-ACK dropped: policy, routing, conntrack exhaustion, asymmetric return path"]

S3["Small requests fine, large ones hang"] --> C3["PMTU blackhole: ICMP type 3 code 4 or ICMPv6 type 2 filtered"]

S4["Intermittent failures only above a load threshold"] --> C4["Accept-queue overflow, ephemeral port or conntrack exhaustion, NIC ring drops"]

S5["Works in browser, fails from the service"] --> C5["Incomplete certificate chain or missing SNI or ALPN mismatch"]

S6["Total simultaneous TLS failure at a round timestamp"] --> C6["Certificate or intermediate expiry, or clock skew"]

S7["Failover done but some clients still hit the old host"] --> C7["Cached DNS beyond TTL, or pooled connections never re-resolved"]

S8["p99 exceeds p50 by roughly one RT0"] --> C8["Tail loss and RT0-based retransmission"]

S9["A few replicas hot, most idle"] --> C9["HTTP/2 or gRPC connection pinning under L4 balancing"]

S10["Backend stays saturated after the trigger is gone"] --> C10["Retry storm and metastable failure"]

S11["External name lookups slow, internal ones fast"] --> C11["ndots search-domain amplification"]

Distributed-systems lens: debugging a fleet you cannot log into

Everything above assumes a shell on the machine where the problem is happening. At fleet scale that assumption fails three ways at once: the failure is a fraction of a percent, so by the time you attach it has moved; there are thousands of candidate machines; and pods are ephemeral, so the container that failed was replaced before you finished typing. Interactive debugging does not scale. Make the network *continuously observable*, so the interactive step, when it comes, is aimed.

Instrument every hop, not just the ends. The minimum viable network telemetry is RED metrics (rate, errors, duration) *per hop*, emitted by the intermediaries themselves — CDN, edge LB, API gateway, ingress proxy, sidecars on both sides of every internal call — plus USE-style saturation signals for the resources that silently drop traffic (accept queues, conntrack tables, connection pools, NIC rings). The point is arithmetic: when hop n reports 12,000 rps and hop $n+1$ reports 11,400,

you have localized 600 lost requests to one link in seconds, without a capture.

The proxy layer is your richest per-hop telemetry, and it is free. A mesh sidecar or L7 proxy sees, for every request, the upstream it chose, whether the connection was reused, TLS details, per-phase timing, and — critically — a *reason code* when things go wrong. Envoy's access-log response flags encode exactly the distinctions this chapter has been drawing: `UF` upstream connection failure, `UH` no healthy upstream, `UO` upstream overflow (circuit breaking), `UT` upstream request timeout, `URX` retry limit exceeded, `NR` no route configured, `DC` downstream connection termination, `SI` stream idle timeout, `LH` local service failed health check. With `%RESPONSE_CODE_DETAILS%` you often get the root cause verbatim:

```
access_log:
  - name: envoy.access_loggers.file
    typed_config:
      "@type": type.googleapis.com/
envoy.extensions.access_loggers.file.v3.FileAccessLog
      path: /dev/stdout
      log_format:
        json_format:
          start_time: "%START_TIME%"
          method: "%REQ(:METHOD)%"
          path: "%REQ(X-ENVOY-ORIGINAL-PATH?:PATH)%"
          protocol: "%PROTOCOL%"
          response_code: "%RESPONSE_CODE%"
          response_flags: "%RESPONSE_FLAGS%"
          response_code_details: "%RESPONSE_CODE_DETAILS%"
          connection_termination_details: "%CONNECTION_TERMINATION_DETA
ILS%"
          duration_ms: "%DURATION%"
          upstream_service_time: "%REQ(X-ENVOY-UPSTREAM-SERVICE-TIME)%"
          upstream_host: "%UPSTREAM_HOST%"
          upstream_cluster: "%UPSTREAM_CLUSTER%"
          upstream_transport_failure_reason: "%UPSTREAM_TRANSPORT_FAILU
RE_REASON%"
          trace_id: "%REQ(X-B3-TRACEID)%"
          request_id: "%REQ(X-REQUEST-ID)%"
```

`UPSTREAM_TRANSPORT_FAILURE_REASON` alone will hand you TLS verification errors in production that would otherwise take an afternoon of `openssl s_client`.

Attribute latency with traces, not guesses. A distributed trace (Volume 11, Chapter 4) is the only artifact that decomposes end-to-end latency across hops owned by different teams. Two practices make traces far more useful here. First, emit a client-side span for *connection acquisition* separately from the request: an `http.connect` span or a `pool.wait` attribute distinguishes "the server was slow" from "we waited 300 ms for a free connection in our own pool." Second, ensure both the client-side and the server-side span exist for each hop; the *difference* between them is, by definition, the network plus queueing on both sides. Beware clock skew: durations are trustworthy, cross-host instants are not.

Correlate the two sides. The most useful cross-cutting question in a distributed network incident is: *does the server have any record of this request?* Propagate a request ID (or reuse the trace ID) and log it on both sides. Then:

- Client timed out, server logged a fast success → the response was lost or delayed on the return path, or the client's event loop was blocked. Look at return-path drops, client GC and scheduling, and — very commonly — a proxy that timed out earlier than the client and swallowed the response.
- Client timed out, server has no record at all → the request never reached the application. It died before `accept()` (accept-queue overflow), in the network (policy drop, conntrack, PMTU), or at a proxy that never routed it (`NR` , `UH`).
- Server logged it slow, client saw it fast → sampling or clock issues in your telemetry; fix the instrument before trusting it.

```
sequenceDiagram
    participant C as Client app
    participant CS as Client sidecar
    participant LB as L4 or L7 load balancer
    participant SS as Server sidecar
    participant S as Server app
    C->>CS: request with trace-id abc123
    CS->>LB: forwarded, request-id preserved
    LB->>SS: forwarded to endpoint 10.42.9.4
    Note over SS,S: app accept queue full, final ACK dropped
    SS --x S: connection never handed to the application
    C->>C: client timeout after 2s, logs abc123 as failed
    Note over SS: sidecar access log shows response flag UF or UT for abc123
    Note over SS: nstat on the node shows TcpExtListenOverflows rising
    Note over C,S: Signature - client failure with no server-side application record
```

Keep a capture capability you can trigger, but treat it as a scalpel.

The mature setup is on-demand: a debug image (`netshoot` or an internal equivalent) in every namespace, a pre-approved procedure for capturing into a pod's network namespace, ring-buffered writes to a bounded location, and a policy that treats captures as sensitive data. Captures contain customer payloads unless TLS protects them, and the `SSLKEYLOGFILE` trick that makes TLS readable makes them radioactive: short retention, restricted access, never a shared bucket.

Instrument the client, not just the server. The most under-observed component in most architectures is the client library: pool size and wait time, resolution latency, retry counts, circuit-breaker state, *per-attempt* rather than *per-call* latency. Without these, pool exhaustion, resolution stalls, and retry amplification are invisible from both ends at once.

Finally, a cultural note that matters more than any tool. The layered method works only because someone wrote down what the layers *are* for your system: the real hop list from browser to database, the timeout at each hop, the telemetry each hop emits. Teams that maintain that document localize failures in minutes; teams that do not rediscover their own topology during every incident, under pressure — exactly when the "it's always DNS" reflex replaces thinking. Write the hop list down, keep each hop's timeout shorter than its caller's (Chapter 11), and make every hop emit a reason code when it fails.

Key takeaways

- Debug by layer in dependency order, stopping at the first stage that misbehaves; then bisect the path by moving your vantage point closer to the server until the failure disappears.
- `curl -w` splits latency into DNS, TCP, TLS, TTFB, and transfer, cumulatively — read the *deltas*. `--resolve` tests one backend at a time with correct SNI and `Host`.
- `dig` speaks DNS; your application speaks NSS. Use `getent hosts` and `resolvectl` to reproduce what the process sees, and remember `+trace` validates the zone, not your cache.
- `ping` and `traceroute` measure ICMP, which routers rate-limit and deprioritize. Middle-hop loss that does not persist to the destination is an artifact, and ECMP and anycast mean `traceroute` often shows a superposition of paths.
- `ss -ti` is a direct read of the TCP control block: `rtt` versus `minrtt` reveals queueing, `bytes_retrans` quantifies loss, `app_limited` tells you not to blame the network, `skmem's d` field counts socket drops. On listeners, `Recv-Q / Send-Q` are accept-queue depth and its limit.
- Counters beat captures for proving drops: `TcpExtListenOverflows`, `TcpExtTCPSynRetrans`, `TcpExtTCPTimeouts`, `rx_missed_errors`, and `conntrack -S`.
- Capture last and capture correctly: right interface and namespace, bounded BPF filters, offloads accounted for, both sides when the question is "did it arrive?" Decrypt with `SSLKEYLOGFILE`, then treat the capture as a secret. eBPF tools and flow observability give the same evidence continuously and fleet-wide.
- Learn the signatures: refused means "reachable, wrong port"; timeout means "silently dropped"; small-works/large-hangs means PMTU blackhole; browser-works/service-fails means an incomplete chain; a few hot replicas means HTTP/2 pinning; load that will not subside means a retry storm.
- At fleet scale, per-hop RED metrics, proxy access logs with reason codes, and distributed traces localize the failing hop before you reach for a shell. A client-side failure with no server-side record is the highest-signal observation in distributed debugging.

Further reading

- `tcpdump` and `pcap-filter` manual pages — <https://www.tcpdump.org/manpages/> — the authoritative BPF filter syntax reference.
- Wireshark User's Guide — <https://www.wireshark.org/docs/> — TCP analysis flags, Expert Information, and TLS decryption with a key log file.
- `ss(8)`, `nstat(8)`, and `ip-route(8)` from `iproute2` — <https://man7.org/linux/man-pages/man8/ss.8.html>.
- Linux kernel networking documentation — <https://docs.kernel.org/networking/> — especially `snmp_counter.rst`, which documents what each `nstat / netstat -s` counter increments on.
- Brendan Gregg, *BPF Performance Tools* (Addison-Wesley, 2019), Chapter 10, for the networking eBPF tools; the BCC repository is at <https://github.com/iovisor/bcc>.
- curl documentation — <https://curl.se/docs/manpage.html> — with the exit-code list at <https://curl.se/libcurl/c/libcurl-errors.html>.
- BIND `dig` documentation — <https://bind9.readthedocs.io/> — and RFC 5001 (NSID).
- RFC 1191 (*Path MTU Discovery*), RFC 8201 (*Path MTU Discovery for IP version 6*), RFC 4821 (*Packetization Layer Path MTU Discovery*), and RFC 8899 (*Datagram PLPMTUD*, used by QUIC).
- RFC 4443 (*ICMPv6*) and RFC 4890 (*Recommendations for Filtering ICMPv6 Messages in Firewalls*).
- RFC 8985 (*RACK-TLP*) — the loss detection behind Linux's tail loss probe.
- Brice Augustin et al., "Avoiding traceroute anomalies with Paris traceroute" (IMC 2006).
- Envoy access logging — https://www.envoyproxy.io/docs/envoy/latest/configuration/observability/access_log/usage — the response-flag table and `RESPONSE_CODE_DETAILS`.
- Cilium and Hubble documentation — <https://docs.cilium.io/> — flow verdicts and drop reasons.
- Nathan Bronson et al., "Metastable Failures in Distributed Systems" (HotOS 2021).

- Volume 2, Chapter 10 (*The Linux Network Stack*) for the SYN and accept queues and the receive path; Volume 2, Chapter 11 (*Performance Analysis: perf, ftrace, and eBPF*) for the tracing infrastructure these tools are built on.
- Volume 11, Chapter 2 (*Metrics and the Golden Signals*), Chapter 4 (*Distributed Tracing and OpenTelemetry*), and Chapter 5 (*Incident Response and On-Call*).
- Chapters 1 through 11 of this volume are the model this chapter tests against. Debugging is nothing more than applying those models one layer at a time until the observations stop matching the model — and that mismatch is always where the bug lives.